



**Faculty of Computer and Data Science,
Alexandria University**

**The graduation project was submitted in partial fulfillment of the B.SC.
Degree**

Traffic Manager (2024-2025)

**A system that aims for less congestion,
and better transportation**

Authors:

Ahmed Ibrahim Hassan

Bichay Adel Ramses

Emad Mohammed Fawzy

Kerolss Bolis Shehata

Marwa Shebl Mohammed

Mayan Islam Mohamed

Mostafa Hussian Mostafa

Remon Ehab Ramses

Taha Fawzy Elshreif

Youssef Ibrahim Hanfy

Supervisors

Dr. Amira A. Alshazly

ACKNOWLEDGMENT

We express our heartfelt gratitude to God for his unwavering support and blessings throughout this journey.

We are deeply thankful to our supervisor, Dr. Amira A. Alshazly, for her invaluable guidance, encouragement, and steadfast assistance, which have been instrumental to the success of this project.

We would also like to extend our sincere appreciation to Dr. Shaimaa Lazem, Dr. Samia Hafez, Eng. Nouran Hisham and Eng. Mohamed Hatem Abdulkader for their support, insightful feedback, and encouragement throughout the course of our work.

Our sincere thanks also go to the authors and contributors of the reference report and paper [1], and [2] whose work provided valuable insights and inspiration for this project.

Finally, we extend our deepest appreciation to our families, whose unwavering support and encouragement have been the cornerstone of our achievements.

LIST OF ABBREVIATIONS

2WaySingle	The area used for benchmark with sumo-RL data	5.4.4.1.2 SUMO-RL DATA Benchmark (2WaySingle)	211
A2C	Advantage Actor-Critic		24
A3C	Asynchronous Advantage Actor-Critic		25
ACC	Accident		57
ACK	Acknowledgement		57
AI	Artificial Intelligence		1
ANOVA	Analysis Of VAriance		115
API	Application Programming Interface		4
Benchmark	Benechmark experiments of our environment		81
CAM	Correlational Attention Mechanism		27
CHK	Check (for internet)		57
CNN	Convolutional Neural Network		30
CV	Computer Vision		29
D2QN /DDQN	Double Deep Q-Network		18
D3QN /Dueling D2QN	Dueling Double Deep Q-Network		19
Data 1/Area 1	Tomart data		64
Data 2/Area 2	Mosheer Ismail		64
Data 3/Area 3	San Stefano		64
DITLCS	Dynamic and Intelligent Traffic Light Control System DITLCS		21
DNN	Deep Neural Network		10
DQN	Deep Q-Network		17
DRIVE MATE	“Name” of mobile app		57

DRL /RL	Reinforcement Learning/Deep	11
EM	Emergency Mode	22
EMR	Emergency	57
Environment Experiments	Experiments of our enviroment in different scenarios	82
EpsDQN	Epsilon-Greedy Deep Q- Network	80
FM	Fair Mode	22
FR	Functional Requirement	267
GAT	Graph Attention Networks	26
GCD	Greatest Common Divisor	68
GPS	Global Positioning System	259
Grouped	Environment with action space groubed by direction	67
GSCTSA-A3C	Graph-Structured Correlation Time-Spatial Attention – Async Actor-Critic	27
GYM	OpenAI gym	4
HighGrouped	Environment of one signal per lanes	67
HMAS	Hierarchical Multi-Agent Systems	20
IoT	Internet of Things	3737
IOU	Intersection Over Union	30
IP	Internet Protocol	61
KL	Kullback–Leibler (Divergence)	29
LAs	Local agent	20
Literature	Literature reward that focus on waiting time	70
LR	Linear Regression	33
LSTM	Long Short-Term Memory	21
MAE	Mean Absolute Error	9
MFDQL-DTCMFDQL-DTC	Mean Field Double Q- Learning with Dynamic Timing Control	23

ML	Machine Learning	5
MPE	Mean Percentage Error	10
MQTT	Message Queuing Telemetry Transport	265
NFR	Non-Functional Requirement	268
NLP	Natural Language Processing	27
NMS	Non-Maximum Suppression	30
NN / DNN	Neural Network /Deep	10
OSM	OpenStreetMap	64
PATM	Process Activity Task Matrix	43
PM	Priority Mode	22
PPO	Proximal Policy Optimization	28
Proposed	Our proposed reward that optimizes for state space	68
QL	Queue length	57
RACS-A2C	Region-Aware Cooperative Strategy A2C	25
RAs	Regional Agents	21
Reference	Reward for reference project	69
RF	Random forest	8
RMSE	Root Mean Square Error	9
SPI	Software Process Improvement	2
SRS	Software Requirements Specification	267
SUMO	Simulation of Urban MObility	3
SumoEnv /Full	Environment with all combination in action space	68
TCP/IP	Transmission Control Protocol / Internet Protocol	57
Temporal Difference /TD	Temporal Difference	11
TraCI	Traffic Control Interface	5
TRPO	Trust Region Policy Optimization	29

V1.0.0	Version 1.0.0	81
V2.0.0	Version 2.0.0	118
V2.0.1	Version 2.0.1	139
XGBoost	Extreme Gradient Boosting	8
Yolo	You Only Look Once	29

DECLARATION

We hereby declare that this graduation project report is the result of our efforts, with guidance from our supervisors. Any external sources used in the work have been appropriately cited, and this document has not been submitted to any other institution for any degree or qualification.

Name	ID	Signature
Ahmed Ibrahim Hassan	20221373950	
Bichay Adel Ramses	20221440637	
Emad Mohammed Fawzy	20221461122	
Kerolss Bolis Shehata	20221374504	
Marwa Shebl Mohammed	20221448840	
Mayan Islam Mohamed	20201323258	
Mostafa Hussian Mostafa	20221449690	
Remon Ehab Ramses	20221459758	
Taha Fawzy Elshreif	20221466557	
Youssef Ibrahim Hanfy	20221454778	

TABLE OF CONTENTS

ACKNOWLEDGMENT	II
LIST OF ABBREVIATIONS	III
DECLARATION	VII
TABLE OF CONTENTS	VIII
LIST OF FIGURES	XV
LIST OF TABLES	XVII
LIST OF EQUATIONS	XIX
ABSTRACT	XX
CHAPTER 1: INTRODUCTION	1
1.1 Project Background	1
1.2 Importance of the Topic	1
1.3 Project Contribution	1
1.4 Report Organization	2
1.5 Conclusion	2
CHAPTER 2: SCIENTIFIC BACKGROUND.....	3
2.1 Simulation	3
2.1.1 GYM.....	4
2.1.2 Simulation of Urban Mobility (SUMO)	4
2.2 Machine Learning	5
2.2.1 Heuristic algorithms	5
2.2.2 Supervised Learning	6
2.2.2.1 Decision Tree	6
2.2.2.1.1 Random Forest.....	8

2.2.2.1.2 XGBoost	8
2.2.2.2 Deep Neural Network (DNN).....	10
2.2.3 Reinforcement Learning	11
2.2.3.2 QN Algorithms.....	16
2.2.3.2.1 Q Network (QN)	16
2.2.3.2.2 Deep Q Network (DQN).....	17
2.2.3.2.3 Double Deep Q Network (D2QN).....	18
2.2.3.2.4 Dual DDQN (D3QN)	19
2.2.3.3 Hierarchical Multi-Agent Systems (HMAS)	20
2.2.3.4 Dynamic and Intelligent Traffic Light Control System DITLCS	21
2.2.3.5 Mean Field Double Q-Learning with Dynamic Timing Control (MFDQL-DTC)	23
2.2.3.6 Actor-Critic Algorithm and Variation.....	24
2.2.3.6.1 Advantage Actor-Critic (A2C).....	24
2.2.3.6.2 Async Advantage Actor critic (A3C)	25
2.2.3.7 Region-Aware Cooperative Strategy A2C (RACS-A2C).....	25
2.2.3.8 Graph-Structured Correlation Time-Spatial Attention – Async Actor-Critic (GSCTSA-A3C)	27
2.2.3.9 Proximal Policy Optimization (PPO).....	28
2.3 You Only look Once (Yolo)	29
2.4 Conclusion	31
CHAPTER 3: LITERATURE REVIEW	32
3.1 Main results of related papers.....	32
3.2 Comparison between related papers	35
3.3 Comparison between similar systems	38
3.4 The Need to Extend Related Work	38
3.5 Scope of Work.....	40
3.6 Conclusion	40
CHAPTER 4: SYSTEM DEVELOPMENT	41
4.1 Software Process Improvement (SPI model)	41
4.1.1 Phases	41
4.1.2 Team Management	41
4.1.3 Iterations.....	42
4.1.4 Results.....	43
4.2 Process Activity Task Matrix (PATM MATRIX)	43
4.3 Requirements	45
4.4 Design diagrams	45

4.4.1 Activity Diagram (Overall Architecture)	46
4.4.2 High-Level Workflow Flowchart (Experimental Architecture)	46
4.4.3 Tools and Environment	48
4.4.4 Use Case Diagram (Mobile App)	49
4.4.5 Class Diagram	50
4.4.6 Database Schema Entity-Relationship Diagram (ERD)	51
4.4.7 Traffic Backend Component diagram	51
4.4.8 Prototype ESP32 Schema	52
4.4.9 Pipeline of traffic management processing model.....	53
4.5 Prototype design	53
4.5.1 Computer Vision Implementation	53
4.5.1.1 State Space estimation.....	53
4.5.1.2 Prototype Video Streaming and Processing Delays	55
4.5.2 Ardinuo Implementation.....	56
4.5.3 Backend Implementation	57
4.5.4 Mobile implementation	57
4.5.5 Performance Monitoring	58
4.5.6 Scaling the prototype	59
4.6 DB Online Prototype	60
4.7 Module decomposition.....	61
4.8 Basic implemented security	61
4.9 Open-Source Availability And License	61
4.10 Conclusion	62
 CHAPTER 5: EXPERIMENTAL DESIGN, EVALUATION, AND SYSTEM PROTOTYPING	 63
5.1 Environment Design	63
5.1.1 Problem and Environment Design.....	63
5.1.2 Data Used	64
5.1.3 State Space.....	66
5.1.4 Action Space	67
5.1.4.1 HighGroupedSumoEnv (HighGrouped).....	67
5.1.4.2 GroupedSumoEnv (Grouped)	67
5.1.4.3 Full (SumoEnv).....	68
5.1.5 Reward	68
5.1.5.1 Proposed Reward.....	68
5.1.5.1.1 Proposed Reward Testing	69
5.1.5.2 Reference Project Reward	69
5.1.5.3 Literature Reward.....	70
5.1.6 Preprocessing	71
5.1.7 Model Used.....	71
5.1.8 Traffic scale.....	71

5.2 Hyperparameter Selection and Tuning	72
5.2.1 PPO and D3QN Hyperparameters.....	72
5.2.2 Parameters from the Benchmark	74
5.2.3 Experiment-Specific Parameters:	74
5.2.4 Sumo Parameters	75
5.2.5 End User Parameters.....	79
5.3 Benchmark Configuration	80
5.4 Experiment Design	81
5.4.1 Version 1.0.0.....	81
5.4.1.1 Parameters Defination of Experiments	81
5.4.1.1.1 SUMO Static light	81
5.4.1.1.2 Benchmark	81
5.4.1.1.3 Environment Experiments	82
5.4.1.2 Metrics	83
5.4.1.3 Experiment Design	84
5.4.1.3.1 Conducted Experiments in Version 1	84
5.4.1.4 Results of Experiments	85
5.4.1.4.1 Results of Benchmark Experiments	85
5.4.1.4.1.1 Plotting for Area 1, Scale Normal.....	86
5.4.1.4.1.2 Plotting for Area 1, Scale Crowded	87
5.4.1.4.1.3 Plotting for Area 2, Scale Normal	88
5.4.1.4.1.4 Plotting for Area 2, Scale Crowded	89
5.4.1.4.2 Results of Environment Experiments.....	90
5.4.1.4.2.1 Effect on Episodes.....	90
5.4.1.4.2.2 Behavior on different Sumo timing	97
5.4.1.4.2.3 Further studying on Data 2.....	103
5.4.1.4.2.4 Further studying on Data 3.....	109
5.4.1.4.2.5 Best Results.....	113
5.4.1.5 Analysis of Variance (ANOVA).....	115
5.4.1.6 Discussion of results	116
5.4.1.6.1 Discussion of the results of Benchmark	116
5.4.1.6.2 Discussion of the results of Environmental Experiments	116
5.4.1.7 Limitation of Version 1.0.0.....	118
5.4.2 Version 2.0.0.....	118
5.4.2.1 Key Modifications.....	118
5.4.2.2 Modified Experiments Design	119
5.4.2.3 Results	120
5.4.2.3.1 Results show effect of episodes	121
5.4.2.3.2 Results with proposed reward.....	125
5.4.2.3.3 Results with literature reward	136
5.4.2.4 Limitation of Version 2.0.0.....	139
5.4.3 Version 2.0.1.....	139
5.4.3.1 Key Modifications.....	139
5.4.3.1.1 Normalizing Reward and Testing It	140
5.4.3.2 Modified Environmental Design	142
5.4.3.2.1 Environmental Experiments	143

5.4.3.2.2 Sumo-RL experiments.....	144
5.4.3.2.3 Metrics	145
5.4.3.3 Results	146
5.4.3.3.1 Results for Mosheer Ismail	146
5.4.3.3.2 Results for San Stefano	176
5.4.3.3.3 Analysis of Variance (Anova)	206
5.4.3.3.4 Analysis of results	207
5.4.3.3.4.1 Performance Results	207
5.4.3.3.4.2 Best Performance Metrics	207
5.4.3.3.4.3 Comparison to Previous Versions	208
5.4.4 Version 2.0.2.....	209
5.4.4.1 Key Modifications.....	209
5.4.4.1.1 Reward Improvement.....	209
5.4.4.1.2 SUMO-RL DATA Benchmark (2WaySingle)	211
5.4.4.1.3 SUMO-RL DATA Experiment Design	212
5.4.4.2 Results	214
5.4.4.2.1 Sumo-RL Bench.....	214
5.4.4.2.1.1 Results	214
5.4.4.2.1.2 Anova.....	219
5.4.4.2.1.3 Discussion	220
5.4.4.2.2 Environmental experiments.....	220
5.4.4.2.2.1 Mosher	220
5.4.4.2.2.2 Stefano	235
5.4.4.2.2.3 Anova.....	250
5.4.4.2.2.4 Discussion	251
5.5 Final Results of System	255
5.9 Conclusion	256
CHAPTER 6: CONCLUSION AND FUTURE WORK.....	258
6.1 Conclusion	258
6.2 Future work.....	258
REFERENCES	260
APPENDIX I: CHALLENGE FACED.....	264
APPENDIX II: SOFTWARE REQUIREMENTS SPECIFICATION (SRS).....	267
APPENDIX III: MINUTES OF MEETING.....	270
APPENDIX IV: WEEKLY TIMESHEET	271

APPENDIX V: JIRA PLAN273

APPENDIX VI: FILE STRUCTURE274

LIST OF FIGURES

Figure 2.1 Visualizing Entropy Function	7
Figure 2.2 Single Neuron	10
Figure 2.3 NN with 4 layers	11
Figure 2.4 Overview of the RL process	13
Figure 2.5 Online and Offline RL interaction	13
Figure 2.6 Q function	17
Figure 2.7 QN and DQN architecture overview	18
Figure 2.8 Figure 2.7 QN and DQN architecture overview	18
Figure 2.9 HMAS architecture	21
Figure 2.10 DITLCS fuzzy logic	22
Figure 2.11 MFDQL-DTC architecture	24
Figure 2.12 Actor-Critic Architecture	25
Figure 2.13 GSCTSA-A3C architecture	28
Figure 2.14 Yolo Typical Neural Network	30
Figure 3.1 Plot of performance of algorithms according to [29]	33
Figure 3.2 Relation between waiting time and simulation time in GSCTSA-A3C according to [3]	34
Figure 3.3 Plotting average wait time, throughput in GSCTSA-A3C according to [3]	34
Figure 3.4 Plotting average queue time, and speed in GSCTSA-A3C according to [3]	35
Figure 4.1 QR code of JIRA plan	42
Figure 4.2 SPI Diagram	43
Figure 4.3 Activity Diagram	46
Figure 4.4 High-Level Workflow Flowchart	48
Figure 4.5 Tools and Environment	48
Figure 4.6 Use Case Diagram	49
Figure 4.7 Class Diagram	50
Figure 4.8 ERD diagram	51
Figure 4.9 traffic backend diagram	52
Figure 4.10 Prototype ESP32 Schema	52
Figure 4.11 Pipeline of traffic management processing model	53
Figure 4.12 Testing prototype on video hosted on local IP address	56
Figure 4.13 Screens of the UI of the prototype mobile app	58
Figure 4.14 Screen of performance monitoring dashboard	59
Figure 4.15 Organizing of files in modular design	61

Figure 4.16 QR code of the project main GitHub	62
Figure 5.1 Data 1 Tomart on google maps (left), SUMO (right)	65
Figure 5.2 Data 2 Mosheer Ismail on google maps (left), SUMO (right).....	65
Figure 5.3 Data 3 San Stefano on google maps (left), SUMO (right).....	65
Figure 5.4 Congestion level in alexandria according to TomTom.....	72
Figure 5.5 Benechmark Plotting for Area 1, Scale Normal	86
Figure 5.6 Benechmark Plotting for Area 1, Scale Crowded	87
Figure 5.7 Benechmark Plotting for Area 2, Scale Normal	88
Figure 5.8 Benechmark Plotting for Area 2, Scale Crowded	89
Figure 5.9 Effect of running many episodes on reward	90
Figure 5.10 Effect of running many episodes on evaluated reward.....	91
Figure 5.11 Effect of running many episodes on waiting time	92
Figure 5.12 Effect of running many episodes on speed	93
Figure 5.13 Effect of running many episodes on depart delay	94
Figure 5.14 Effect of running many episodes on timeloss.....	95
Figure 5.15 Effect of running many episodes on number of waiting cars	96
Figure 5.16 Effect of different sumo timing on reward.....	97
Figure 5.17 Effect of different sumo timing on evaluated reward.....	97
Figure 5.18 Effect of different sumo timing on waiting time	98
Figure 5.19 Effect of different sumo timing on speed.....	99
Figure 5.20 Effect of different sumo timing on depart delay	100
Figure 5.21 Effect of different sumo timing on timeloss.....	101
Figure 5.22 Effect of different sumo timing on number of waiting cars.....	102
Figure 5.23 Performance comparison across algorithms and environments on reward in area 2	103
Figure 5.24 Performance comparison across algorithms and environments on evaluated reward in area 2.....	104
Figure 5.25 Performance comparison across algorithms and environments on waiting time in area 2	104
Figure 5.26 Performance comparison across algorithms and environments on speed in area 2	105
Figure 5.27 Performance comparison across algorithms and environments on depart delay in area 2	106
Figure 5.28 Performance comparison across algorithms and environments on timeloss in area 2.....	107
Figure 5.29 Performance comparison across algorithms and environments on number of waiting cars in area 2.....	108
Figure 5.30 Performance comparison across algorithms and environments on reward in area 3	109
Figure 5.31 Performance comparison across algorithms and environments on evaluated reward in area 3.....	110

Figure 5.32 Performance comparison across algorithms and environments on waiting time in area 3	110
Figure 5.33 Performance comparison across algorithms and environments on speed in area 3	111
Figure 5.34 Performance comparison across algorithms and environments on depart delay in area 3	111
Figure 5.35 Performance comparison across algorithms and environments on timeloss in area 3.....	112
Figure 5.36 Performance comparison across algorithms and environments on number of waiting cars in area 3.....	113
Figure 5.37 Best results at Area2 with Normal scale.....	113
Figure 5.38 Best results at Area2 with Crowded scale.....	114
Figure 5.39 Best results at Area3 with Normal scale.....	114
Figure 5.40 Best results at Area3 with Crowded scale.....	115
Figure 5.41 Results show effect of episodes at V2.0.0	124
Figure 5.42 Results with proposed reward at V2.0.0.....	136
Figure 5.43 Results with literature reward at Version 2.0.0	139
Figure 5.44 Results of Version 2.0.1 at Mosheer Ismail.....	175
Figure 5.45 Results of Version 2.0.1 at San Stefano	206
Figure 5.46 Area used in sumo-rl Bench	212
Figure 5.47 Results of Sumo-RL Bench.....	219
Figure 5.48 Results of Mosheer at version 2.0.2	235
Figure 5.49 Results of Stefano at version 2.0.2	250
Figure III.1 screen of minutes of meeting	270
Figure IV.1 screen of a weekly timesheet	271
Figure V.1 screen of a JIRA Plan.....	273

LIST OF TABLES

Table 3.1 algorithms result according to [19].....	32
Table 3.2 algorithms result according to [28].....	32
Table 3.3 Results of algorithms according to [29]	33
Table 3.4 Comparison between related papers	35
Table 3.5 Comparison between similar systems	38
Table 4.1 PATM Matrix	43
Table 5.1 State Space Defination.....	66
Table 5.2 D3QN hyperparameters	73
Table 5.3 PPO hyperparameters	73
Table 5.4 randomTrips hyperparameters	75

Table 5.5 sumo cmd hyperparameters	76
Table 5.6 End User Hyperparameters	79
Table 5.7 SUMO experiment parameter defination.....	81
Table 5.8 Benchmark experiment parameter defination.....	81
Table 5.9 Environment experiment parameter defination.....	82
Table 5.10 Metrics defination	83
Table 5.11 Anova results	115
Table 5.12 Modified Environmental Experiments for Version 2.0.0	119
Table 5.13 Testing modified proposed reward for V2.0.1	141
Table 5.14 Modified Environmental Experiments for Version 2.0.1	143
Table 5.15 Sumo-RL Experiment Design for Version 2.0.1.....	144
Table 5.16 Modified Metrices for Version 2.0.1	145
Table 5.17 Anova results for Version 2.0.1	206
Table 5.18 Validation of the reward of version 2.0.2	210
Table 5.19 SUMO-RL DATA Experiment Design	212
Table 5.20 Anova for Sumo-rl Bench	219
Table 5.21 Anova for environmental experiments at version 2.0.2.. Error! Bookmark not defined.	
Table 5.22 Discussion of environmental results	254

LIST OF EQUATIONS

Equation 2.1 Entropy	7
Equation 2.2 Information gain equation.....	7
Equation 2.3 Mean absolute error.....	9
Equation 2.4 R-squared	9
Equation 2.5 Root Mean Square Error.....	10
Equation 2.6 Mean Percentage Error.....	10
Equation 2.7 Basic cumulative reward in RL	12
Equation 2.8 Cumulative reward with discount factor.....	12
Equation 2.9 State Value Function	14
Equation 2.10 Action Value Function.....	14
Equation 2.11 Soft update rule	15
Equation 2.12 Monte Carlo update rule	15
Equation 2.13 Temporal Difference update rule	15
Equation 2.14 Bellman equation	16
Equation 2.15 Advantage function.....	19
Equation 2.16 correlation score between nodes in RACS	26
Equation 2.17 SoftMax normalize of weight	26
Equation 2.18 Average Weight of Intersection in RACS.....	26
Equation 2.19 Correlation with a neighbor in RACS	27
Equation 2.20 LSTM for temporal pattern analysis	27
Equation 2.21 ratio between old policy and updated policy in PPO.....	29
Equation 2.22 CLIP probability loss in PPO.....	29
Equation 4.1 Estimated occupancy	54
Equation 4.2 Estimated queue length	54
Equation 4.3 Estimated throughput.....	54
Equation 4.4 duration without arduino error	56
Equation 5.1 Action space for HighGrouped Sumoenv	67
Equation 5.2 Action space for Grouped Sumoenv	67
Equation 5.3 Action space for Full Sumoenv.....	68
Equation 5.4 Proposed Reward for Version 1.0.0	68
Equation 5.5 Reference Reward	69
Equation 5.6 Literature Reward	70
Equation 5.7 Modified Proposed Reward (Normalized)	140
Equation 5.8 Modified reward of version 2.0.2.....	210

ABSTRACT

Urban traffic congestion remains a major issue that disrupts daily life, delays operations, and negatively impacts the environment, public health, and overall productivity. In densely populated areas such as Egypt, it significantly affects commuting and everyday activities. Despite substantial yearly investments in infrastructure projects such as road and bridge expansions, the problem continues to persist.

This study introduces an optimized intelligent traffic management system based on dynamic control of traffic signals. In contrast to traditional systems that lack adaptability and often contribute to congestion, the proposed framework is trained to determine best decisions at different situations.

This approach aims to reduce vehicle waiting times, improve overall traffic flow, and ensure fairness among road users by leveraging advanced reinforcement learning algorithms. Additionally, the system is designed for easy deployment, monitoring and manually controlling, and continuous enhancement through real-world data.

The framework also aspires to provide a scalable, open-source solution that reduces delays, minimizes energy consumption, and mitigates environmental impacts while remaining flexible for future development and upgrades.

INTRODUCTION

In this chapter, the urban traffic congestion problem is described, with a focus on its specific impact in Egypt. The limitations of traditional traffic systems are highlighted, and the for dynamic, AI-driven control is explained. The key contributions of the project are introduced, including a reinforcement learning-based signal optimization approach and a mechanism need for prioritizing emergency vehicles.

1.1 Project Background

The rapid increase in the number of vehicles on the road has resulted in a significant rise in traffic congestion, leading to numerous challenges for urban areas in Egypt. The smooth functioning of daily life and work routines for millions of people has been disrupted due to this congestion. As cities continue to grow and vehicle ownership increases, greater pressure is placed on the existing traffic infrastructure. Moreover, external factors such as unpredictable weather conditions add another layer of complexity to the management of traffic flow. Delays are exacerbated, road safety is reduced, and maintaining efficient transportation systems in urban environments is made even more difficult [1],[3].

1.2 Importance of the Topic

Considerable challenges are posed by traffic congestion, including negative impacts on the environment, public health, and productivity. The management of traffic effectively requires dynamic, real-time responses to address these issues. Traditional models have been found to be limited in their ability to optimize energy consumption and traffic flow, highlighting the need for innovative solutions [3].

1.3 Project Contribution

Advanced AI techniques, specifically Reinforcement Learning (RL), are utilized by our system to train a model that optimizes traffic flow by simulating traffic conditions in Alexandria's urban streets. Traffic signals are dynamically adjusted based on these simulations. Additionally, an innovative solution is introduced for registered emergency vehicles, such as ambulances and fire trucks, allowing them to log in, select a route, and send this information to the system. The traffic signals along the selected route are identified by the system and adjusted by extending green light durations and shortening red light durations to facilitate smoother passage. To validate the overall structure and functionality of the system, a small-scale prototype simulating a single intersection has been developed. The project's concept, deployment feasibility, and integration with a mobile application are tested through this prototype.

1.4 Report Organization

The structure of the next chapters is organized as follows:

Chapter 2: Scientific Background – The scientific foundations of traffic optimization are reviewed, including both traditional methods and modern advancements.

Chapter 3: Literature review– The existing literature is reviewed, various papers and studies are compared, and the findings are analyzed to identify gaps or areas requiring further exploration, thereby defining the scope of the work and necessary extensions.

Chapter 4: System Requirements and Design – In this chapter, the design of the project will be discussed in detail. The overall system flow will be explained, along with the stages of the SPI model and the software engineering methodologies that were applied.

Chapter 5: Experimental Design, Evaluation, and System Prototyping – In this chapter, the design of the experiments is presented, and their results are analyzed.

Chapter 6: Conclusion and Future Work – A summary of the project is provided, and potential directions for future work are discussed.

1.5 Conclusion

In this chapter, the background of the problem and its impact on Egypt have been discussed. Its significance has been elaborated, followed by an explanation of how the issue is addressed by our project. Finally, the structure of the subsequent chapters has been outlined. The next chapter will dive into the scientific background of algorithms used in the project.

SCIENTIFIC BACKGROUND

In the last chapter, the issue of traffic congestion in urban areas has been introduced, emphasizing the challenges brought about by urbanization and the increasing number of vehicles.

In this Chapter, the key algorithms, solutions, and foundational concepts relevant to related work, as well as those suitable for this project, are discussed.

- Section 1: Simulation – The importance of simulation in traffic management is explained, and SUMO, a commonly used framework for traffic simulation, is introduced.
- Section 2: Machine Learning – The background of machine learning algorithms proposed in related work or considered for this project is covered. The discussion progresses from basic heuristic algorithms through unsupervised and supervised learning, concluding with reinforcement learning, including approaches such as Deep Q-Networks (DQN) and GSCTSA-A3C.
- Section 3: Computer Vision – A background on computer vision, which is utilized in the proposed solution, is provided.

2.1 Simulation

[4]

A simulation is a representation of how an existing or proposed system operates, designed to assist in decision-making by testing various scenarios or process changes. Simulations play a crucial role in performance enhancement, process optimization, safety assurance, theoretical testing, personnel training, and even entertainment through video games. By scientifically modeling systems, simulations allow users to analyze how different conditions and actions impact system performance.

Simulations are particularly valuable when the actual system is inaccessible, too dangerous to study directly, or still in its conceptual stages. A safe and cost-effective means to explore and experiment with different scenarios is provided.

The accuracy of a simulation is heavily dependent on the quality and reliability of the data used in its model construction. As a result, ongoing research focuses on improving methods for verifying and validating simulation models, particularly in computer-based environments.

Modern simulation tools leverage specialized software to create detailed visual models of processes, incorporating factors such as timing, rules, resource allocation, and constraints. For instance, a supermarket simulation might predict customer behavior during peak hours, offering insights to optimize staffing, store layouts, and inventory management.

Several simulation tools are available for addressing traffic congestion:

- **SUMO:** An open-source framework for simulating traffic networks, widely used in research. Due to its free availability, it has been selected for this project.

- **PTV Vissim:** A paid framework offering advanced features such as 3D visualization, pedestrian simulation, and integration with other PTV software.
- **Gymnasium (Gym):** An OpenAI framework that provides various basic environments for training reinforcement learning (RL) agents, with integration capabilities for SUMO, making it useful for RL-based traffic optimization experiments.

2.1.1 GYM

[5]

OpenAI Gym is an open-source Python toolkit designed for developing and comparing reinforcement learning (RL) algorithms. It provides a standardized API for communication between learning algorithms and environments, facilitating development.

Key Features:

- **Standardized API:** Gym offers a consistent interface for various environments, enabling seamless integration with RL algorithms.
- **Diverse Environments:** A wide range of environments is included, allowing for comprehensive testing and benchmarking of RL models.
- **Community Support:** As an open-source project, Gym has an active community that provides extensive resources, tutorials, and support.

Gym can be integrated with traffic simulation tools like SUMO to create environments for training RL agents in traffic management scenarios. This integration allows researchers to develop and test algorithms for optimizing traffic flow, signal control, and other transportation-related tasks within a simulated environment.

2.1.2 Simulation of Urban Mobility (SUMO)

[6]

Simulation of Urban Mobility (SUMO) is an open-source traffic simulation platform designed to model, analyze, and optimize urban mobility within complex road networks. Its microscopic simulation approach enables a highly detailed representation of individual vehicle behaviors, including acceleration, deceleration, lane changes, and route selection. The platform supports various vehicle types—cars, buses, bicycles, and pedestrians—and their interactions with infrastructure components such as traffic signals, pedestrian crossings, and road closures.

One of SUMO's most notable features is its flexibility in integrating with external tools and algorithms, particularly in the domains of artificial intelligence (AI) and machine learning. For instance, SUMO can be combined with reinforcement learning algorithms to design and evaluate adaptive traffic control strategies. This capability transforms SUMO into a dynamic training environment for AI models aimed at optimizing real-time traffic flows, reducing congestion, and minimizing travel times by simulating realistic traffic dynamics.

Key Features of SUMO include.

- **Extensive Data Output Capabilities:** SUMO facilitates detailed analysis and visualization by generating metrics such as vehicle travel times, queue lengths, and fuel consumption. These outputs provide valuable insights for identifying traffic patterns, inefficiencies, and potential solutions.
- **Real-Time Interaction via TraCI:** SUMO provides the Traffic Control Interface (TraCI), allowing users to interact with the simulation in real-time using programming languages like Python. Traci enables external applications to:
 - Retrieve live simulation data, such as vehicle positions, speeds, and waiting times.
 - Dynamically modify simulation parameters, including traffic signal phases, vehicle routing, and lane speed limits.
- TraCI's API includes commands for real-time simulation control and customization, offering flexibility to test innovative traffic management strategies tailored to specific urban challenges.
- By integrating advanced simulation features with AI-driven tools, SUMO provides a versatile platform for researchers and practitioners to develop smarter, more efficient traffic management systems.

2.2 Machine Learning

[7]

Machine learning (ML) is recognized as a subset of artificial intelligence (AI), focusing on the creation of systems capable of learning from data and making decisions or predictions. Rather than being explicitly programmed for every task, ML models are trained to learn patterns and rules from data. It is defined as the field of study that enables computers to learn without explicit programming. As implied by its name, it allows computers to acquire learning capabilities, making them more human-like.

2.2.1 Heuristic algorithms

Heuristic algorithms are designed to efficiently address complex optimization problems by providing approximate solutions when exact methods are impractical.

This approach is particularly useful in scenarios where traditional algorithms are computationally prohibitive. The handling of uncertainty, incomplete information, and large-scale optimization tasks is facilitated by heuristics, making them widely utilized.

Heuristic algorithms are categorized into two approaches based on the complexity of the problem and solution: Heuristics and Metaheuristics.

Heuristics

Heuristic problem-solving techniques are employed to find satisfactory or "good enough" solutions within a reasonable time frame when computing exact solutions is too costly or time-

consuming. Simple rules of thumb, educated guesses, or intuitive judgments are typically utilized by these algorithms, aiding in efficient navigation through complex problem spaces.

[7]

Metaheuristics

Metaheuristics are defined as higher-level procedures or strategies that guide heuristic algorithms in solving complex optimization problems. These methods are flexible and general-purpose, allowing adaptation to various fields such as engineering, AI, and economics.

[8]

There exist many algorithms of heuristics algorithms, genetic algorithms, Simulated Annealing Algorithms, and Local Search Algorithms (Hill-Climbing), the most common one algorithm is genetic algorithms.

Genetic algorithms

These algorithms are inspired by the process of natural evolution. Genetic algorithms use principles such as selection, crossover (recombination), mutation, and survival of the fittest are utilized to iteratively search for optimal solutions in a problem space.

[8]

2.2.2 Supervised Learning

Supervised learning is recognized as a foundational approach in machine learning, in which models are trained on labeled datasets. This method is applied to predict outcomes based on input features and is widely used in various domains, such as spam detection, image classification, and medical diagnosis.[3]

2.2.2.1 Decision Tree

[9].[10]

A decision tree is widely utilized as a machine learning model that employs a tree-like structure to represent decisions and their possible consequences, including outcomes, resource costs, and utility. Each internal node is used to test an attribute, each branch represents the outcome of the test, and each leaf node contains a class label. This structure facilitates classification and regression tasks and is easily interpretable.

Entropy:

Entropy is regarded as a key concept in information theory, introduced by Claude Shannon. It measures the amount of randomness or impurity present in a dataset.

$$H(S) = - \sum_{i=1}^n P_i(P_i)$$

Equation 0.1 Entropy

Where:

P_i is the probability of target class i given a feature in the data

During decision tree splitting, the maximization of purity is aimed at ensuring effective classification. Entropy is utilized to quantify the uncertainty of a random variable and to determine how well a feature splits the data.

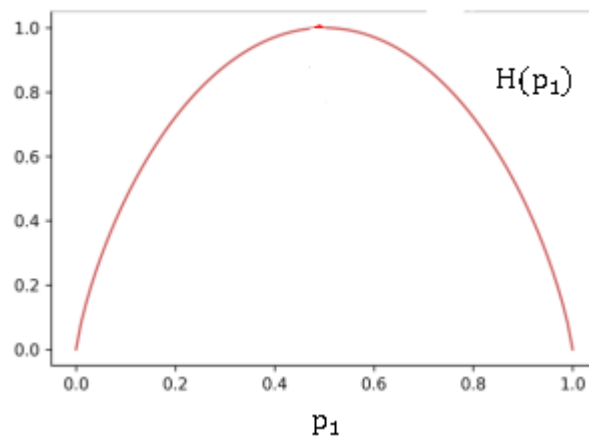


Figure 0.1 Visualizing Entropy Function

And so, the decision to choose a feature with maximum information gain

$$H(P_1^{\text{root}}) - (w^{\text{left}} H(P_1^{\text{left}}) + w^{\text{right}} H(P_1^{\text{right}}))$$

Equation 0.2 Information gain equation

Where H is the entropy function

w is a fraction of instances with a specific feature value

The regression tree A regression tree is recognized as a type of decision tree that is applied to continuous target variables. Instead of splitting based on entropy, variance reduction is utilized as the criterion, and the mean value of the target variable for each group is predicted.

However, decision trees are prone to overfitting, necessitating the use of more robust methods. Common techniques employed to address this include tree ensembles, sampling techniques, and cumulative loss, as applied in XGBoost.

However, decision trees tend to overfit, so more robust methods are required. Common approaches to address this include tree ensembles, sampling techniques, and cumulative loss (as used in XGBoost).

Tree Ensembles

Tree ensembles are utilized to enhance robustness and reduce error by combining multiple decision trees. Rather than relying on a single tree, predictions are aggregated across many trees, thereby improving stability and accuracy.

Sampling in Tree Ensembles

Sampling techniques are incorporated into tree ensembles to enhance computational efficiency and minimize error. Instead of working with the entire dataset, subsets of data are used. The algorithms that implement these techniques include Random Forest and XGBoost, which are discussed in the subsequent sections.

2.2.2.1.1 Random Forest

[9],[10]

Random Forest (RF) is identified as a variation of the decision tree algorithm that employs an ensemble of trees. The method involves:

- **Sampling:** Each tree is trained on a random subset of the data.
- **Bagging:** The final decision is determined by aggregating the outputs of individual trees, utilizing majority voting for classification or averaging for regression.

Bagging: Bagging is recognized as a technique used in tree ensembles. The predictions of individual trees are combined by:

- Taking the majority vote for classification tasks.
- Calculating the mean for regression tasks.

2.2.2.1.2 XGBoost

[9],[10]

XGBoost (Extreme Gradient Boosting) is acknowledged as an advanced variation of the decision tree algorithm. The method involves:

- The utilization of tree ensembles with sampling.

- **Boosting:** Instead of merely aggregating outputs, errors from previous trees are optimized to enhance performance.

XGBoost improves the gradient boosting algorithm, providing enhanced speed and efficiency. Due to its robustness, it is extensively applied in competitions and real-world implementations.

Boosting

Boosting is employed to improve ensemble learning through the sequential training of trees. Unlike bagging, predictions are not simply aggregated (e.g., by majority vote or mean). Instead, adjustments are made for errors encountered by previous trees, optimizing the overall loss function.

Regularization is incorporated within XGBoost to mitigate overfitting, ensuring better generalization to unseen data and increasing reliability in predicting traffic delays under varying conditions.

Parallel processing is supported by XGBoost, expediting training and making it well-suited for large-scale simulations and extensive datasets.

Tree pruning is employed in XGBoost to reduce model complexity and enhance interpretability.

To assess the performance of regression trees, evaluation metrics commonly utilized include:

Mean absolute error (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Equation 0.3 Mean absolute error

R squared:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Equation 0.4 R-squared

Root Mean Square error (RMSE):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y - \hat{y}_i)^2}$$

Equation 0.5 Root Mean Square Error

Mean Percentage Error (MPE):

$$MPE = \frac{1}{n} \sum_{i=1}^n \frac{y_i - \hat{y}_i}{y_i} \times 100$$

Equation 0.6 Mean Percentage Error

2.2.2.2 Deep Neural Network (DNN)

[9]

A Deep Neural Network (DNN) is recognized as an AI algorithm designed to mimic the human brain. It is widely applied in fields such as speech recognition and image processing.

DNNs consist of neurons, where each neuron functions similarly to a regression/logistic regression model with an activation function.

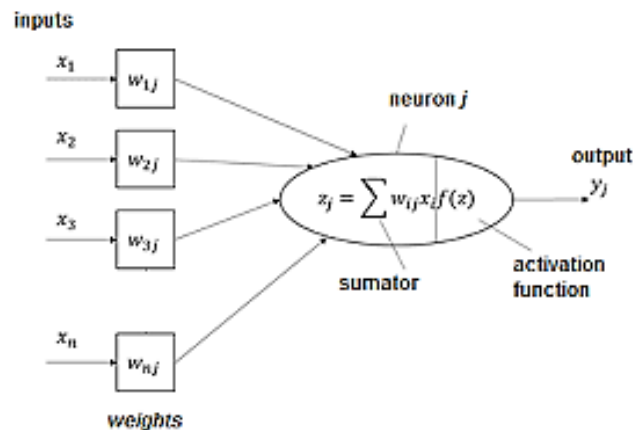


Figure 0.2 Single Neuron

[11]

Using many of these enables the NN to be able to learn more about the data making it a powerful algorithm.

Layers:

- **Input Layer:** Receives raw data.

- **Hidden Layers:** Intermediate layers where computations occur; multiple hidden layers contribute to the "deep" aspect of DNNs.
- **Output Layer:** Produces the final prediction or classification.

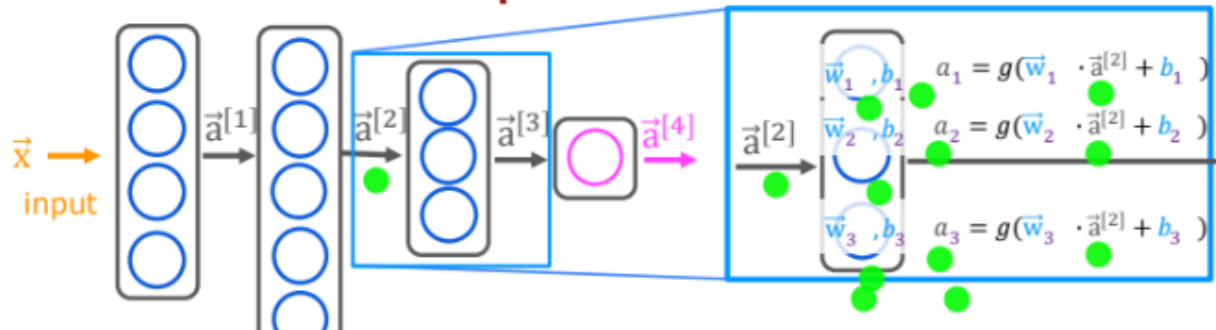


Figure 0.3 NN with 4 layers

2.2.3 Reinforcement Learning

Reinforcement learning (RL) is a machine learning methodology in which decisions are learned by an agent through interaction with its environment via trial and error. Feedback is provided in the form of rewards or penalties, guiding the agent to optimize its behavior. This approach is modeled after how humans learn from experiences to improve actions and achieve desired outcomes. The primary goal of a reinforcement learning agent is to maximize its expected cumulative reward, often referred to as the expected return. Although both supervised learning and reinforcement learning involve mapping inputs to outputs, their feedback mechanisms differ significantly. In supervised learning, feedback is provided as correct actions or labels, whereas in reinforcement learning, feedback comes in the form of rewards and punishments, encouraging positive behavior and discouraging undesirable actions.

This introduction explains the main concepts of reinforcement learning (RL), beginning with the key components of any RL model. It is followed by the discount rate, which is used to calculate rewards more effectively, and the ways in which data is collected by the agent (online and offline). Next, the types of algorithms, including policy-based and value-based methods, are discussed. The section also explores soft updates, and the amount of data collected, with a focus on Temporal Difference (TD) and Monte Carlo methods. Finally, the Bellman function is introduced.

Key Components of Reinforcement Learning [12]:

- **Agent:** The decision-maker who learns by interacting with the environment, taking action, and receiving feedback.

- **Environment:** The external system in which the agent operates, defined by specific rules and conditions.
- **State:** The representation of the environment at a given moment. For instance, in a video game, the state could be a single frame, while in a trading system, it could be the current value of a stock.
- **Action:** A decision made by the agent in each state. The set of all possible actions is referred to as the action space.
- **Reward:** The feedback signals the agent receives from the environment after acting. Rewards quantify the desirability of the agent's actions, with the goal being to maximize cumulative rewards over time.

The reward is fundamental in RL because it's the only feedback for the agent. Thanks to it, our agent knows if the action taken was good or not.

$$R(\tau) = \sum_{k=0}^{\infty} (r_{t+k+1})$$

Equation 0.7 Basic cumulative reward in RL

Cumulative rewards are the total rewards accumulated by the agent over time.

However, in practice, we can't just add them directly, we add a factor that determines how much agents consider long-term rewards.

The discount rate(γ) determines the importance of future rewards:

Rewards received earlier (at the start of the game) are more likely to occur as they are more predictable than long-term future rewards.

- A higher γ means the agent considers long-term rewards more heavily.
- A lower γ means the agent focuses on short-term rewards.

$$\begin{aligned} R(\tau) &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots \\ &= \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \end{aligned}$$

Equation 0.8 Cumulative reward with discount factor

Where:

τ is trajectory (sequence of state/action)

γ is the discount factor

The Overview of the RL process:

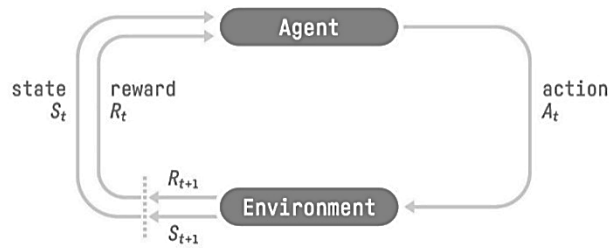


Figure 0.4 Overview of the RL process

Reinforcement learning algorithms can collect data from the environment using two approaches: online or offline.

In online reinforcement learning, the agent gathers data directly: it collects a batch of experiences by interacting with the environment. Then, it uses this experience immediately (or via some replay buffer) to learn from it (update its policy).

On the other hand, in **offline reinforcement learning**, the agent only uses data collected from other agents or human demonstrations. It does not interact with the environment [13],[14].

The approaches can also be viewed from another perspective: in offline training, the model is trained without real-time interaction with the environment, while online training involves collecting data after training and using it to directly improve the model from the environment.

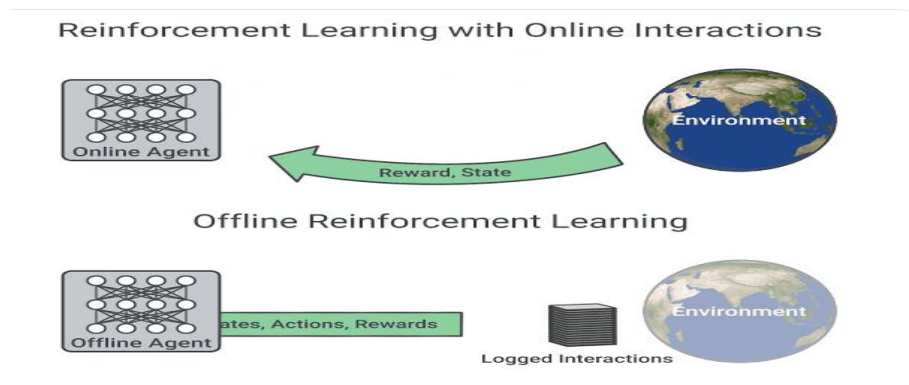


Figure 0.5 Online and Offline RL interaction

Reinforcement learning algorithms can be trained using two approaches: one involves training to predict cumulative rewards and selecting actions with the highest reward (value-based), while the other predicts actions directly.

Policy-Based Methods [12]

Policy-based methods focus on learning a **policy function** that directly maps states to optimal actions.

- **Deterministic Policies:** The agent consistently chooses the same action for a given state.
- **Stochastic Policies:** The policy generates a probability distribution over possible actions for each state, allowing for action variability.

Value-Based Methods

Value-based methods aim to learn a **value function** that estimates the expected return for being in a state or performing an action in that state.

- **State Value Function (V(s)):** Represents the expected cumulative reward starting from state s and following policy π :

$$V^{\pi}(s) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_t = s \right]$$

Equation 0.9 State Value Function

- **Action Value Function (Q(s, a)):** Represents the expected cumulative reward starting from states, taking action a, and then following policy π :

$$Q^{\pi}(s, a) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_t = s, a_t = a \right]$$

Equation 0.10 Action Value Function

The frequency with which an RL algorithm collects data from the environment can be either over the entire episode or just a temporal part of it. First, the term "episode" is explained, followed by soft update rule, which is used in exploring laws, then explanation of Monte Carlo and Temporal Difference methods.

An **episode** refers to a complete cycle of interaction, starting from an initial state, progressing through a series of actions and rewards, and ending in a final state. For example, an episode could represent one game or an attempt to achieve a specific goal.

Soft Update [12]

In reinforcement learning, **soft updates** gradually adjust the parameters of a target network, commonly used in algorithms like Deep Q-learning.

Formula:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha X$$

Alternatively:

$$Q(s, a) = Q(s, a) + \alpha(X - Q(s, a))$$

Equation 0.11 Soft update rule

- This gradual adjustment improves stability in training compared to hard updates, which copy parameters directly.

Temporal Difference (TD) and Monte Carlo Methods [12]

- **Monte Carlo Methods:**

- Learn from entire episodes of experience before updating.
- Use the total return (G_t) at the end of an episode to update the value function.
- Formula:

$$V(s_t) = V(s_t) + \alpha[G_t - V(s_t)]$$

Equation 0.12 Monte Carlo update rule

Where:

$V(s_t)$: value of state

α : learning rate

G_t : return from episode

- **Temporal Difference (TD) Methods:**

- Learn incrementally after each step.
- Update the value function based on the TD target.
- Formula:

$$V(s_t) = V(s_t) + \alpha[R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Equation 0.13 Temporal Difference update rule

- Where:

$[R_{t+1} + \gamma V(s_{t+1})]$ is called TD target

The full update rule in Q-learning algorithms is represented by the Bellman equation, which first selects an action and then follows the optimal behavior.

Bellman Equation:

[12]

The Bellman equation is a fundamental equation in reinforcement learning that helps an agent figure out the best way to maximize its total rewards over time.

It's not a new thing, it's a simplified way of saying if an agent takes an action, it is rewarded immediately (based on $R(s, a)$), and then it expects to behave optimally later to maximize the total reward, which is why the future rewards are considered through the $\text{MAX}_{a'} V(s')$ term.

In other words, the Bellman equation in RL does suggest that an agent should take an action in the current state that will maximize its future expected rewards, weighted by γ . The agent considers both the immediate reward and the potential future rewards (looking ahead and behaving optimally)

$$Q(s, a) = R(s) + \gamma \max_{a'} (Q(s', a'))$$

Equation 0.14 Bellman equation

[15], [16]

2.2.3.2 QN Algorithms

This section explains the first RL algorithm, Q-learning (QNetwork), along with its variations, most of which are used in the related work.

2.2.3.2.1 Q Network (QN)

[12],[16]

Q-learning (Q-Network /QN) is a fundamental form of learning that serves as the foundation for more advanced techniques. It has been widely used in both research and industry applications. Q-learning is a reinforcement learning method that enables agents to learn optimal actions through temporal difference (TD) learning. In this process, the agent takes an action in a given state, receives a reward or penalty, and updates its understanding of that state based on the feedback received.

Algorithm Steps:

- **Initialize the Q-table:** Set up a table with Q-values for each state-action pair. Initially, these values can be set to zero or small random values.
- **Choose an action using the ϵ -greedy strategy:** With probability ϵ , choose a random action to explore different possibilities (exploration). Otherwise, choose the action with the highest Q-value for the current state (exploitation).
- **Act at get reward R_{t+1} and observe next state S_{t+1} :** The agent performs the chosen action in the environment, receives a reward, and transitions to a new state.

- **Update $Q(S_t, A_t)$** Use the Q-learning update rule to adjust the Q-value for the current state-action pair based on the received reward and the maximum expected future rewards.

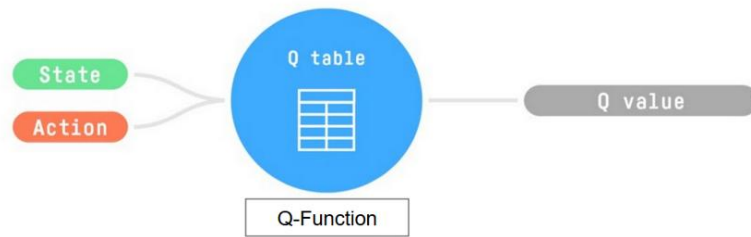


Figure 0.6 Q function

The limitation Q-Learning is a **tabular method**, meaning it stores Q-values for each state-action pair in a table. This becomes a significant limitation when the state or action spaces are **large**, requiring an enormous table size, or when the spaces are **continuous**, making it infeasible to represent them using arrays or tables.

To overcome Q-Learning's scalability issues, **Deep Q-Learning** replaces the table with a deep neural network to approximate Q-values, explained in the next section.

2.2.3.2.2 Deep Q Network (DQN)

[17]

To address Q-Learning's scalability issues, Deep Q-Learning (DQN) replaces the table with a deep neural network to approximate Q-values.

Before explaining this, it's important to introduce a key concept: **Experience Replay**. Experience Replay improves the efficiency of learning by creating a Replay Buffer, which stores experience tuples collected during interactions with the environment. Later, small random batches of these tuples are sampled from the buffer for training. This approach prevents the network from focusing solely on recent actions and outcomes, ensuring a more stable and comprehensive learning process. It enables the efficient utilization of experiences, helps prevent forgetting, and reduces correlation (mitigating catastrophic forgetting).

The training algorithm for DQN involves two main phases:

- **Sampling:**
 - The agent interacts with the environment by taking actions and stores the observed experiences as tuples in the replay memory. Each tuple typically includes:
 - s: Current state
 - a: Action taken
 - r: Reward received
 - s': Next state
- **Training:**
 - A small, random batch of experience tuples is selected from replay memory.

- The agent learns from this batch by performing a gradient descent update step to minimize the Q-value prediction error.

In some cases, the input may proceed, for example when the input is Images or Video frames.

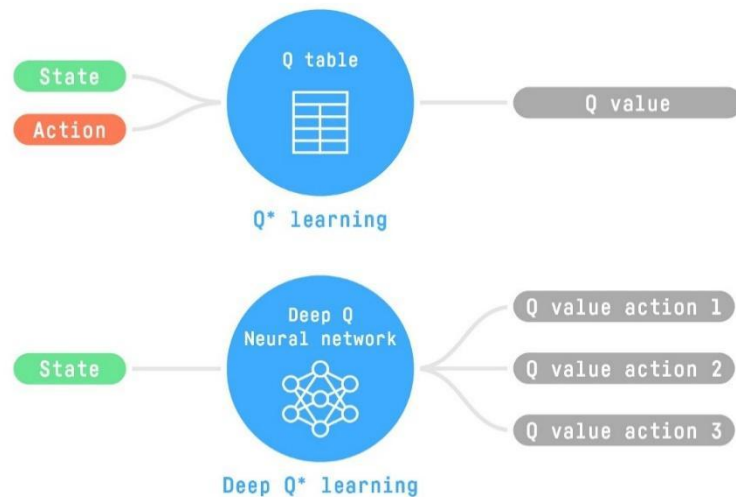


Figure 0.7 QN and DQN architecture overview

2.2.3.2.3 Double Deep Q Network (D2QN)

[12]

When calculating the TD error (also known as the loss), we compute the difference between the TD target (Q-Target) and the current Q-value (the estimation of Q).

However, the challenge arises because the same parameters (weights) are used for estimating both the TD target and the Q-value. As a result, there is a significant correlation between the TD target and the parameters we are adjusting. This means that, at each training step, both the Q-values and the target values shift. While we are getting closer to the target, the target itself is also moving. It's like chasing a moving target! This dynamic can lead to significant oscillations during training.

To illustrate this with an analogy: imagine you are a cowboy (representing the Q-value estimation) trying to catch a cow (representing the Q-target). Your goal is to get closer to the cow (reduce the error). However, at every time step, as you try to approach the cow, it keeps moving (because you are using the same parameters for both the Q-value and the TD target). This constant movement of the cow makes it difficult to catch, causing instability in the training process.

The solution to this issue is to decouple the action selection from the target Q-value generation by using two separate networks.

Specifically, we:

- Use our **DQN network** to select the best action to take for the next state (the action with the highest Q-value).

- Use our **Target network** to calculate the target Q-value for taking that action at the next state.

By using two networks, we prevent the target from constantly shifting with the Q-value estimates, thereby reducing the oscillation and instability in training. This helps stabilize the learning process.

Advantages of the Algorithm:

- Reduced Bias: Minimizes Q-value estimation errors for better decision-making.
- Stability: Target networks and experience replay ensure stable training.
- Performance: Excels in complex environments due to its robust architecture.

Limitations of the Algorithm:

- Implementation Complexity: Requires significant computational resources and expertise.
- Dependence on Reward Structure: Effectiveness heavily relies on the quality of the reward design.
- Hyperparameter Sensitivity: The DDQN algorithm's performance and stability are highly sensitive to changes in key hyperparameters,

2.2.3.2.4 Dual DDQN (D3QN)

[18],[19]

This algorithm is like DDQN, but with the addition of the Advantage function to improve stability.

Advantage function:

We can stabilize learning further by using the Advantage function

The idea is that the Advantage function calculates the relative advantage of an action compared to the others possible at a state: how taking that action at a state is better compared to the average value of the state.

- **If $A(s, a) > 0$:** our gradient is pushed in that direction.
- **If $A(s, a) < 0$:** (our action does worse than the average value of that state), our gradient is pushed in the opposite direction.

$$A(s,a) = Q(s,a) - V(s)$$

Equation 0.15 Advantage function

Where:

V: is the average value of the state

The idea of the D3QN algorithm:

Dueling Architecture: In dueling DQN, instead of directly calculating the difference with the advantage function, we **separate** the two components:

1. **State value ($V(s)$):** This tells you how good a state is.
2. **Advantage function ($A(s, a)$):** This tells you how much better a specific action is compared to others in that state.

Advantages of the algorithm:

- **Better Policy Stability:** Reduces overestimation bias, resulting in more stable and reliable policies.
- **Improved Learning Efficiency:** Achieves faster convergence and better sample efficiency, reducing the need for extensive training.
- **Enhanced Decision-Making:** Excels in environments with many similar states, where certain actions are more critical to optimal performance many similar states where certain actions are more critical.

Disadvantages of the algorithm:

- **Higher Computational Requirements:** The added complexity of the architecture leads to increased computational demands and requires significantly more resources due to the additional streams, approximately 2.5 to 3 times that of DQN.
- **Complexity in Training:** The dual-stream architecture and combination of Double Q-Learning and Dueling Network require more advanced techniques and longer training times.
- **Not Ideal for High-Dimensional State Spaces:** The algorithm may struggle with environments that have extremely high-dimensional state spaces.
- **Requires Extensive Tuning:** Finding the optimal hyperparameters (e.g., learning rate, exploration rate) is time-consuming but crucial for achieving good performance.

2.2.3.3 Hierarchical Multi-Agent Systems (HMAS)

[3], [20]

This algorithm exemplifies how hierarchical and predictive systems can improve urban traffic management; the algorithm consists of **two levels of control**:

- **Local Agents (LAs):**
 - Each intersection has its local agent managing traffic lights.

- Uses **Reinforcement Learning** to learn how to reduce traffic delays based on past experiences.
- **Regional Agents (RAs):**
 - Each region (a group of intersections) has a regional agent overseeing traffic.
 - Uses **LSTM** to predict future traffic conditions based on data from all the local agents.

Communication Between Levels:

- Local agents send real-time traffic data (e.g., queue lengths, delays) to the regional agent.
- Regional agents decide when coordination is needed and guide local agents in managing traffic.

Reinforcement Learning (RL): Helps local agents adapt to traffic conditions at their intersections.

LSTM Neural Networks: Allow regional agents to predict traffic jams before they happen.

Two Modes of Operation:

- Independent Mode: Manages the intersection autonomously.
- Cooperative Mode: Coordinates with the Regional Controller when there's heavy traffic.

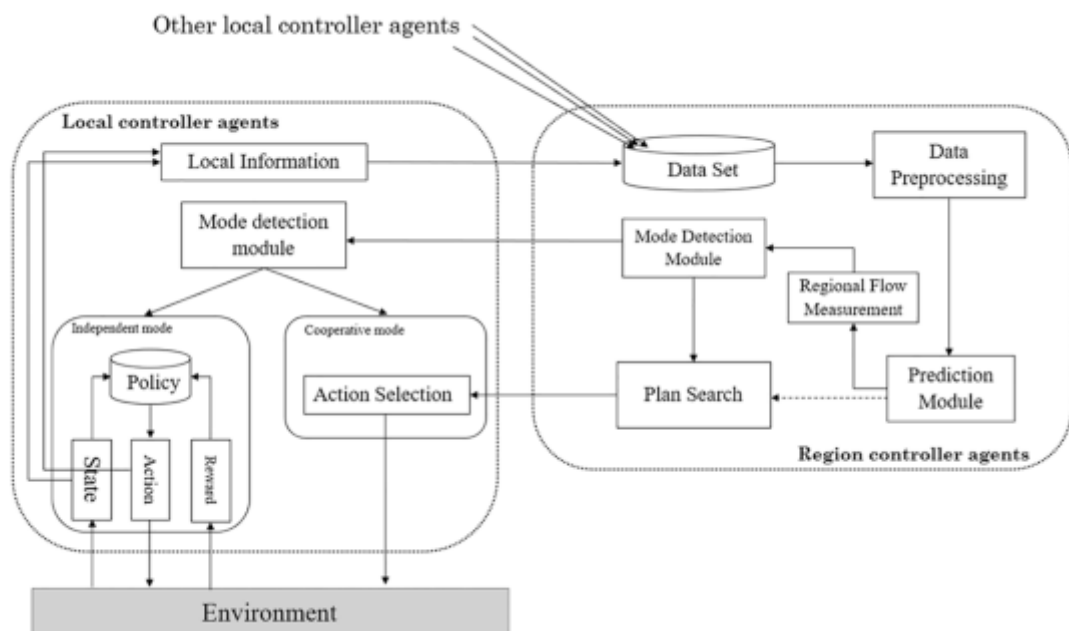


Figure 0.9 HMAS architecture

2.2.3.4 Dynamic and Intelligent Traffic Light Control System DITLCS [21]

The Dynamic and Intelligent Traffic Light Control System (DITLCS) is a proposed algorithm designed to enhance urban traffic management through a combination of deep reinforcement learning and fuzzy logic.

Modes of Operation:

- **Fair Mode (FM):** Treats all vehicles equally, aiming to minimize the overall average waiting time.
- **Priority Mode (PM):** Assigns different priorities to vehicles, reducing waiting times for higher-priority vehicles (e.g., buses).
- **Emergency Mode (EM):** Gives the highest priority to emergency vehicles like ambulances and fire trucks.

Algorithm Workflow:

Mode Selection:

The fuzzy inference system (if condition-fuzzy logic) selects the most appropriate mode based on traffic conditions.

For example, if emergency vehicles are detected, EM is activated.

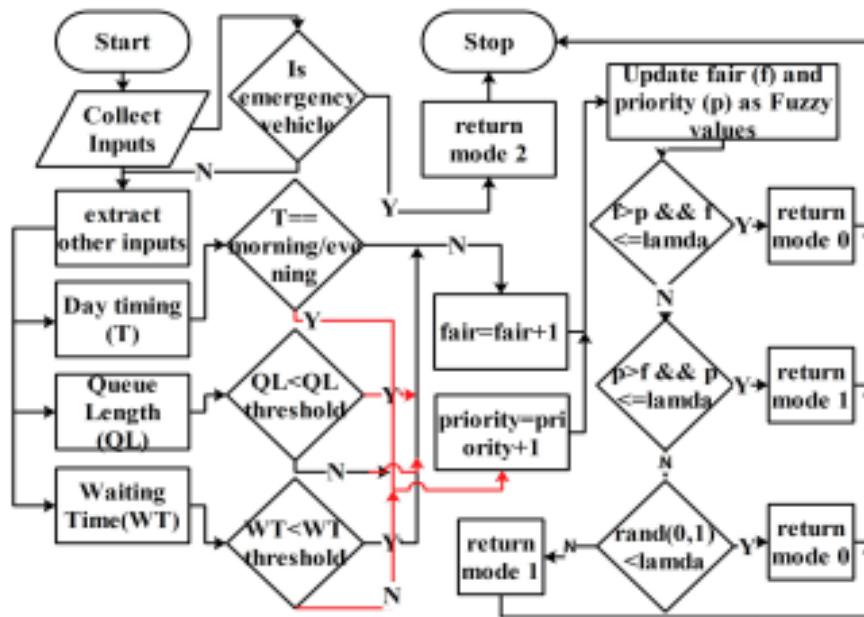


Figure 0.10 DITLCS fuzzy logic

Action Selection: The DRL agent analyzes traffic states to decide whether to change the traffic light phase or extend the current phase duration.

Phase Adjustment: Based on the DRL output, signal durations are dynamically adjusted for each traffic direction to optimize flow.

Advantages of the algorithm:

- **Efficiency:** Minimizes average waiting times, queue lengths, and delays.
- **Scalability:** Adapts to heterogeneous traffic, including priority and emergency vehicles.
- **Environmental Benefits:** Reduces fuel consumption and carbon emissions by decreasing idling times.

2.2.3.5 Mean Field Double Q-Learning with Dynamic Timing Control (MFDQL-DTC)

[22]

The MFDQL-DTC algorithm (Mean Field Double Q-Learning with Dynamic Timing Control) is an algorithm for dynamic traffic signal control that integrates reinforcement learning and other AI for traffic optimization.

The algorithm steps:

For each episode:

1. Start with an initial observation of the local intersection.
2. Predict the traffic flow β_t for the current timestep using the RNN.
3. Compute the optimal phase duration D_t dynamically using a revised Webster's formula.

The **Webster's Formula** calculates how long the green light should stay on.

Example:

- If lots of cars are coming: Green light lasts longer.
 - If fewer cars are coming: The green light lasts shorter.
4. Each agent (uses the DQN algorithm) selects an action (a green phase from a predefined set) based on the "Boltzmann" SoftMax policy.
 5. Execute the selected actions for the computed duration D_t .
 6. Observe the rewards and next state observations.

7. Calculate the mean action of neighboring agents (**Mean Field**), The **mean field** is the **average effect** that a group of many agents (or things) has on a single agent.
8. Instead of calculating the influence of each agent individually, we take their **average behavior** use that as a simplified representation, and then store transitions in replay buffers.

Then update values of DQN.

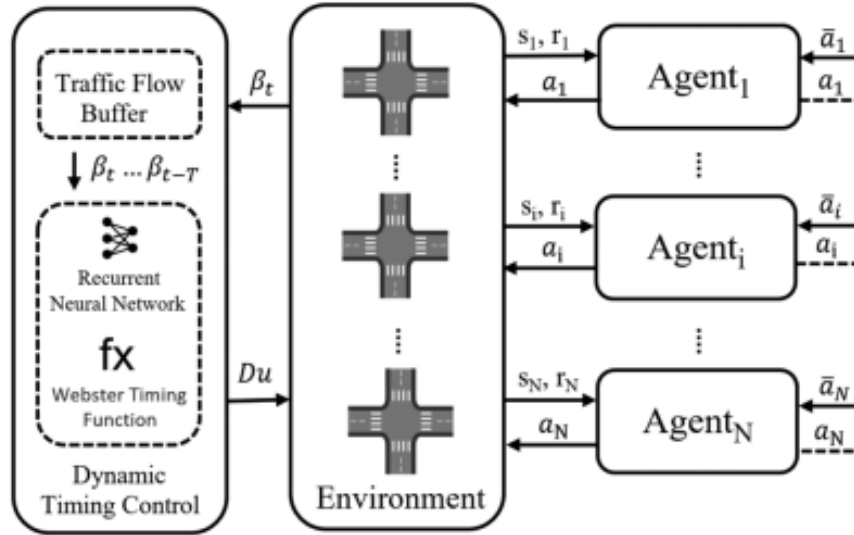


Figure 0.11 MFDQL-DTC architecture

2.2.3.6 Actor-Critic Algorithm and Variation

2.2.3.6.1 Advantage Actor-Critic (A2C)

In Policy-Based methods, we aim to optimize the policy directly without using a value function. More precisely, Reinforce is part of a subclass of *Policy-Based Methods* called *Policy-Gradient methods*. This subclass optimizes the policy directly by estimating the weights of the optimal policy using Gradient Ascent [12].

Given the stochasticity of the environment (random events during an episode) and stochasticity of the policy, trajectories can lead to different returns, which can lead to high variance. Consequently, the same starting state can lead to very different returns. Because of this, the return starting at the same state can vary significantly across episodes [23].

The solution is to mitigate the variance by using a large number of trajectories, hoping that the variance introduced in any one trajectory will be reduced in aggregate and provide a “true” estimation of the return [23].

Actor-critic methods, a hybrid architecture combining value-based and Policy-Based methods that helps to stabilize the training by reducing the variance using:

- An *Actor* that controls how our agent behaves (Policy-Based method)

- A Critic that measures how good the taken action is (Value-Based method)

The details of algorithms:

At each timestep, t , we get the current state S_t from the environment and pass it as input through our Actor and Critic, The Policy takes the state and outputs an action A_t , Then the critic takes that action also as input and, using S_t and A_t , computes the value of taking that action at that state: the Q-value.

the model then executes the action then updates the parameters of the actor and critic [18].

2.2.4.6.2 Advantage Actor-Critic (A2C)

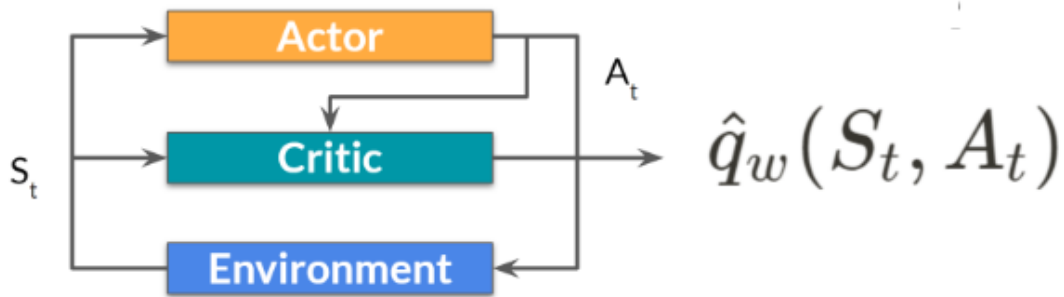


Figure 0.12 Actor-Critic Architecture

It's based on A2C with Advantage function as discussed in section 2.2.4.2.3

Combining the Actor critic with the advantage function results in better algorithm called Advantage Actor critic (A2C) [18].

2.2.3.6.2 Async Advantage Actor critic (A3C)

[24]

The A3C algorithm is an asynchronous version of the advantage actor-critic (A2C) algorithm.

A key innovation of A3C is the use of multiple worker agents running in parallel. Each worker interacts with its own environment and performs updates to the global model asynchronously. This reduces the correlation between training data and helps prevent overfitting.

A3C includes an entropy term in the objective function to encourage exploration and prevent the policy from becoming too deterministic too early in training.

2.2.3.7 Region-Aware Cooperative Strategy A2C (RACS-A2C)

[24]

The proposed Region-Aware Cooperative Strategy (RACS) based on the Advantage Actor-Critic (A2C) algorithm is a decentralized approach designed to improve adaptive traffic signal control in multi-intersection scenarios.

The paper uses an A2C algorithm with an extension Region-Aware Cooperative Strategy (RACS).

Region-Aware Cooperative Strategy (RACS):

To operate within a decentralized framework, where each intersection is controlled by a local Reinforcement Learning (RL) agent and **captures** spatial correlations between different intersections by assigning weights to nearby nodes (intersections): The paper introduced **Region-Aware Cooperative Strategy (RACS)**, this is implemented by using **Graph Attention Networks (GAT)**.

The algorithm calculates the weighted neighbor features between neighbors of the intersection

$$f_{ui} = \text{LeakyReLU}(b^T \cdot [g_u; g_i'])$$

Equation 0.16 correlation score between nodes in RACS

Where:

f_{ui} : correlation score between node u and I

b: Learnable weight vector

$$b_{u_i} = \frac{\exp(f_{u_i})}{\sum_{L=1} \exp(f_{uL})}$$

Equation 0.17 SoftMax normalize of weight

$$g'_u = \frac{1}{u-1} \sum_{g' \in \text{CW}} b_{ji} \times g'_j$$

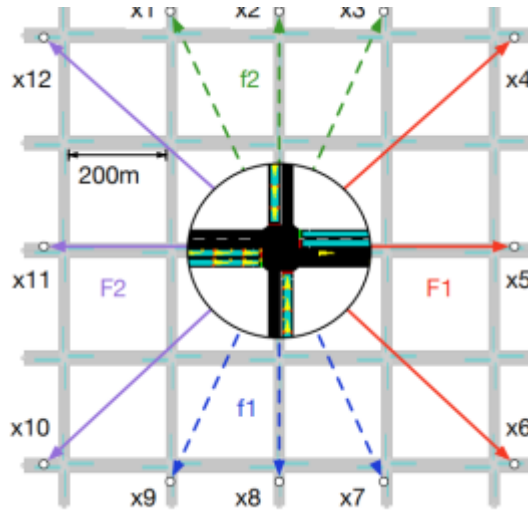
Equation 0.18 Average Weight of Intersection in RACS

Where:

g_u : Final updated feature for intersection u

b_{ui} : Weight showing how many neighbors i influence u

g'_i : Neighbor i's feature



Equation 0.19 Correlation with a neighbor in RACS

2.2.3.8 Graph-Structured Correlation Time-Spatial Attention – Async Actor-Critic (GSCTSA-A3C)

The algorithm is a combination of RL with other algorithms aimed at measuring the traffic flow in more than one view trying to capture features.

The algorithm is a combination of:

- 1- Graph Attention Networks (GAT): The paper used GAT, allowing it to measure **Spatial** info, inspired by paper [24].
- 2- Correlational Attention Mechanism (CAM): Dynamically prioritizes features (e.g., traffic volume, weather) based on their relevance at different times [3].

This is inspired by the attention mechanism widely used in deep learning, especially in natural language processing (NLP)

- 1- Long Short-Term Memory (LSTM): Models the temporal dependencies in traffic patterns

$$\text{Output} = \text{LSTM}(\text{current traffic}, \text{previous memory})$$

Equation 0.20 LSTM for temporal pattern analysis

- 2- Asynchronous Advantage Actor-Critic (A3C) is a reinforcement learning algorithm, a variation of A2C developed by OpenMind to learn policies in a decentralized architecture.

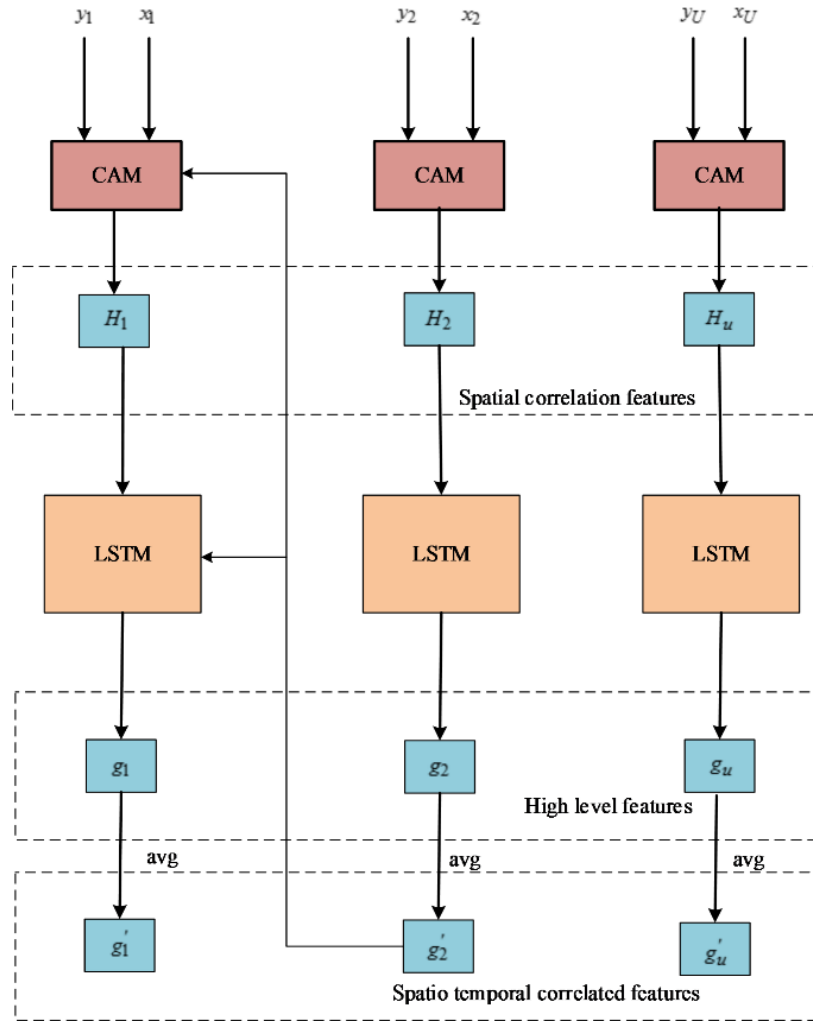


Figure 0.13 GSCTSA-A3C architecture

The weight b in this algorithm is usually normalized (by softmax to make all weights sure all sum to 1)

Limitations of this Algorithm [3]:

- **Scalability and Complexity:** Due to the decentralized nature of A3C, scaling the model to large, real-world environments may introduce significant complexity.

2.2.3.9 Proximal Policy Optimization (PPO)

[25]

The idea with Proximal Policy Optimization (PPO) is that we want to improve the training stability of the policy by limiting the change you make to the policy at each training epoch: we want to avoid having too large of a policy update.

. To do so, we need to measure how much the current policy changed compared to the former one using a ratio calculation between the current and former policy. We clip this ratio in a range $[1-\epsilon, 1+\epsilon]$ meaning that we remove the incentive for the current policy to go too far from the old one (hence the proximal policy term).

At first, the algorithm calculates the ratio between the old policy and the updated policy (This ratio can replace the log probability in the policy objective function)

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

Equation 0.21 ratio between old policy and updated policy in PPO

- If $r_t(\theta) > 1$, the action a_t , at state s_t is more likely in the current policy than the old policy.
- If $r_t(\theta)$ is between 0 and 1, the action is less likely for the current policy than for the old one.

So, this probability ratio is an easy way to estimate the divergence between old and current policy

However, without a constraint, if the action taken is much more probable in our current policy than in our former, this would lead to a significant policy gradient step and, therefore, an excessive policy update.

Consequently, we need to constrain this objective function by penalizing changes that lead to a ratio far away from 1 (in the paper, the ratio can only vary from 0.8 to 1.2).

By clipping the ratio, we ensure that we do not have a too large policy update because the current policy can't be too different from the older one.

To do that, we have two solutions:

- *TRPO (Trust Region Policy Optimization)* uses KL divergence constraints outside the objective function to constrain the policy update. However, this method is complicated to implement and takes more computation time.
- *PPO* clip probability ratio directly in the objective function with its Clipped surrogate objective function.

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta)\widehat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\widehat{A}_t)]$$

Equation 0.22 CLIP probability loss in PPO

2.3 You Only look Once (Yolo)

[26]

Computer vision (CV) is the science and technology of enabling machines to interpret and understand visual data. It involves developing theoretical and algorithmic methods to acquire, process, analyze, and understand images or videos and use this information to create meaningful representations and interpretations of the world.

YOLO (You Only Look Once) is a fast and accurate object detection model introduced by Joseph Redmon and his team in 2016.

How YOLO Works.

YOLO divides an input image into an $S \times S$ grid. If the center of an object falls within a grid cell, that cell becomes responsible for detecting that object. Each grid cell predicts B bounding boxes, each with a confidence score. This score tells us how sure the model is that a box contains an object and how accurate its position is

The Main concepts of YOLO are listed below.

Bounding Box Prediction

YOLO allows each grid cell to predict multiple bounding boxes. During training, we want only one box predictor to handle each object. So, YOLO chooses the predictor with the highest IOU (Intersection over Union) with the true object location. This approach helps each predictor specialize in certain object types or shapes, leading to better performance overall.

Non-Maximum Suppression (NMS)

When YOLO detects objects, it often creates multiple overlapping boxes for the same object. To fix this, it uses a technique called Non-Maximum Suppression (NMS). NMS removes extra or less accurate boxes and keeps the one with the highest confidence, so each object is detected only once.

Network Architecture

YOLO processes the input image using a deep convolutional neural network (CNN). This CNN directly detects objects by predicting class probabilities and bounding box coordinates in one go.

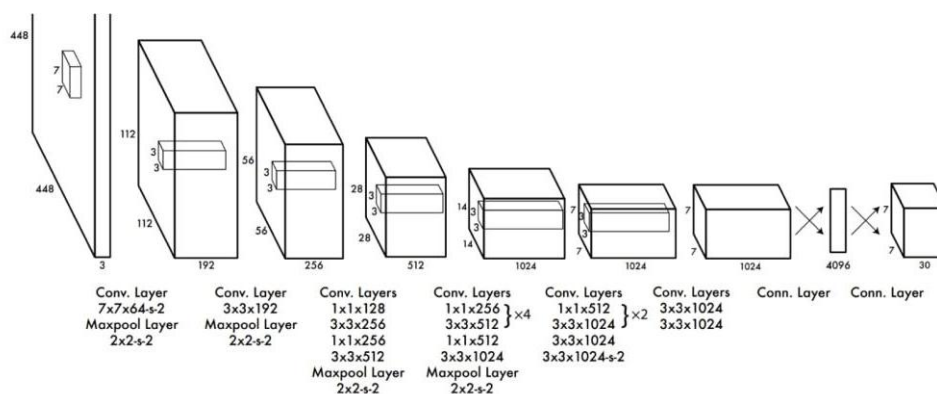


Figure 0.14 Yolo Typical Neural Network

Object Detection

Object detection is a task in computer vision where a model identifies and locates objects within images or videos. It is used in many fields, including security, autonomous vehicles, and robotics.

[27]

Speed Estimation

Speed estimation is performed as part of the object tracking process. Objects are detected and tracked across video frames, and their movement is measured in pixels. When the video frame rate and real-world scale are provided, the object's speed is calculated based on the distance traveled over time. The estimated speed is then displayed for each tracked object.

2.4 Conclusion

In this chapter, the scientific background necessary for understanding the algorithms and techniques used in the project was presented. The next chapter will explore related work, providing comparisons between existing approaches and identifying the scope of the current study.

LITERATURE REVIEW

In last chapter , we covered the scientific background relevant to the problem, including simulation, algorithms, and computer vision.

In this chapter, the results of the algorithms presented in each paper will be compared, followed by a comparison between the papers and related work. Finally, the need for extending current work will be highlighted, and the scope of the project will be defined.

3.1 Main results of related papers

Based on the results of a study of the algorithms DQN, DDQN, D3QN in [19], D3QN is chosen

Table 0.1 algorithms result according to [19]

	DQN	DDQN	D3QN	Fixed time
Average reward	-222.76	-213.38	-211.22	-
Avg delay	25.06	24.03	23.3	40.17

Based on the results of a study of the algorithms DQN, DDQN in [28], DDQN is chosen

Table 0.2 algorithms result according to [28]

Measure	DQN (low traffic)	DDQN (low traffic)	DQN (high traffic)	DDQN (high traffic)
total negative reward	-3367.6	-4440.8	-113320.2	-77260
total accumulated wait time	14025.8	20139	6222282.6	2822351
expected wait time per car	14.228	20.6837	180.8533	136.3796
average queue length	0.53202	0.7551	22.8937	18.0516

Based on the results of a study of the RF, DNN, XGBoost, Linear Regression (LR) algorithms in [29], XGBoost is chosen

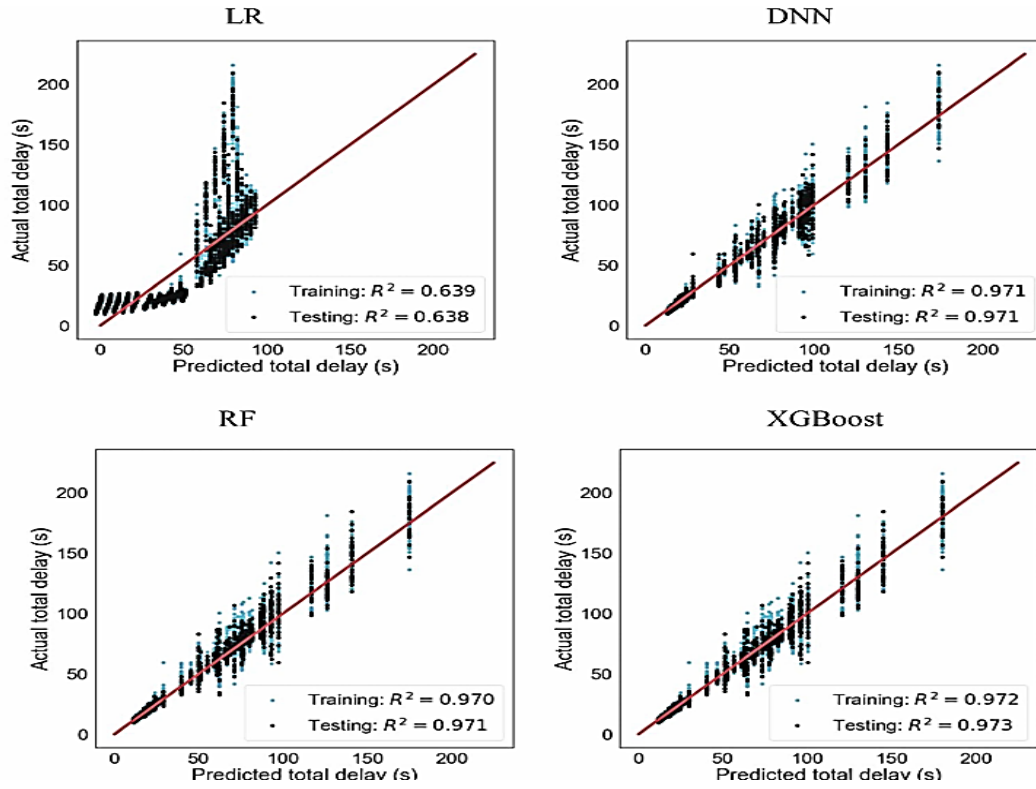


Figure 0.1 Plot of performance of algorithms according to [29]

Table 0.3 Results of algorithms according to [29]

Model	Mean Absolute Error (MAE)	Root Mean Square Error (RMSE)	Mean Percentage Error (MPE)	R^2	Inference time (s per instance) Processing time
DNN	3.609	6.409	0.080	0.971	3.119E-05
RF	3.035	5.991	0.061	0.971	1.111E-05
XGBoost	2.919	5.614	0.060	0.973	4.604E-06

Based on the results of a study of the HMAS, MFDQL-DTC, RACS-A2C, DITLCS, GSCTSA-A3C algorithms in [3], GSCTSA-A3C is chosen

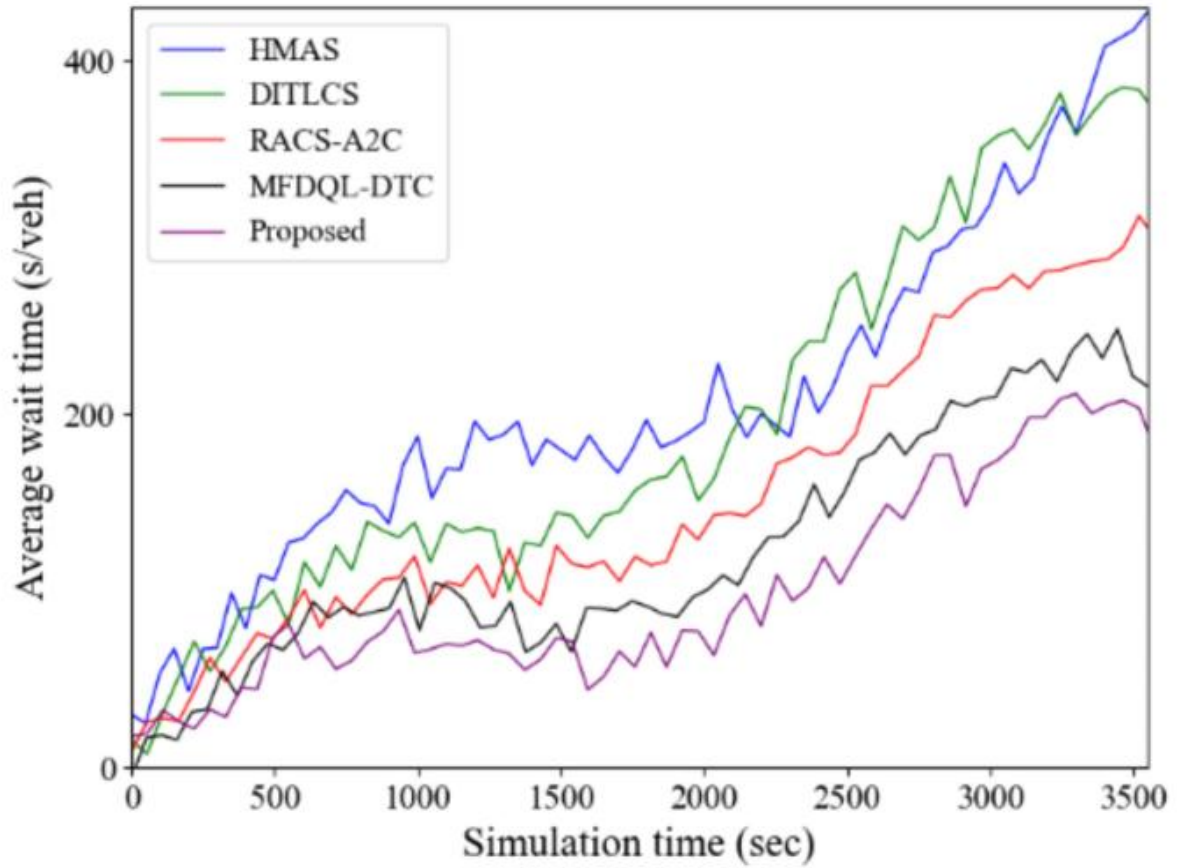
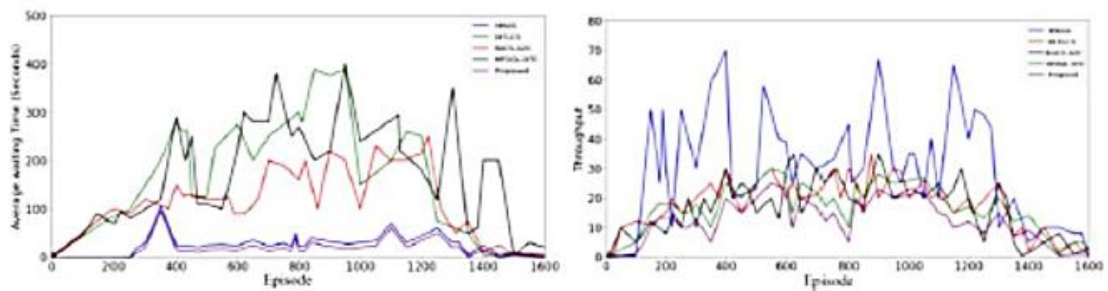


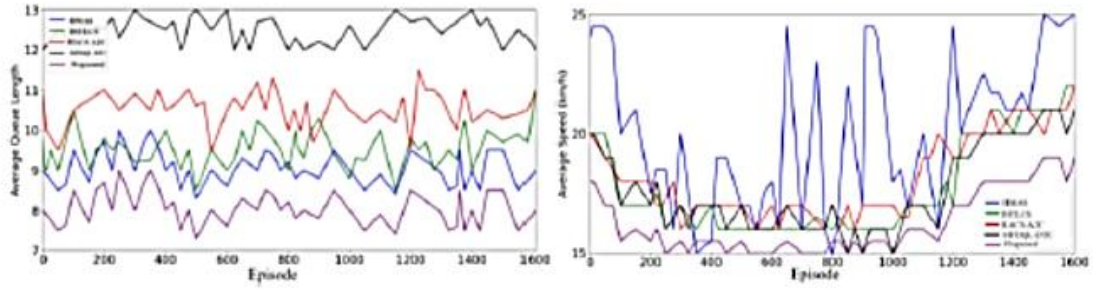
Figure 0.2 Relation between waiting time and simulation time in GSCTSA-A3C according to [3]



i) Average waiting time

ii) Throughput

Figure 0.3 Plotting average wait time, throughput in GSCTSA-A3C according to [3]



iii) Average queue length

iv) Average speed

Figure 0.4 Plotting average queue time, and speed in GSCTSA-A3C according to [3]

3.2 Comparison between related papers

1) Environment and Characteristics:

The first table compares papers in terms of Year, journal rank, availability of source, environment used in training (as described in Section 2.1), characteristics of the dataset, objective, and learning paradigm (which refers to the type of learning used).

Table 0.4 Comparison between related papers

		[3]	[30]	[19]	[29]	[28]	[2]	[1]	Ours
Year		2024	2022	2024	2024	2024	2019	2023	2025
Journal rank		Q1	Q1	conference	Q1	Q2	package	Graduation Report	Graduation Report
Survey		✓	✓	✓	✓	✓	✓	✓	✓
Open source		No	No	Yes	No	No	Yes	Yes	Yes
Environment	GYM	✗	✗	✗	✗	✗	✓	✗	✓
	SUMO	✓	✓	✗	✗	✓	✓	✓	✓
	VISSIM	✗	✗	✓	✓	✗	✗	✗	✗
Characteristic s of dataset	Length of line	✓	✗	✓	✗	✓	✗	✓	✓
	Flow of traffic	✓	✓	✓	✗	✓	✓	✓	✓
	Weather	✓	✗	✗	✗	✗	✗	✗	✗
	Pedestrians	✓	✓	✗	✗	✗	✗	✗	✗
	Time of day	✓	✓	✗	✗	✗	✗	✗	✗
Objective	Decrease delay	✓	✓	✓	✓	✓	✓	✓	✓
	Fairness	✓	✓	✗	✗	✓	✓	✓	✓
	Energy	✓	✗	✗	✗	✗	✗	✗	✗

Learning Paradigms	Reinforcement learning	✓	✓	✓	✗	✓	✓	✓	✓
	supervised learning	✗	✗	✗	✓	✗	✗	✗	✗

Characteristics of dataset:

- Length of Line: The distance of vehicles waiting in a queue.
- Flow of Traffic: The movement rate of vehicles on a road.
- Weather: Conditions like rain, sun, or fog that affect driving.
- Pedestrians: People walking near or crossing the road.
- Time of Day: The specific hour, like morning, afternoon, or night.

Objective:

- Decrease Delay: Reducing waiting time for vehicles.
- Fairness: Ensuring all road users get equal priority.
- Energy: The fuel or power used by vehicles.

2) Models:

The second table compare between algorithms used in each paper and the one that got best result of each paper.

			[3]	[30]	[19]	[29]	[28]	[2]	[1]	Ours
Model	ML	Heuristic	✗	✗	✗	✓	✗	✗	✗	✗
		Random forest	✗	✗	✗	✓	✗	✗	✗	✗
		Extreme Gradient Boosting	✗	✗	✗	✓	✗	✗	✗	✗
	DL	DNN	✗	✗	✗	✓	✗	✗	✗	✗
	RL	A2C	✗	✗	✓	✗	✗	✗	✗	✗
		A3C	✓	✗	✗	✗	✗	✗	✗	✗
		HMAS	✓	✗	✗	✗	✗	✗	✗	✗
		DITLCS	✓	✗	✗	✗	✗	✗	✗	✗
		RACS-A2C	✓	✗	✗	✗	✗	✗	✗	✗
		MFDQL-DTC	✓	✗	✗	✗	✗	✗	✗	✗
		GSCTSA-A3C	✓	✗	✗	✗	✗	✗	✗	✗
		DQN	✗	✗	✓	✗	✓	✓	✓	✓

		DDQN	X	X	✓	X	✓	✓	X	X
		D3QN	X	✓	✓	X	X	X	X	✓
		PPO	X	X	X	X	X	✓	X	✓
Best Model			GSCTSA - A3C	D3QN	D3QN	XGBoost	DDQN	PPO	DQN	D3QN

3) Other Characteristics

The third table compares additional characteristics of the experiments conducted in each paper, such as features, applications (mobile, IoT), and sensors used.

Characteristics

- Edge: A small device or system that processes data near its source.
- Feedback: Information returned to improve a system.
- Handle Accidents: Managing road incidents to reduce their impact.
- Priority for Emergency Vehicle: Giving emergency vehicles, like ambulances, the right of way.
- Predict Traffic: Estimating future traffic conditions.
- Notify with Certain News: Sending alerts about specific updates or events.

Sensor used

- Camera: A device to capture images or video for monitoring.
- Inductive Loops: Sensors in roads that detect vehicles.
- Weather: Environmental conditions like rain, wind, or snow

		[3]	[30]	[19]	[29]	[28]	[2]	[1]	Ours
Characteristics	Edge	✓	X	X	X	X	X	X	✓
	Feedback	X	✓	X	X	X	X	X	X
	Handle accidents	✓	✓	X	X	X	X	X	✓
	priority for emergency vehicle	X	X	X	X	X	X	X	✓
	predict traffic	✓	X	X	X	X	X	✓	X
	Notify with certain news	X	X	X	X	X	X	X	X
App	Mobile	X	X	X	X	X	X	X	✓
	IoT	✓ (theoretical)	X	✓	X	X	X	X	✓

Sensors	Camera	✓	✓	✓	✗	✗	✗	✗	✓
	Inductive loops	✓	✗	✓	✗	✓	✗	✗	✗
	Weather	✓ (Rain-time of day)	✗	✗	✗	✗	✗	✗	✗

3.3 Comparison between similar systems

The following table compares the features of real-world systems.

- **Cybersecurity:** Protecting systems and data from cyber threats.
- **Edge:** Local devices or systems that process data near the source.
- **Road Safety (Accidents):** Measures to prevent or reduce accidents on the road.
- **Tunnels:** Underground passageways for vehicles or pedestrians.
- **Predict:** To estimate or forecast future events or conditions.
- **Waterproof:** Resistant to water damage.
- **Identify and Track Traffic Violators:** Detecting and monitoring people who break traffic rules.
- **Pedestrian Monitoring:** Observing and tracking people walking near or across roads.

Table 0.5 Comparison between similar systems

	[31, Intellias]	[32, Telegra]	[33, Iteris]	[34, Kapsch]	Ours
Cybersecurity	✓	✗	✓	✓	✗
edge	✓	✓	✓	✓	✓
Road safety (accidents)	✓	✓	✓	✓	✓
tunnels	✗	✓	✓	✗	✗
predict	✓	✗	✓	✓	✗
Waterproof	✗	✓	✓	✓	✗
Identify and track traffic violators	✗	✗	✗	✗	✗
Pedestrian monitoring	✗	✗	✗	✗	✗

3.4 The Need to Extend Related Work

After a thorough review of the tables, the following conclusions were drawn:

1. No implemented open-source system is adequately documented to explain the technologies or methods used.

2. Only one of the existing papers addresses the Egyptian context, and the latest project report [1] fails to document the entire case.
3. The traffic optimization model is intended to be trained using real-world data, although most related work has been based on simulations due to their cost-effectiveness and suitability for training purposes.
4. The system's simulation should include key factors such as line length and traffic flow, which are commonly considered in similar systems. Additionally, elements like pedestrian presence, weather conditions, and time of day should be incorporated to enhance the model's accuracy.
5. The primary focus of the system should be minimizing traffic delays and ensuring fairness among vehicles. In some cases, energy consumption can be directly considered as an optimization factor.
6. In the latest related work on traffic optimization, reinforcement learning (RL) is used, yielding better results compared to traditional machine learning (ML).
7. Recent research also attempts to consider state temporal and spatial information, consider nearby intersections (by using GAT, LSTM), and optimize the model as much as possible [3],[22],[24].
8. Baseline modeling should involve using an available online model as a baseline, followed by performance improvements through customizing rewards, enhancing the model, training multiple models, or applying techniques such as grid search or Optuna.
9. Deploying the system in real-world IoT environments is essential. Common traffic monitoring sensors, including cameras, inductive loops, and weather sensors, should be utilized when environmental conditions are considered.
10. The system should be designed to be interactive, allowing user feedback to be provided for continuous improvement. Additionally, relevant updates and alerts should be sent to users.
11. Continuous refinement should be conducted by leveraging real-world data collected after deployment, enabling the system to adapt to changes such as government road improvements or the deterioration of road conditions due to factors like rain.
12. Accidents must be handled effectively by ensuring road sections are halted when incidents occur to maintain safety [32],[34],[35].
13. Traffic prediction capabilities should be integrated so that congestion can be anticipated and traffic flow optimized accordingly [32],[34],[35].
14. Cybersecurity measures must be implemented to ensure system safety and compliance with cyber regulations [32],[34],[35].
15. Fast response times should be ensured for both users and traffic conditions, minimizing the impact of factors such as distance and internet latency [3].
16. Traffic violators should be identified and tracked to enforce regulations effectively [32],[34],[35].
17. Support for emergency vehicles must be provided, and awareness of accidents should be maintained to ensure quick and safe passage.

3.5 Scope of Work

According to section 3.4 the scope of work:

- Make a documented open-source project.
- Train on crowded area in Egypt.
- Simulate using SUMO.
- Include main road measurements such as traffic flow and speed in the simulation.
- Minimize delays and ensure fairness.
- Use reinforcement learning (PPO, D2QN).
- Baseline modeling using open-source data from Egypt.
- Develop a small prototype for a single intersection using video analysis with computer vision.
- Latency should be optimized with a near real-time processing architecture.
- A mobile application should be developed.

3.6 Conclusion

In this chapter, the key results from related papers were presented, followed by a comparison of each paper's algorithms, environment, and other characteristics. Existing systems were also compared, and potential extensions were discussed based on the analysis of the tables. Finally, the scope of the work was outlined. The next chapter will focus on system requirements and design, where the project's architecture, system flow, SPI development stages, and applied software engineering methodologies will be discussed in detail.

SYSTEM DEVELOPMENT

In last chapter the key results from related papers were presented, followed by a comparison of each paper's algorithms, environment, and other characteristics. Existing systems were also compared, and potential extensions were discussed based on the analysis of the tables. Finally, the scope of the work was outlined

In this chapter, we will delve deeper into the project by detailing its life cycle, presenting diagrams, and discussing the software model.

4.1 Software Process Improvement (SPI model)

[35]

The Software Process Improvement (SPI) model was selected due to its iterative structure, which allows evolving requirements to be addressed and complex challenges to be divided across suitable iterations. It will be covered in the areas of phases, team management, iteration, and results.

4.1.1 Phases

The SPI model was implemented through four main phases:

1. **Initiation:** Reference documents and research papers were reviewed during the first term.
2. **Planning:** The system environment was designed, and software requirement specifications (SRS) were prepared.
3. **Execution and Monitoring:** Code was written, reviewed, and technical problems were addressed.
4. **Closure:** Final documentation was completed, and the project was finalized.

4.1.2 Team Management

SPI roles were adapted for the project's limited scope:

1. **Project Management:** Role assignments and progress monitoring were conducted.
2. **Product Development:** Code implementation was performed.
3. **Peer Reviews:** Code logic was internally reviewed.
4. **Quality Assurance:** Feedback was provided by supervisors.

5. **Configuration Management:** File organization was handled, and final outputs were published on GitHub.

- **Meeting Minutes:** Key discussion points, decisions, and responsibilities were documented to ensure alignment and clarity across the team [Appendix III: Minutes of meeting].
- **Weekly Timesheets:** Work hours and tasks were tracked weekly to monitor individual contributions and project progress [Appendix IV: Weekly Timesheet].
- **JIRA Implementation Progress** The following QR is link to our project JIRA plan, also mentioned in [APPENDIX V: JIRA PLAN].



Figure 0.1 QR code of JIRA plan

4.1.3 Iterations

The SPI model followed across four main iterations:

- 1 Research on ideas and review of reference papers (First Term).
- 2 Initial environment design and study of related projects.
- 3 Completion of environment design.
- 4 Data modification, experiment execution, and prototype development.
- 5 Improvement of Results by benchmarking SUMO-RL (Version 2.0.0)
- 6 Improvement of Results by benchmarking SUMO-RL (Version 2.0.1), and implementation of RL online.

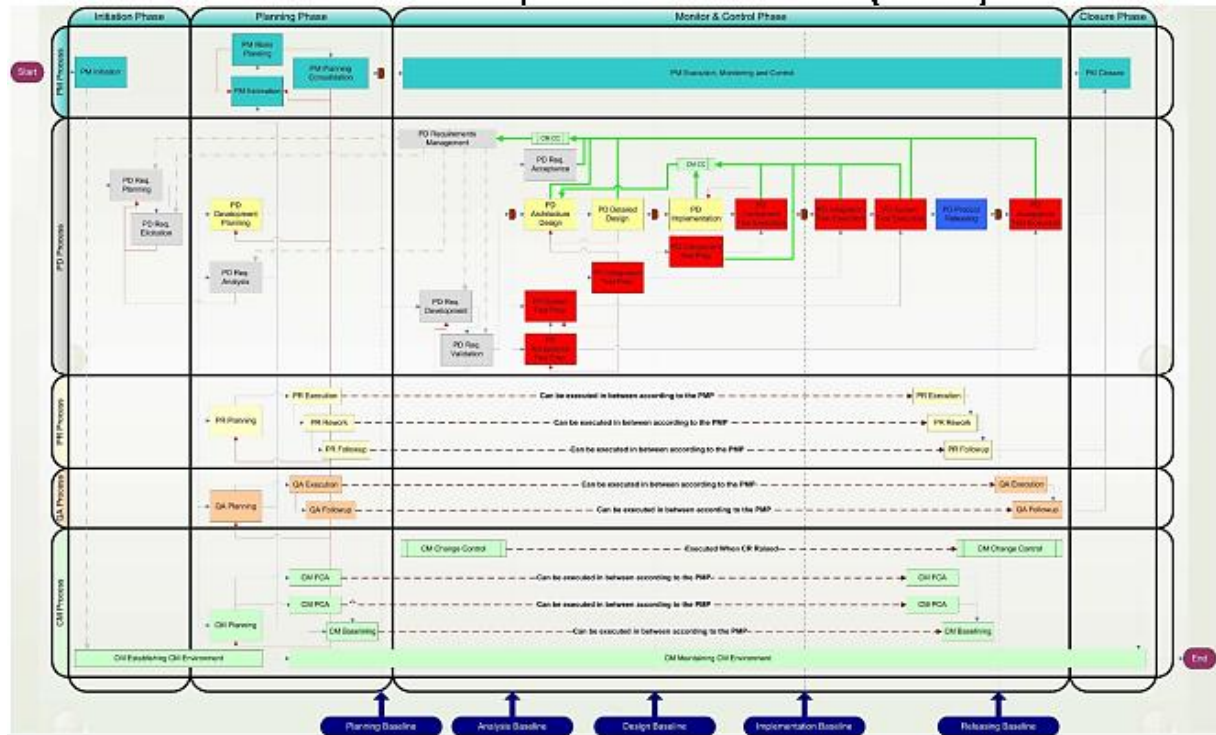


Figure 0.2 SPI Diagram

4.1.4 Results

- A realistic prototype was developed with the focus on training an effective model.
- Future iterations may involve accuracy improvements, use of real data, and optimization for deployment.
- The project was assessed to be at a "Managed" or nearly "Defined" maturity level.

4.2 Process Activity Task Matrix (PATM MATRIX)

In this section, the Process Activity Task Matrix (PATM) is presented, through which the system was analyzed to its main processes. Then, each process was analyzed to its activities. Finally, each activity was analyzed to its tasks

Table 0.1 PATM Matrix

Process	Activity	Task
---------	----------	------

Baseline	Baseline with reference project	Run a portion of the simulations.
		Define baseline parameters.
		Compare results.
	Implement The Model	Create a modified environment.
		Perform data preprocessing and analysis.
		Conduct experiments at different scales.
		Analyze results
	Create modified data	Modify the data.
	Create a prototype model.	Choose the best model results and save them.
		Set up ESP32 as a single-lane traffic light.
		Analyze one video, estimating it as a single lane.
	Implement RL online	Implement prototype connection and test environment on it.
Emergency	Maintaining emergency vehicle	Create account
		Log in to the account.
		Select vehicle route
		Make a database for connecting to the server
		Manage Database using Backend Service
		Send place to server database.
Publication	Documentation	Documentation
	Open source	Publish public repo on GitHub.

4.3 Requirements

Functional Requirements

The proposed intelligent traffic management system will provide the following core functionalities:

- Adaptive Traffic Signal Control (FR-01):
Uses Reinforcement Learning (RL) to dynamically adjust traffic light timings in real-time, aiming to reduce vehicle waiting time, delay, and time loss while improving traffic speed.
- IoT-Based Data Collection (FR-02):
Integrates cameras and computer vision to detect vehicles and collect traffic data continuously for analysis and model training.
- Emergency Vehicle Priority Handling (FR-03):
Authenticated emergency vehicles can log in, report incidents, and receive dynamic signal prioritization along their route without disrupting overall flow.
- Accident-Triggered Signal Adjustment (FR-04):
Authenticated users can report accidents with location/severity, and the system will respond by adjusting nearby signals to restrict traffic and enhance safety.
- Simulation-Driven Optimization & Performance Monitoring (FR-05):
Uses SUMO for simulation and offline experience data for model training, followed by real-time validation. The system also tracks its own performance using live metrics such as traffic efficiency and model accuracy.

For full Software Requirements Specification (SRS), see [appendix II, Software Requirements Specification (SRS)]

4.4 Design diagrams

In this section, various diagrams will be presented to clearly illustrate the project workflow. These will include the plan, design, experimental setup, prototype, mobile application, the steps to be followed, and the tools to be utilized.

4.4.1 Activity Diagram (Overall Architecture)

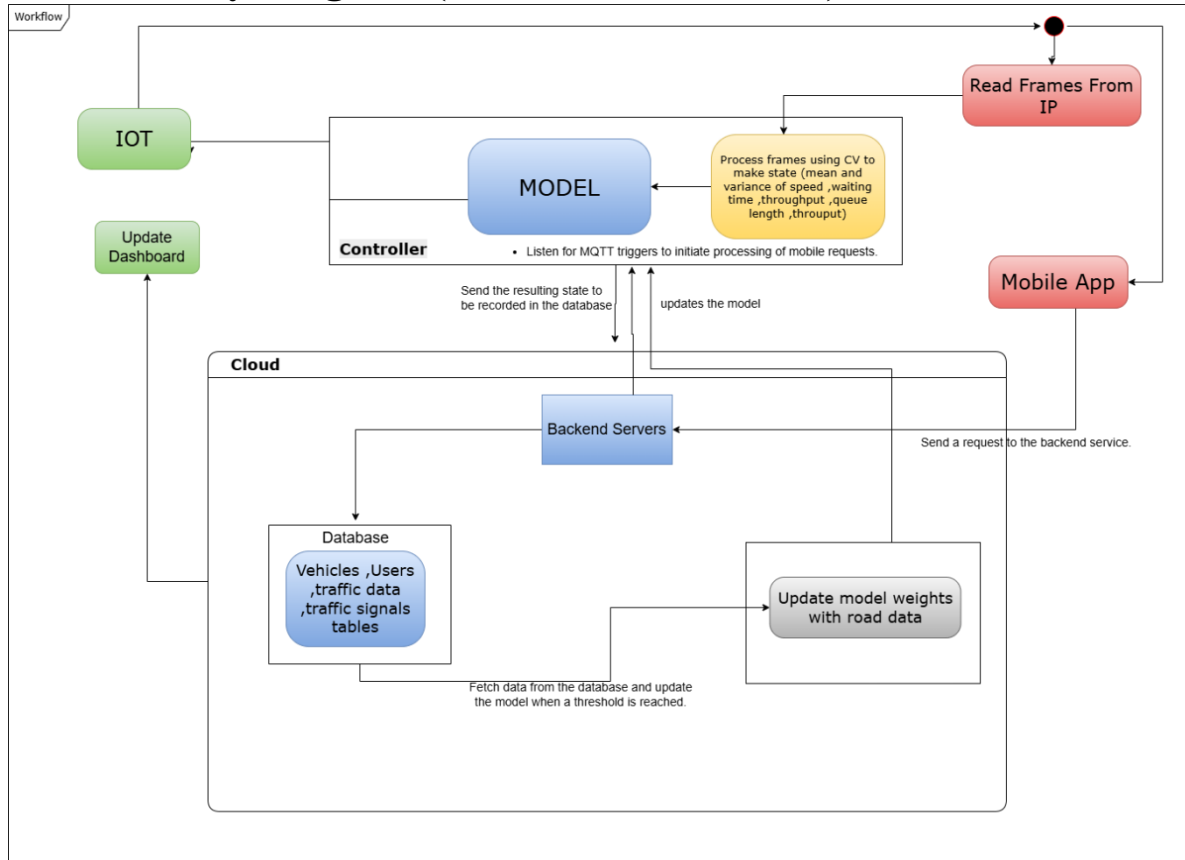


Figure 0.3 Activity Diagram

4.4.2 High-Level Workflow Flowchart (Experimental Architecture)

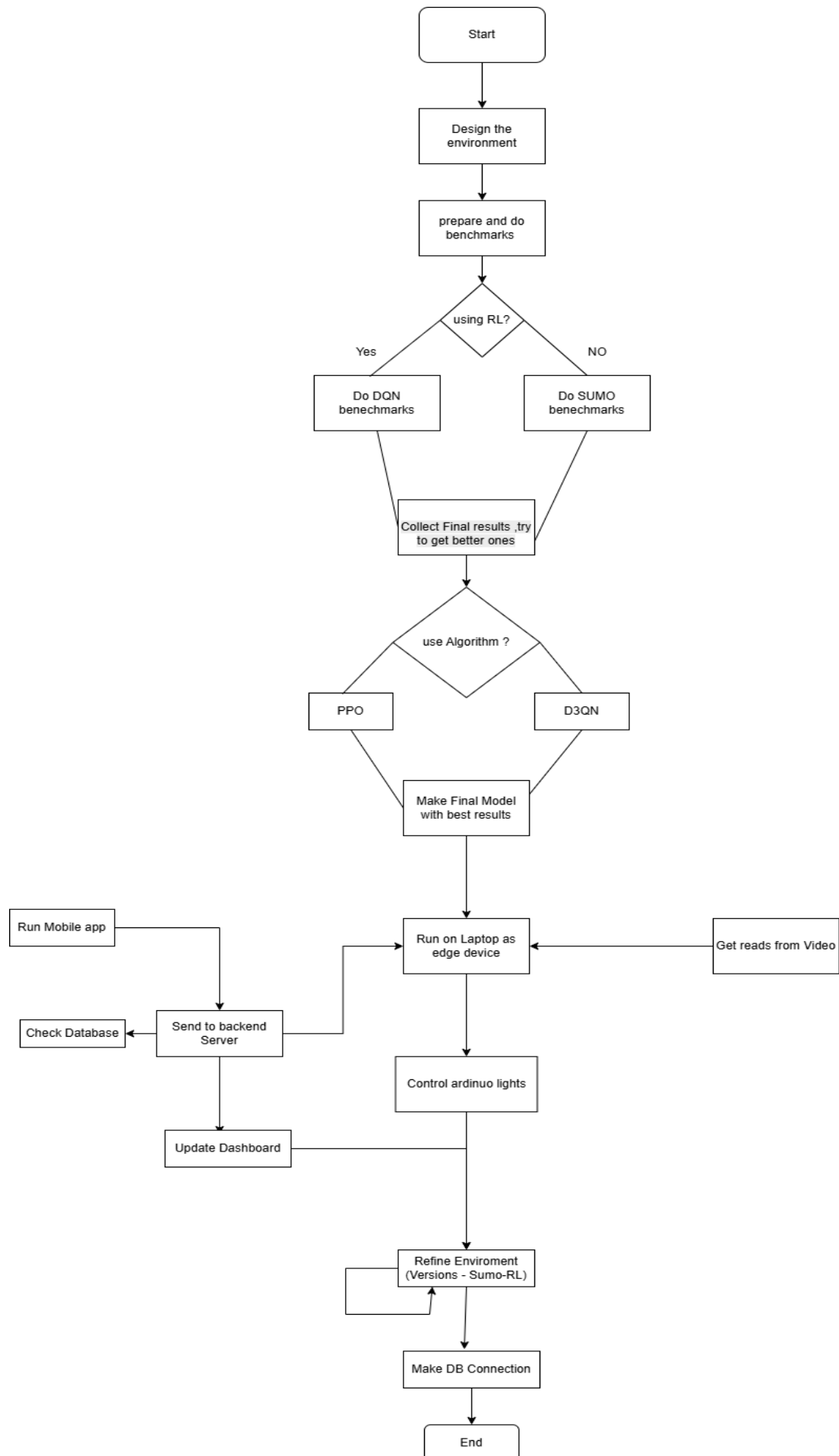


Figure 0.4 High-Level Workflow Flowchart

4.4.3 Tools and Environment

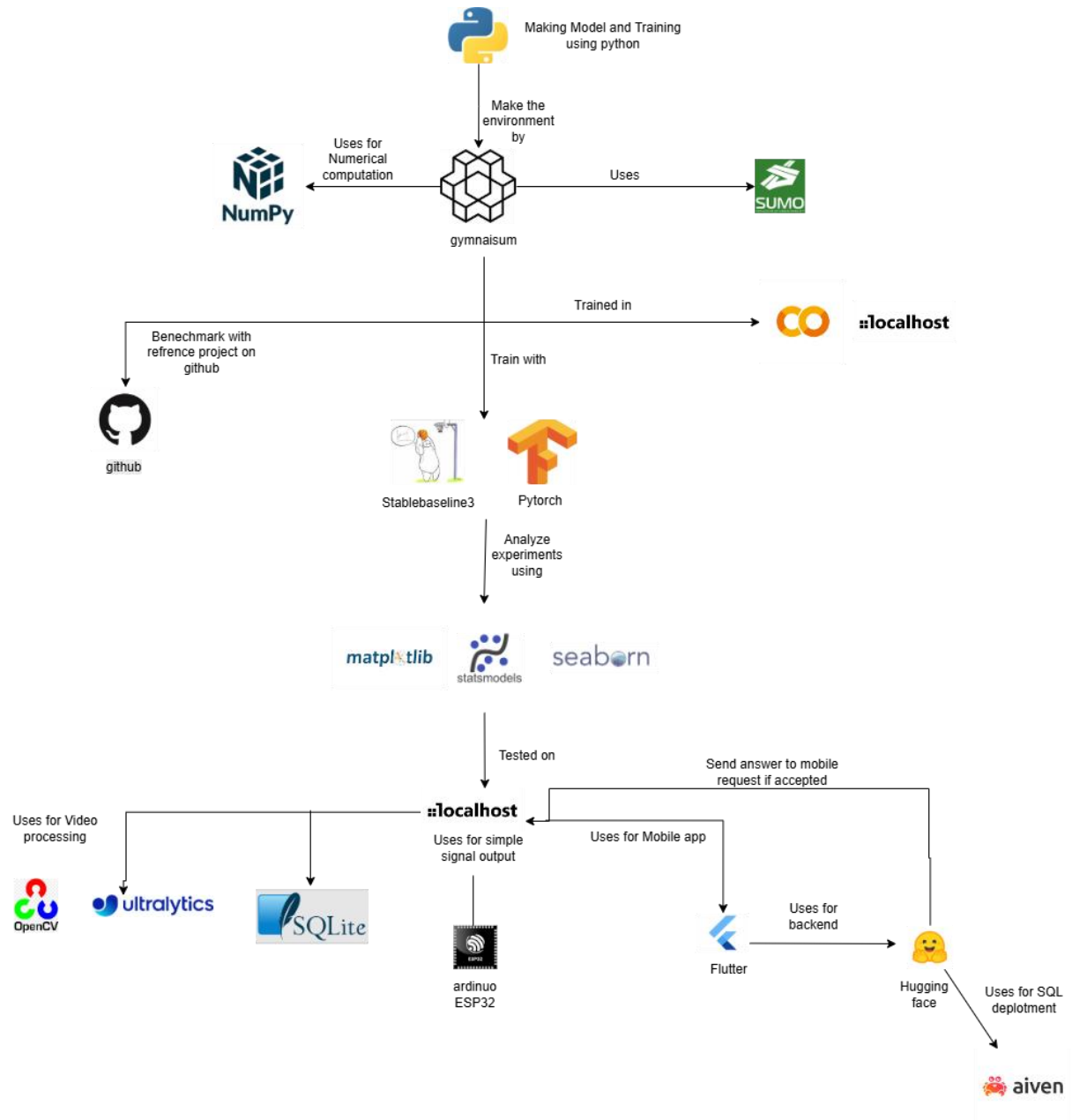


Figure 0.5 Tools and Environment

4.4.4 Use Case Diagram (Mobile App)

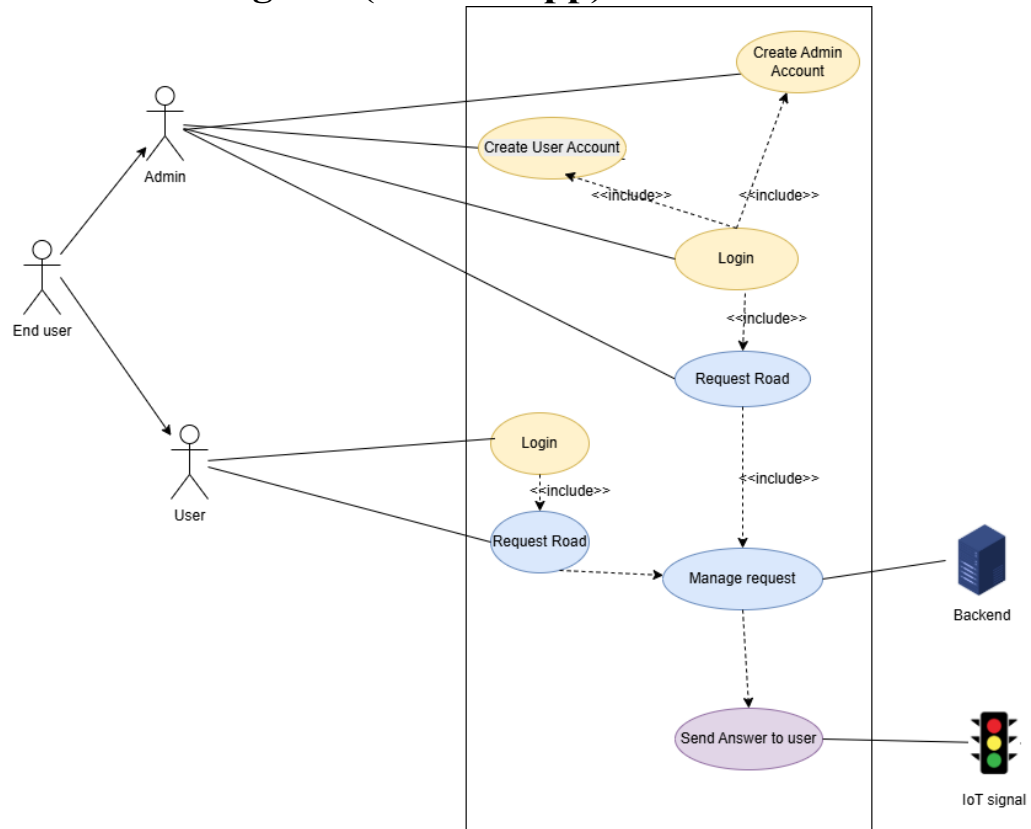


Figure 0.6 Use Case Diagram

4.4.5 Class Diagram

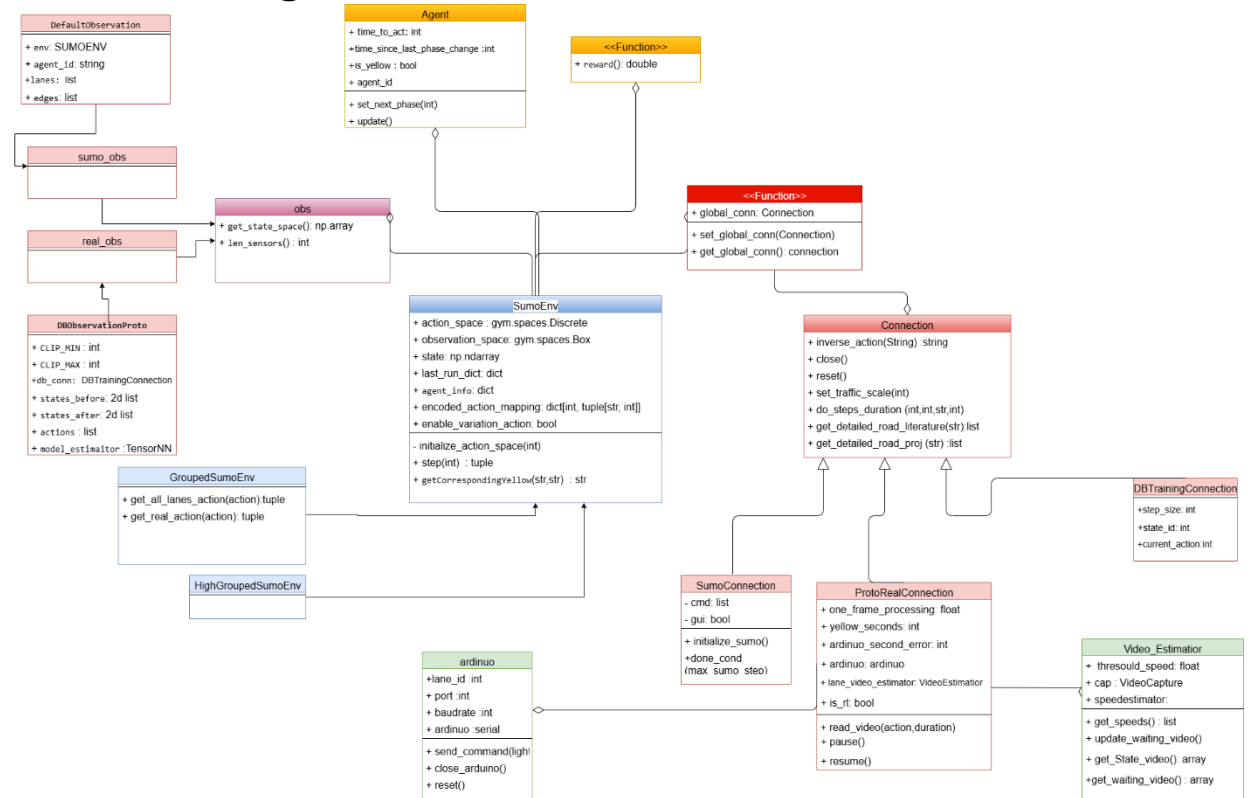


Figure 0.7 Class Diagram

4.4.6 Database Schema Entity-Relationship Diagram (ERD)

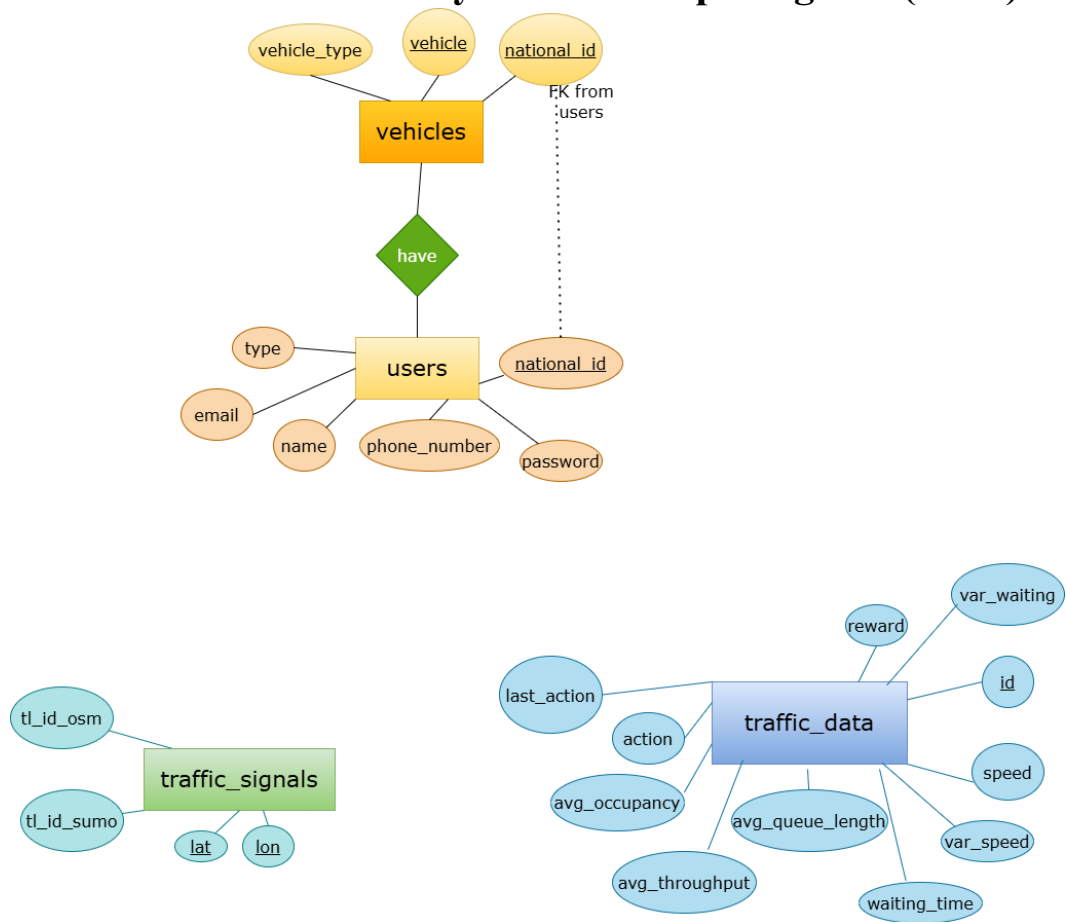


Figure 0.8 ERD diagram

4.4.7 Traffic Backend Component diagram

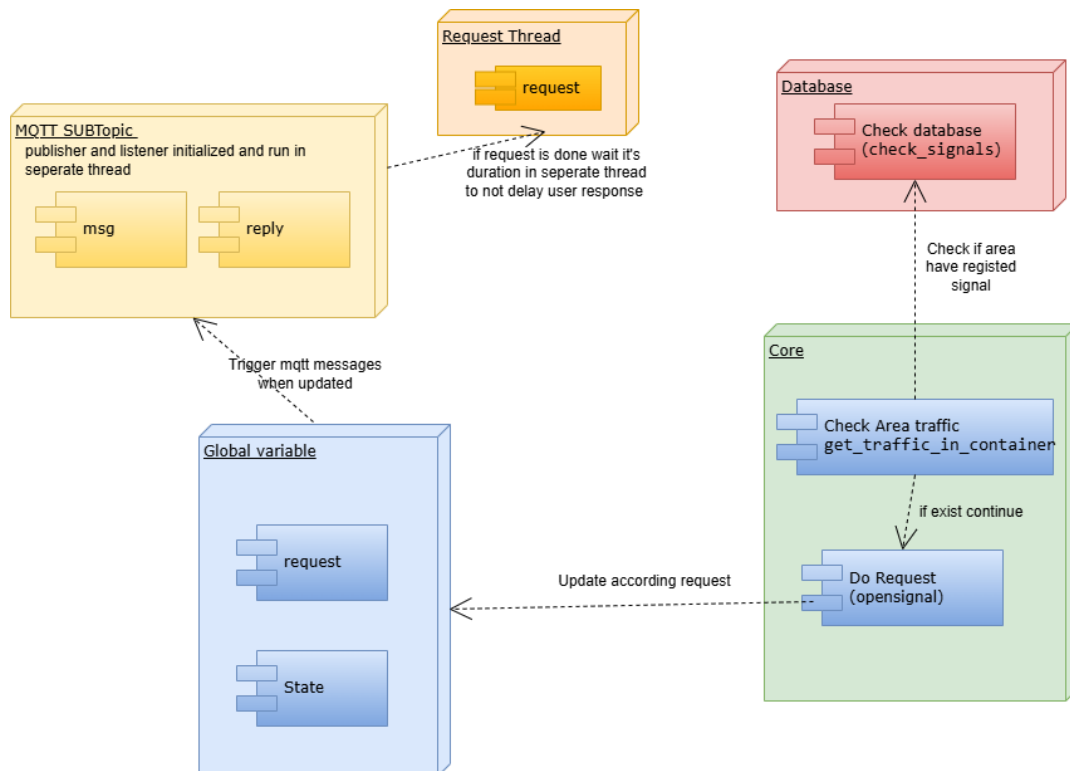


Figure 0.9 traffic backend diagram

4.4.8 Prototype ESP32 Schema

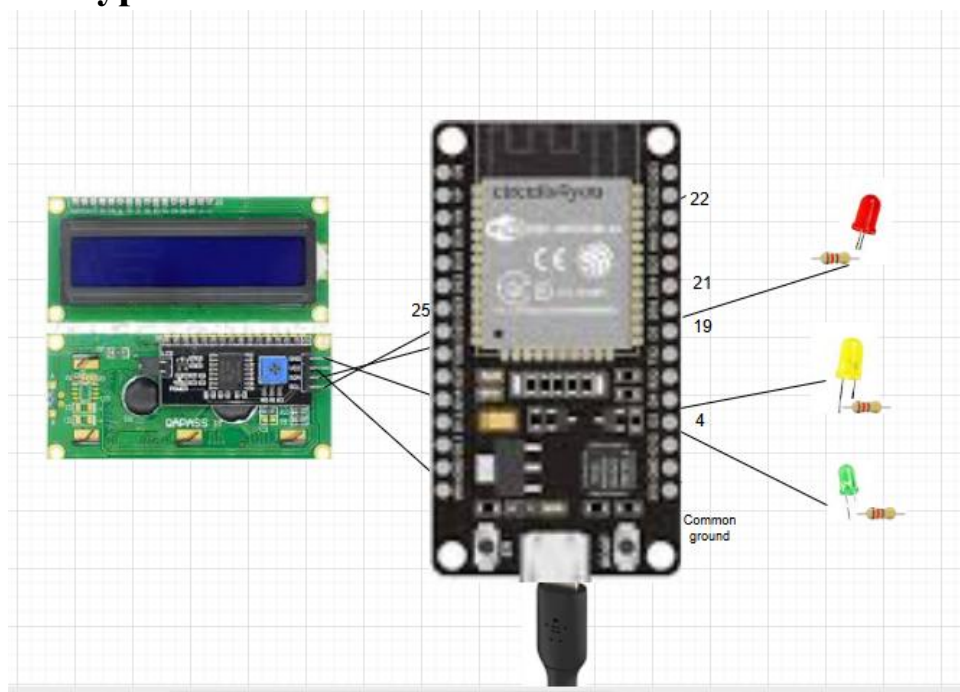


Figure 0.10 Prototype ESP32 Schema

4.4.9 Pipeline of traffic management processing model

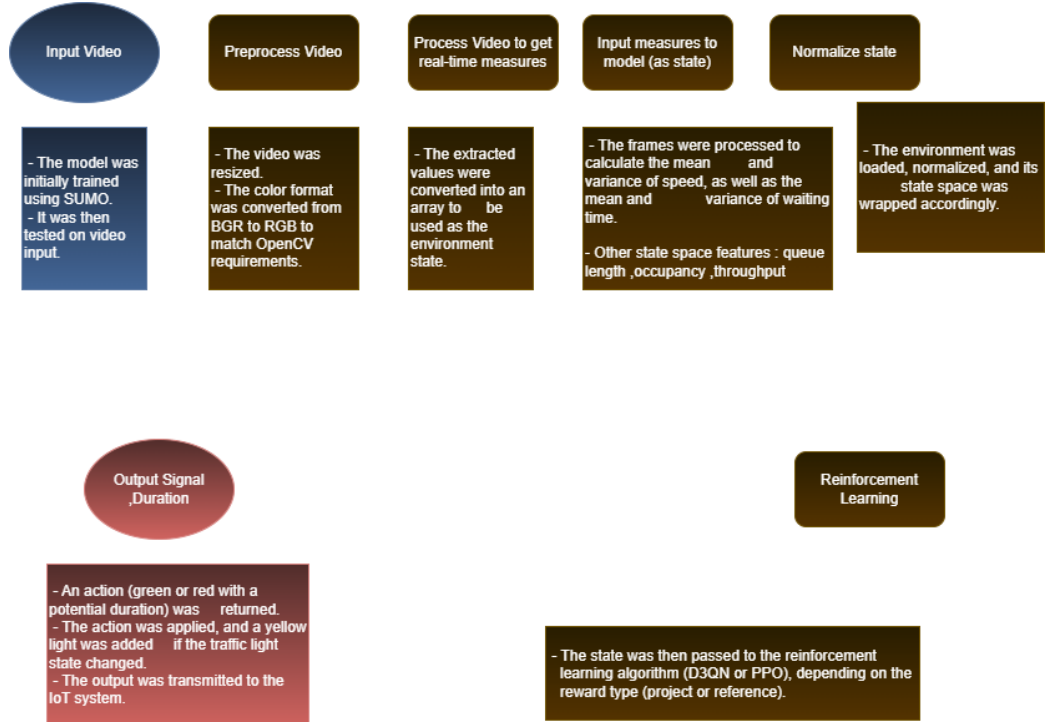


Figure 0.11 Pipeline of traffic management processing model

4.5 Prototype design

In the previous section, the environment design, experiment setup, and experimental results were discussed. In this section, the developed prototype is described, which utilizes the model that achieved the best performance during the analysis phase following training.

To implement the system, traffic measures are intended to be obtained from controlled lanes and averaged. However, in this prototype, the focus was limited to estimating one lane using individual measurements and controlling a single traffic light via a hardware-based setup.

A very important code details is that all Prototype is made with phase 1 of code, just for general prototype (Github pushed in June 2025), for later phases may not work.

4.5.1 Computer Vision Implementation

4.5.1.1 State Space estimation

As defined in the [section 5.1.3 State Space], the environment state includes 7 road measures:

- Mean speed
- Speed variance
- Mean waiting time
- Waiting time variance
- Occupancy
- Throughput
- Queue length

These features are used to inform the model's actions based on current road conditions.

In the prototype, due to time constraints, only **speed** and **waiting time** were directly measured. The remaining measures were estimated as follows:

- **Mean speed** and **speed variance** were computed based on detected vehicles.
- **Waiting time** was estimated by measuring the duration each vehicle's speed remained below a threshold (as in SUMO definitions). The logic used was:
 - For each frame:
 - If a vehicle was still in a "processing" state (not yet exited), the global waiting time was increased by 1/fps.
 - If the frame was not processed but the vehicle had not exited, the same increment was applied (this challenge is detailed in [Section 4.5.1.2 Prototype Video Streaming and Processing Delays]).
 - If the light was red/yellow, the waiting time was also incremented accordingly.
 - When a vehicle exited (i.e., newly processed), it was removed from the waiting list.
- **Mean and variance of waiting time** were computed from the accumulated individual waiting times.

The other features were **estimated** using the following formulas:

$$\text{occupancy} = \min\left(\frac{n_{car}}{10}, 1\right) * 100$$

Equation 0.1 Estimated occupancy

Assuming max 10 vehicles per lane

$$\text{queue_length} = n_{\text{waiting_car}}$$

Equation 0.2 Estimated queue length

$$\text{throughput} = n_{car} * fps$$

Equation 0.3 Estimated throughput

where fps is number of frames per second

This equivalent to Rate of passing vehicles

The ultralytics library was used for object detection (YOLOv11n).

4.5.1.2 Prototype Video Streaming and Processing Delays

For the prototype, a FastAPI app was developed to stream random videos from the internet, due to limited access to real-life data. The app was configured to run on a local IP (localhost), and the video feed was simulated to mimic a connected camera.

The prototype is runned on best results [section 5.7.3 Best Observed Results] got from **Data 3, Scale Normal** (as suitable for video we have)

Streaming API

The details of the API are outlined below. Its primary function is to generate frames from a given video and stream them.

- Small delays are introduced between frames to prevent processor overload.
- Pause and resume functionalities are included; however, they do not perform meaningful thing if real video but are implemented to simulate traffic signals given video (e.g., stopping or resuming flow after a stop).
- A locking mechanism is applied to ensure thread safety, particularly when the API is accessed simultaneously by a browser and the pause/resume commands are triggered by the application, preventing potential conflicts.

During testing, a frame was captured and held until the processing was completed. YOLOv11 was used for processing, and a maximum processing time per frame of approximately 0.56 seconds was observed. This value, referred to as `processing_time_per_frame`, was determined based on the performance of the test device, which included an Intel i7 7th generation processor and 16 GB of RAM.

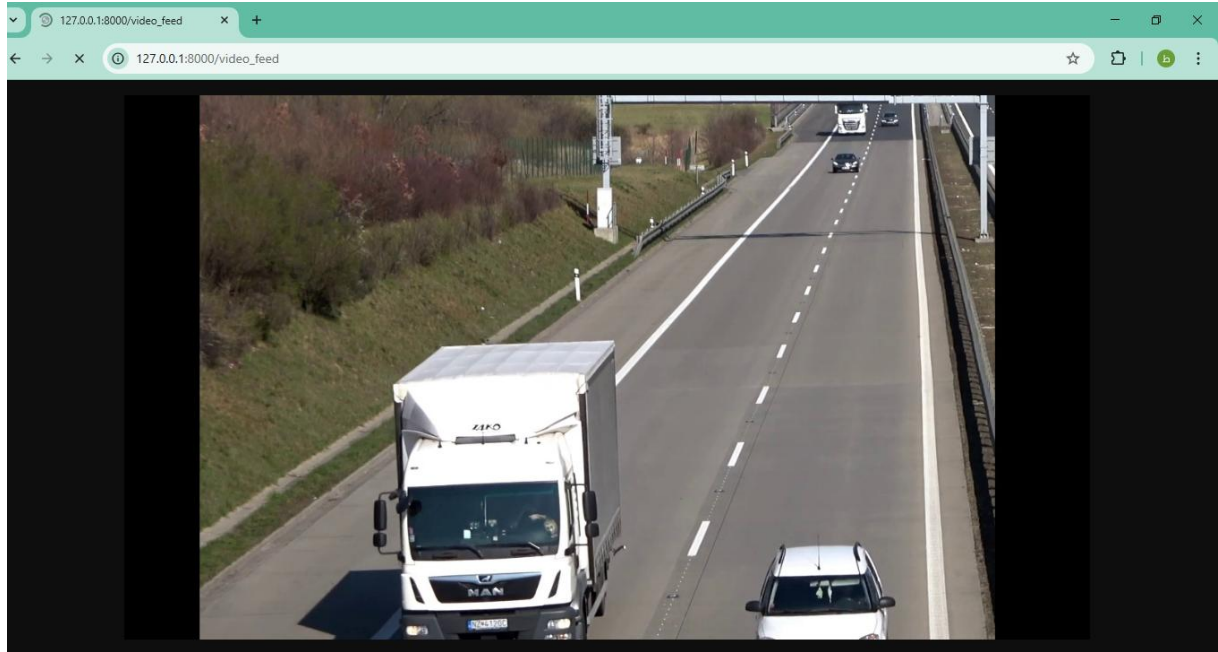


Figure 0.12 Testing prototype on video hosted on local IP address

4.5.2 Arduino Implementation

The Arduino was programmed using the C language and the LiquidCrystal_I2C library. Communication with the PC (LOCALHOST /Edge) was handled through pyserial[36]. The Python script handled model predictions and transmitted the corresponding traffic light actions via serial.

Several sources of error were identified in the Arduino component of the system. The primary source was the delay in sending signals to the Arduino. In this project, communication was handled via a serial connection, resulting in an average delay of approximately 0.5 seconds.

The process followed was as follows:

1. A command was sent to the Arduino specifying the full duration.
2. The timing discrepancy caused by the Arduino's delay was used to adjust the video reading offset.

$$\text{duration_wanted_without_arduino} = \text{duration} - \text{arduino_second_error}$$

Equation 0.4 duration without arduino error

This cycle ensured that the intended duration x was accurately achieved, compensating for any delays introduced by the Arduino.

An additional source of error was observed during the Arduino reset process. While the initial reset occurred without issue, subsequent resets triggered by application requests introduced minor timing inaccuracies. Due to our time constraints and real-time processing limitations, further reduction of this error was not feasible. However, the impact remained minimal and only occurred when handling specific requests.

4.5.3 Backend Implementation

The backend of the project is structured into three main components:

Traffic_Account

This component is hosted on Hugging Face and implemented using FastAPI.
Link: [<https://taha454-trafficmanager-account.hf.space>]

It is responsible for managing login functionality for both users and administrators.

- As described in [Section 4.4.4 Use Case Diagram (Mobile App)], account creation is restricted to administrators. This restriction ensures that only verified users are granted access to the mobile application.

Traffic_Manager

This component is also hosted on Hugging Face and built using FastAPI.
Link: [<https://taha454-trafficmanager.hf.space>]

It is designed to handle communication between the mobile application and IoT devices, as detailed in Section 4.4.7 (Traffic Backend Component Diagram).

A known issue discussed: the potential for unreliable internet access at signal locations. This has been addressed using a TCP/IP-like protocol with the following message sequence:

CHK → ACK → DONE QL? → QL.... → EMR/ACC ...duration....

After the availability check, a request for queue length is issued, as opening the signal would be ineffective in cases of severe congestion. However, in certain scenarios, the system may not behave as expected; addressing these cases is left for future work (Section 6.2).

4.5.4 Mobile implementation

The "DRIVE MATE" app is designed to facilitate emergency vehicle passage and manage road accidents. It achieves this by opening the traffic signal for emergency vehicles—provided the queue length is less than or equal to 15, as longer queues would render the intervention ineffective—or by closing the signal if an accident is requested that blocks the road.

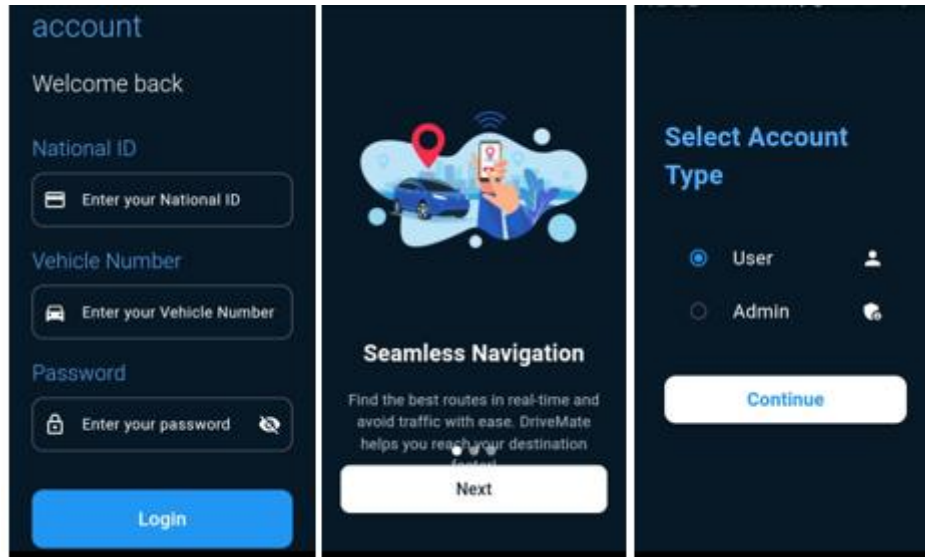


Figure 0.13 Screens of the UI of the prototype mobile app

The whole processes as follows:

1. The user first login, with verification described in [4.5.3 Backend Implementation]
2. A user selects a point, and an retrieved the shortest path.
3. The coordinates are checked against the system's traffic signal database. This is performed by identifying the minimum bounding square that includes the selected path and checking for any registered signals. However, the API may return two signals with opposite directions. Identifying and selecting the signal that matches the user's direction is considered future work (**Section 6.2**).
4. If a valid signal is found, it is activated. The duration and delay are provided by the user; however, future implementations aim to estimate these values automatically based on real-time positioning (**Section 6.2**).

4.5.5 Performance Monitoring

Streamlit [37] was used to develop a simple dashboard for monitoring the system's behavior. This tool enabled real-time observation of system interactions, which facilitated easier debugging and performance evaluation.

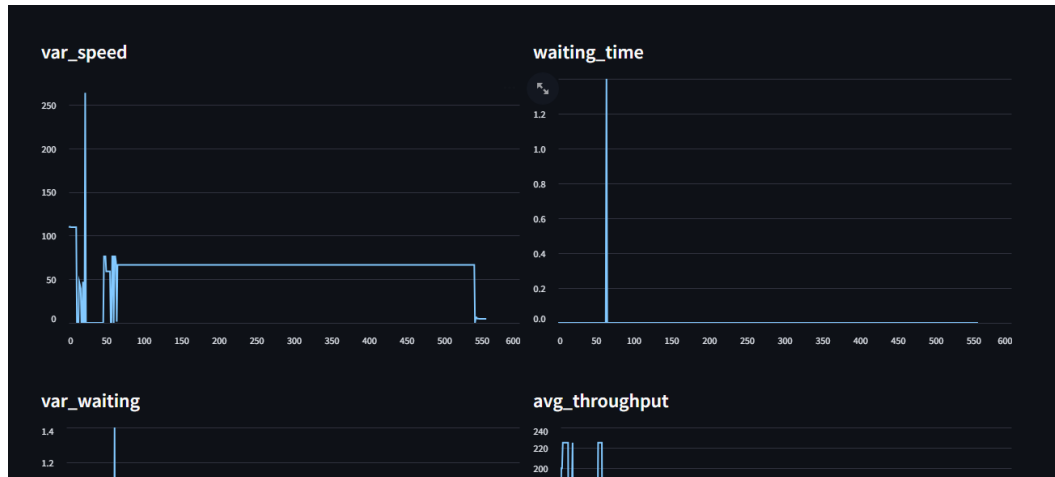


Figure 0.14 Screen of performance monitoring dashboard

4.5.6 Scaling the prototype

The prototype demonstrates a feasible pipeline:

Model training → **Analysis** → **Best model selection** → **Deployment**.

Applying the System in Real Scenarios (Single Traffic Light Area)

1. The SUMO map should be captured using OpenStreetMap (OSM).
2. The hyperparameters should be refined to match the selected area by modifying **Section 5.2.4 (SUMO Parameters)**, or the default values can be used. *(Basic SUMO hyperparameters can be modified through the SumoConnection attributes.)*
3. Data should be processed by generating routes and random trips from the previous step.
4. The code in **automated_experiments_running.py.py** under *enable_editing* should be executed to generate *info_road.xlsx*, if a different area is used.
5. As stated in **Section 5.5 (Final Results of System)**, the best configuration identified was **PPO**, *highgrouped*, and *literature reward*, optimized for the *San Stefano / Mosheer* areas.
 - This configuration may be reused, or alternatively, *automated_experiments_running* and related files can be modified for the new area to re-run the experiment and obtain a new best model.
 - Any other algorithm supported by *Gymnasium* (besides PPO or D3QN) may also be applied, with custom rewards or environment levels defined.
 - Metrics can be added or removed as needed.
 - Finally, the analysis can be conducted (as shown in the *Results V2.0.1* folder) to select the best model. *(The SUMO-RL benchmark step can be skipped if desired.)*

6. Once the model has been trained in SUMO, new connections suitable for deployment should be added, and the environment can then be executed on the chosen setup.
7. After deployment, usage data should be collected and monitored. Online reinforcement learning should then be used periodically to improve performance using real-world data.
 - It's very recommended to refine on real-time data before deployment, as reinforcement learning models are known to be sensitive to environment changes [38].
 - The proposed architecture in [4.4.1 Activity Diagram (Overall Architecture)] should be followed to ensure an optimal deployment workflow, including deploy on edge devices like Raspberry Pi, optimizing low-resource environments.
8. The project can be further improved by cloning and modifying the existing implementation.

4.6 DB Online Prototype

This prototype was developed for version 2.0.1. It represents a small database-based prototype, distinct from the previously mentioned one. The purpose of this version is to simulate the system's usage, allowing future improvements to be made based on the data collected (like the process implemented in the previously described prototype dashboard).

The operational flow can be summarized as follows:

1. Data is collected by the system.
2. The system is then trained online using the collected data.

Online Database Version

The online version of the database primarily focuses on database connectivity and its integration with the SUMO environment module.

Key Design Aspects

- A simple MySQL database with randomly generated data was used, as the system has not yet been deployed.
- The workflow proceeds as follows:
 - A) After deployment, data are collected in the form (*state before – action taken – state after*).
 - B) For training purposes, each state must be mapped to a possible action. A simple neural network model was therefore trained to predict the *next state* given the *current state* and *action*.
 - C) The collected data are iteratively processed and fed into the model; all encapsulated within the database connection module.

A minor issue has been observed: the system may add approximately seconds of yellow light time even when the action remains unchanged.

4.7 Module decomposition

In addition to [section 4.4.6 Class Diagram], a modular design was adopted by organizing the project into separate files, each responsible for a distinct function. This structure provided a clear delineation of the system's components, making it easier to understand, maintain, and scale. By isolating functionalities into well-defined modules, development and debugging processes were streamlined, and the overall architecture of the project was made more transparent and extensible.

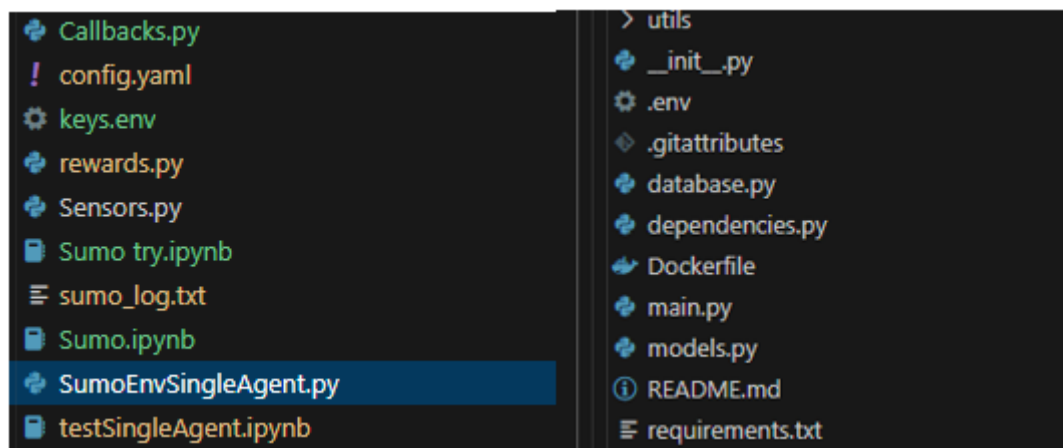


Figure 0.15 Organizing of files in modular design

4.8 Basic implemented security

1. Passwords were securely stored using hashing with bcrypt.
2. Access to the mobile application was restricted to verified users, as discussed in [section 4.5.3 (Backend Implementation)].
3. In accordance with modular design principles, secret keys were stored in separate configuration files.

Further security enhancements—such as restricting database access to a closed IP range and performing vulnerability assessments—are considered future work

4.9 Open-Source Availability And License

This project is open source and available for public access and collaboration. You can explore the full code and resources by scanning the following QR code:



Figure 0.16 QR code of the project main GitHub

An MIT License was added to the project to ensure that the software remains freely available as open source. This license was selected to allow broad usage, modification, and distribution, while explicitly requiring attribution to the original authors of the work.

4.10 Conclusion

In this chapter, we explore the project in more detail, including its requirements and lifecycle. We present relevant diagrams, the JIRA plan, and the software model. These elements provide a structured overview of the project's development. The chapter aims to clarify the project's scope and planning. The next chapter will further elaborate on the experimental design and present the results of it.

EXPERIMENTAL DESIGN, EVALUATION, AND SYSTEM PROTOTYPING

In the previous chapter, the project was delved into in more depth, its life cycle was detailed, diagrams were presented, and the software model was discussed.

In this chapter, the environment design, data used, state space, action space, reward structure, and the chosen model will be presented. The selection of hyperparameters and the detailed environment setup will then be explained. This will be followed by the presentation of the environment's results and the implementation of the system prototype.

5.1 Environment Design

5.1.1 Problem and Environment Design

As explained in (Section 3.4, The Need to Extend Related Work), the goal was to implement an area traffic optimization system, designed to control traffic with optimal results—minimizing or eliminating waiting time and delays, and achieving the highest possible speeds as allowed by the road conditions. Additionally, the system was intended to be efficient for deployment, with no latency, and capable of adapting as road states change. However, this objective is challenging, as area-wide control constitutes a multi-agent problem. For realistic scenario, factors such as duration and clear space for deploying were considered essential.

In this project, the focus was placed on training a single-agent model to achieve the best experimental outcomes. Traffic was modeled under default scenarios (the default without RL), and performance insights were drawn accordingly. A prototype was also developed and tested in a simple scenario using a mobile application.

As detailed in [Section 5.1.4 Action Space], the training focused on the single-agent problem. However, traffic lights typically control multiple lanes, leading to three options for action space design:

- The first option involves assigning independent green/red signals to each lane. While this approach is valid, it tends to introduce noise and may become computationally expensive for intersections with many lanes. Furthermore, it may exceed the capabilities supported by the Gymnasium (Gym) API used in this project.
- The second option groups lanes by direction, which is commonly used in research [22], [28].
- The third option involves controlling all lanes simultaneously. Although less optimal, this reflects the default traffic system design in Egypt, which is the context of this study.

To accommodate these cases, a flexible model was designed. The system supports three configurations: full control (which used in [1]), grouped lanes (used for applying advised strategies), and high-level control as the default scenario.

To prevent the system from getting stuck on a single action, a boolean flag `enable_variation_action` was introduced. This flag appends the current action to a queue, and if the same action is repeated three times, the action is reversed (e.g., green is changed to red) for each lane.

5.1.2 Data Used

As explained in (Section 3.4, The Need to Extend Related Work), the current work has been focused on the Egyptian region. However, the only found available project with data for Egypt is that used in [1]. In that referenced project, training was conducted on a sample of traffic lights that were controlled using a simple technique—based on looping logic rather than a gym environment—which may pose limitations in scalability for more advanced algorithms and restricts the algorithm's ability to control most areas effectively.

To address this, new Egyptian traffic data was prepared from two sources:

- Data 1: A traffic light was selected at random from the existing dataset, specifically one that controls a high number of lanes, to ensure thorough testing of the code (Tomart).
- Data 2: The traffic lights were estimated to cover approximately the same area as real ones, based on available geographical references, specifically in the Sidi Gaber and Mosher Ismail areas.
- Data 3: Another real dataset was obtained for the San Stefano area.

Using OpenStreetMap (OSM) [39] the system can be used with any other area containing more realistic traffic light configurations (or map is modified to include the signal if not).

Data for a 48-hour SUMO random trip was preprocessed at Version 1.0.0, for V1.2 Only scope of time, which is used in experiments, is used in preprocessing (generate trip file).

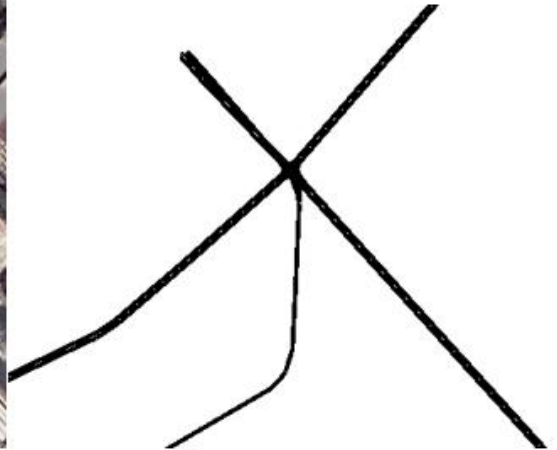


Figure 0.1 Data 1 Tomart on google maps (left), SUMO (right)

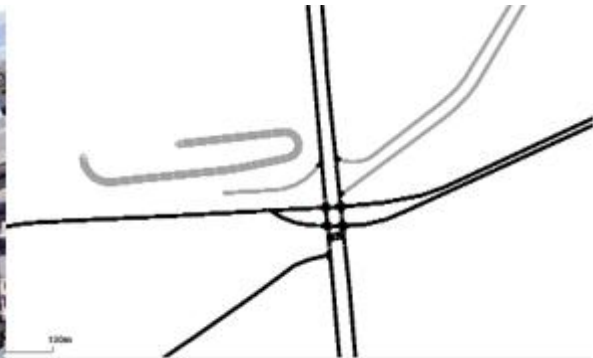


Figure 0.2 Data 2 Mosheer Ismail on google maps (left), SUMO (right)



Figure 0.3 Data 3 San Stefano on google maps (left), SUMO (right)

5.1.3 State Space

The state space was designed to include key measures of the main road, along with their variances (reflecting deviation around the mean to ensure fairness in later evaluations). It was also intended to be easily measurable during deployment. The following features were proposed:

Table 0.1 State Space Defination

Measure	Defination	Way Calculated
avg_speed (float)	The mean speed of all vehicles observed across controlled lanes.	The average speed is calculated by taking the average of all vehicle speeds within the monitored region during a given time step. Additionally, speed variance is computed to assess the variability of traffic flow.
var_speed (float)	The variance in vehicle speeds observed across controlled lanes.	-
avg_waiting_time (float)	The mean waiting time accumulated by vehicles across controlled lanes.	For each vehicle, the accumulated waiting time is tracked, ensuring that time spent across multiple lanes is not double counted. The average waiting time and its variance are then computed across all monitored lanes.
var_waiting_time (float)	The variance in the waiting time across vehicles in the controlled lanes.	-
avg_throughput (float)	The average number of vehicles passing through a lane per time step.	It is computed as the number of vehicles crossing the observed segments within a time step, averaged across all monitored sections.
avg_queue_length (float)	The average number of vehicles halted or waited in the queue across controlled lanes.	The queue length for each lane is determined by counting the number of halted vehicles during a time step. The overall value is expressed as the average queue length across all controlled lanes.
avg_occupancy (float)	The average percentage of lane occupancy represents how much of the lane is occupied by vehicles.	It is calculated as the percentage of lane or segment space occupied by vehicles during a specific observation period and averaged across all monitored segments.

In addition, as part of benchmarking, it was considered that other methods could be called from the connection interface to retrieve further necessary information. This approach is consistent with the logic used in [3], where the reward function differs from the state space.

The state space was designed to capture main road measures that affect the model and was also made easily integrable into the real world.

The measures were averaged from the camera for each controlled lane to maintain a static state space size.

5.1.4 Action Space

As described in the problem definition, each environment level was assigned its own action space.

For each lane, the action was defined as either green or red. To simulate a more realistic scenario, a duration parameter was also introduced into the environment. This parameter consisted of a list of possible durations, from which the algorithm was allowed to select during training.

These action-duration combinations were then encoded as a list and structured as a space to be used later for analysis. The Gym MultiDiscrete space was not used for actions, as some of the intended training libraries do not support it.

The final environment configurations (levels of action space/ environment) will be explained in next sections.

5.1.4.1 HighGroupedSumoEnv (HighGrouped)

$$A = \{(s,d) | s \in \{"r" * n, "g" * n\}, d \in D\}, D = \{d_1, d_2, \dots, d_k\}$$

Equation 0.1 Action space for HighGrouped Sumoenv

- Where n is the number of lanes
- D is the set of possible durations
- Each action is a pair: signal string s and duration d
- g,r : traffic signal green, red

5.1.4.2 GroupedSumoEnv (Grouped)

$$A = \{(s,d) | s \in \{r,g\}^4, d \in D\}, D = \{d_1, d_2, \dots, d_k\}$$

Equation 0.2 Action space for Grouped Sumoenv

- D is the set of possible durations
- Each action is a pair: signal string s and duration d
- g,r : traffic signal green, red

5.1.4.3 Full (SumoEnv)

$$A = \{(s,d) | s \in \{r,g\}^n, d \in D\}, D = \{d_1, d_2, \dots, d_k\}$$

Equation 0.3 Action space for Full Sumoenv

- D is the set of possible durations
- Each action is a pair: signal string s and duration d
- g,r : traffic signal green, red

Each action was then encoded as an integer index within a Discrete Gym space of size |A|, and a mapping was maintained from each index to its corresponding (signal, duration) tuple

To accelerate training in SUMO, the greatest common divisor (GCD) of the given duration array was computed and used as the SUMO step interval. The total number of steps was then calculated by dividing the original durations by this step size.

5.1.5 Reward

As will be discussed in the [5.4 Experiment Design] section, the proposed reward will be used, along with literature-based rewards in the experimental design. A comparison will be made between our work and the reference project's reward. Below is the definition of the rewards.

5.1.5.1 Proposed Reward

$$R = S_s \left(\log \left(1 + \frac{\underline{v}}{\sigma_v^2 + \eta} \right) \right) - S_w \left(\frac{1}{1 + \underline{w} + \sigma_w^2} \right) + S_e \left(w_1 \underline{t} - w_2 \underline{q} \right)$$

Equation 0.4 Proposed Reward for Version 1.0.0

Where:

$\underline{v}$ is average speed, σ_v^2 is variance of speed, $\underline{w}$ is the average waiting time, σ_w^2 is variance of waiting time, $\underline{t}$ is average of throughput, $\underline{q}$ is average of queue length, η is small number to prevent division by zero(1e-6).

S_s, S_w, S_e, w_1, w_2 : is weight for terms, configured when testing the reward ($w_1 = .6, w_2 = .4, S_s = 0.233, S_w = 0.667, S_e = 0.1$), $(w_1 \underline{t} + w_2 \underline{q})$ is efficiency term

The proposed reward is most suitable for a state space that considers all state factors. The main objective is to maximize speed, minimize waiting time, and maximize throughput while minimizing queue length, using a weighted sum approach. Variance is incorporated to ensure

fairness. Additionally, a logarithmic function is used to prevent excessively large rewards (with a maximum reward of 5 for most experiments).

5.1.5.1.1 Proposed Reward Testing

1. Mathematical Concept Testing

In this phase, the function was assessed to ensure appropriate behavior under different input values.

For example,

- Small values corresponding to a poor traffic state (1,1,20,20,0,20,1) produced a reward of -0.639 , whereas a better state (15,1,0.5,0.5,30,1,0.1) resulted in a higher reward of 1.87.
- For an average case such as (10,1,0,1,0,0,0), the output was found to be acceptable -0.107 , confirming that high speed contributes positively but is outweighed by the waiting time penalty.
- When all state values were zero (0,0,0,0,0,0,0), the reward was -0.6 . In cases with mixed values such as (10,0,5,0,10,2,0), no error occurred, but a relatively high value (3.6) was obtained, which is unrealistic under normal conditions.

2. Benchmark Testing

The second type of evaluation involved benchmarking the proposed reward function against established literature-based reward functions within a simulated environment. This was conducted through experimental comparisons to assess performance under realistic traffic scenarios [5.5.3 Experiment Design].

5.1.5.2 Reference Project Reward

$$R(s,a,a') = (\text{occupancy}/\text{haltingNumber}) - \text{hamming}(a,a') - \text{emergencyStops}$$

Equation 0.5 Reference Reward

The reward used here achieved the best results in the reference project. The second reward function is designed to minimize sudden actions that cause rapid fluctuations by calculating the Hamming distance between consecutive actions. This approach aims to reduce unnecessary fluctuations. Additionally, the model is penalized for emergency stops. The reward function is defined as follows:

The first term in the equation primarily focuses on rewarding the model for high traffic flows. Specifically, the higher the occupancy and the lower the halting number, the greater the reward the model receives.

5.1.5.3 Literature Reward

[3]

$$reward = \frac{1}{n_{wt}} \sum_{n=1}^{n_{wt}} \lambda \left[1 - \left(\frac{wt_n}{T_{wt}} \right)^\alpha \right]$$

Equation 0.6 Literature Reward

where n_{wt} denotes the number of vehicles waiting at the intersection, wt_n represents the waiting time of vehicle n , T_{wt} is the acceptable waiting time, and λ and α are constants. T_{wt} is set to 20, λ is set to 0.15, and α is set to 2.

The objective of the reward is to minimize the average vehicle delay while reducing energy consumption and enhancing junction traffic flow. The reward increases as the waiting period decreases. At the same time, fairness is ensured to prevent a small number of vehicles from experiencing excessively long waiting times.

The approach focuses on minimizing the average waiting time for all vehicles, divided by T_{wt} to enable comparison with the acceptable waiting time (indicating that shorter waiting times for all vehicles are desirable). Additionally, the parameters λ and α are used to increase the significance of the reward, making it more impactful.

Sensitivity Analysis

- **Inverse Relationship to Waiting Time:** The reward provides a clear negative gradient with respect to wt . The agent is directly incentivized to reduce the waiting time for all tasks.
- **Impact of Exponent α :** The exponent α dictates the **shape** of the penalty curve (the reward landscape) as a function of $\frac{wt_n}{T_{wt}}$.
- **Target Time Flexibility T_{wt} :** Increasing T_{wt} *always* increases the reward. This is logical: relaxing the target makes the current performance look better relative to the target.
- **Overall Scaling (λ):** λ controls the magnitude of the reward signal, but not the relative priorities or the direction of the optimization. Increasing λ simply makes the system respond more strongly (positively or negatively) to the current state.

5.1.6 Preprocessing

Since reinforcement learning is heavily influenced by normalization [40], the following methods were used:

1. **Normalization was performed using:**
 - **VecNormalize** (for Stable-Baselines PPO): Automatically normalizes observations and returns to stabilize learning.
 - **NormalizeObservation** (for D3QN using Gym): Scales state observations to a standard range for more stable convergence.
2. **The environment was wrapped using DummyVecEnv:**
Allows vectorized environments to be used by algorithms that expect batched inputs, even when using a single environment instance.

5.1.7 Model Used

- **DQN** was selected, as it was used in the benchmark project [1], to allow for result comparison see [Section 5.4.1.2 Benchmark], Using Package StableBaseline3 [41].
- **D3QN** was chosen, as it achieved better results in [19], [30]. algorithm code is inspired by [9]
- **PPO** was employed as most reinforcement learning algorithms easily overfit to one environment and become unstable in new ones, so PPO solves this by controlling how much the policy can change in each update, and was also utilized in [2], Using Package StableBaseline3 [41].

5.1.8 Traffic scale

To measure traffic under different scenarios, experiments were conducted at two scales: Normal and Crowded [section 5.4.1 Parameters Definition of Experiments].

In SUMO, the scale is defined using an integer value; however, for this project, it needed to be estimated from real-world data. In the reference project [1], large scale values such as 25 or 50 were used, which are suitable for extensive areas. However, since this project focuses on a single intersection, such large values were deemed unnecessary [6].

As mentioned in [42], it can be estimated that the congestion level in Alexandria is approximately 14% under normal conditions and around 38% during crowded scenarios.

Based on this, scale values of 1 ($14\% \times 10$) and 3 ($38\% \times 10$) were selected for the Normal and Crowded scenarios, respectively [Version 1.0.0].

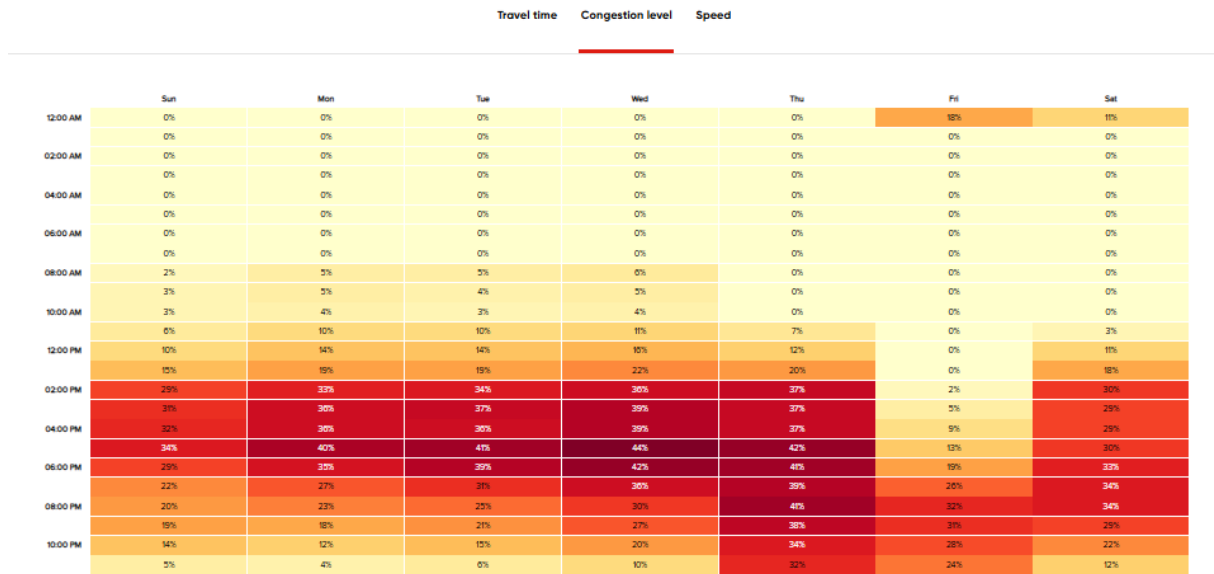


Figure 0.4 Congestion level in alexandria according to TomTom

5.2 Hyperparameter Selection and Tuning

For this project, the parameters were selected with the goal of achieving the highest possible performance. These parameters are categorized as follows:

- Algorithm Hyperparameters: These were determined through a hyperparameter optimization process.
- Benchmark Parameters: These were adopted as-is to ensure fair and consistent comparison with existing results [1].
- Experimental Parameters: These were used to compare the performance of different algorithms and to select the one that performed best.
- Sumo Parameters: Parameters that sumo itself used to control traffic, we tuned it in version 2.0.1 to best simulate real traffic
- End User Parameters: These are main parameters for using our environment.

Further details are provided in the following sections.

5.2.1 PPO and D3QN Hyperparameters

For tuning the PPO and D3QN algorithms, Optuna was employed to automatically search for optimal configurations.

- For D3QN, the following parameters were tuned:
learning_rate, gamma, tau, l2_reg, epsilon_decay, and batch_size.

Table 0.2 D3QN hyperparameters

Parameter	Meaning	Range
learning_rate	Step size for updating network weights	1e-6 to 1e-3
gamma	Discount factor for future rewards	0.9 to 0.9999
tau	Soft update rate for target network	0.8 to 1.0
l2_reg	L2 regularization factor to prevent overfitting	0.001 to 0.01
epsilon_decay	Rate at which exploration decreases	0.0001 to 0.4
batch_size	Number of samples per training batch	[32, 64, 128, 256, 512]

- For PPO, the following parameters were tuned:
learning_rate, gamma, gae_lambda, ent_coef, clip_range, batch_size, and net_arch.

Table 0.3 PPO hyperparameters

Parameter	Meaning	Range
learning_rate	Step size for updating network weights	1e-6 to 1e-3
gamma	Discount factor for future rewards	0.9 to 0.9999
gae_lambda	Smoothing factor for Generalized Advantage Estimation	0.8 to 1.0
ent_coef	Entropy coefficient for exploration	0.0 to 0.1
clip_range	Range for clipping policy updates to prevent large steps	0.1 to 0.4
batch_size	Number of samples per training batch	[32, 64, 128, 256, 512]
net_arch	Number of neurons in hidden layers of the neural network	[32, 64, 128, 256, 512]

- The parameter ranges were estimated, as no publicly available source explicitly defined where these values should be drawn from.
- The objective function was designed to maximize the derivative of cumulative reward (from last to first episode) to ensure efficient learning.
- Three different random seeds were tested for each configuration, and the average derivative was used as the optimization target in Optuna to ensure robustness.
- Each trial was run for 10 episodes, using 3 seeds and 20 trials, resulting in a total of $10 \times 20 \times 3 = 600$ episodes.
The number of trials (20) was selected as a middle ground to balance between computational cost and result quality.
Durations tested: [15, 30, 60, 90] seconds, with a maximum of 500 SUMO steps per episode[Version 1.0.0].
- Optuna tuning was repeated for each level (in the proposed experiments), for each of the two reward types, and for each algorithm, resulting in: $2 \text{ reward types} \times 2 \text{ levels} \times 2 \text{ algorithms} = 8$ hyperparameter sets. These are considered critical, as the action space, reward function, and algorithm significantly influence the results.
- Training was conducted using Dataset 2, and the best parameters were validated on Dataset 3.
Final configurations were saved in a config file.

5.2.2 Parameters from the Benchmark

The same hyperparameters used in the benchmark project were adopted to ensure a fair comparison. Minor modifications were made to adapt them to the new environment, as detailed in [Section 5.3 Benchmark Configuration].

5.2.3 Experiment-Specific Parameters:

Additional parameters were varied during experiments to evaluate the system's performance under different conditions, see [section 5.4.3 Experiment design]. These were used to analyze environmental behavior, gain insights, and identify the best setup for deployment.

5.2.4 Sumo Parameters

Routes (RandomTrips) file hyperparameters

[43]

Table 0.4 randomTrips hyperparameters

Parameter	Definition	Chosen Value for V 1.0.0/ 2.0.0	Chosen Value for V 2.0.1	Reason for Choosing the Value for V2.0.1
-n	Map file to use for the simulation	<Map file>	<Map file>	Specifies the network or area for the traffic simulation.
-b	Simulation start time	0 s	0 s	Start from time zero at this file.
-e	Simulation end time	604800 s	604800 s	In routes file, for generalization it's run on 7 days.
--seed	Random seed for the experiment	(Default) 1	2	Allows reproducibility and controlled variation between simulation runs.
--SPEED-EXPONENT	Weight given to faster edges in route calculation	(Default) 1	2	Assigns more importance to faster roads, so vehicles preferentially use higher-speed edges.
--ANGLE-FACTOR	Influences trip alignment with general direction	(Default) 1.0	1.0 (San Stefano), 3.0 (Mosheer)	Higher values favor straighter trips. Lower value in San Stefano allows more indirect routes; higher in Mosheer to favor direct routes in congested areas.
--RANDOM-FACTOR	Adds stochasticity to route selection	(Default) 1	2	Ensures not all vehicles choose the optimal path; value reflects average over multiple seeds.
--min-distance	Minimum distance for a trip to be considered valid	(Default) 0	20 m	Trips shorter than 20 m are ignored, avoiding trivial or unrealistic trips.
--allow-fringe	Allow trips to start/end at fringe edges of the network	(Default) false	true	Enables realistic trips that originate or terminate at network edges.
--fringe-factor	Weight adjustment	(Default) 1	2	Gives lower priority to fringe edges while still allowing their

	for fringe edges			use, focusing trips on the main study area.
--pedestrians	Include pedestrian traffic in simulation	(Default) false	true	Adds realism by modeling pedestrian flows alongside vehicles.
--persontrips	Generate intermodal trips (vehicles + pedestrians)	(Default) false	true	Ensures trips reflect realistic multimodal behavior in the network.
--period / -p	Time between successive trip departures	(Default) 1.0	20 s (San Stefano), 12 s (Mosheer)	Corresponds to 3 vehicles/min in San Stefano and ~5 vehicles/min in Mosheer (Poisson-based estimate from literature); reflects typical and peak traffic conditions [44].

Sumo file hyperparameters

[6]

Table 0.5 sumo cmd hyperparameters

Parameter	Meaning	Chosen Value V1.0/V2.0.0	Chosen Value V2.0.1	Reason for Choosing the Value for V2.0.1
-c	Configuration file for the simulation	<Config file>	<Config file>	Specifies the SUMO configuration to be used.
-b	Simulation start time	0	10800 s (3 hours)	Delayed start allows some existing vehicles to already be in the network, improving realism.
-e	Simulation end time	500s – 1500s – 2000s – 4000s (as explained at sections 5.4.1.1/5.4.1.3.1/5.4.2.2)	13800 s / 10800 s	Set approximately 1 hour after starting for training purposes; longer for

				testing larger time periods.
--step-length	Simulation step length	N	N	Typically modified when creating training factors; adjusted if a greatest common factor (GCF) exists between durations.
--time-to-teleport	Max waiting time before stuck vehicle is teleported	300(default for V1.0.0), -1 (V2.0.0)	1000 s	Chosen logically via experiment; no direct data available, ensures vehicles do not block network indefinitely.
--max-num-teleports	Maximum number of teleports allowed per vehicle	Default (300)	Default (300)	Keeping default allows simulation to continue without interruption; avoids arbitrary limits.
--max-depart-delay	Maximum delay a vehicle can wait at departure	-1	300 s	Chosen logically via experiment represents realistic waiting without data.
--scale	Traffic demand scaling factor	.2 (basic), .38 (high) {which is later multiplied by 10 to be used as integer (1,3)}	1.2 (basic), 1.4 (high)	Scales traffic volume; 1.2 for typical demand, 1.4 for heavy demand. Values reflect ~20–40% increase over baseline

				according to TomTom [42], [45].
--collision.action	Action taken on vehicle collision	(Default) Teleport	Warn	Provides sufficient notification while maintaining realism.
--collision.check-junctions	Enable collision checking at junctions	(Default) True	True	Ensures realistic detection of conflicts at intersections.
--collision.mingap-factor	Minimum gap factor between vehicles	(Default) - 1	0.1	Keeps small distance between vehicles, logical for realistic car spacing (relate to mingap described here [46]).
--pedestrian.striping.mingap-to-vehicle	Minimum gap between pedestrian and vehicle at crossings	0.25 (default)	0.25 (default)	Default value is logical and reflects realistic pedestrian behavior.
--weights.random-factor	Adds randomness to route choice	(Default) 1	1.5	Introduces small stochastic variation; prevents all vehicles from choosing the exact same path.
--threads	Number of threads for simulation	(Default) 1	1	Single thread reduces overhead during training; no additional threads needed for evaluation.

--log	Log file	<FILE>	<FILE>	Stores debug and simulation data for analysis.
--------------	----------	--------	--------	--

5.2.5 End User Parameters

Table 0.6 End User Hyperparameters

Parameter	Meaning	Chosen Value V1.0.0/V2.0.0	Chosen Value V2.0.1	Reason for Choosing the Value V2.0.1
is_gui	Enable or disable the SUMO graphical interface	false	false	GUI disabled for Colab to save resources and allow headless execution.
duration	Episode duration in seconds	15,30,60,90	17, 30, 60, 90	Multiple durations allow testing agent performance over short and long episodes; helps evaluate stability and learning.
yellow_time	Duration of yellow traffic signal	0s (V1.0, V2.0.1 – in training)	30 s	Represents realistic yellow-light duration at intersections.
enable_variation_action	Enable action variation to prevent stuck environment	True	True	Prevents the environment from freezing if traffic signals repeat the same action for many steps.
see_progress_each	Frequency to print progress during simulation	10 steps	10 steps	Allow monitoring progress without overloading the console.
max_steps	Maximum steps per episode	10000	10000	Large value to prevent stuck to infinity.
n_env	Number of parallel environments	1	1	Can be set for parallel training; single or multiple environments depending on resources.

enable_gcd	Enable Green-Coordinate-Dependent logic (if used)	True	True (as future work bug, so used durations without common factor)	Determines whether traffic signals coordinate based on global cycle; enables realistic signal coordination.
-------------------	---	------	--	---

In addition to those, Sumo Parameters, described in section 5.2.4 can also be modified from SumoConnection before running any experiment.

5.3 Benchmark Configuration

As described in [section 3.4 The Need to Extend Related Work], few projects are available for Egypt. The only available opensource project was [1], which has issues in fairly applying the model to the area. These issues include:

- Working on a subset of agents while obtaining results for the entire area.
- Using a for loop that may not capture dependencies between agents or facilitate effective learning from experiences [section 5.1.1 Problem and Environment Design].

However, this was the only available resource to compare Egypt's performance and using it for comparison with environments designed differently might not be ideal. Therefore, the following modifications were made to the code to ensure it aligns with the requirements of our project and allows for a fair comparison:

- The project was modified to work with a single agent (e.g., by limiting the loop to the target agent).
- The constructor was modified to accept a seed parameter.
- The code was updated to accept action spaces for SumoEnv, GroupedSumoEnv, and HighGroupedSumoEnv.
- The getSensor method implementation was slightly modified to ensure compatibility with our environment.

In addition to these modifications for our environment:

- The same algorithm hyperparameters from the previous project were used.
- This included defining (epsilon DQN) EpsDQN and (Root Mean Square Propagation) RMS_DQNPolicy, where:
 - EpsDQN inherited from Stable-Baselines3's DQN to enable epsilon-greedy with the same minimum, maximum, and decay rate as in the previous project.
 - RMS_DQNPolicy inherited from Stable-Baselines3's DQNPolicy to use RMSProp as the optimizer, as in the previous implementation.
- The model was run for one episode with a duration of 1 (i.e., 1 second per step).

5.4 Experiment Design

The model will be trained on different cases to study its behavior under varying reward structures, action spaces, and other conditions. The configuration that performs best across these scenarios will be selected. Details of the conducted experiments are provided in the following sections.

5.4.1 Version 1.0.0

In this section first version of project is detailed.

5.4.1.1 Parameters Definition of Experiments

5.4.1.1.1 SUMO Static light

Table 0.7 SUMO experiment parameter definition

Parameter	Meaning	Real meaning	Range for experiment
Data	The dataset or environment used	Country-specific traffic data	Area 1 (Tomart)-Area2 (Mosheer Ismail)
Max sumo steps	Maximum number of simulation steps in SUMO	Represents the simulation time span—longer steps simulate longer real-world hours	(500-4000)
Traffic Scale	Traffic load intensity	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowded (.38)

5.4.1.1.2 Benchmark

Table 0.8 Benchmark experiment parameter definition

Parameter	Meaning	Real meaning	Range for experiment
Data	The dataset or environment used	Country-specific traffic data	Area 1 (Tomart)-Area2 (Mosheer Ismail)
Action space	The scope of available agent actions	Reflects how detailed or flexible traffic signal	Full (SumoEnv)-GroupedSumoEnv

		decisions are in the real world	HighGroupedSumoEnv
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	Reference Project(project)
Max sumo steps	Maximum number of simulation steps in SUMO	Represents the simulation time span—longer steps simulate longer real-world hours	(500-4000)
Run	Experiment repetition count	The number of times the experiment is repeated to compare environments and ensure consistency	Reference Project (Project) -Proposed
Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2
Traffic Scale	Traffic load intensity	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowded (.38)

5.4.1.1.3 Environment Experiments

Table 0.9 Environment experiment parameter definition.

Parameter	Meaning	Real meaning	Range for experiment
Area	The dataset or environment used	Country-specific traffic data	Area2 (Mosheer Ismail) – Area 3 (San Stefano)
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	Literature - Proposed
Environment type	Type of SUMO environment used	Level of traffic control detail and intersection grouping	GroupedSumoEnv HighGroupedSumoEnv

Algorithm	RL algorithm applied	Different learning strategies used to optimize traffic control	D3QN -PPO
Episodes	Number of training episodes	Number of training cycles (longer = more learning)	200,1000 ,10000
Max sumo steps	Maximum number of simulation steps in SUMO	The number of times the experiment is repeated to compare environments and ensure consistency	500-1500 -2000-4000
Traffic Scale	Traffic intensity load	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowding (.38)
Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2,3

5.4.1.2 Metrics

The metrics aimed to be optimized include the reward obtained in the final training episode, the evaluation reward to ensure generalization beyond training, the reward derivative to confirm learning progress, and road-level performance metrics to assess the impact of model deployment. Further details are presented in the following table.

Table 0.10 Metrics definition

Metric	Defination	Direction
Reward of last episode	The total accumulated reward obtained by the agent during the final episode of training	Max
Average Reward for evaluated episodes	The mean of total rewards collected across multiple evaluation episodes.	Max/Equal Reward of last episode
Waiting Time	The average time spent standing (seconds)	Min

Speed	The average trip speed (m/s)	Max
Depart Delay	The average time vehicles had to wait before starting their journeys (seconds)	Min
Time loss	The average time lost due to driving slower than desired (including waiting time) (seconds)	Min
Waiting Car	Number of vehicles with delayed insertion that were still waiting for insertion at simulation end	Min
Means of Experiments	The average of a specific metric (all metrics) across multiple independent experiment runs.	Max
Variance of Experiments	A measure of how spread out the experiment results are. High variance indicates inconsistent results across runs.	Min
Total Time	The total time taken to complete the full training or evaluation process, from start to finish. (Minutes)	-
Derivative of reward	Measures on how reward is changing over episodes — tells if the agent is improving.	Max

5.4.1.3 Experiment Design

After conducting the initial experiments, we will apply experimental design techniques to gain deeper insights. The most suitable approach is the **Full or Fractional Factorial Design**, as it allows us to systematically study multiple factors and their interactions—even when the number of experiments needs to be limited due to time constraints.

This design will help us answer key questions such as:

- What is the effect of the algorithm on average waiting time or other performance metrics?
- How does increasing the number of episodes impact the results?
- What are the mean differences across various experimental setups?

5.4.1.3.1 Conducted Experiments in Version 1

Due to time constraints for the version 1, Main experiment are conducted to get meaningful results.

1. Re-ran initial test experiments from the reference project (22 runs).
2. Executed all SUMO test experiments (8 runs).
3. Ran experiments with the old configuration (without seed), which led to unreliable results (18 runs).
4. Performed full benchmark tests using the final configuration (48 runs).

5. Episode Length Impact
2 datasets $\times$ 2 algorithms $\times$ 3-episode settings = 12 runs
6. SUMO Max Time Impact
2 datasets $\times$ 2 algorithms $\times$ 3-sumo time settings = 12 runs
7. Experiments Using Fixed Settings
Episode = 350
SUMO Time = 2000s ($\approx$ 33.3 minutes)
→ Total runs with these fixed settings = 163
8. Additional Analytical Focus
9. Algorithm Performance Analysis
2 datasets $\times$ 2 algorithms $\times$ 2 scales $\times$ 4 seeds
10. Reward Function Variation
2 datasets $\times$ 2 algorithms $\times$ 2 reward types
11. Traffic Signal Level Testing
2 datasets $\times$ 2 algorithms $\times$ 2 signal levels

Total Experiments Conducted: 259

5.4.1.4 Results of Experiments

In this phase, the primary focus was to establish a benchmark to ensure that the proposed system achieved performance comparable to existing approaches and demonstrated superiority over traditional traffic control methods. Additionally, an initial analysis was conducted to examine the behavioral patterns of the model and to assess the influence of key parameters on the obtained results.

5.4.1.4.1 Results of Benchmark Experiments

5.4.1.4.1. / Plotting for Area 1, Scale Normal

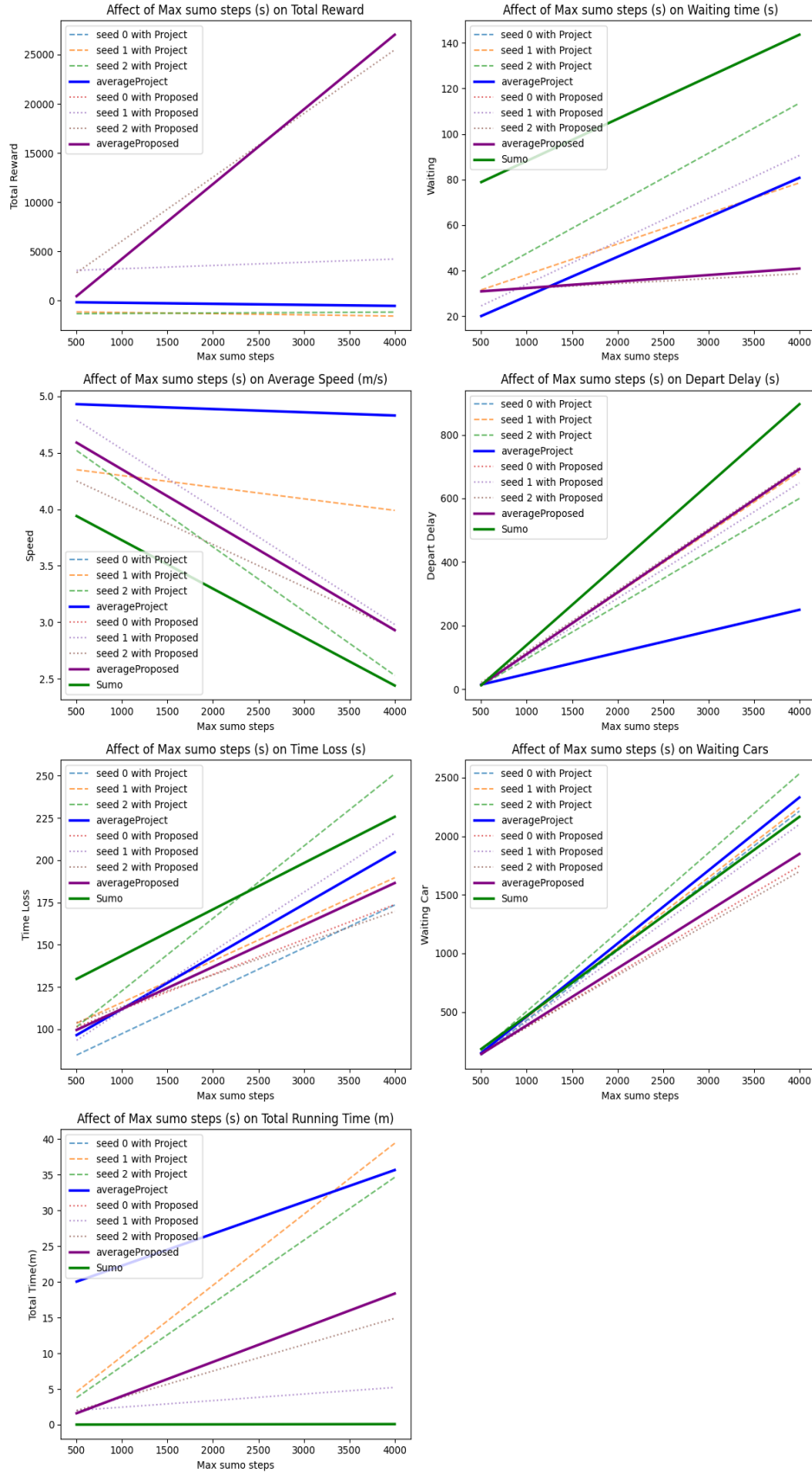


Figure 0.5 Benechmark Plotting for Area 1, Scale Normal

5.4.1.4.1.2 Plotting for Area 1, Scale Crowded

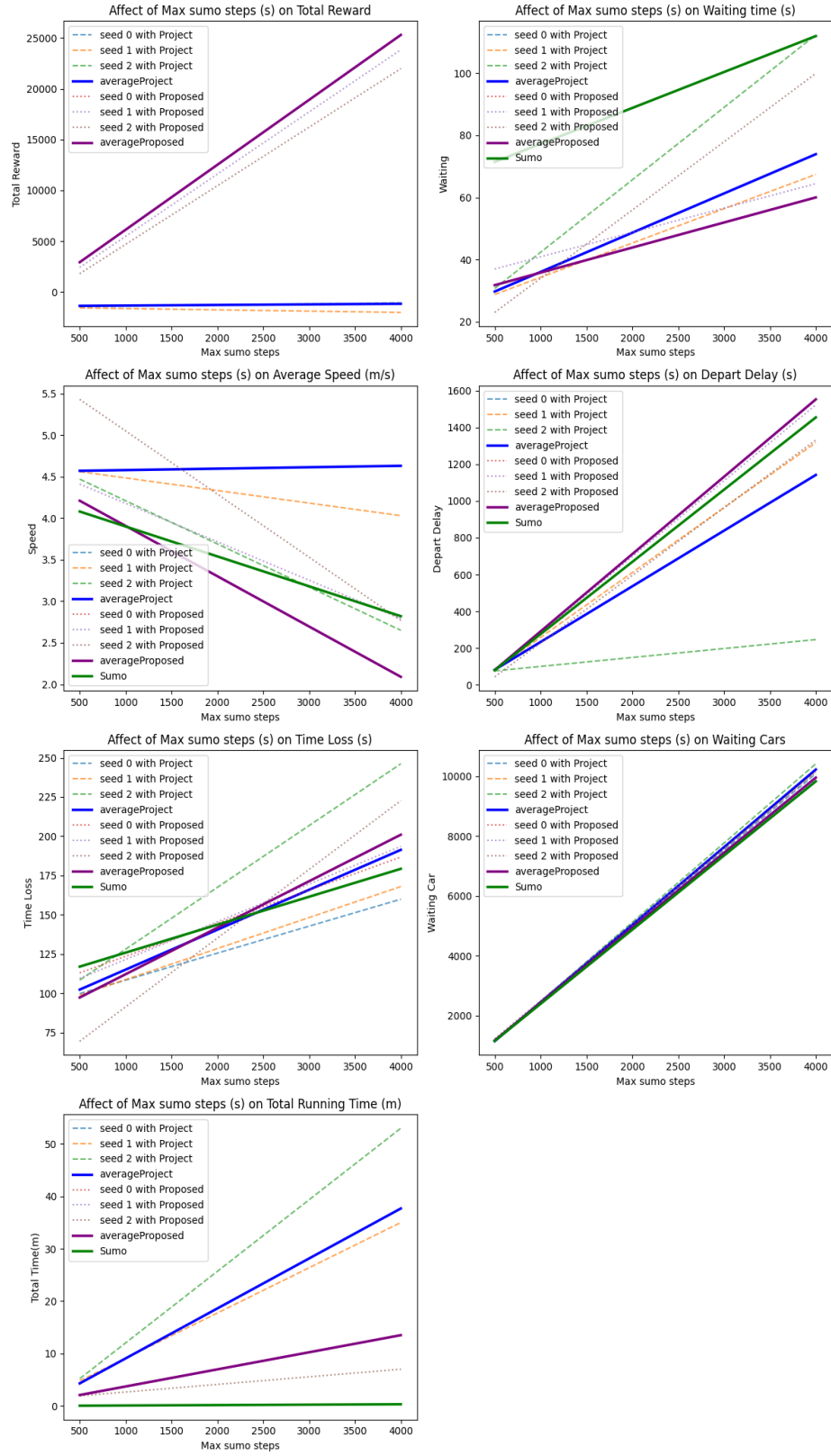


Figure 0.6 Benechmark Plotting for Area 1, Scale Crowded

5.4.1.4.1.3 Plotting for Area 2, Scale Normal

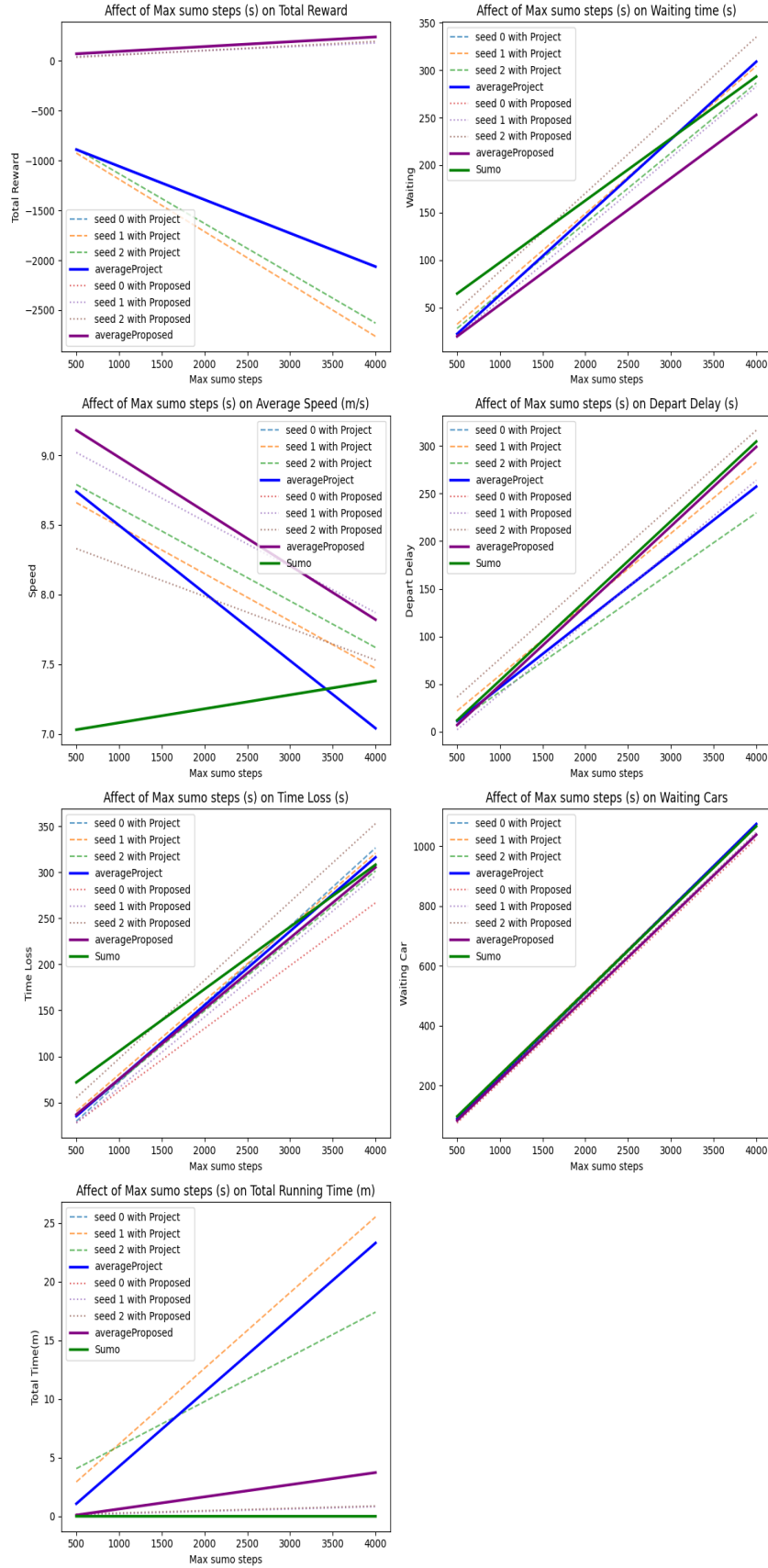


Figure 0.7 Benechmark Plotting for Area 2, Scale Normal

5.4.1.4.1.4 Plotting for Area 2, Scale Crowded

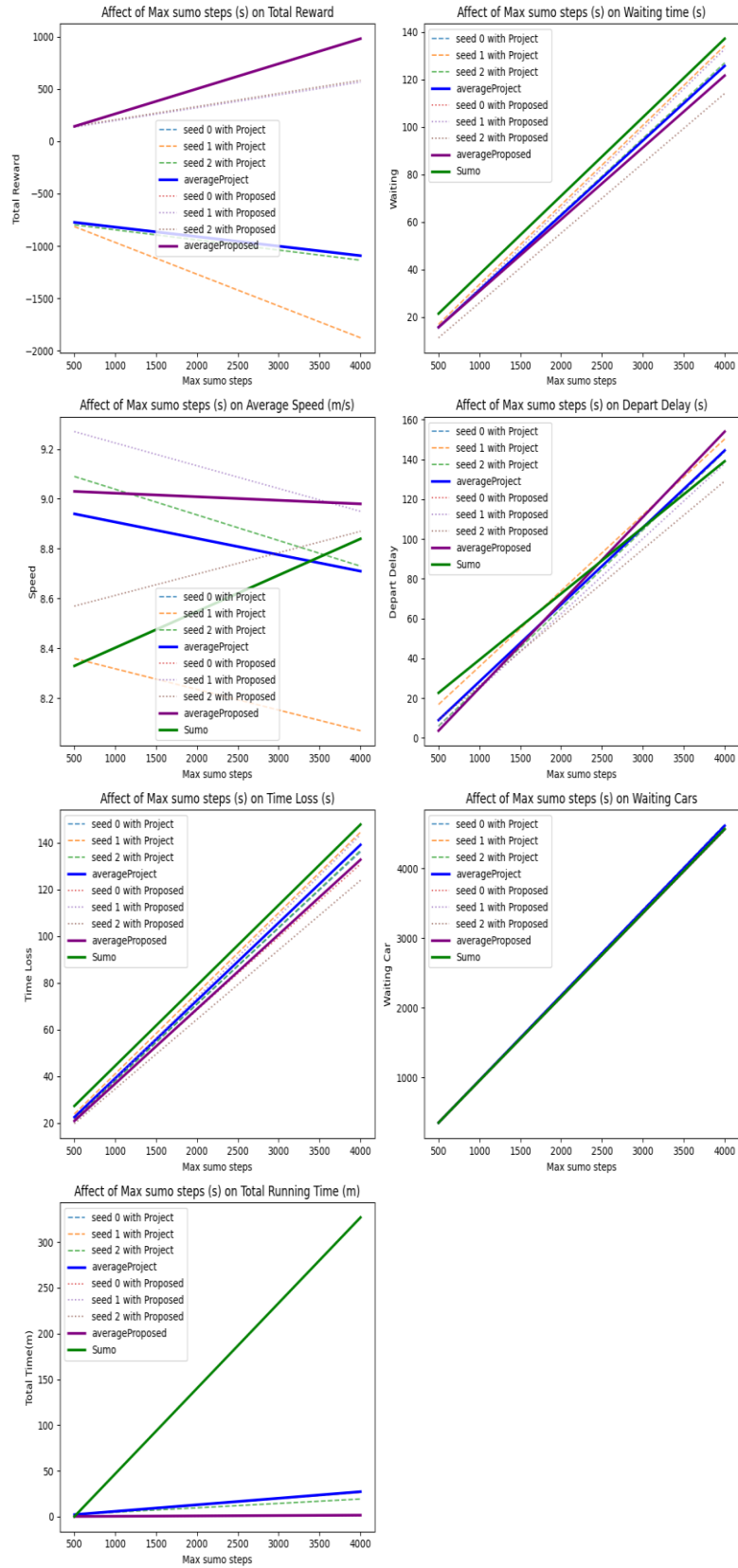


Figure 0.8 Benechmark Plotting for Area 2, Scale Crowded

5.41.4.2 Results of Environment Experiments

5.41..4.2.1 Effect on Episodes

Performance Comparison Across Different number of episodes on Reward of Last Episode

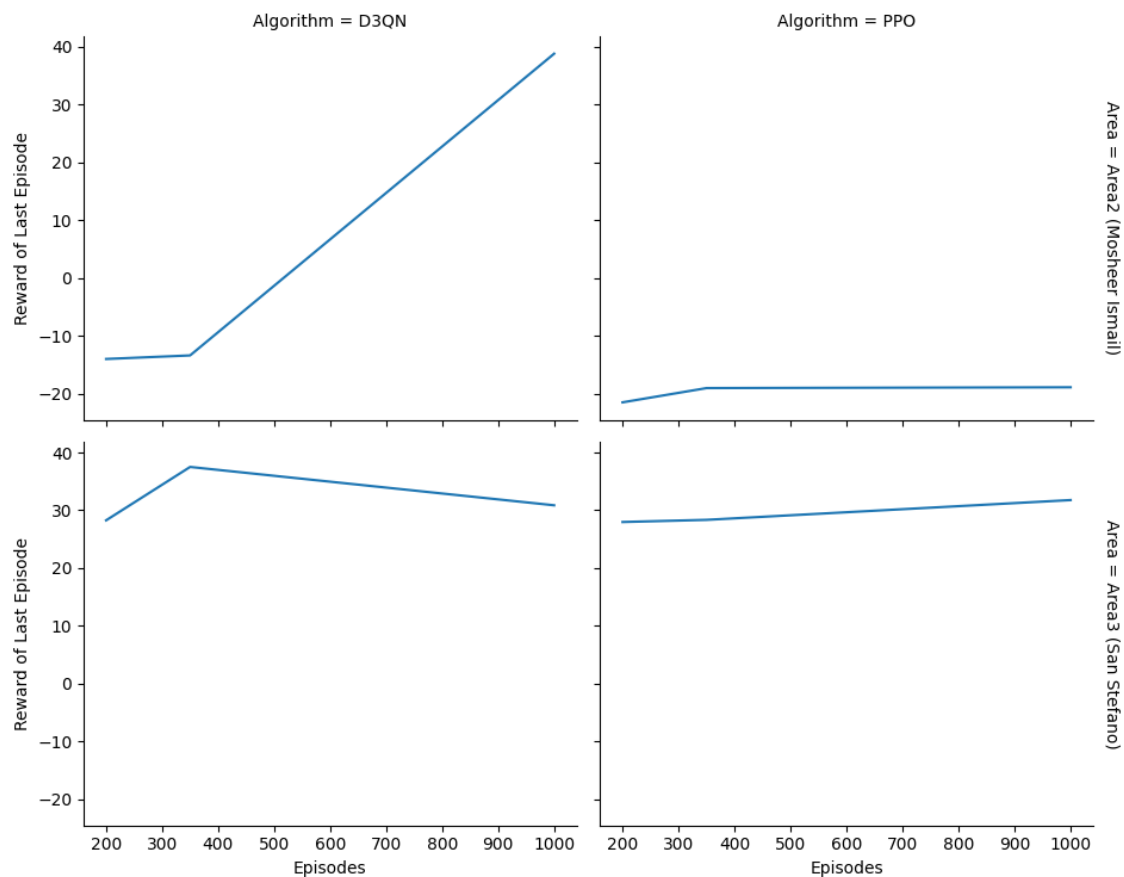


Figure 0.9 Effect of running many episodes on reward

Performance Comparison Across Different number of episodes on Average Reward for Evaluated Episodes

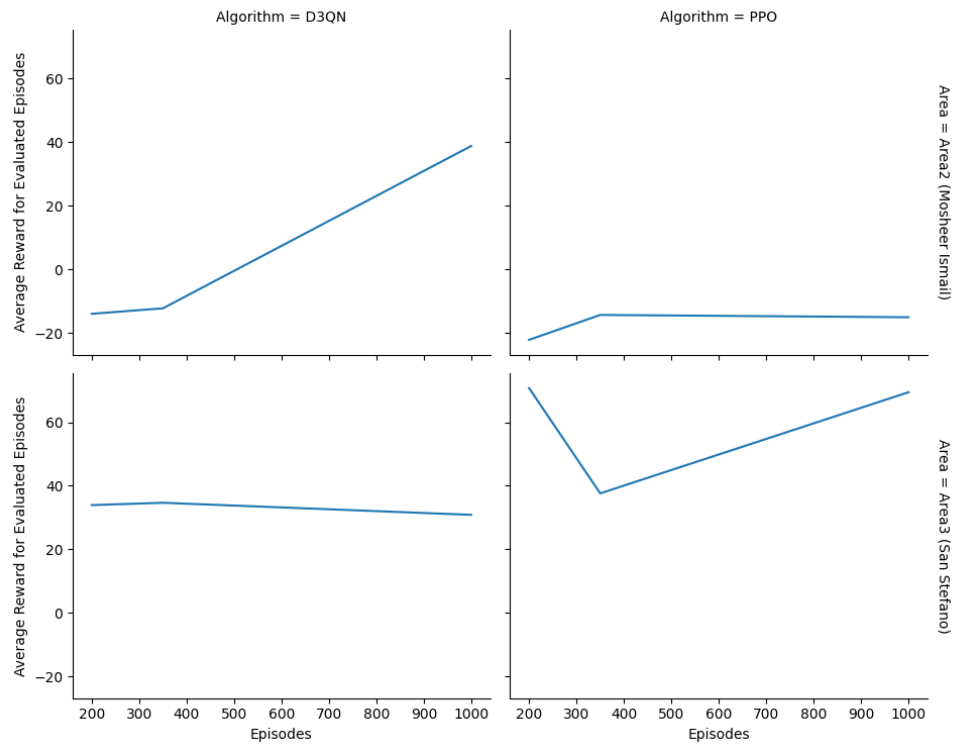


Figure 0.10 Effect of running many episodes on evaluated reward

Performance Comparison Across Different number of episodes on Waiting Time (s)

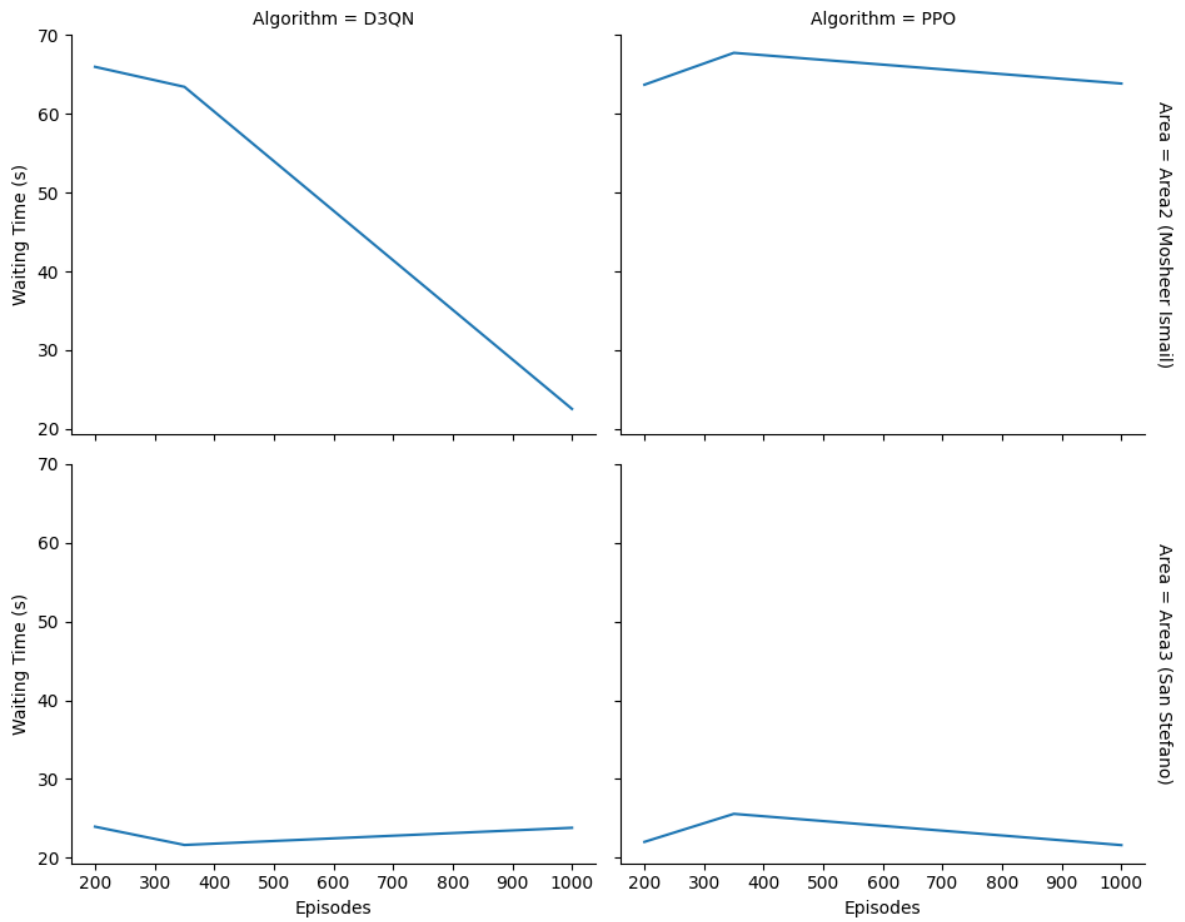


Figure 0.11 Effect of running many episodes on waiting time

Performance Comparison Across Different number of episodes on Speed (m/s)

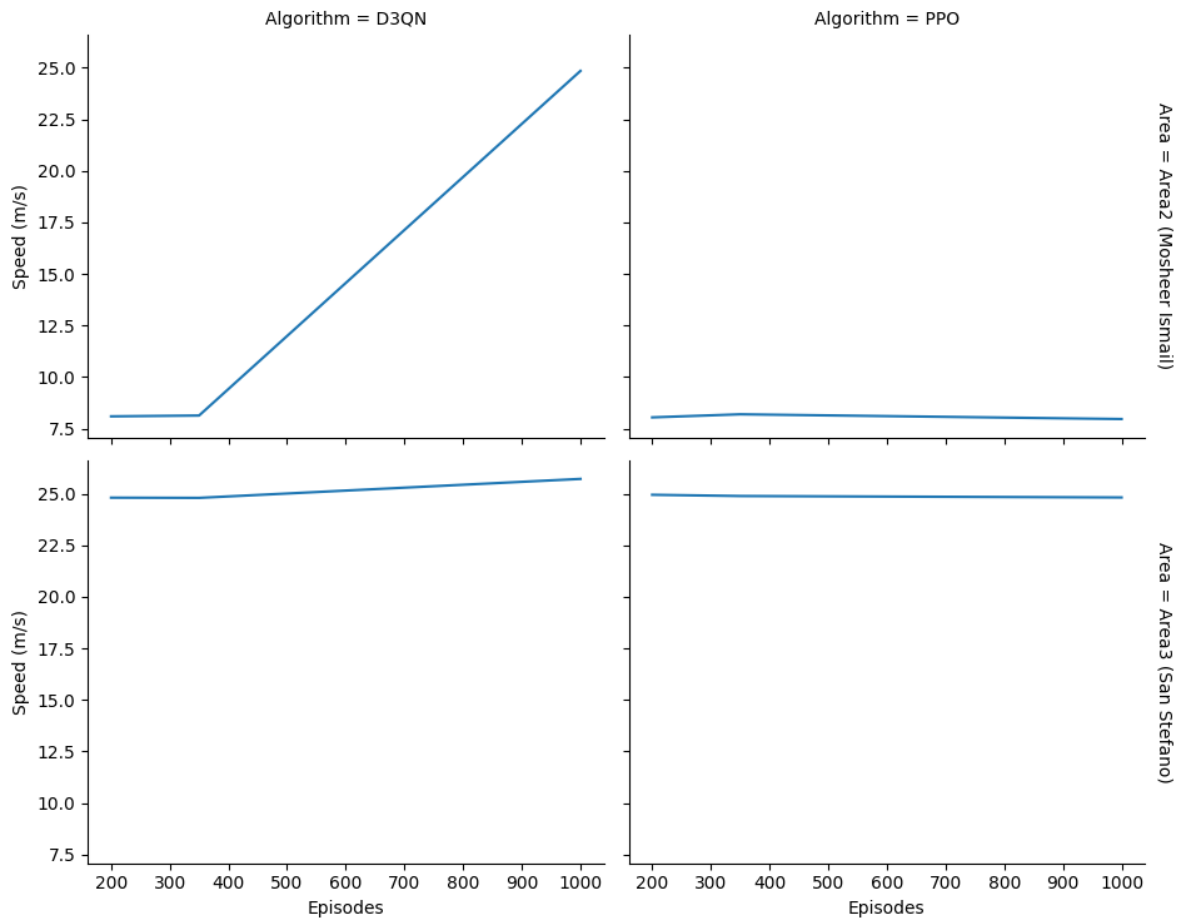


Figure 0.12 Effect of running many episodes on speed

Performance Comparison Across Different number of episodes on Depart Delay (s)

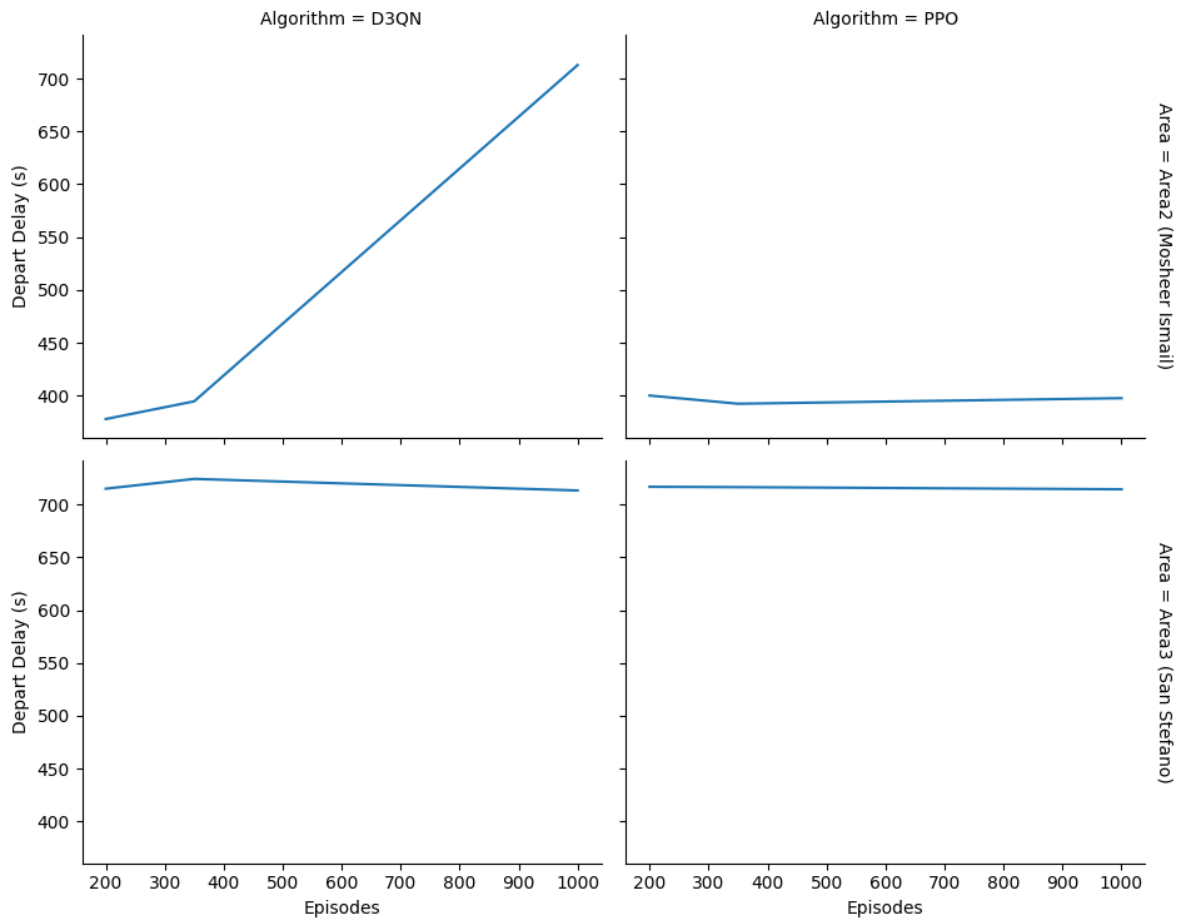


Figure 0.13 Effect of running many episodes on depart delay

Performance Comparison Across Different number of episodes on Time Loss (s)

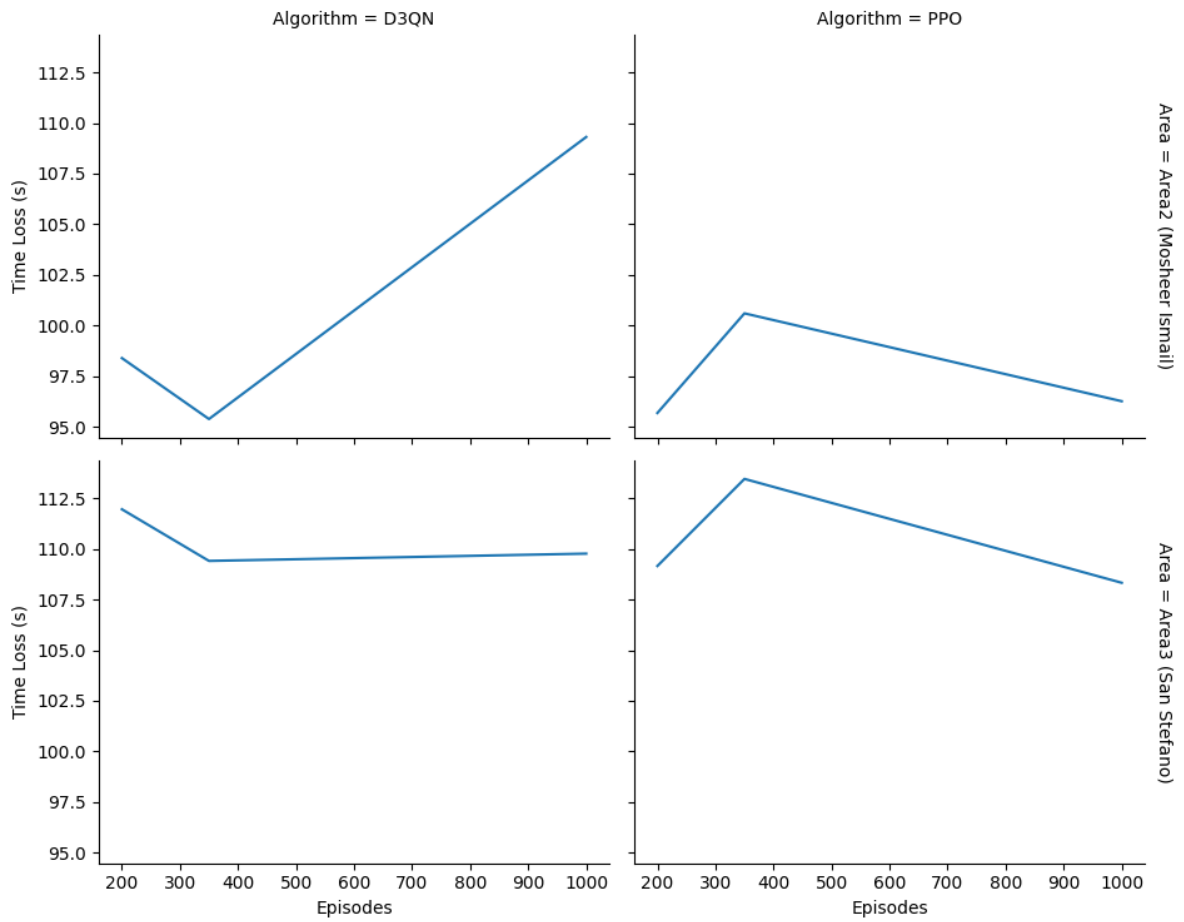


Figure 0.14 Effect of running many episodes on timeloss

Performance Comparison Across Different number of episodes on Waiting Car

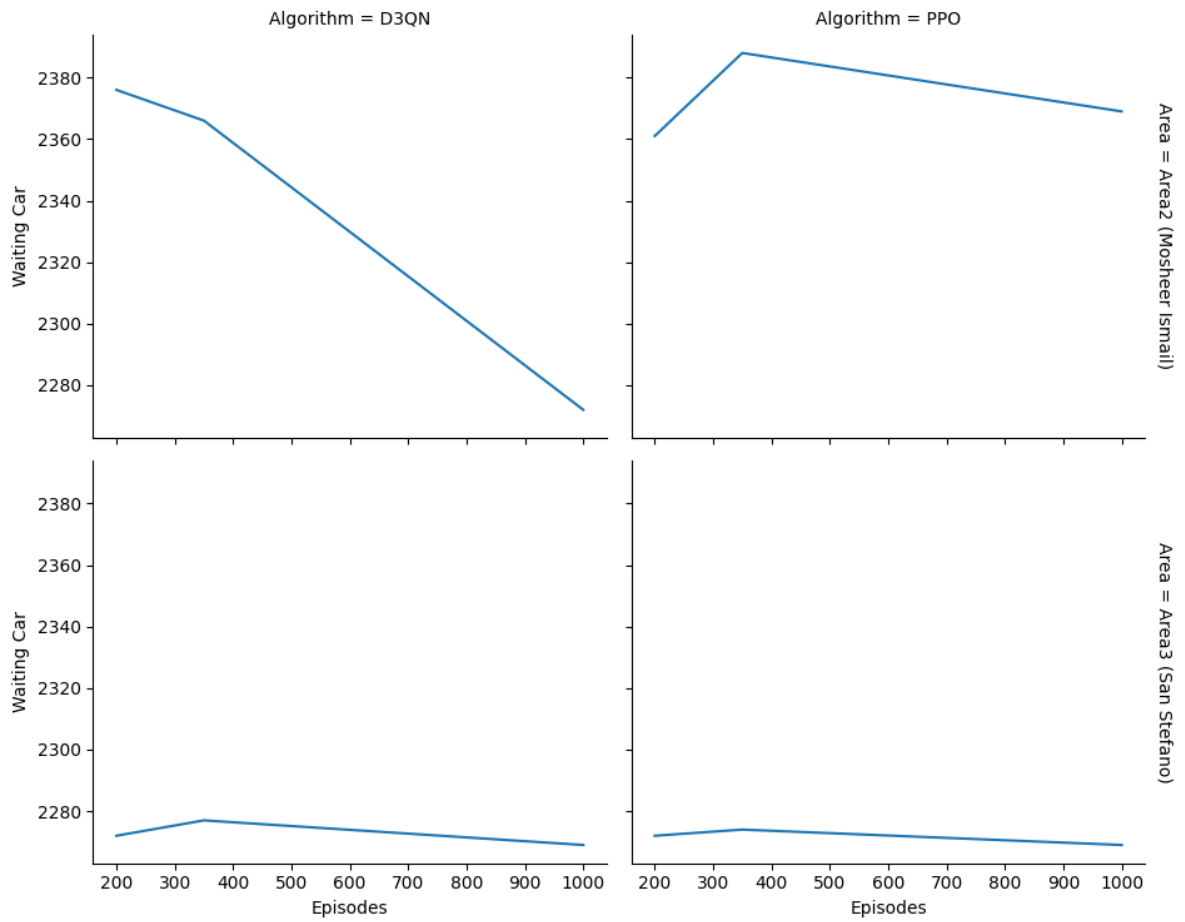


Figure 0.15 Effect of running many episodes on number of waiting cars

5.41..4.2.2 Behavior on different Sumo timing

Performance Comparison Across Different period of running simulation on Reward of Last Episode

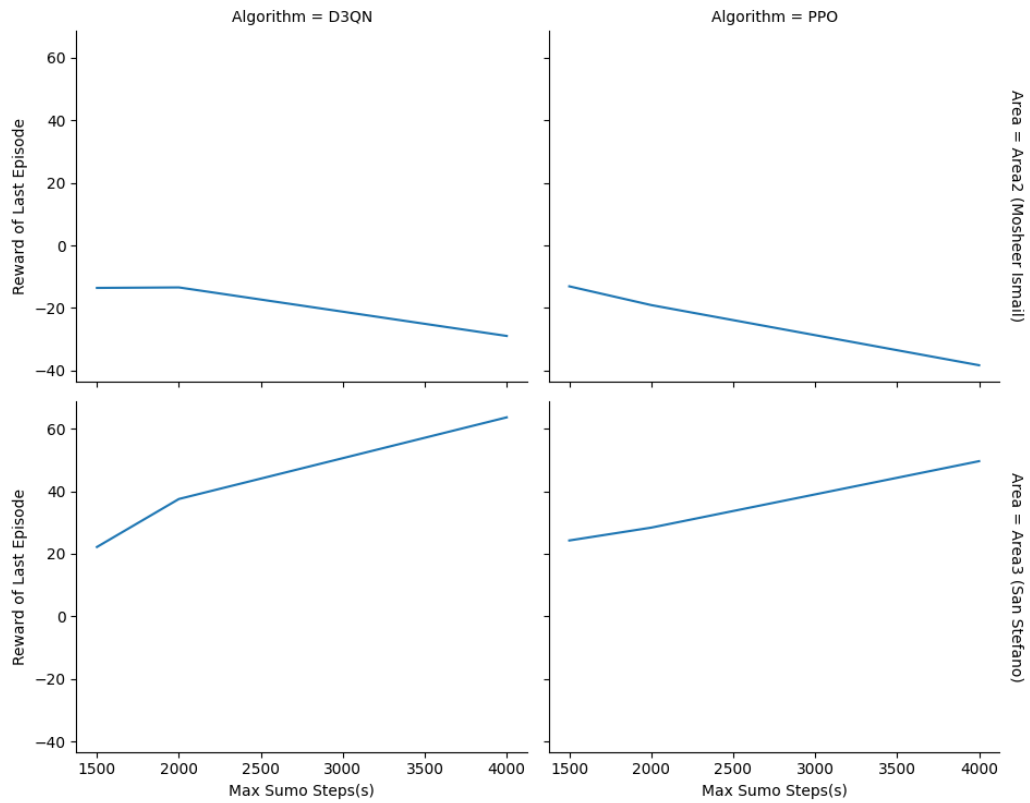


Figure 0.16 Effect of different sumo timing on reward

Performance Comparison Across Different period of running simulation on Average Reward for Evaluated Episodes

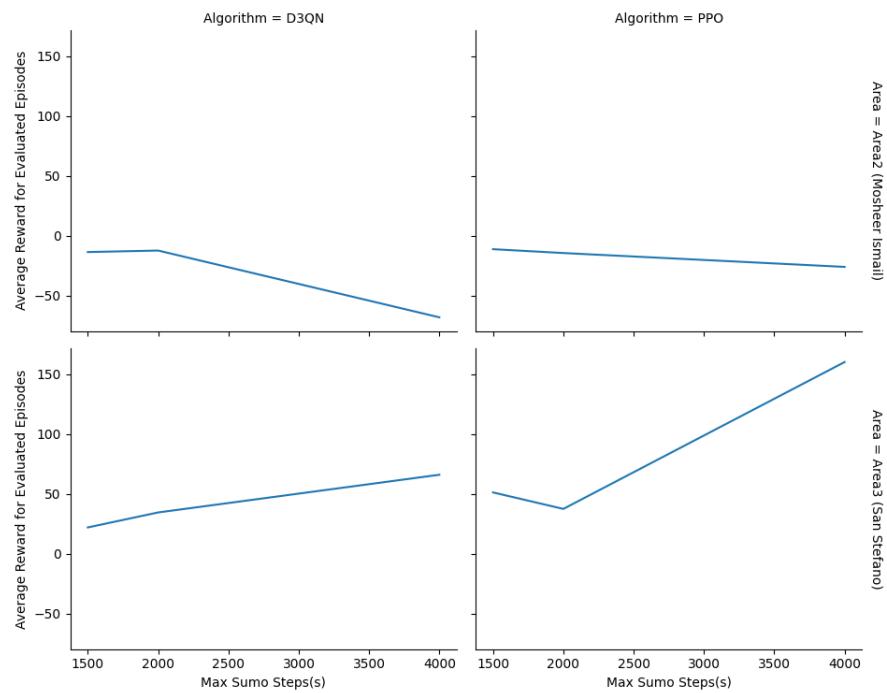


Figure 0.17 Effect of different sumo timing on evaluated reward

Performance Comparison Across Different period of running simulation on Waiting Time (s)

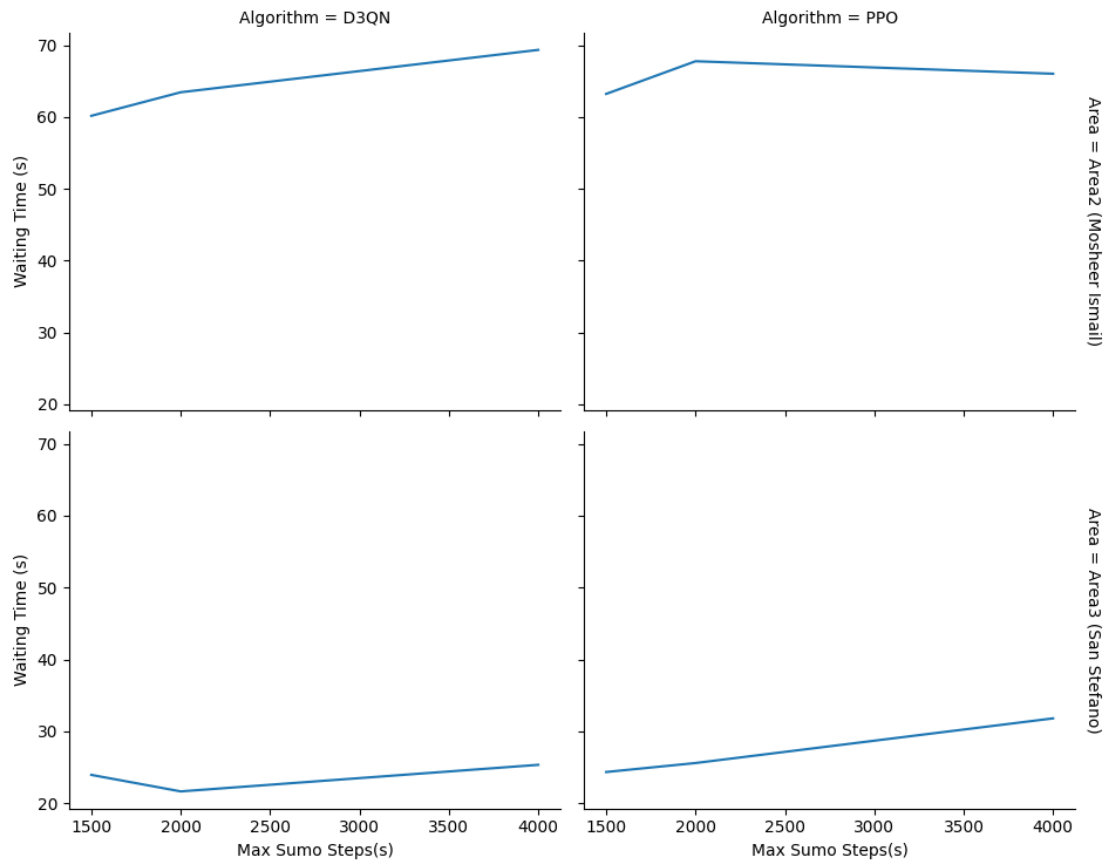


Figure 0.18 Effect of different sumo timing on waiting time

Performance Comparison Across Different period of running simulation on Speed (m/s)

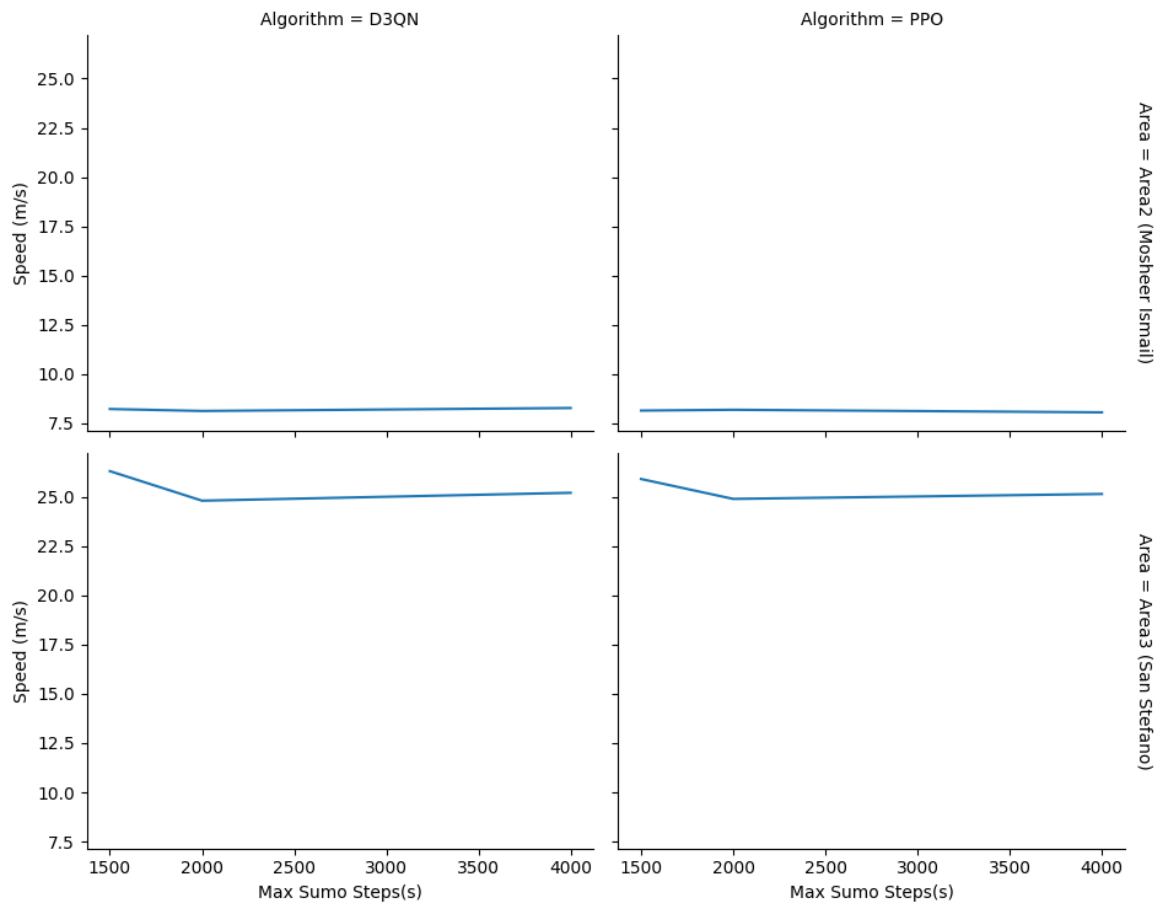


Figure 0.19 Effect of different sumo timing on speed

Performance Comparison Across Different period of running simulation on Depart Delay (s)

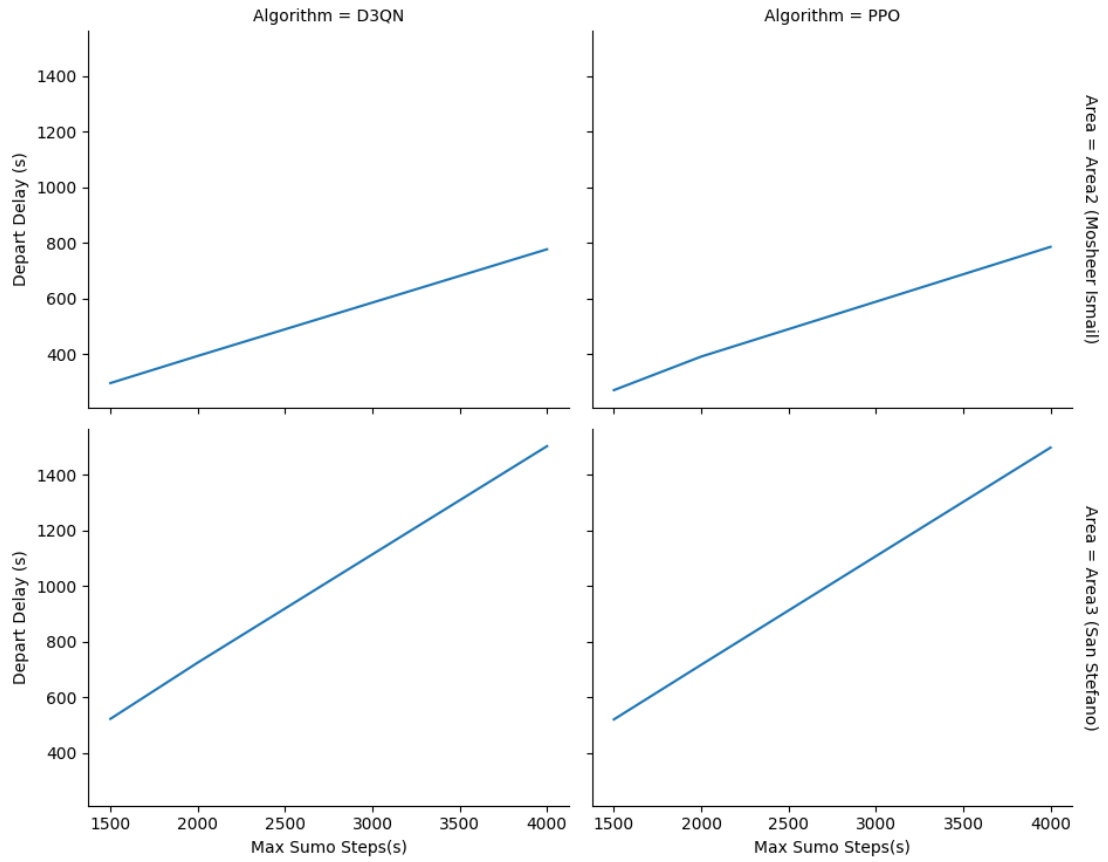


Figure 0.20 Effect of different sumo timing on depart delay

Performance Comparison Across Different period of running simulation on Time Loss (s)

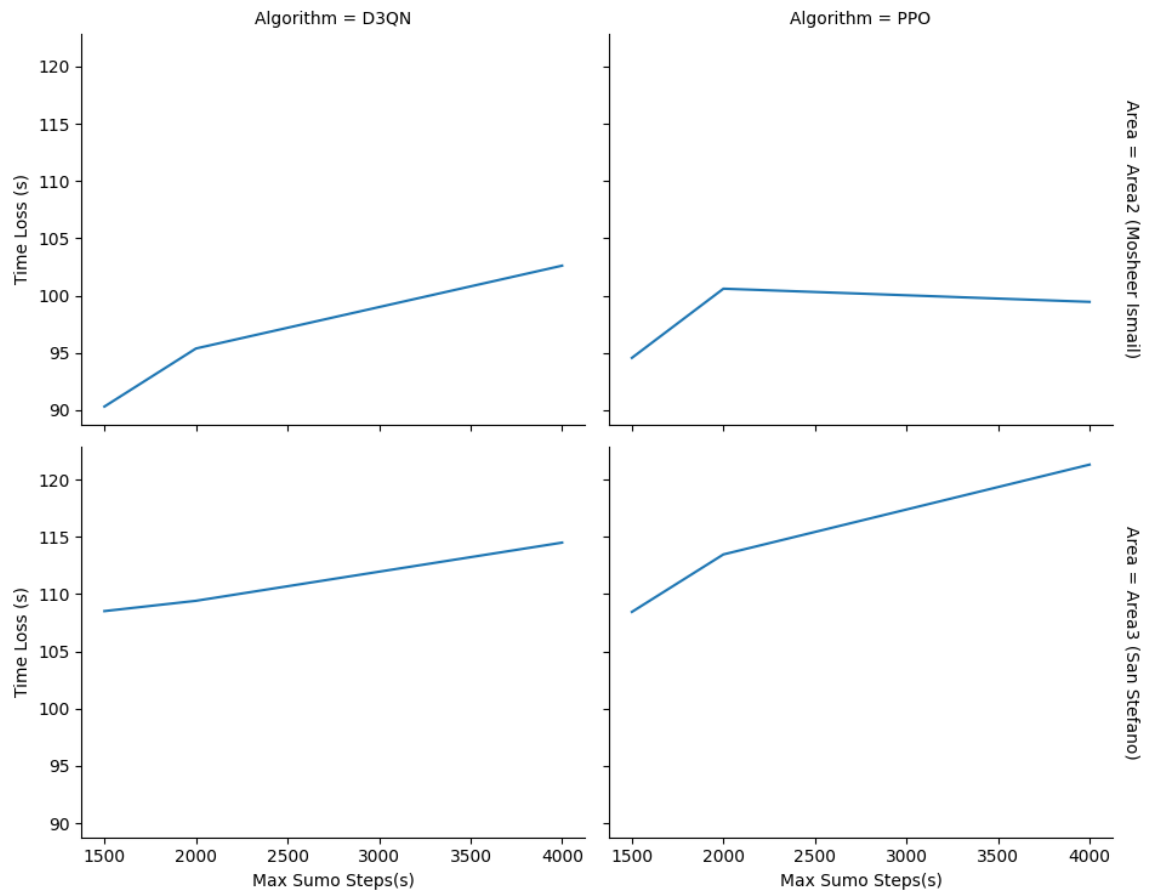


Figure 0.21 Effect of different sumo timing on timeloss

Performance Comparison Across Different period of running simulation on Waiting Car

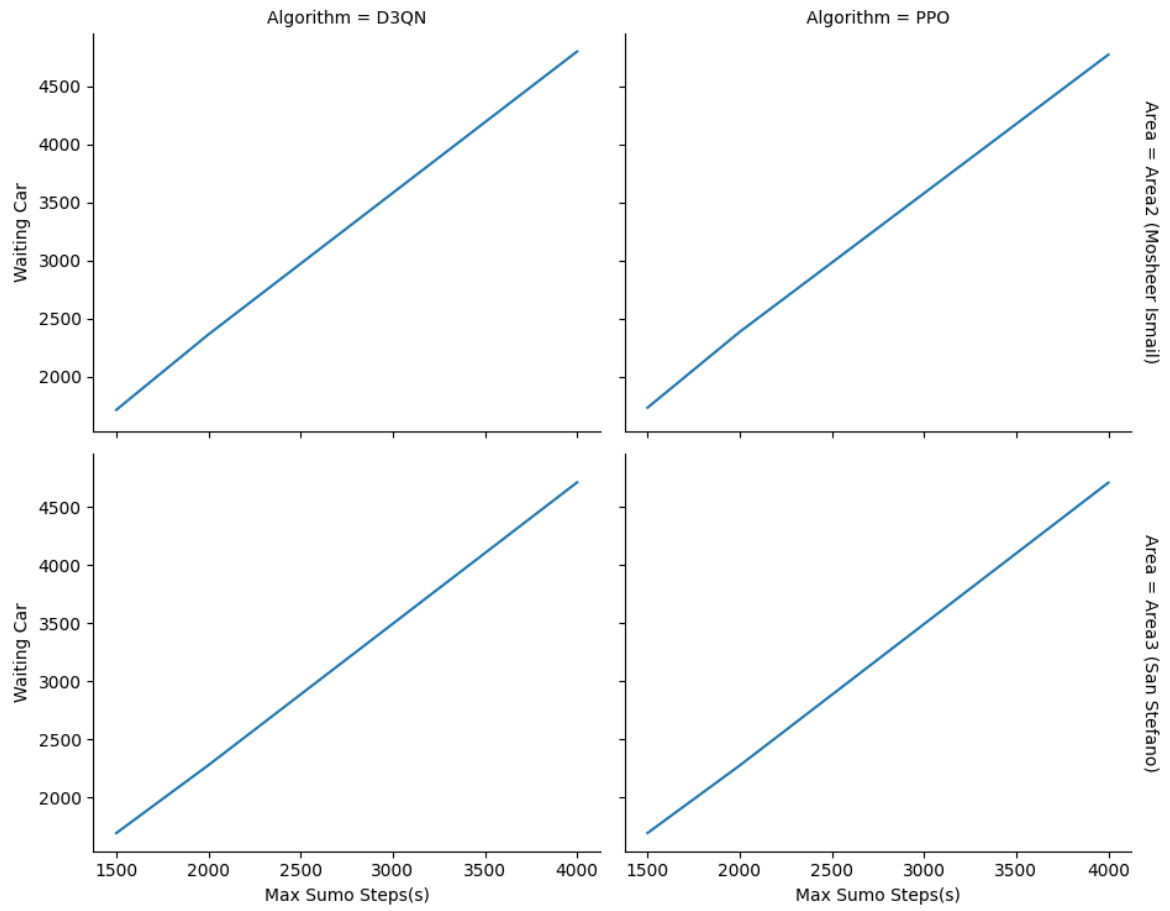


Figure 0.22 Effect of different sumo timing on number of waiting cars

5.4.1.4.2.3 Further studying on Data 2

Performance Comparison Across Algorithms and Environments on Reward of Last Episode

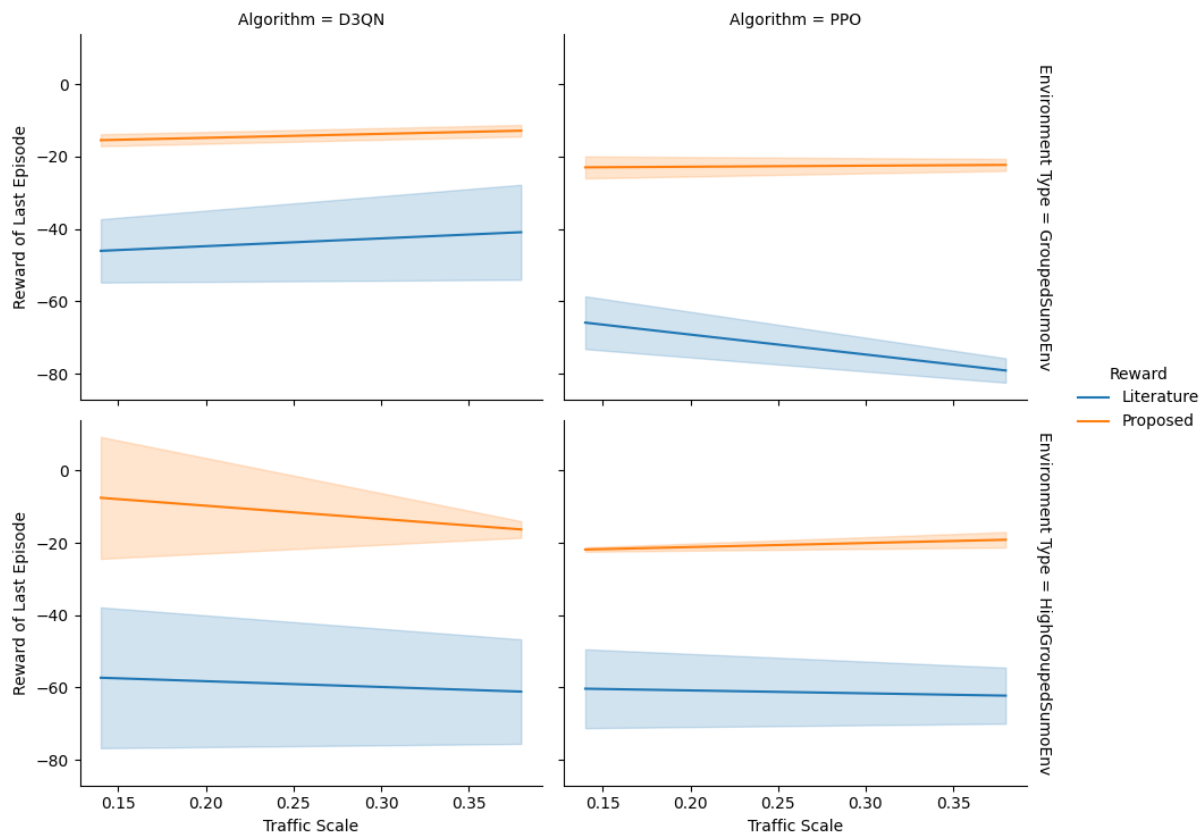


Figure 0.23 Performance comparison across algorithms and environments on reward in area 2

Performance Comparison Across Algorithms and Environments on Average Reward for Evaluated Episodes

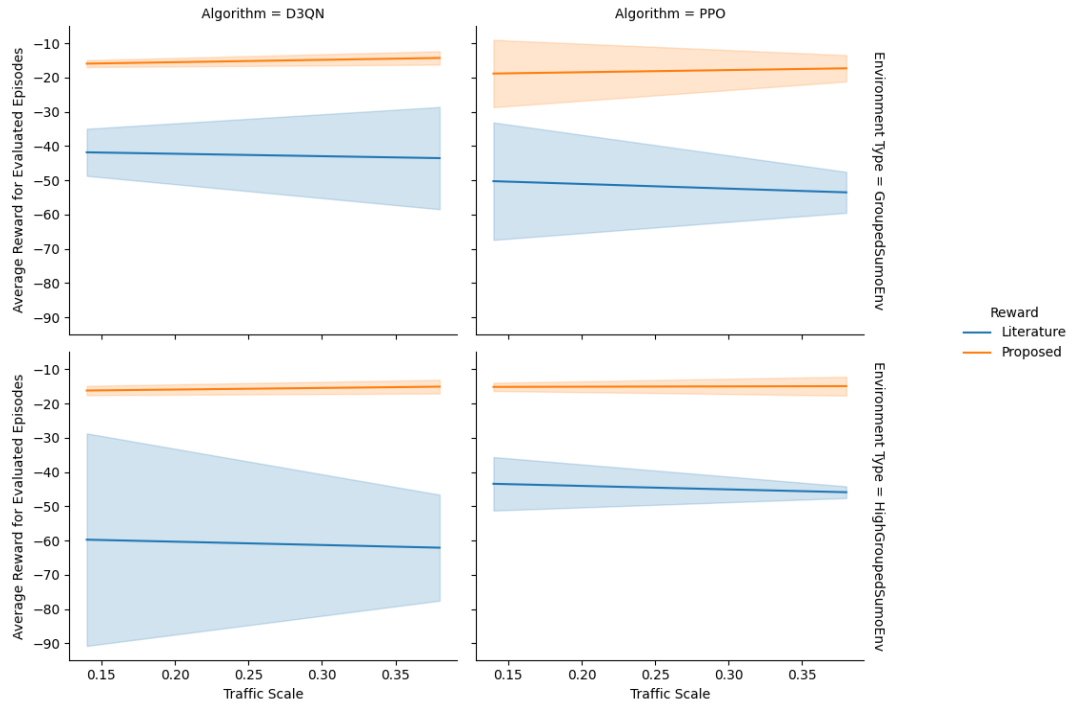


Figure 0.24 Performance comparison across algorithms and environments on evaluated reward in area 2

Performance Comparison Across Algorithms and Environments on Waiting Time (s)

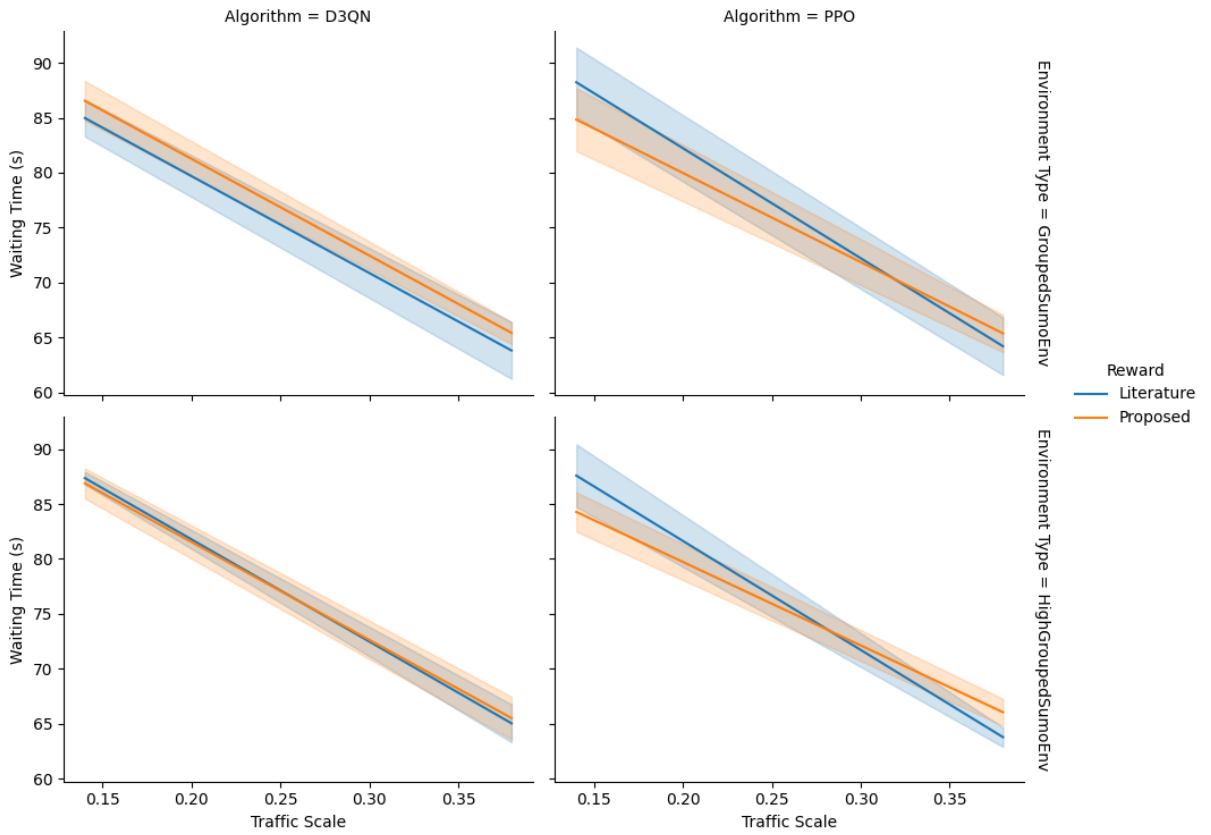


Figure 0.25 Performance comparison across algorithms and environments on waiting time in area 2

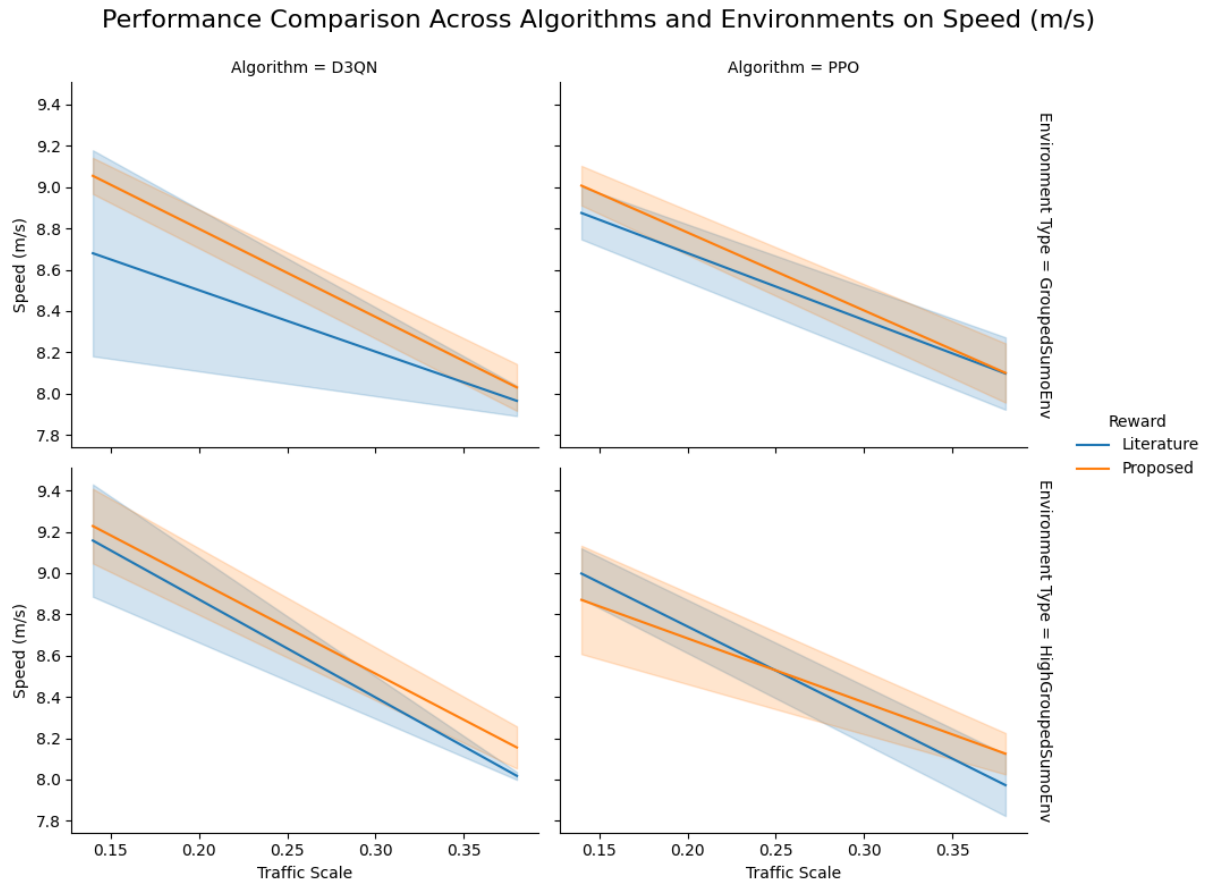


Figure 0.26 Performance comparison across algorithms and environments on speed in area 2

Performance Comparison Across Algorithms and Environments on Depart Delay (s)

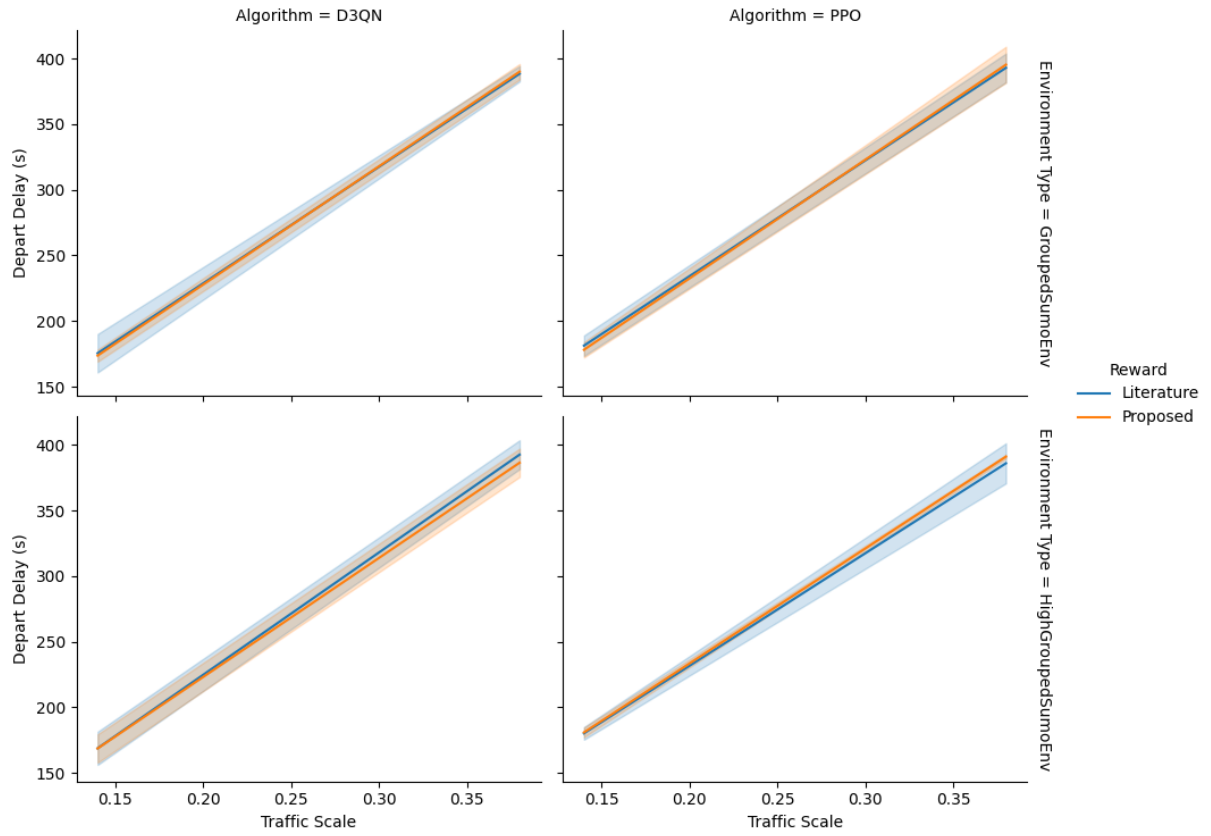


Figure 0.27 Performance comparison across algorithms and environments on depart delay in area 2

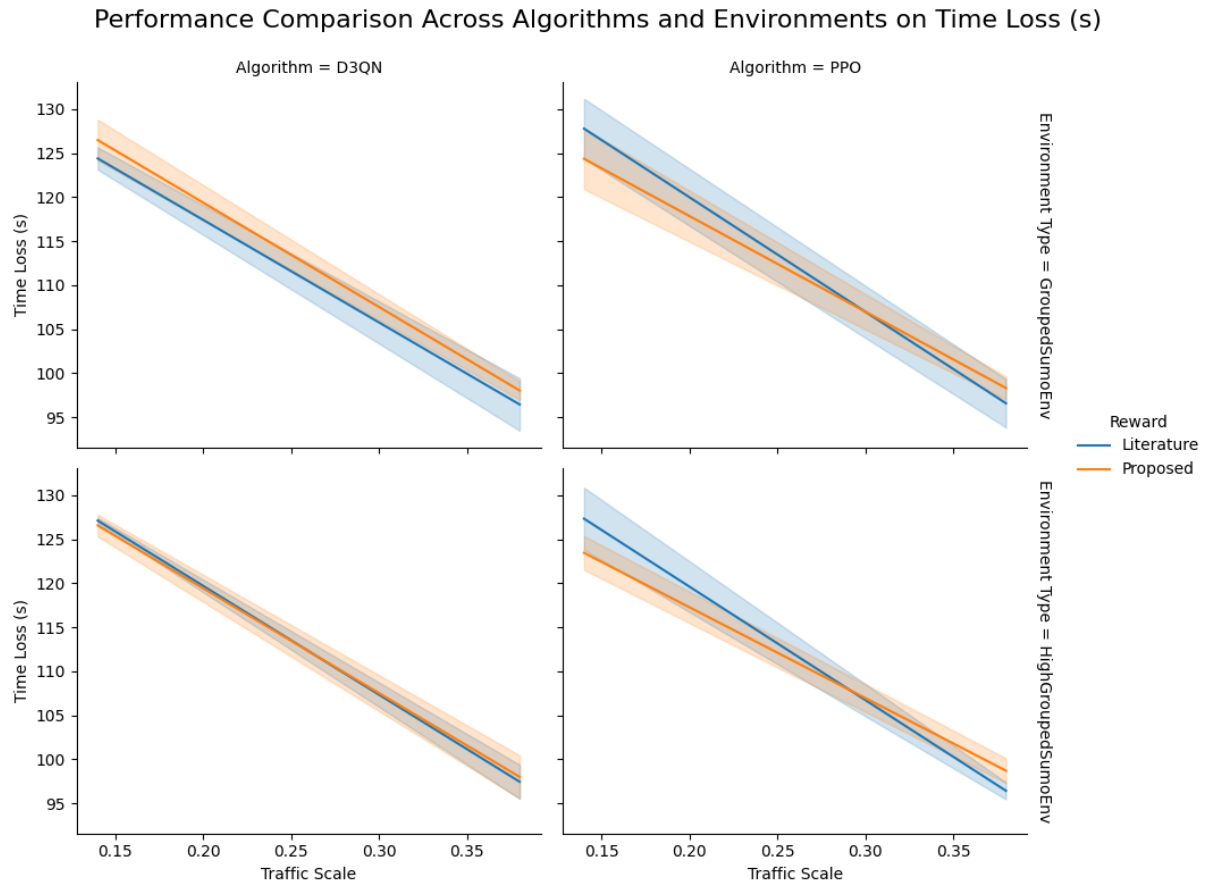


Figure 0.28 Performance comparison across algorithms and environments on timeloss in area 2

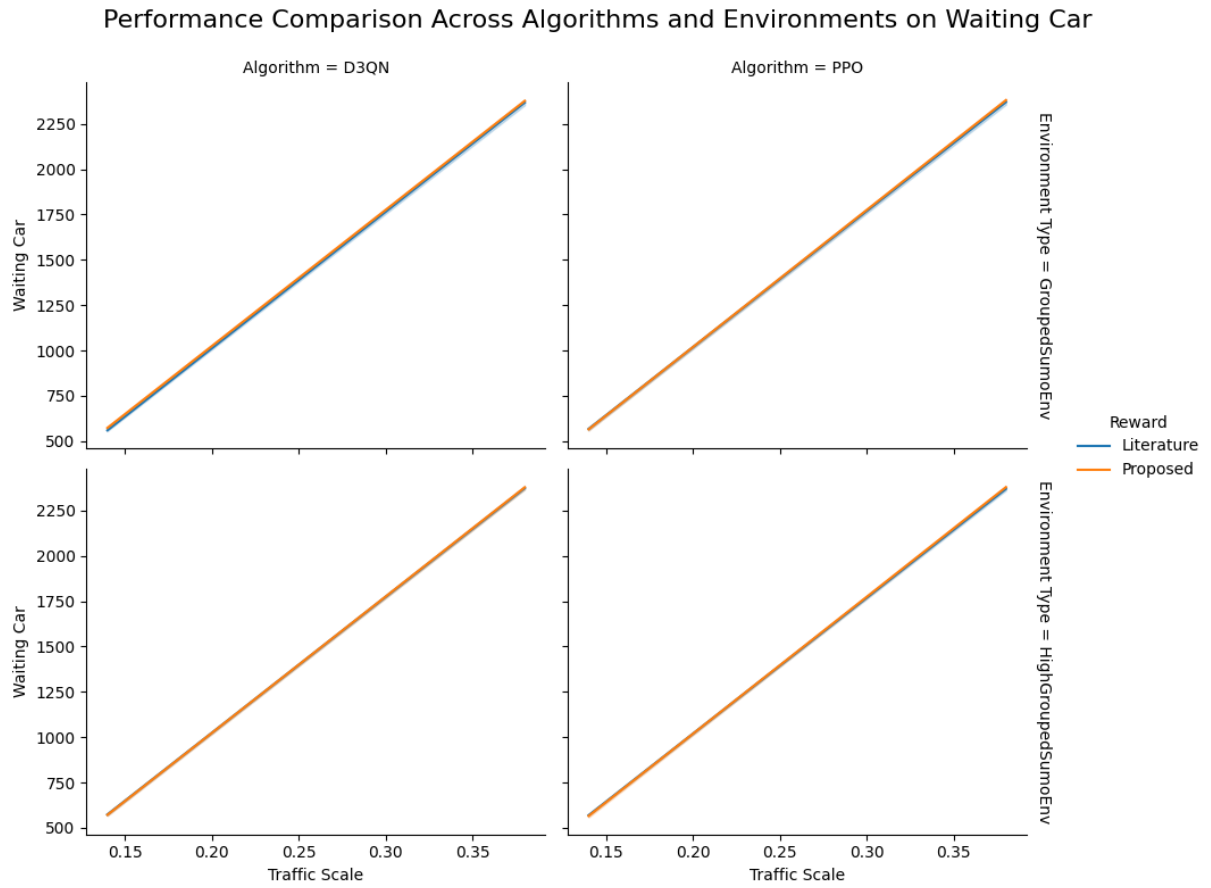


Figure 0.29 Performance comparison across algorithms and environments on number of waiting cars in area 2

5.4.1.4.2.4 Further studying on Data 3

Performance Comparison Across Algorithms and Environments on Reward of Last Episode

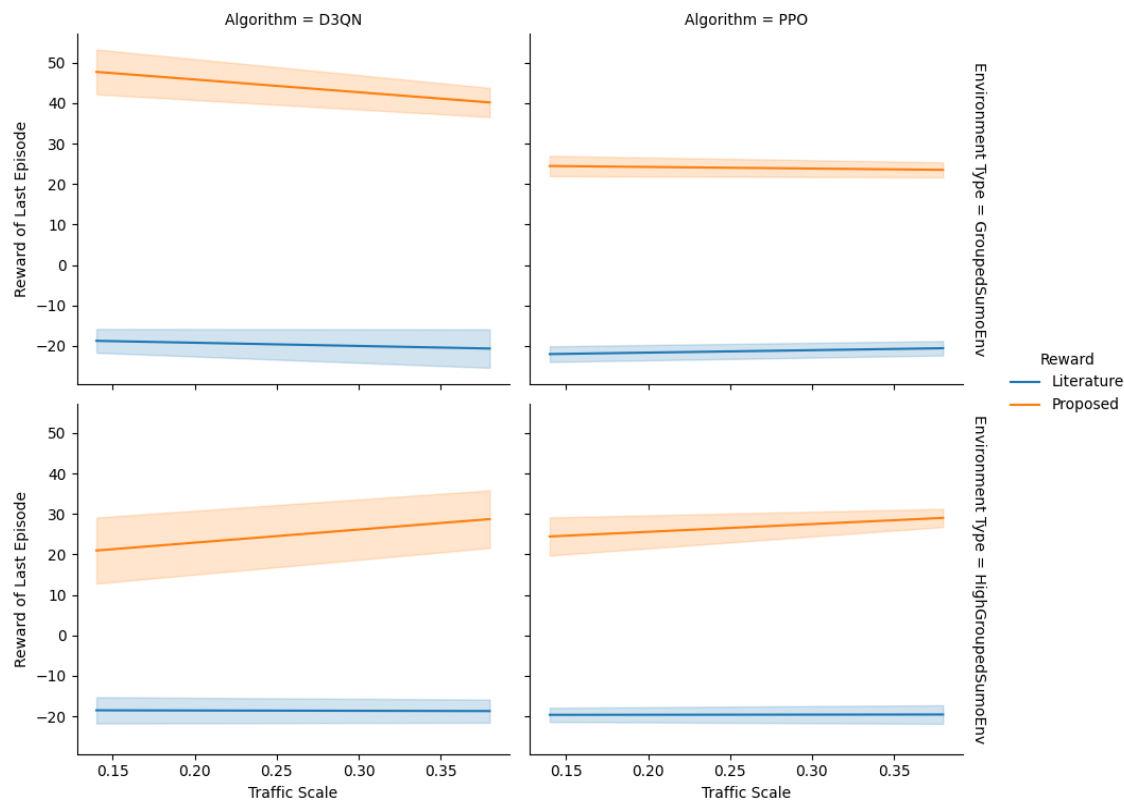


Figure 0.30 Performance comparison across algorithms and environments on reward in area 3

Performance Comparison Across Algorithms and Environments on Average Reward for Evaluated Episodes

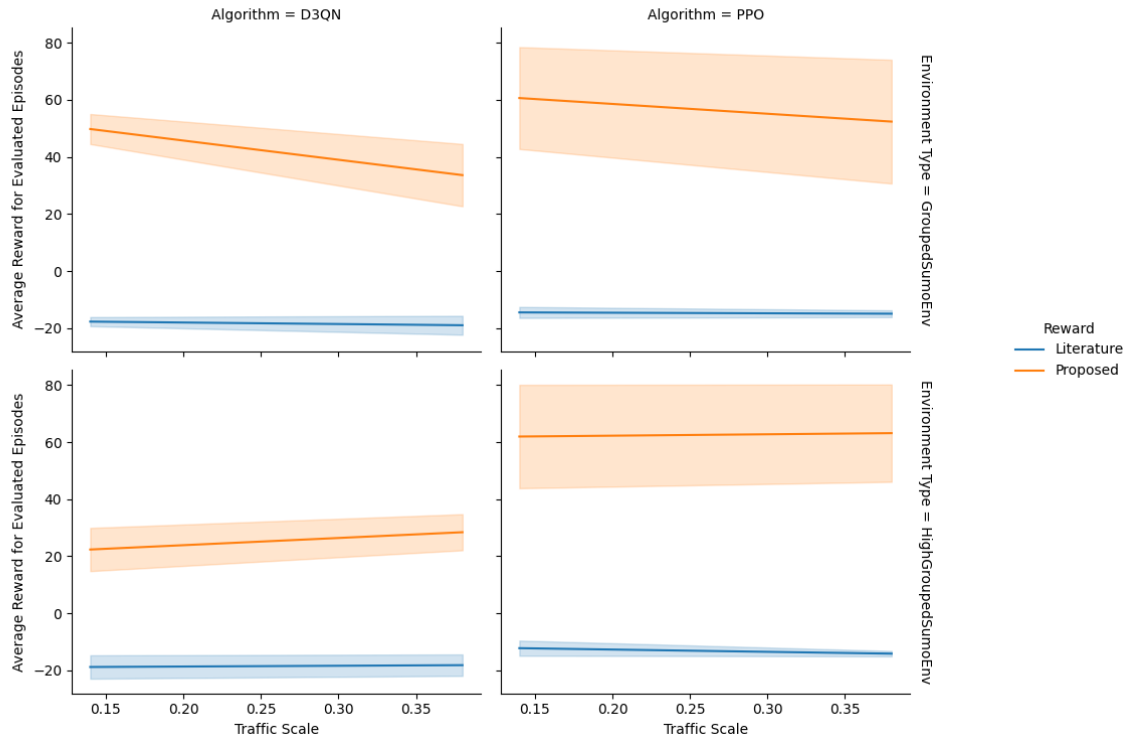


Figure 0.31 Performance comparison across algorithms and environments on evaluated reward in area 3

Performance Comparison Across Algorithms and Environments on Waiting Time (s)

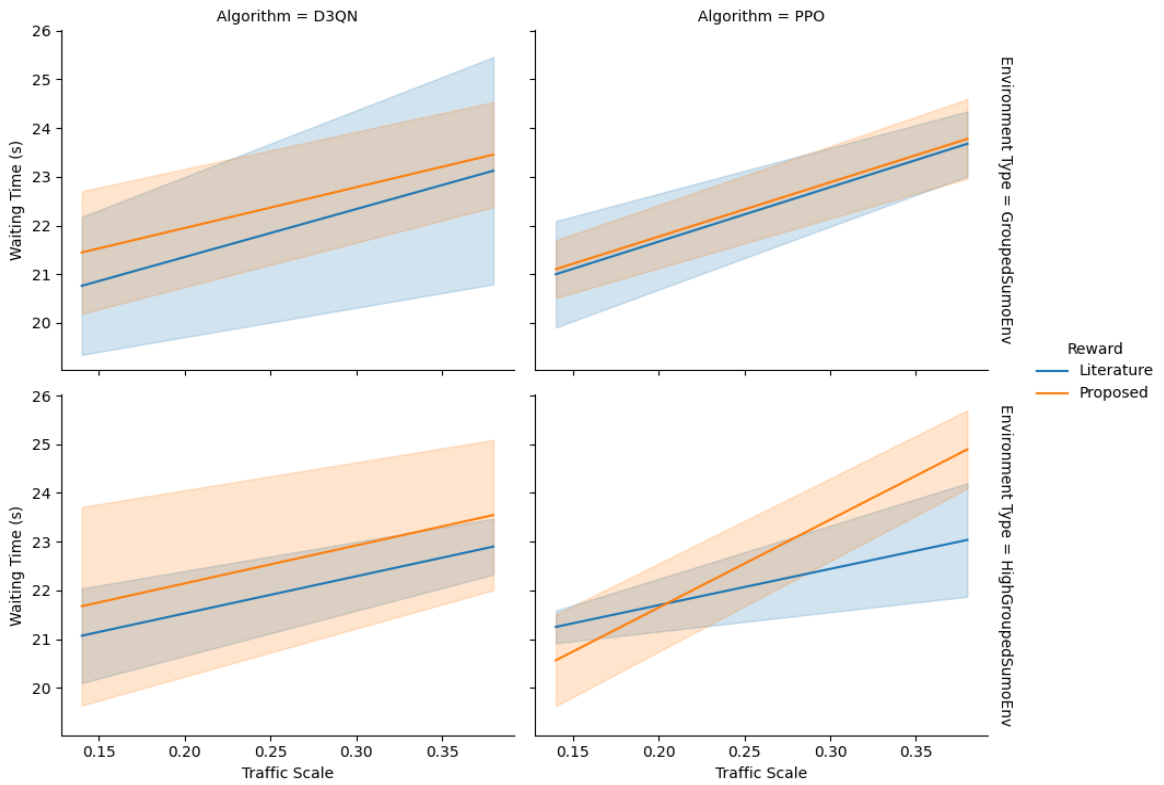


Figure 0.32 Performance comparison across algorithms and environments on waiting time in area 3

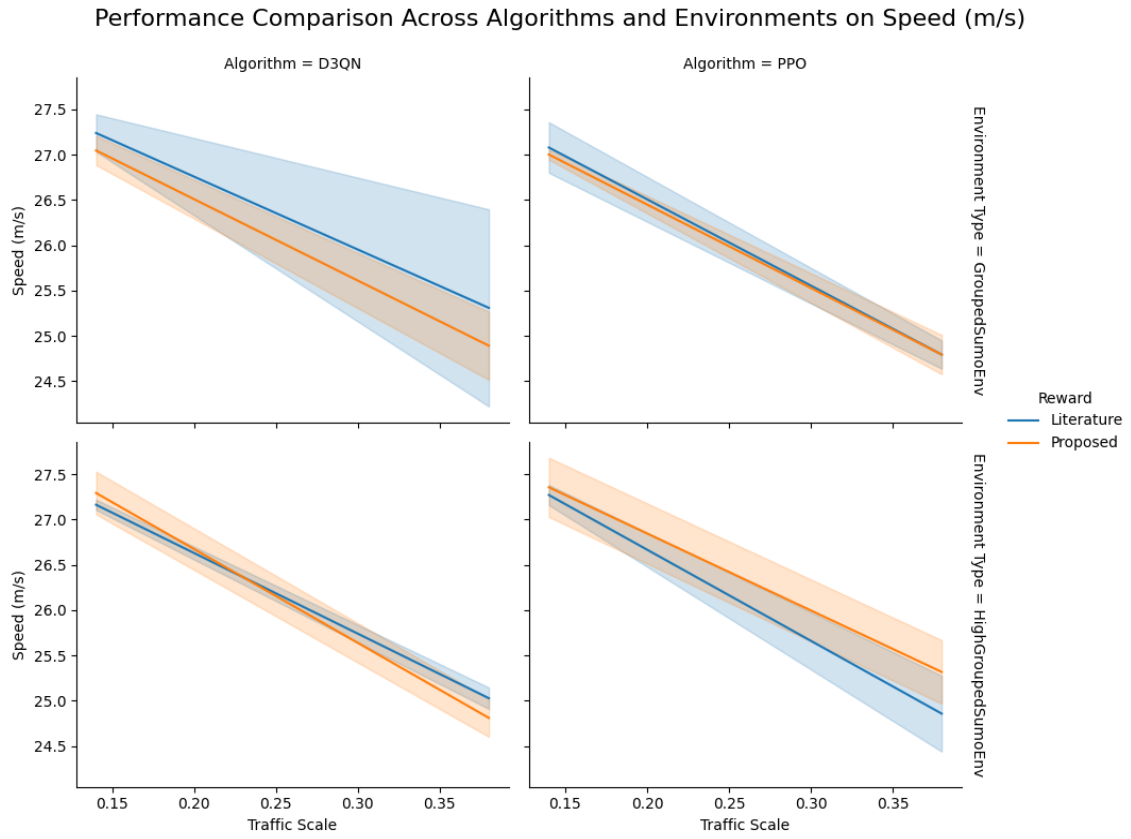


Figure 0.33 Performance comparison across algorithms and environments on speed in area 3

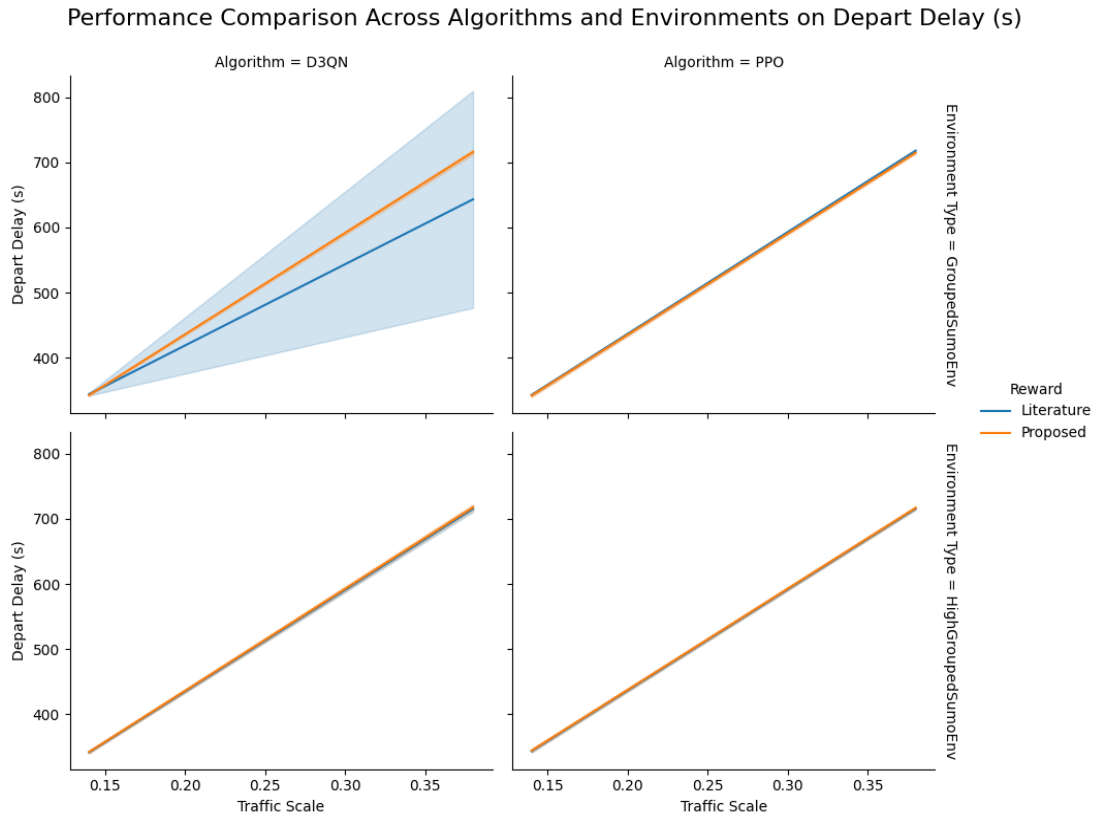


Figure 0.34 Performance comparison across algorithms and environments on depart delay in area 3

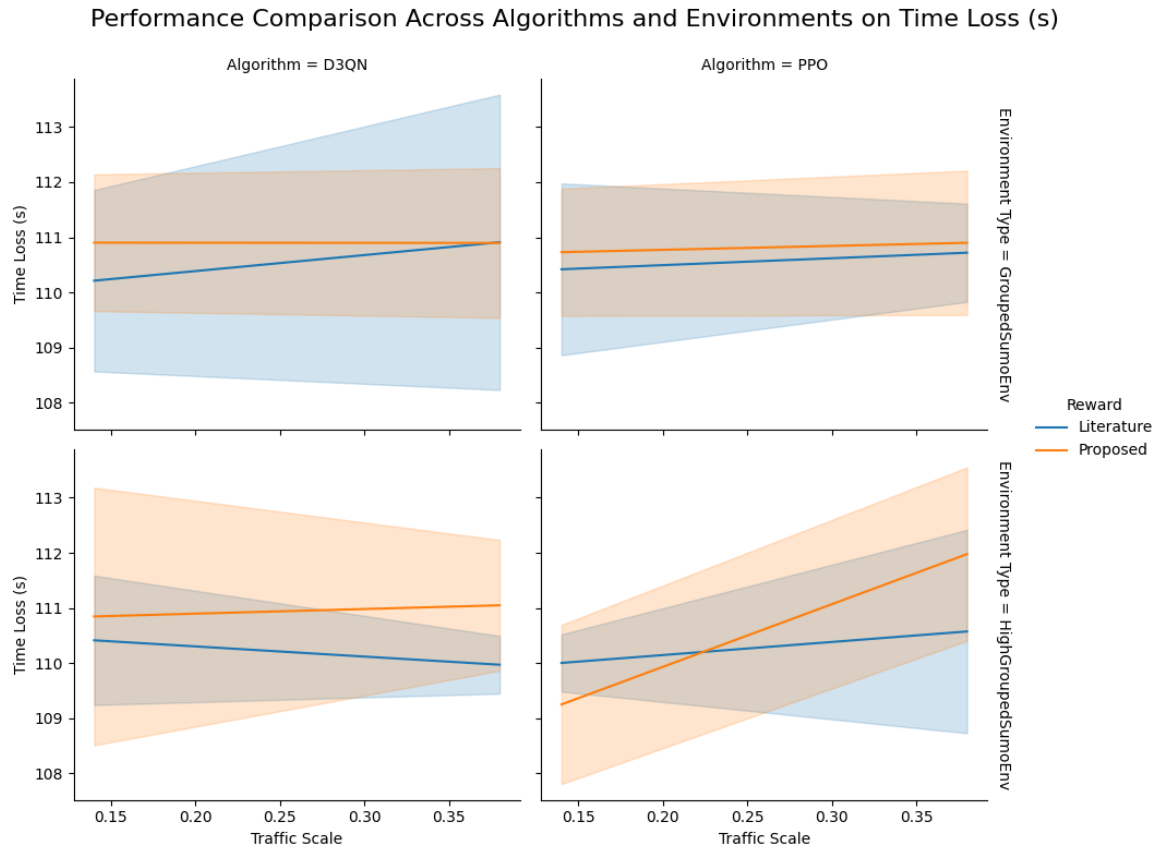


Figure 0.35 Performance comparison across algorithms and environments on timeloss in area 3

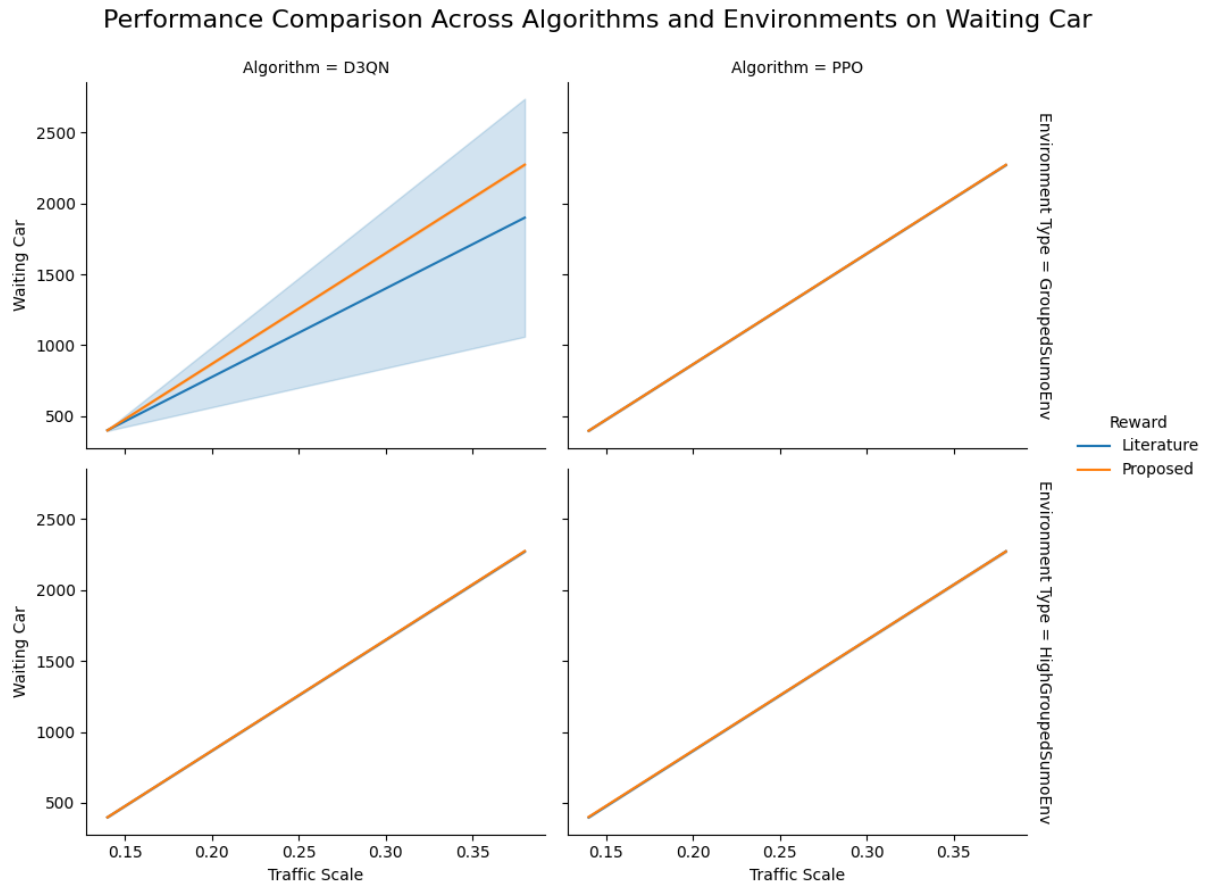


Figure 0.36 Performance comparison across algorithms and environments on number of waiting cars in area 3

5.4.1.4.2.5 Best Results

Summary of seeds at Area2 (Mosheer Ismail) ,Scale Normal AT 33.3M



Figure 0.37 Best results at Area2 with Normal scale

Summary of seeds at Area2 (Mosheer Ismail) ,Scale Crowded AT 33.3M



Figure 0.38 Best results at Area2 with Crowded scale

Summary of seeds at Area3 (San Stefano) ,Scale Normal AT 33.3M

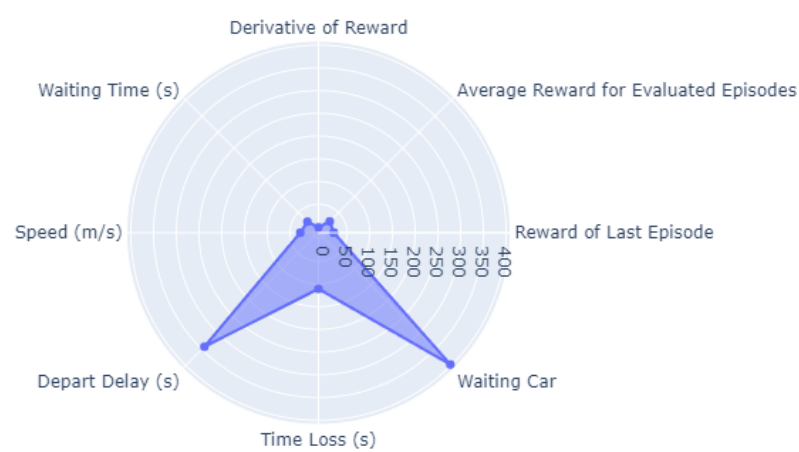


Figure 0.39 Best results at Area3 with Normal scale

Summary of seeds at Area3 (San Stefano) ,Scale Crowded AT 33.3M



Figure 0.40 Best results at Area3 with Crowded scale

5.41..5 Analysis of Variance (ANOVA)

Analysis of Variance (ANOVA) is a statistical method used to determine whether there are significant differences between the means of three or more independent groups. It works by analyzing the variability within groups and between groups to assess whether the observed differences in sample means are likely due to actual effects or just random chance [47].

After the visualization was completed, the data was later analyzed using ANOVA to address the question: *Does using a high-grouped environment yield better results compared to a grouped one?*

Table 0.11 Anova results

Area	Reward	Traffic Scale	Algorithm	Reward of Last Episode	Waiting Time (s)	Speed (m/s)	Depart Delay (s)	Time Loss (s)	Waiting Car
Area3 (San Stefano)	Literature	0.14	PPO	0.211171	0.673872	0.252849	0.624318	0.631029	0.839421
Area2 (Mosheer Ismail)	Literature	0.14	PPO	0.49756	0.771623	0.214706	0.823133	0.86741	0.643226
Area3 (San Stefano)	Proposed	0.14	PPO	0.919798	0.37047	0.0773891	0.22451	0.160713	0.0398628
Area2 (Mosheer Ismail)	Proposed	0.14	PPO	0.48622	0.753739	0.363079	0.451698	0.66866	0.965445
Area3 (San Stefano)	Literature	0.38	PPO	0.321063	0.376744	0.789099	0.333174	0.891971	0.323796
Area2 (Mosheer Ismail)	Literature	0.38	PPO	0.0501503	0.766994	0.319131	0.48455	0.930467	0.95313
Area3 (San Stefano)	Proposed	0.38	PPO	0.464083	0.0984053	0.0444354	0.215673	0.332536	0.625921
Area2 (Mosheer Ismail)	Proposed	0.38	PPO	0.356003	0.546678	0.786843	0.557952	0.675678	0.765449

Area3 (San Stefano)	Literature	0.14	D3QN	0.688833	0.743306	0.494866	0.114162	0.855852	0.192344
Area2 (Mosheer Ismail)	Literature	0.14	D3QN	0.301997	0.0358339	0.143797	0.516375	0.00580131	0.00456948
Area3 (San Stefano)	Proposed	0.14	D3QN	0.00101296	0.853829	0.136805	0.805795	0.968167	0.413561
Area2 (Mosheer Ismail)	Proposed	0.14	D3QN	0.749427	0.77916	0.136595	0.426554	0.946116	0.806054
Area3 (San Stefano)	Literature	0.38	D3QN	0.732422	0.858222	0.625868	0.420143	0.517794	0.409174
Area2 (Mosheer Ismail)	Literature	0.38	D3QN	0.134828	0.460697	0.214563	0.53454	0.585142	0.431086
Area3 (San Stefano)	Proposed	0.38	D3QN	0.442762	0.924835	0.714651	0.543875	0.871118	0.562764
Area2 (Mosheer Ismail)	Proposed	0.38	D3QN	0.581691	0.933766	0.152576	0.578887	0.992819	0.877516

With a significant level of 97%, most results were found not to show significant improvement. This was initially observed from the plot. The lack of improvement may be attributed to the nature of the road being in a single direction.

5.4.1.6 Discussion of results

5.4.1.6.1 Discussion of the results of Benchmark

- The proposed environment was observed to outperform both basic SUMO and the reference project in many scenarios, and in most cases overall.
- The performance improvement may stem from multiple low-level design choices and default configurations embedded within Stable-Baselines3. These include critical hyperparameters such as `target_update_interval`, `train_freq`, and `learning_starts`, which were not explicitly set in the earlier implementation. Furthermore, the transition to PyTorch and potential architectural differences in the environment or training loop could also contribute. Reinforcement learning is inherently stochastic and highly sensitive to these configurations, meaning even subtle differences can lead to significant changes in learning dynamics and final performance.
- Certain performance metrics were found to be highly sensitive to random seeds, indicating variability across runs [42].

5.5.1.6.2 Discussion of the results of Environmental Experiments

Effect of Episode Count Increasing the number of episodes was observed to improve several performance metrics—specifically, waiting cars, waiting time, speed, and reward—particularly in Dataset 2 With D3QN. A less significant impact was noted in Dataset 3. However, results remained noisy, as the experiments were conducted with a single random seed due to time constraints. The configuration used included: the proposed reward function, a traffic

scale of 0.38, 500 SUMO steps, the 'highgrouped' level, and seed 0. These conditions are candidates for future work with more comprehensive seeding and evaluation.

Effect of SUMO Steps An increase in the number of SUMO steps logically resulted in higher waiting time and time loss, yet the reinforcement learning algorithm was able to adapt and achieve better reward performance. The derivative of average waiting time supported this learning behavior. Again, results showed high variability due to the single-seed constraint. This configuration involved: the proposed reward function, 1000 episodes, seed 0, traffic scale of 0.38, and 'highgrouped' setting. Further work should explore multi-seed evaluations to mitigate variance.

Reward Function Comparison The literature-defined reward achieved better results in Dataset 3, although the differences were not substantial. In Dataset 2, results were close between both reward functions.

Seed Variability As typical in reinforcement learning, high variance was observed between different random seeds, underlining the need for multi-seed validation [42].

Learning Efficiency The proposed reward function facilitated better learning, especially in Dataset 3, where positive rewards were obtained. In contrast, Dataset 2 showed large reward derivatives across episodes and often produced negative rewards. Notably, the reference paper for the proposed reward also reported initial negative rewards, eventually improving performance by incorporating additional road-related parameters (e.g., neighboring roads, time-spatial features).

Algorithm Comparison Both algorithms converged to similar final metrics. However, D3QN exhibited greater stability, both in terms of reward derivative and training-test performance consistency. The reduced noise in training plots further supports this observation.

Effect of Grouping The grouping configuration highgrouped did not yield significant differences, possibly due to the structure of Egyptian roads, where lanes often flow in the same direction.

Difference between Areas

- Better performance was achieved in Dataset 3 San Stefano, where reward values were generally non-negative. This may be attributed to differences in road infrastructure, which affected the number of cars passing through.
- Future Work is to address geographical differences in simulation difficulty in data 2 (Mosheer Ismail). It was observed that Alexandrian roads posed greater challenges due to narrower lanes and lower vehicle speeds (e.g., ~ 7 m/s in AlST vs. ~ 18 m/s in LuST), which made the simulation both harder and slower. This should be considered in engineering applications aiming for realistic traffic modeling.

Overall Results

- The overall waiting time was achieved to be very good (20:90 s for half of hour), and for speed was max possible speed
- High number of waiting cars, with small waiting time and timeloss indicate many waited but with fairness which is wanted
- Depart delay was a larger, need further learning (can be 1.5M :13.3M)

5.4.1.7 Limitation of Version 1.0.0

Although acceptable performance was observed, several issues were identified, including

- high departure delay, increased time loss, and extended vehicle waiting times.
- The issues were clearer in the Mosheer Ismail environment.
- Certain metrics—such as the effect of episode count and training duration—were not thoroughly evaluated due to time constraints.

In the next version, alternative environment (e.g., the SUMO-RL package) will be examined, and the implementation will be refined to achieve improved results.

5.4.2 Version 2.0.0

In this section modified version (V2.0.0) of project is detailed.

5.4.2.1 Key Modifications

The results of Phase 1 were good; however, several problems were identified, including large departure delays, time losses, waiting vehicles, and inconsistent rewards.

In the following phases (versions), efforts were made to improve these results.

The core solution in the later phases was based on benchmarking against publicly available implementations, specifically the SUMO-RL package [2].

Main improvements in this phase included enhancing the `step()` function to ensure accurate metric calculation and refining the waiting time computation following the SUMO-RL approach.

In this approach:

- The intended action is assigned to the agent by the `step()` function.
- The agent's decision regarding which action to take and when to stop is then determined.
 - If a new phase is required but the minimum green time has not yet been reached, the current phase is continued.
 - If the same phase is selected, it is maintained, potentially with an updated duration.
 - When a different phase is to be introduced, a yellow phase is first applied. This yellow phase, considered the opposite of the current one, is automatically calculated whenever two lanes are changed and is applied at the start of `time_since_last_phase_change`.
- The timing of the next action is then determined, and the process continues until the next action duration is reached (this is done instead of taking the whole action in one step as Version 1.0.0).

However, some unexpected behaviors were observed when the greatest common divisor (GCD) of the duration was calculated in [5.1.4 Action Space]; this issue has been noted for future work. In addition, the first action was always executed as an initial action rather than through the `step()` function. [6.2 Future Work].

Although this added some complexity, it is expected to simplify future implementation of multi-agent reinforcement learning.

-

5.4.2.2 Modified Experiments Design

Table 0.12 Modified Environmental Experiments for Version 2.0.0

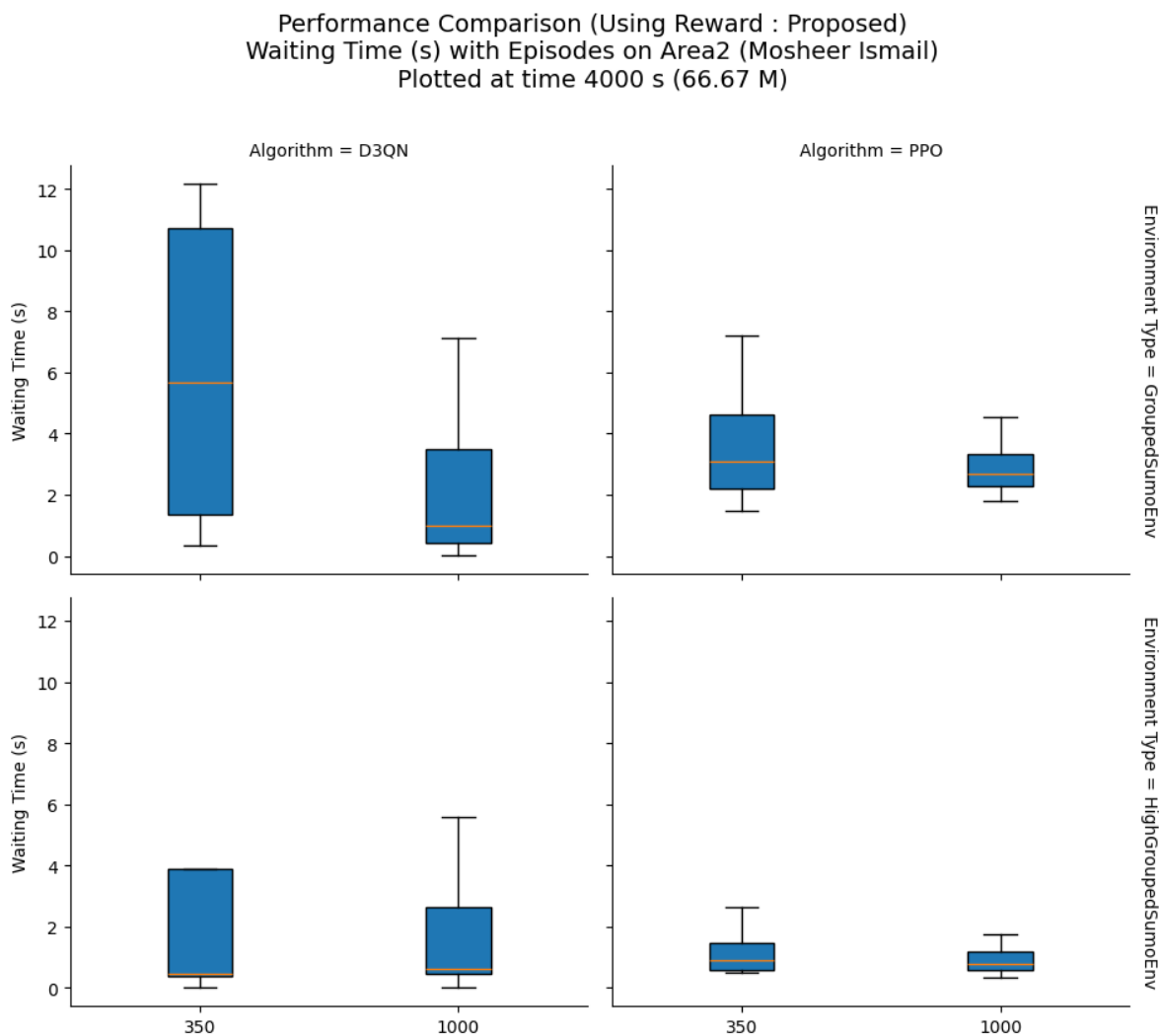
Parameter	Meaning	Real meaning	Range for experiment
Area	The dataset or environment used	Country-specific traffic data	Area2 (Mosheer Ismail) – Area 3 (San Stefano)
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	Literature - Proposed
Environment type	Type of SUMO environment used	Level of traffic control detail and intersection grouping	GroupedSumoEnv HighGroupedSumoEnv

Algorithm	RL algorithm applied	Different learning strategies used to optimize traffic control	D3QN -PPO
Episodes	Number of training episodes	Number of training cycles (longer = more learning)	350,1000
Max sumo steps	Maximum number of simulation steps in SUMO	The number of times the experiment is repeated to compare environments and ensure consistency	500-2000 -4000
Traffic Scale	Traffic intensity load	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowding (.38)
Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2,3

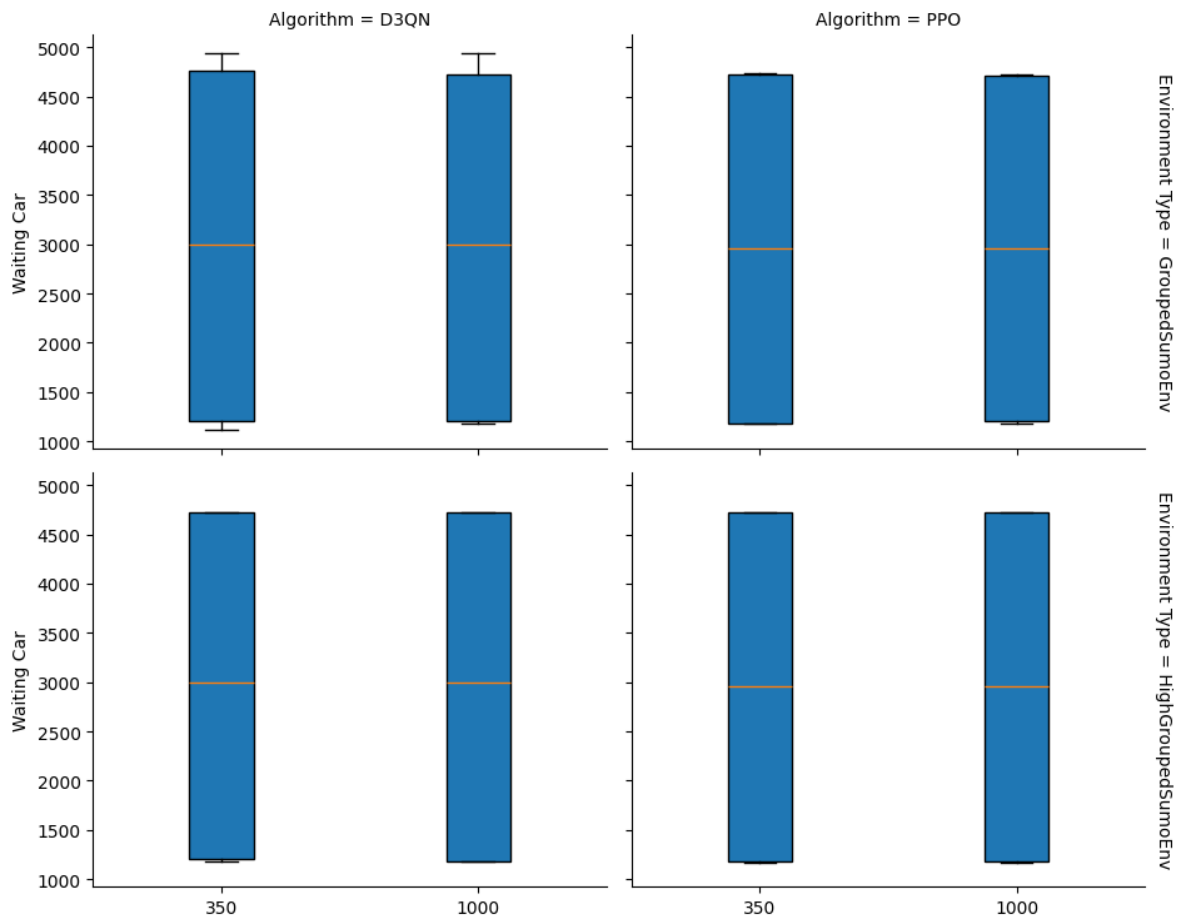
5.4.2.3 Results

In this stage, the focus was directed toward improving the performance of the previous version by integrating a standardized reinforcement learning package that was supported by a referenced research paper. However, during the experimental phase, several metric inconsistencies and technical issues were encountered, which prevented the completion of a full set of experiments and a comprehensive analysis, so in this section the main plots that only correspond to result will be shown here (Only Mosheer Ismail and the average include episodes as not differ and for cleaner plots).

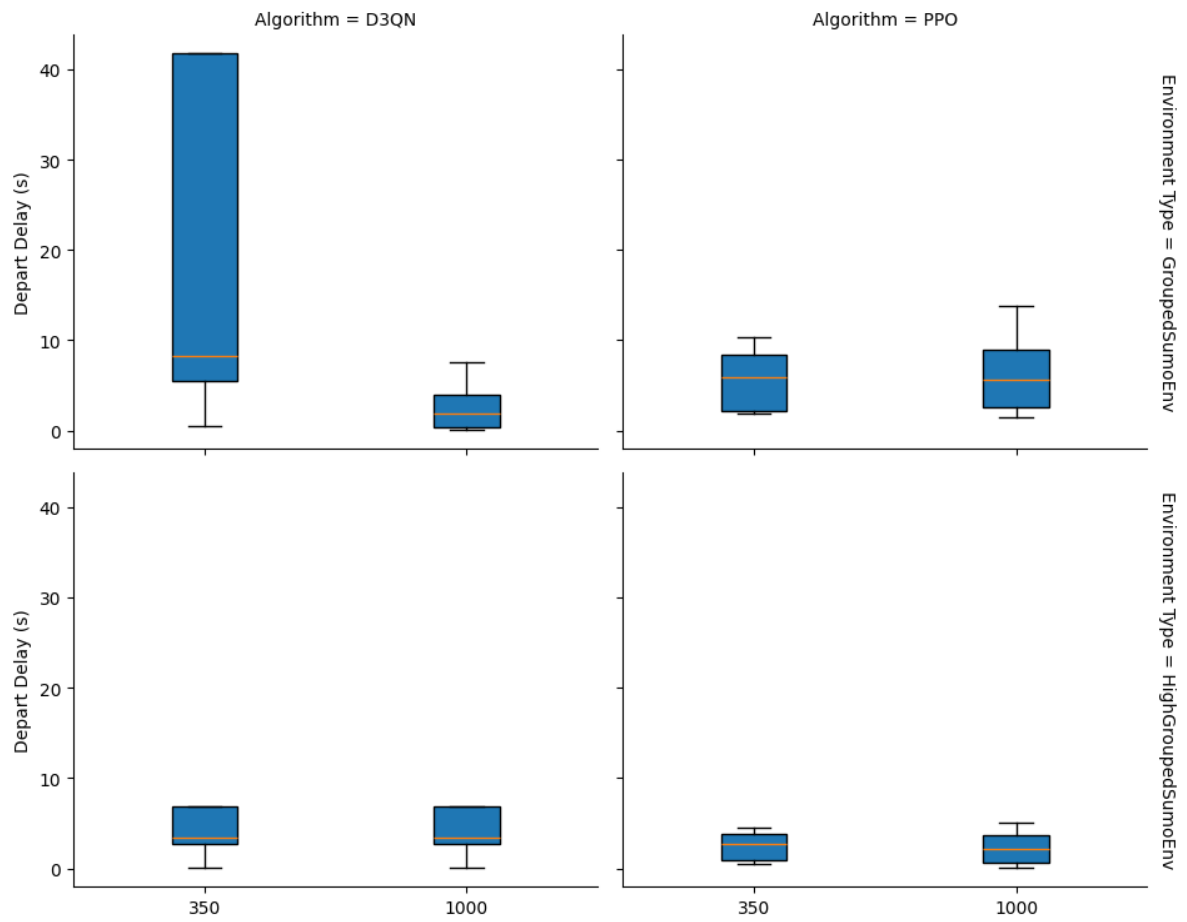
5.4.2.3.1 Results show effect of episodes



Performance Comparison (Using Reward : Proposed)
 Waiting Car with Episodes on Area2 (Mosheer Ismail)
 Plotted at time 4000 s (66.67 M)



Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) with Episodes on Area2 (Mosheer Ismail)
 Plotted at time 4000 s (66.67 M)



Performance Comparison (Using Reward : Proposed)
Time Loss (s) with Episodes on Area2 (Mosheer Ismail)
Plotted at time 4000 s (66.67 M)

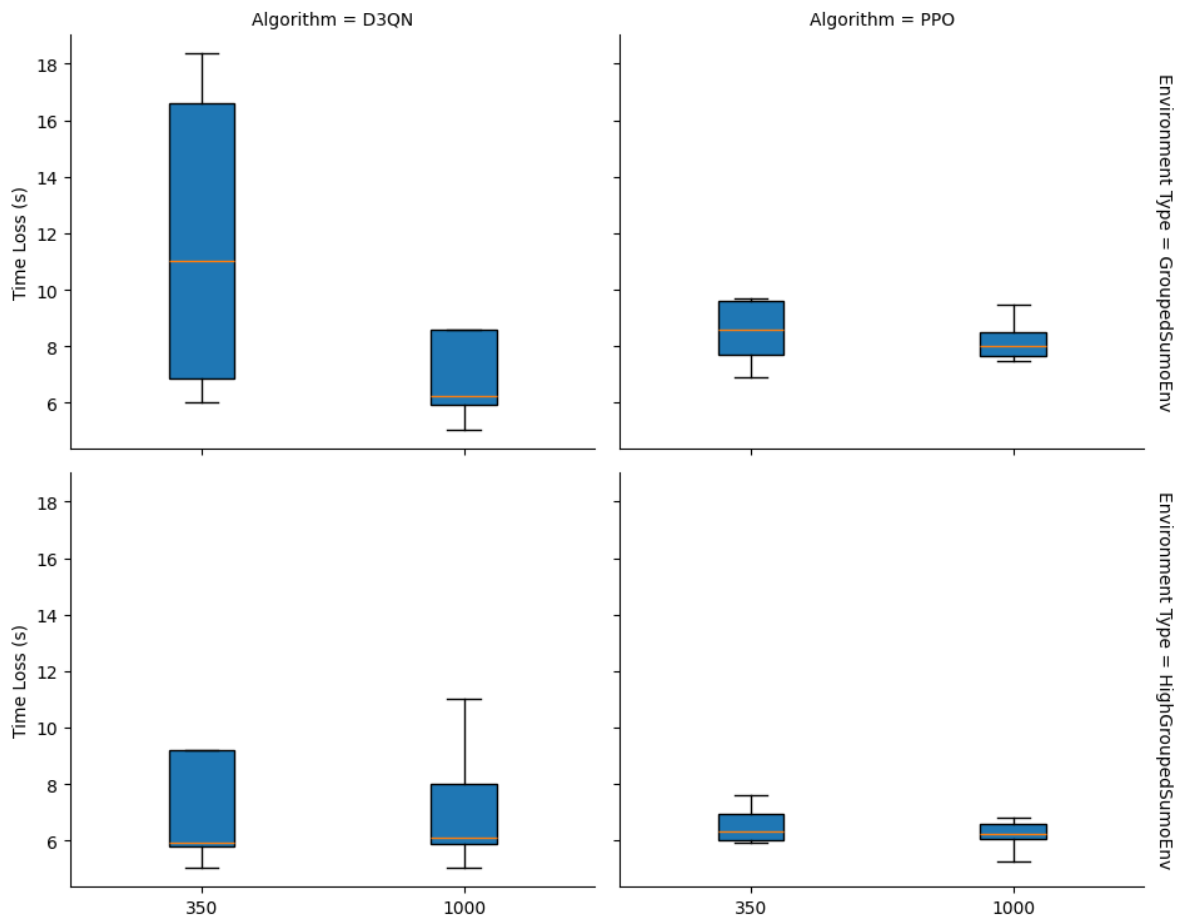
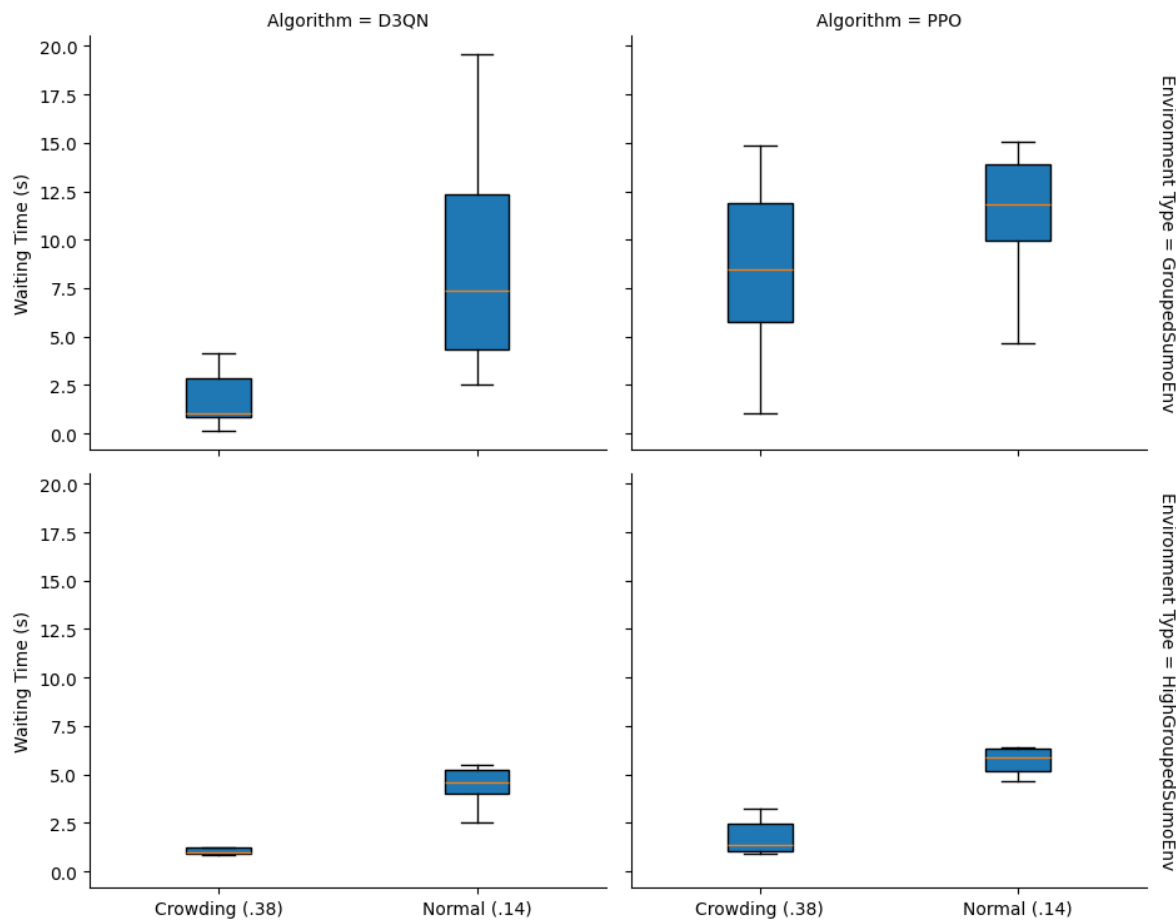


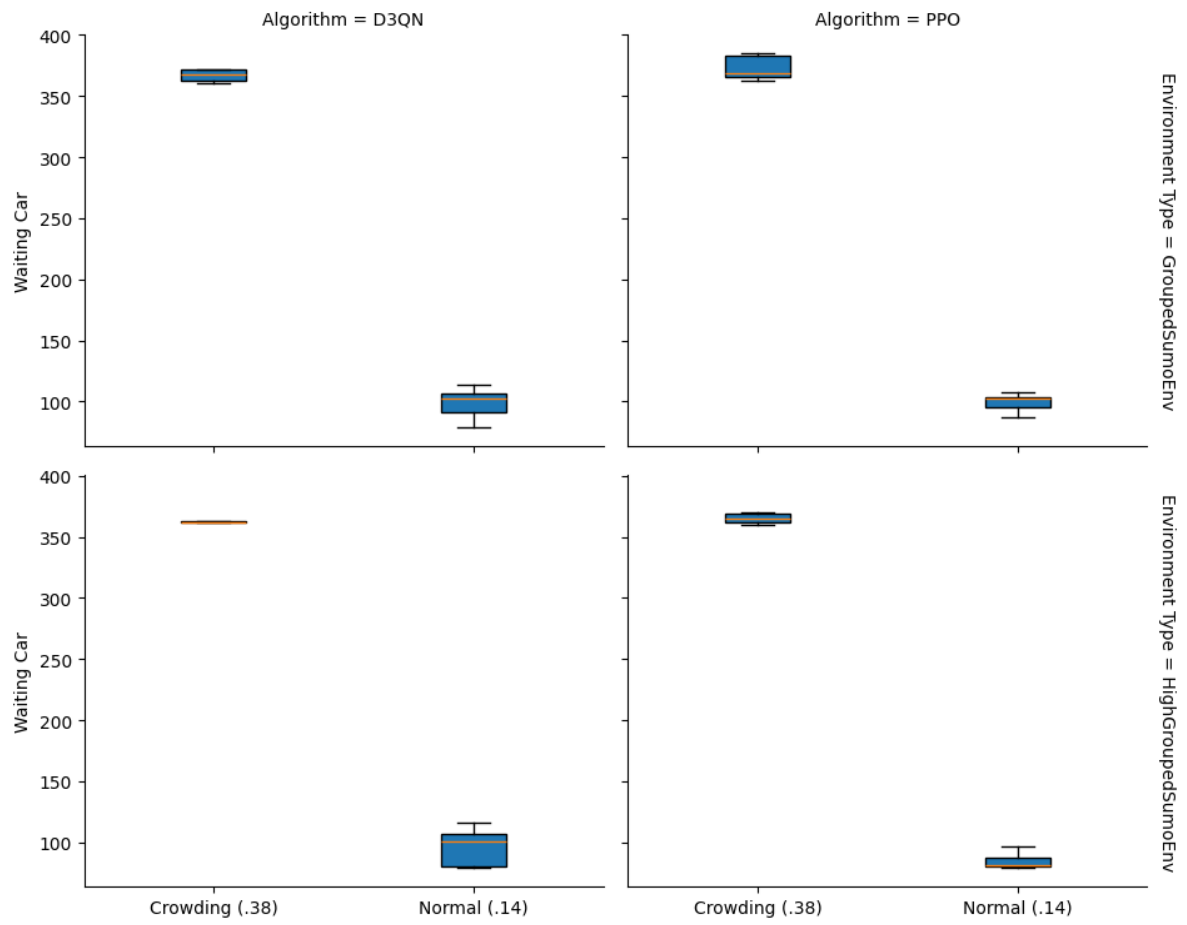
Figure 0.41 Results show effect of episodes at V2.0.0

5.4.2.3.2 Results with proposed reward

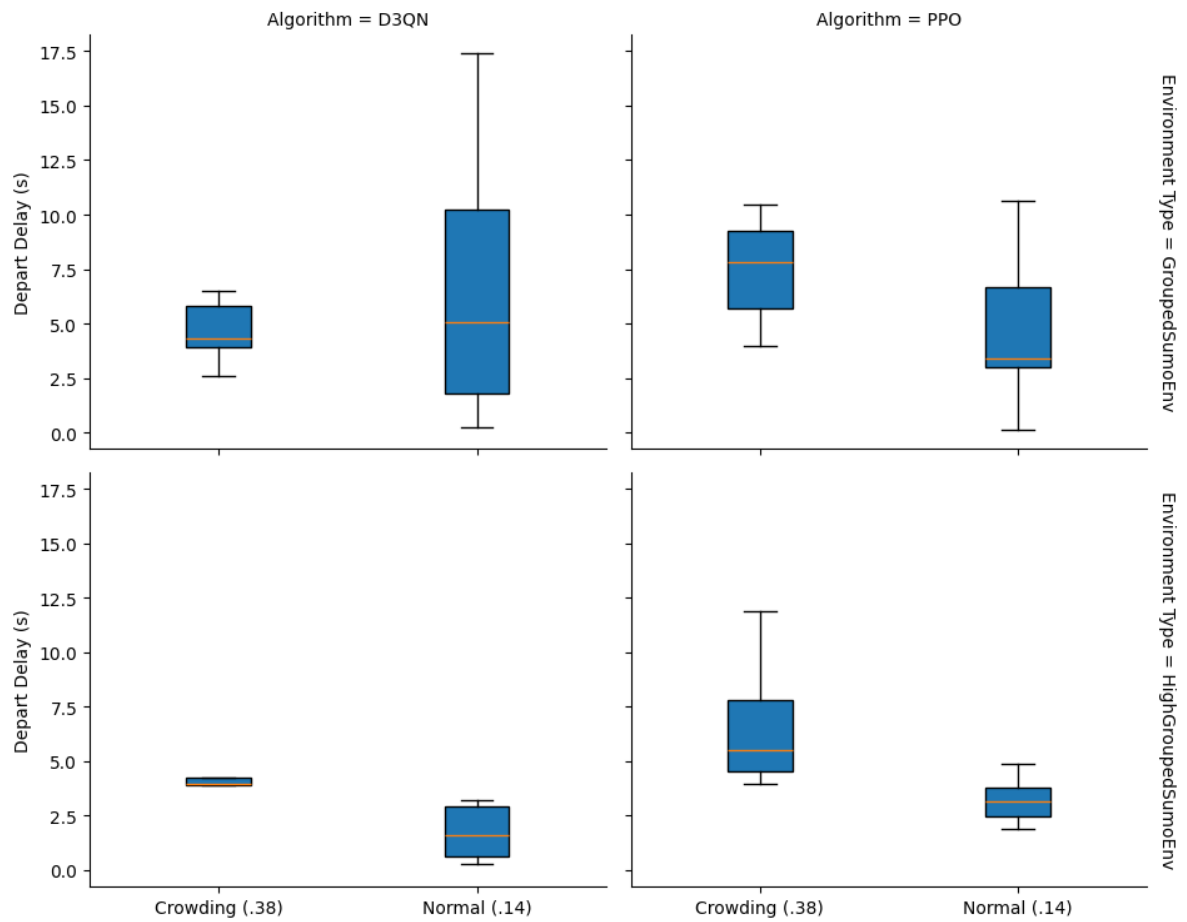
Performance Comparison (Using Reward : Proposed)
Waiting Time (s) with Traffic Scale on Area2 (Mosheer Ismail)
Plotted at time 500 s (8.33 M)



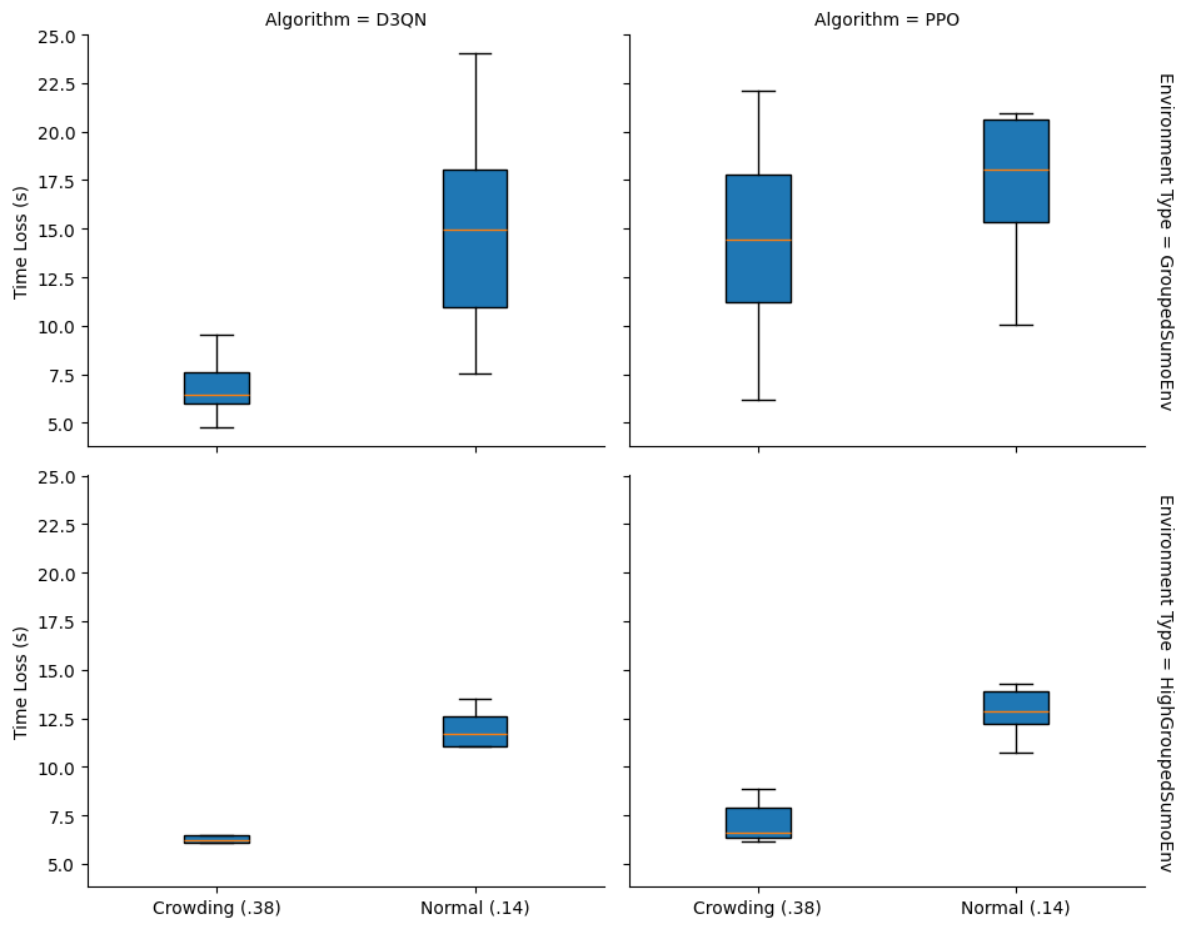
Performance Comparison (Using Reward : Proposed)
 Waiting Car with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 500 s (8.33 M)



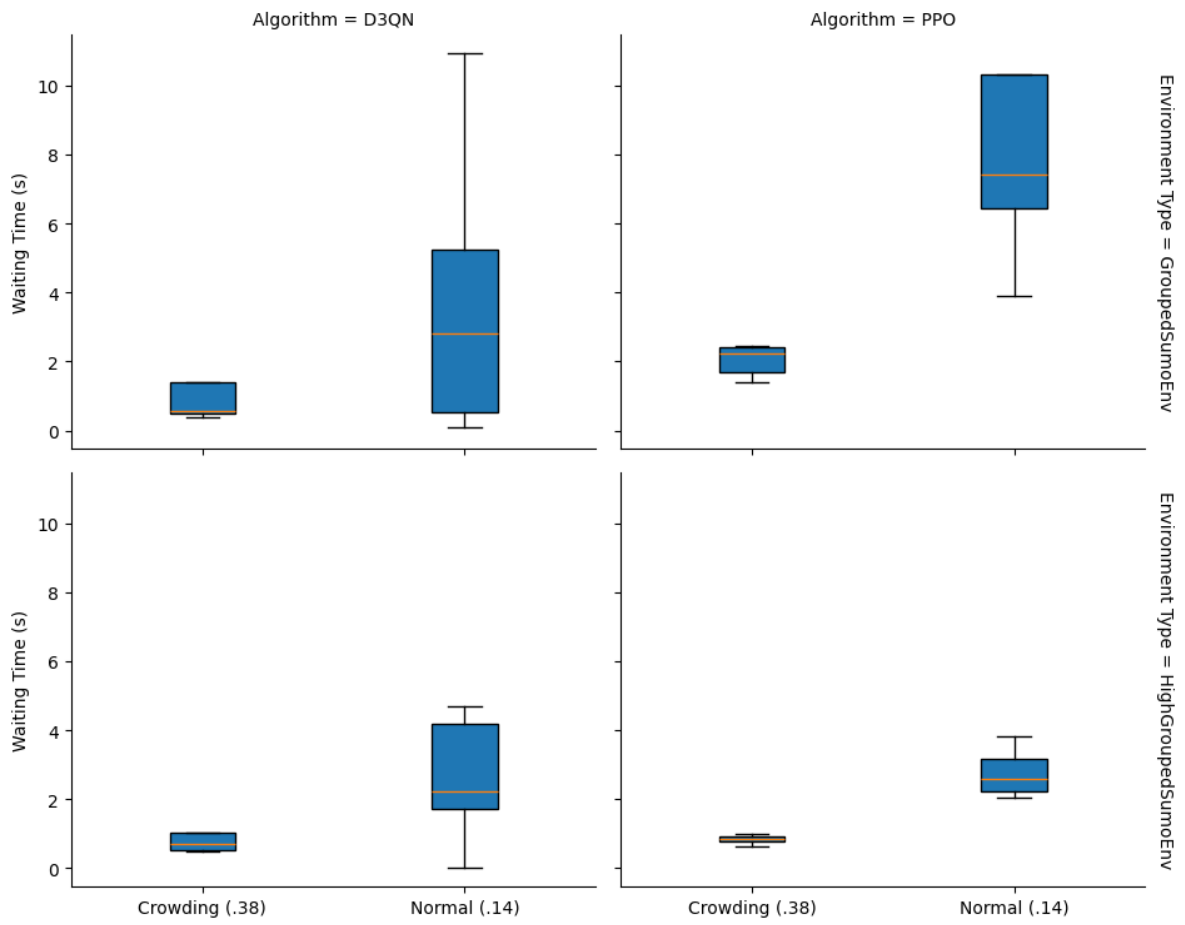
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 500 s (8.33 M)



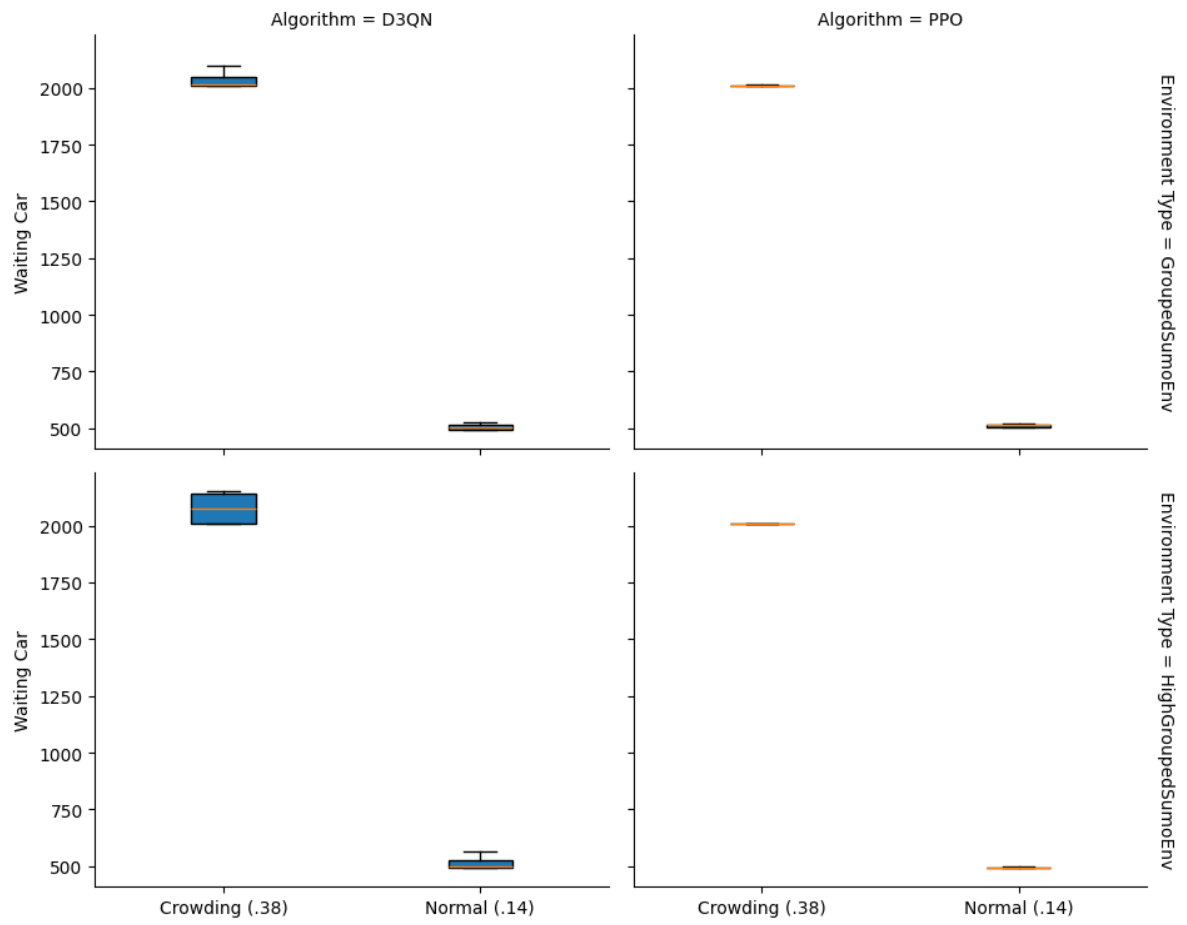
Performance Comparison (Using Reward : Proposed)
Time Loss (s) with Traffic Scale on Area2 (Mosheer Ismail)
Plotted at time 500 s (8.33 M)



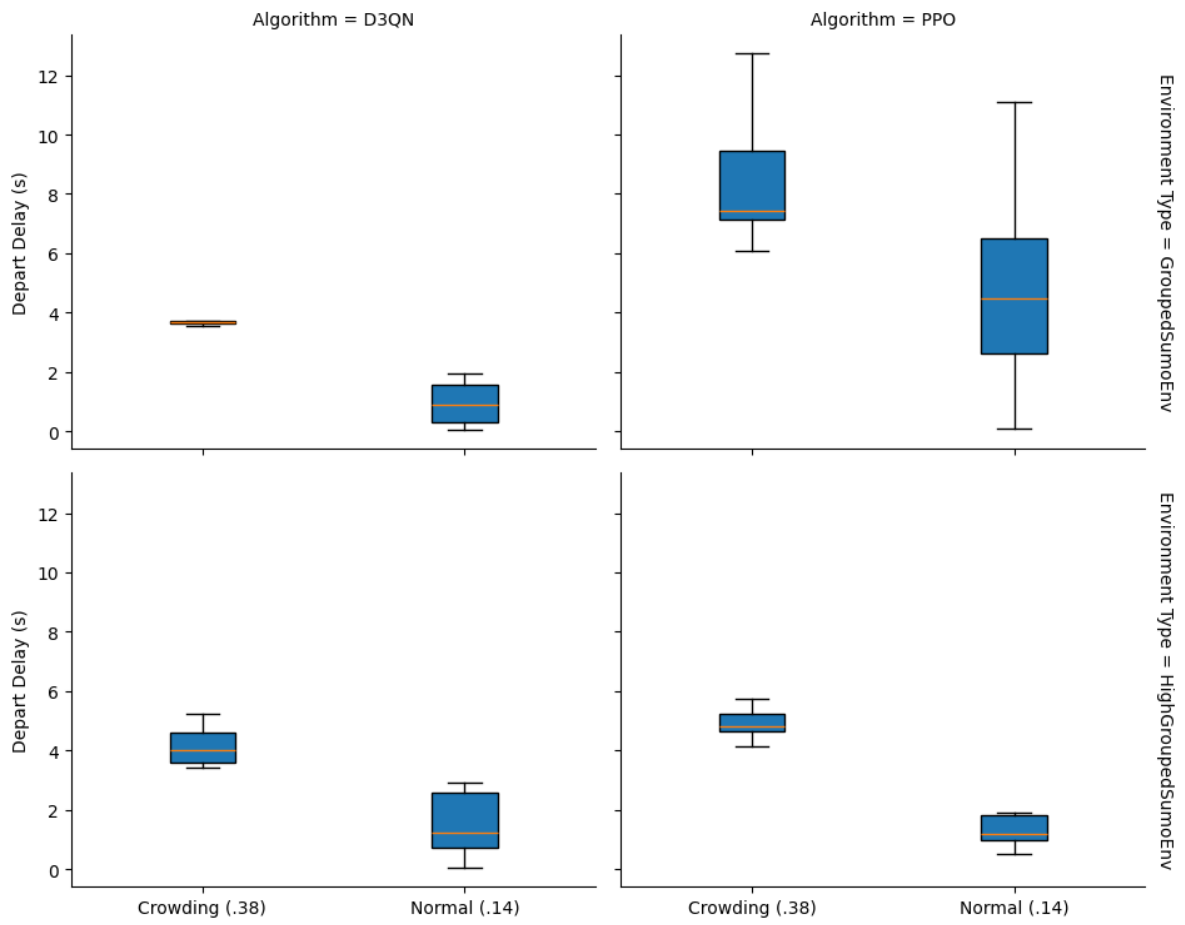
Performance Comparison (Using Reward : Proposed)
 Waiting Time (s) with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 2000 s (33.33 M)



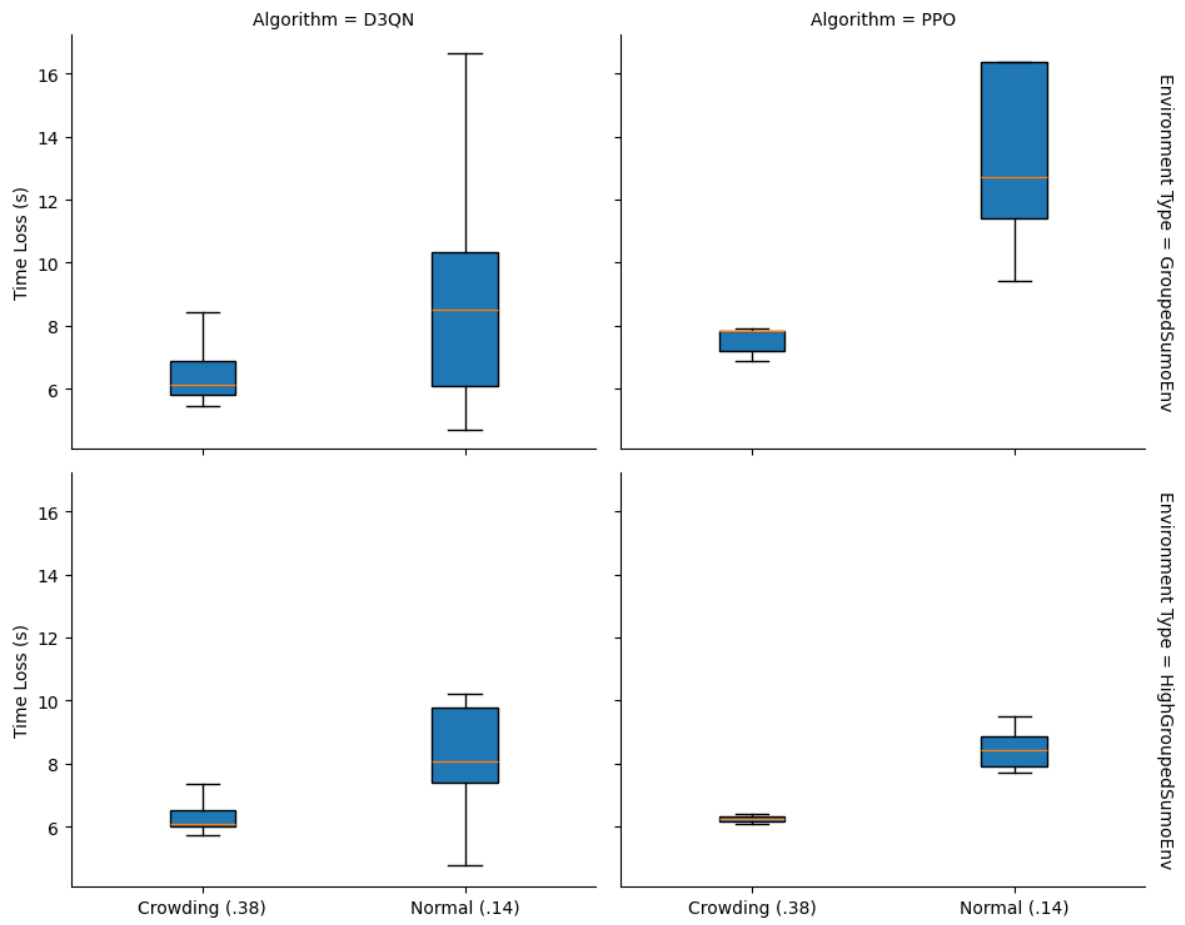
Performance Comparison (Using Reward : Proposed)
 Waiting Car with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 2000 s (33.33 M)



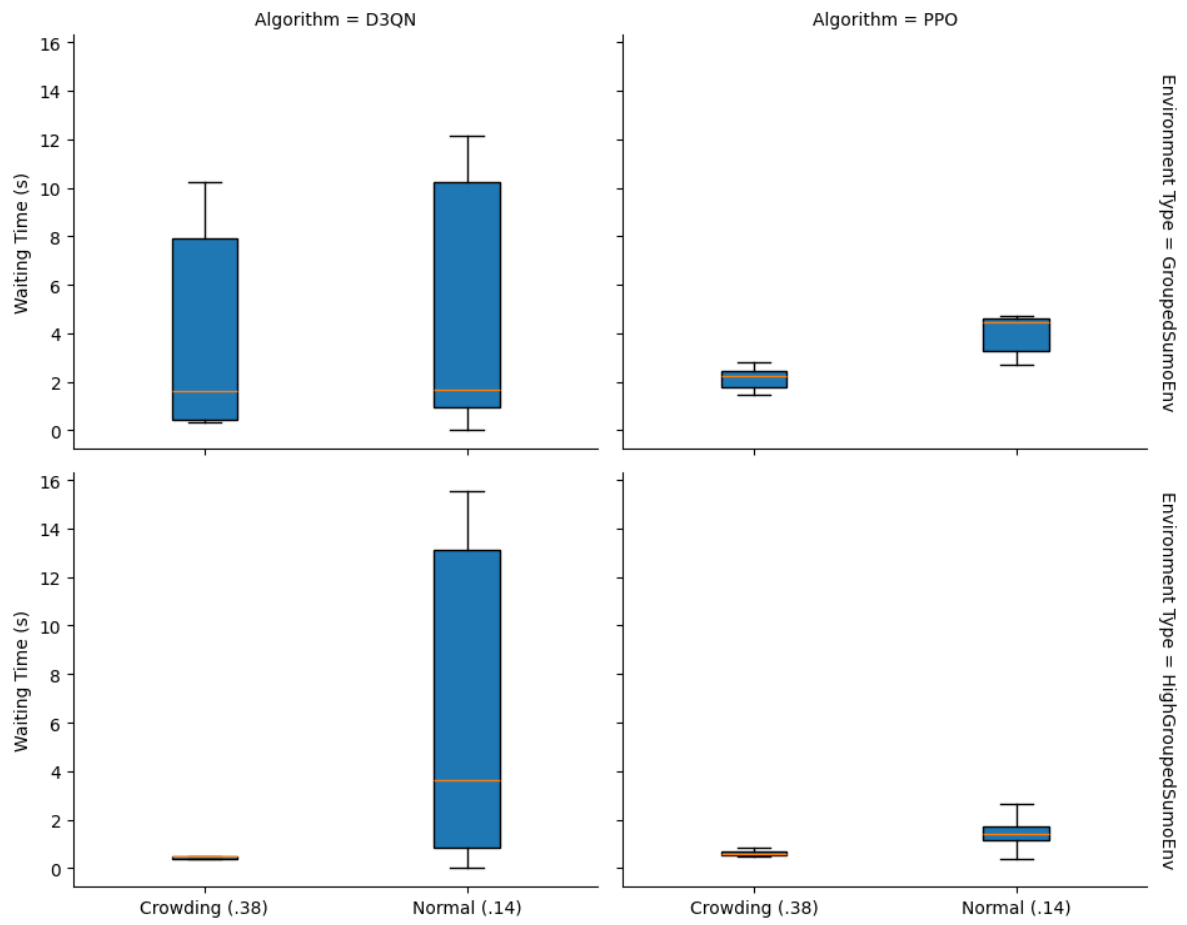
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 2000 s (33.33 M)



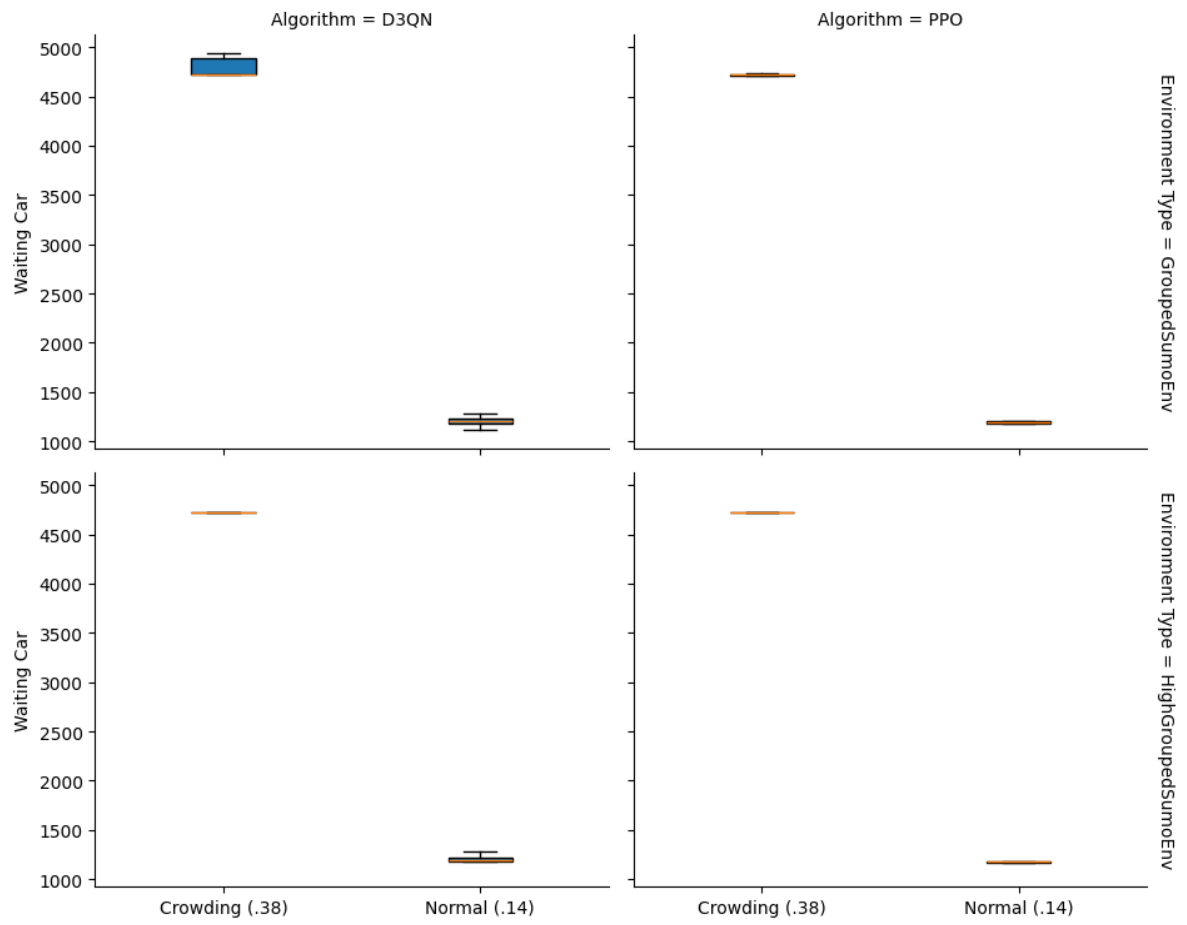
Performance Comparison (Using Reward : Proposed)
Time Loss (s) with Traffic Scale on Area2 (Mosheer Ismail)
Plotted at time 2000 s (33.33 M)



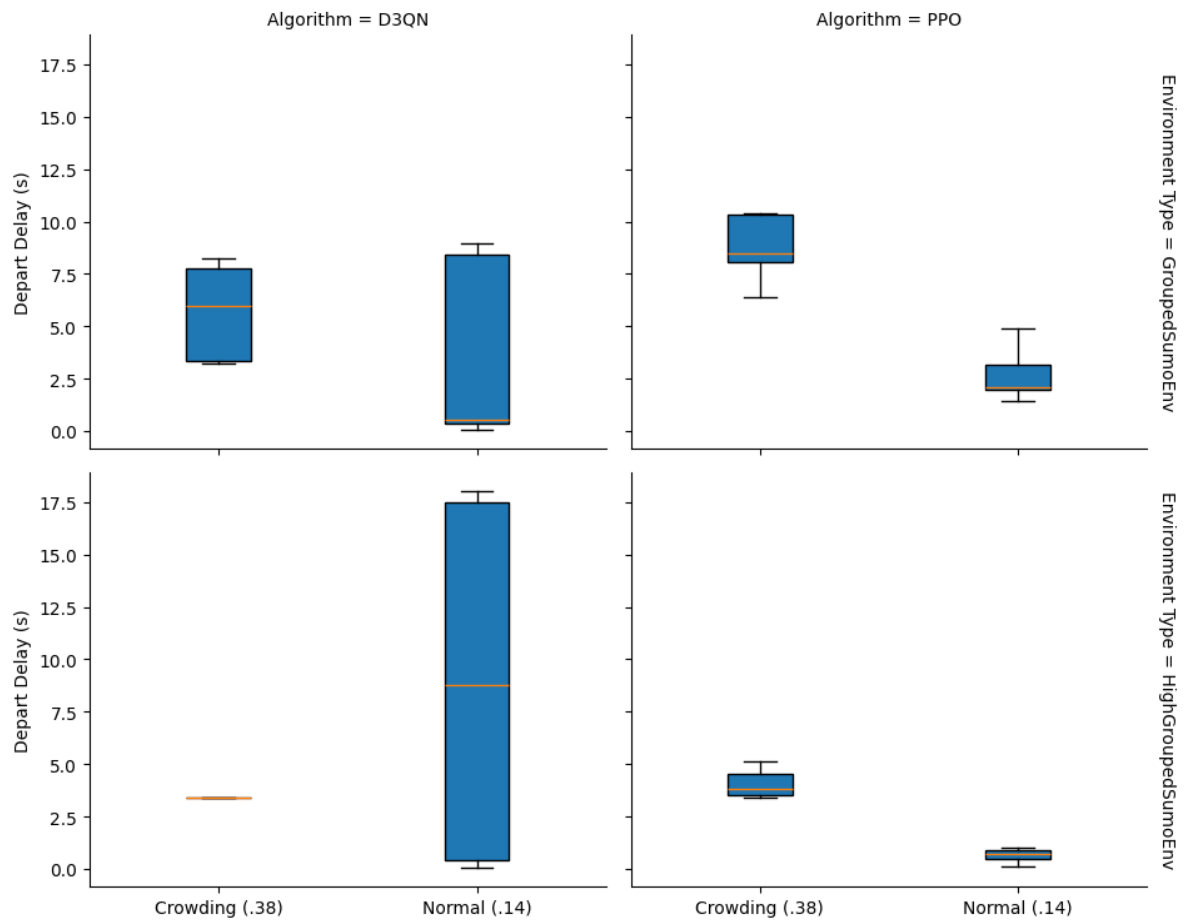
Performance Comparison (Using Reward : Proposed)
 Waiting Time (s) with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 4000 s (66.67 M)



Performance Comparison (Using Reward : Proposed)
 Waiting Car with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 4000 s (66.67 M)



Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) with Traffic Scale on Area2 (Mosheer Ismail)
 Plotted at time 4000 s (66.67 M)



Performance Comparison (Using Reward : Proposed)
Time Loss (s) with Traffic Scale on Area2 (Mosheer Ismail)
Plotted at time 4000 s (66.67 M)

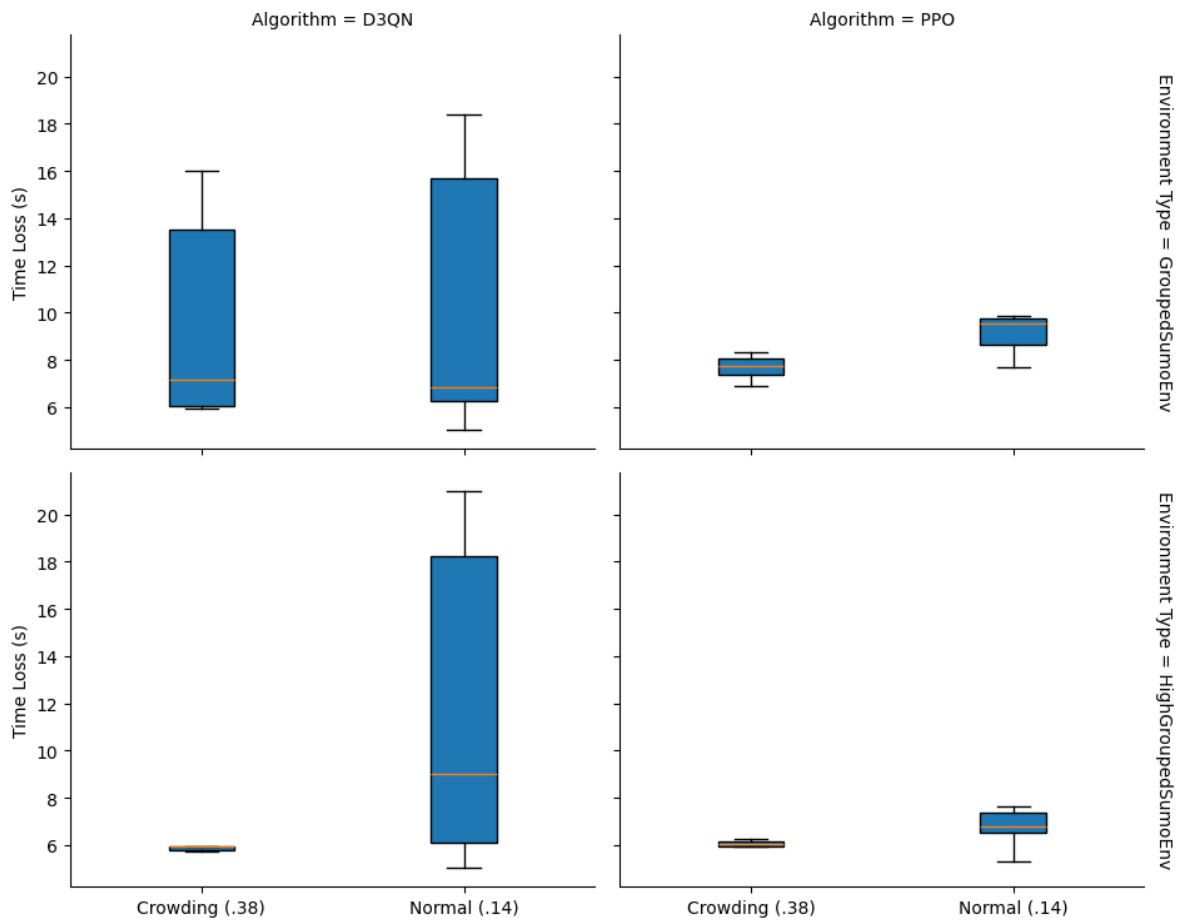
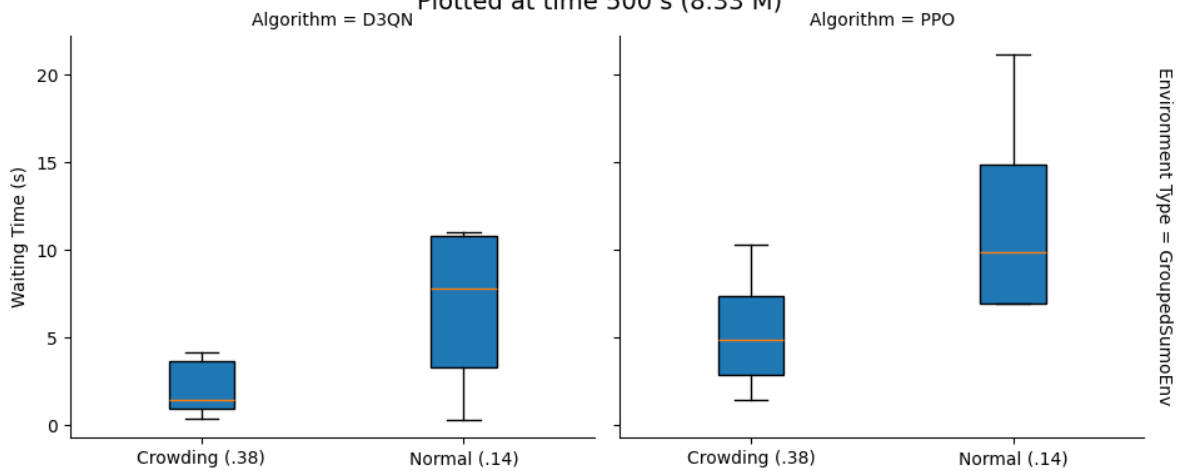
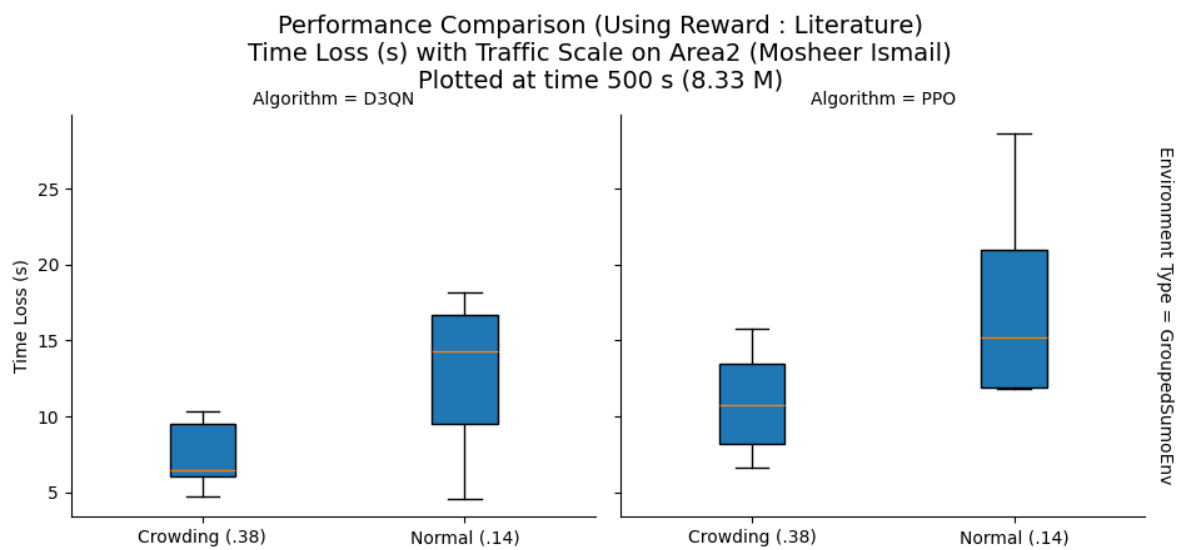
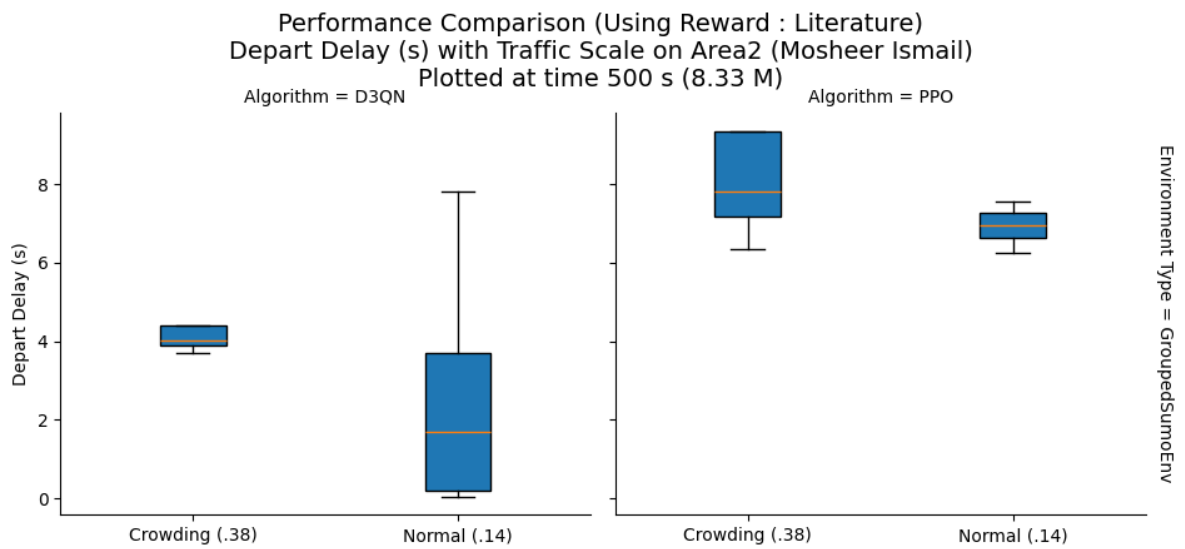
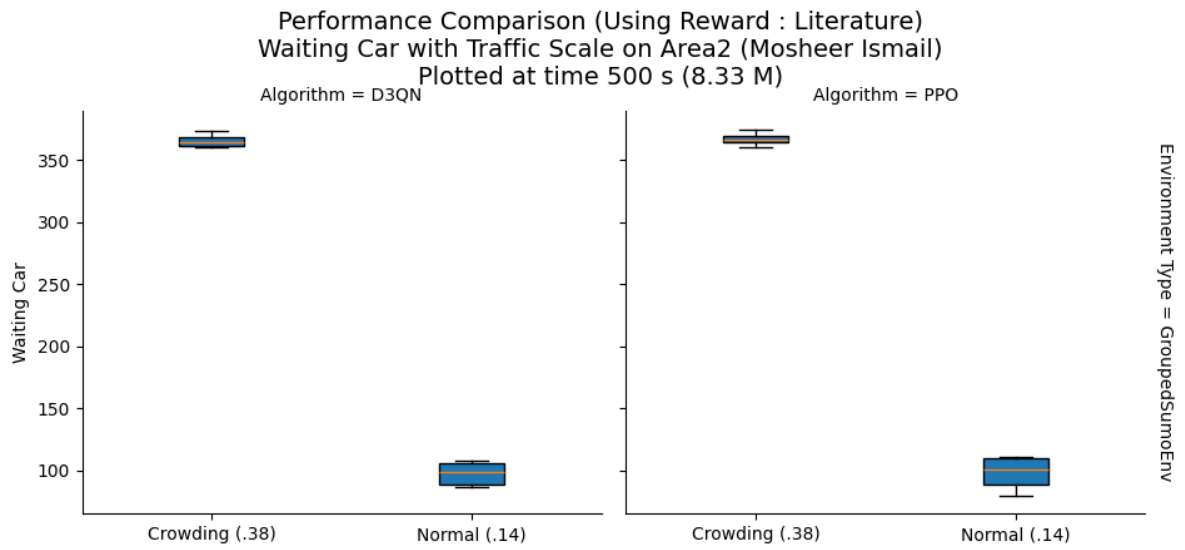


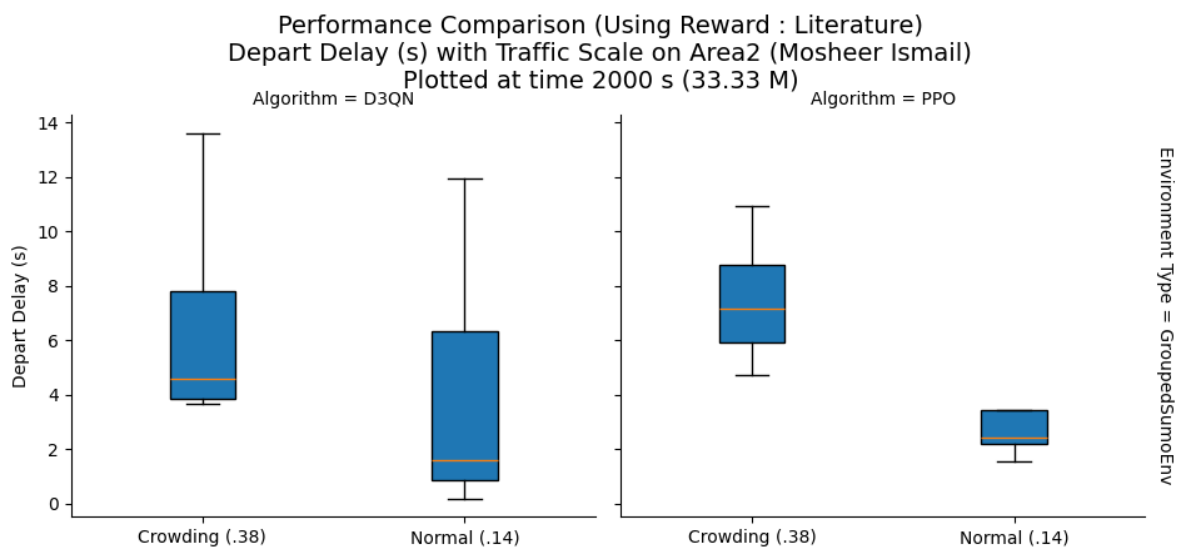
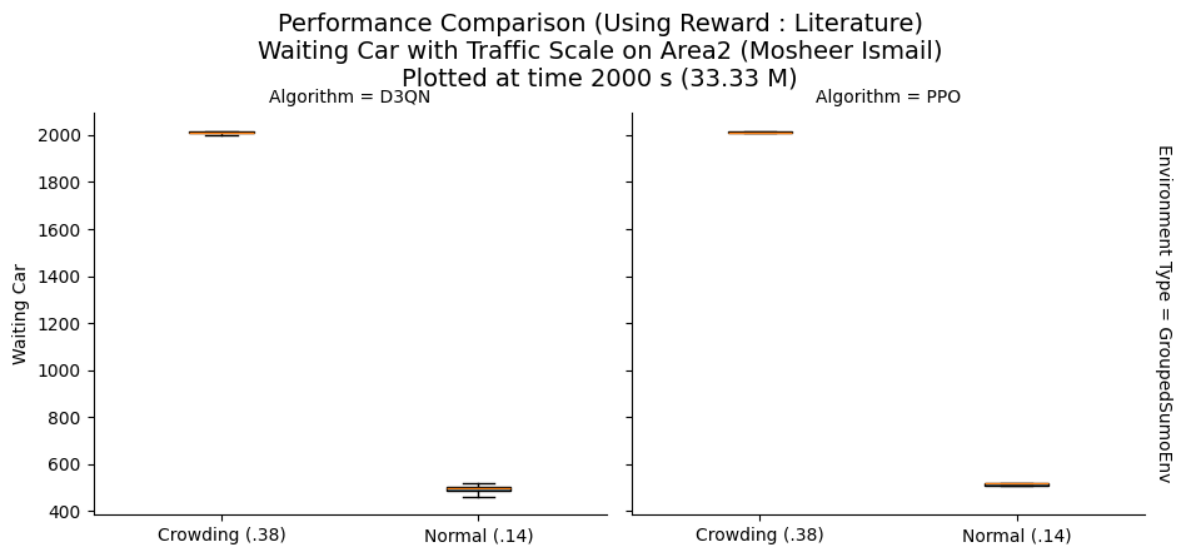
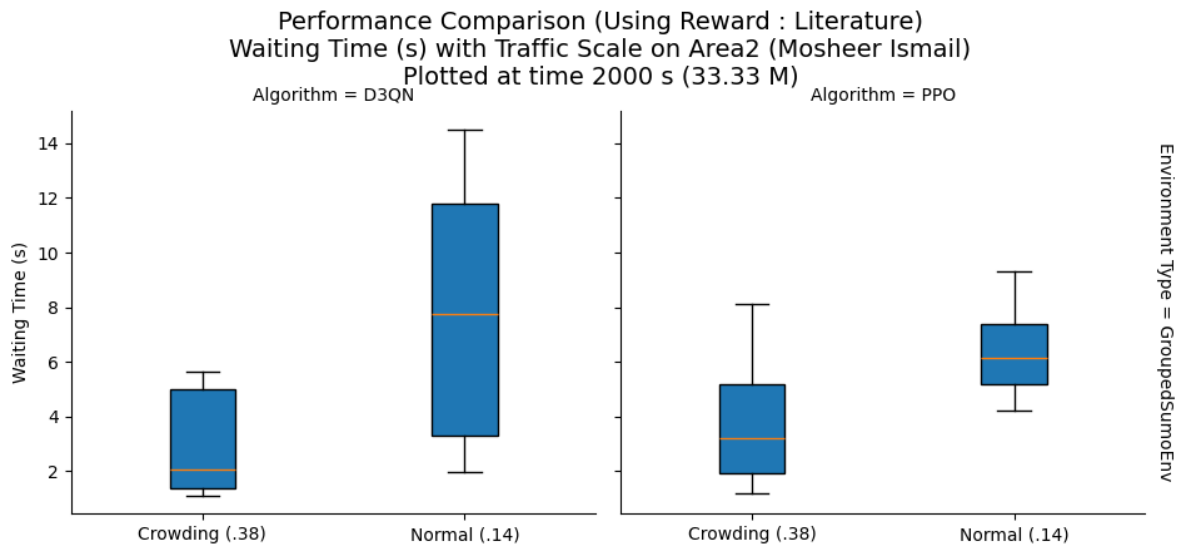
Figure 0.42 Results with proposed reward at V2.0.0

5.4.2.3.3 Results with literature reward

Performance Comparison (Using Reward : Literature)
Waiting Time (s) with Traffic Scale on Area2 (Mosheer Ismail)
Plotted at time 500 s (8.33 M)







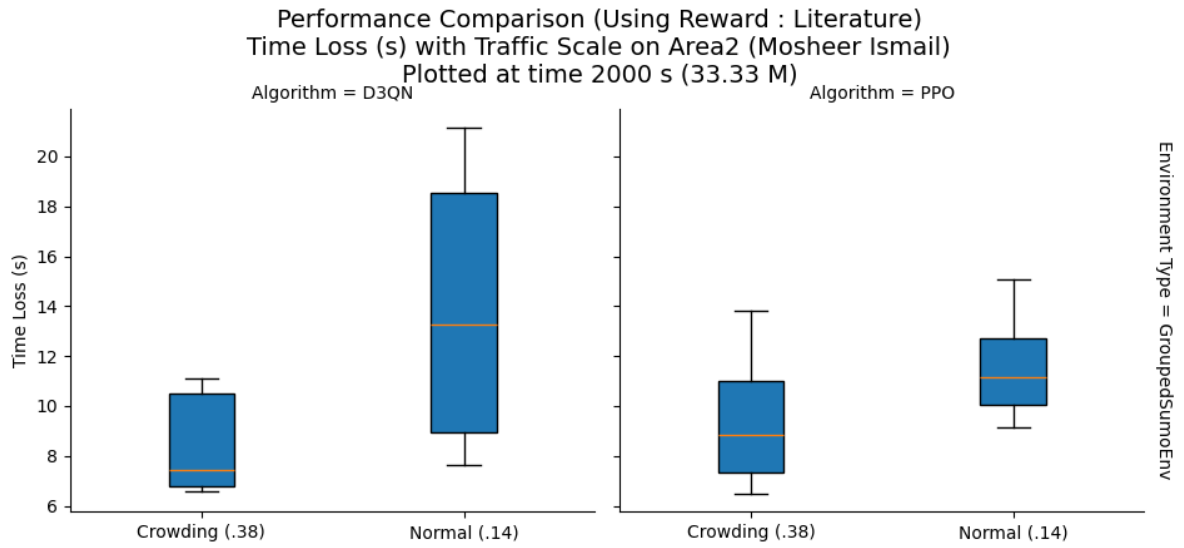


Figure 0.43 Results with literature reward at Version 2.0.0

5.4.2.4 Limitation of Version 2.0.0

Although the version showed improvements in results, some issues were found:

- Prolonged training durations (up to six hours) were recorded in some experiments without noticeable improvement in performance.
- Departure delay, time loss, and vehicle waiting values were found to be unrealistically low, while the number of waiting cars were very large indicating that most cars are not entered the map particularly in the Mosheer Ismail scenario.

In the following version, benchmarking with SUMO-RL will be conducted, callback mechanisms will be introduced, SUMO parameters will be adjusted to enhance realism, and reward normalization will be incorporated.

5.4.3 Version 2.0.1

In this section modified version (V2.0.1) of project is detailed.

5.4.3.1 Key Modifications

Up to this point, the results have not been very logical. Therefore, further improvements were attempted.

1. Additional normalization was added to the reward function to improve accuracy and achieve better scaling. This modification is referred to as the *proposed reward*.
2. The SUMO-RL package was introduced to enhance the benchmarking and ensure compatibility with established implementations.

3. Since the training process required excessive time (up to approximately six hours) and often showed limited improvement, early stopping was implemented. By doing so, the model was trained effectively within a smaller number of episodes, reducing the total training time to just a few minutes.
4. During experiments, it was observed that the departure delay was very low and that most vehicles did not pass through the environment as expected.

To address this issue, the SUMO parameters were modified to make the simulation more realistic. For more accurate results, Optuna optimization was re-run after these modifications.

A minor bug was observed during experiment execution: the max step was set to 100, which is smaller than the maximum duration plus yellow phase, which may cause the system to operate faster than intended.

5.4.3.1.1 Normalizing Reward and Testing It

$$R(s) = \alpha \cdot \left(\frac{\underline{v}}{v_{max}} \cdot \frac{1}{1 + \sigma_v^2} \right) - \beta \left(\frac{\underline{w}}{w_{max}} \cdot \frac{1}{1 + \sigma_w^2} \right) + \gamma \left(\frac{w_1 \cdot \underline{t}}{t_{max}} - \frac{w_2 \underline{q}}{q_{max}} \right)$$

Equation 0.7 Modified Proposed Reward (Normalized)

Where:

$\underline{v}$ is average speed, σ_v^2 is variance of speed, $\underline{w}$ is the average waiting time, σ_w^2 is variance of waiting time, $\underline{t}$ is average of throughput, $\underline{q}$ is average of queue length.

$w_1, w_2, \alpha, \beta, \gamma$:is weight for terms, configured when testing the reward ($w_1 = .6, w_2 = .4, \alpha, \beta, \gamma = 1$), $v_{max}, w_{max}, q_{max}, t_{max}$ is max estimated speed, waiting time, queue length and throughput accordingly, $v_{max} = 60, w_{max} = 120, q_{max} = 30, t_{max} = 50$.

Key Modification

- Replaced log by normalization with estimated max value
- Refined the reward function to produce more logically consistent values by decomposing the single speed term into two multiplicative components.
- Retuned the weights

Reward Function Validation

To validate the behavior of the proposed reward function, six representative traffic states were defined and evaluated. Each state vector corresponds to specific traffic conditions (e.g.,

vehicle count, waiting time, and flow indicators). The reward values were computed using the function `reward_proposed(state)` to confirm its consistency with the intended design principles.

Defined Test States

Table 0.13 Testing modified proposed reward for V2.0.1

State	State Explain	Description	Expected Behavior	Observed Reward
State 1: (0, 0, 0, 0, 0, 0)	No speed, no variance, no waiting, no throughput, no queue, empty lanes → represents total standstill because no vehicles exist or move.	Represents complete inactivity, where no vehicles are moving.	A base penalty should be applied due to total stagnation.	0.0000
State 2: (10, 1, 0, 1, 0, 0)	Moderate avg speed with stable speeds, no waiting, small variance, zero throughput/queues → light traffic beginning to flow.	Indicates some traffic flow at moderate speed.	Reward should increase slightly, though waiting penalties remain dominant.	0.0833
State 3: (5, 1, 1, 1, 20, 1, 0)	Medium speed, consistent flow, small waiting, high throughput, short queues, no occupancy issues → efficient controlled movement.	Represents efficient traffic flow.	A positive reward is expected, reflecting optimal conditions.	0.2642
State 4: (10, 0, 5, 0, 10, 2, 0)	High speed but zero variance, moderate waiting time, decent throughput, moderate queues → balanced but mildly congested.	Balanced state with moderate congestion.	The reward should approach neutrality.	0.2183
State 5: (1, 1, 20, 20, 0, 20, 1)	Very low speed, huge waiting, no throughput, long queues, high occupancy → classic heavy	Highly congested scenario.	A strong penalty should be applied to reflect poor conditions.	-0.2663

	congestion/jam condition.			
State 6: (15, 1, 0.5, 0.5, 30, 1, 0.1)	High stable speed, almost no waiting, strong throughput, tiny queue, low occupancy → ideal traffic performance.	Represents optimal flow and minimal waiting.	The highest reward value is expected.	0.4689

Analysis

The obtained results confirm that the **reward function behaves as intended**. Scenarios representing efficient flow received higher positive rewards, while congested or idle conditions were penalized accordingly. Specifically, *State 6* produced the maximum reward (0.4689), demonstrating the system's ability to incentivize smooth traffic flow, whereas *State 5* yielded the minimum reward (−0.2663), correctly identifying severe congestion.

Sensitivity analysis

- **Weight Dominance:** The parameters act as linear multipliers. An increase in any of the primary weights α, β, γ will proportionally increase the importance of its corresponding term.
- **Velocity vs. Workload Trade-off:** The ratio α, β is paramount. If α is significantly larger than β , the agent will prioritize high velocity even at the cost of high waiting time.
- **Terminal Reward:** The term γ links the policy to the long-term goal defined by the t/q trade-off. A high γ means the agent will heavily prioritize throughput and queue length.

And an anomaly observed later is that the derivative $\frac{\partial R}{\partial \sigma_w^2}$ is positive, indicating that the agent is rewarded for instability σ_w^2 [6.2 Future Work].

5.4.3.2 Modified Environmental Design

As a result of the recent modifications:

- Only a limited number of episodes were used, with a maximum of **350**.
- For the SUMO experiments, the built-in action space was utilized; therefore, the parameters of GroupedSumoEnv and HighGroupedSumoEnv were not required.

- The **scale calculation** was improved by referencing TomTom traffic data [t] instead of using the previous estimation of $10 * 0.38$, which lacked a clear source. The new definition adjusts the scale by either **0.14** or **0.38** above the default SUMO scale ($1 + \text{scale}$).
- The **time calculation** was refined to better represent realistic traffic conditions while reducing excessive simulation time.

5.4.3.2.1 Environmental Experiments

Table 0.14 Modified Environmental Experiments for Version 2.0.1

Parameter	Meaning	Real meaning	Range for experiment
Area	The dataset or environment used	Country-specific traffic data	Area2 (Mosheer Ismail) – Area 3 (San Stefano)
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	Literature - Proposed
Environment type	Type of SUMO environment used	Level of traffic control detail and intersection grouping	GroupedSumoEnv HighGroupedSumoEnv
Algorithm	RL algorithm applied	Different learning strategies used to optimize traffic control	D3QN -PPO
Episodes	Number of training episodes	Number of training cycles (longer = more learning)	350
Max sumo steps	Maximum number of simulation steps in SUMO	The number of times the experiment is repeated to compare environments and ensure consistency	3000
Traffic Scale	Traffic load intensity	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowding (.38)

Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2,3
--------------	---------------------------------	--	---------

5.4.3.2.2 Sumo-RL experiments

Table 0.15 Sumo-RL Experiment Design for Version 2.0.1

Parameter	Meaning	Real meaning	Range for experiment
Area	The dataset or environment used	Country-specific traffic data	Area2 (Mosheer Ismail) – Area 3 (San Stefano)
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	Literature - Proposed
Algorithm	RL algorithm applied	Different learning strategies used to optimize traffic control	D3QN -PPO
Episodes	Number of training episodes	Number of training cycles (longer = more learning)	350
Max sumo steps	Maximum number of simulation steps in SUMO	The number of times the experiment is repeated to compare environments and ensure consistency	3000
Traffic Scale	Traffic load intensity	Adjusts the number of vehicles generated in the simulation	Normal (.14) - Crowding (.38)
Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2,3

--	--	--	--

5.4.3.2.3 Metrics

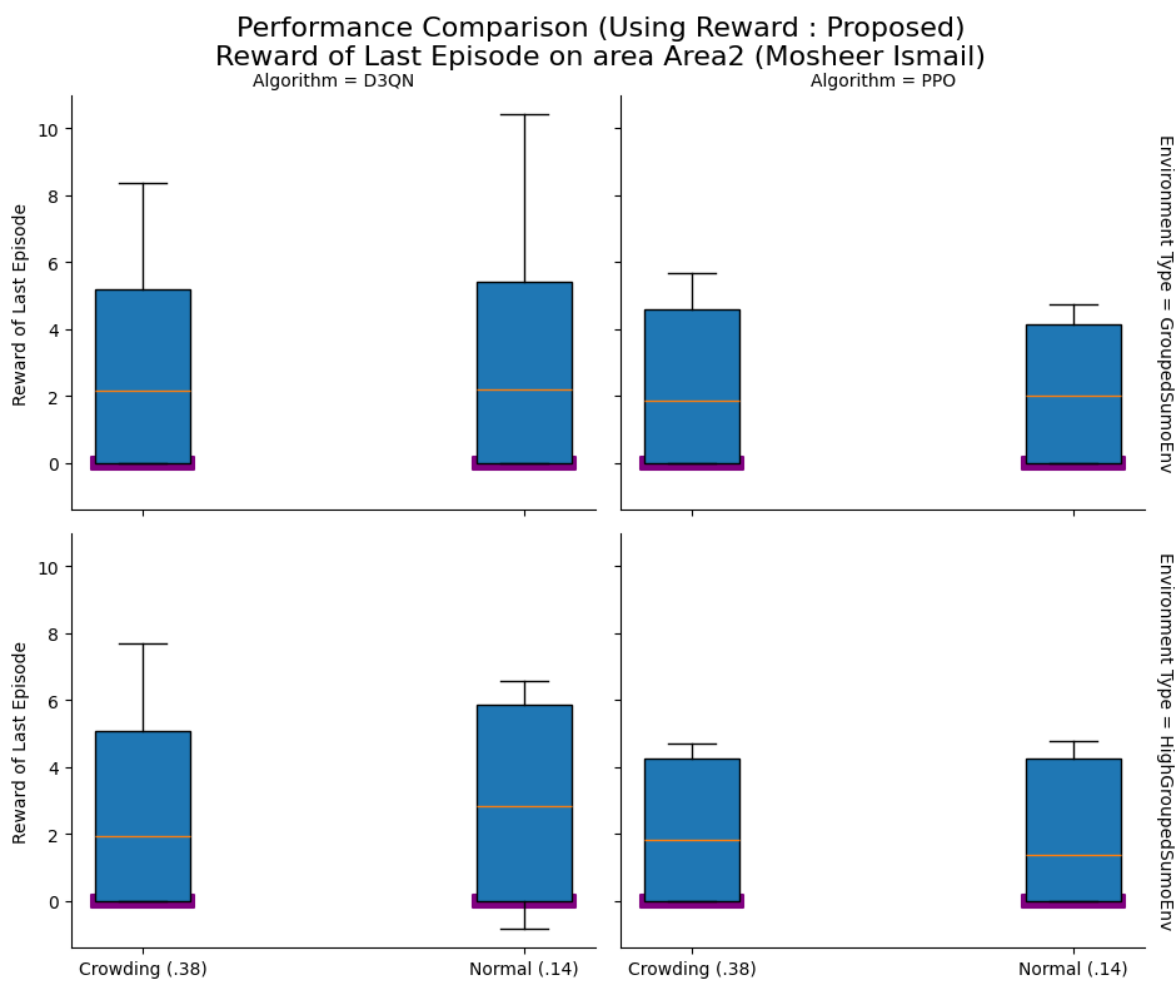
Table 0.16 Modified Metrics for Version 2.0.1

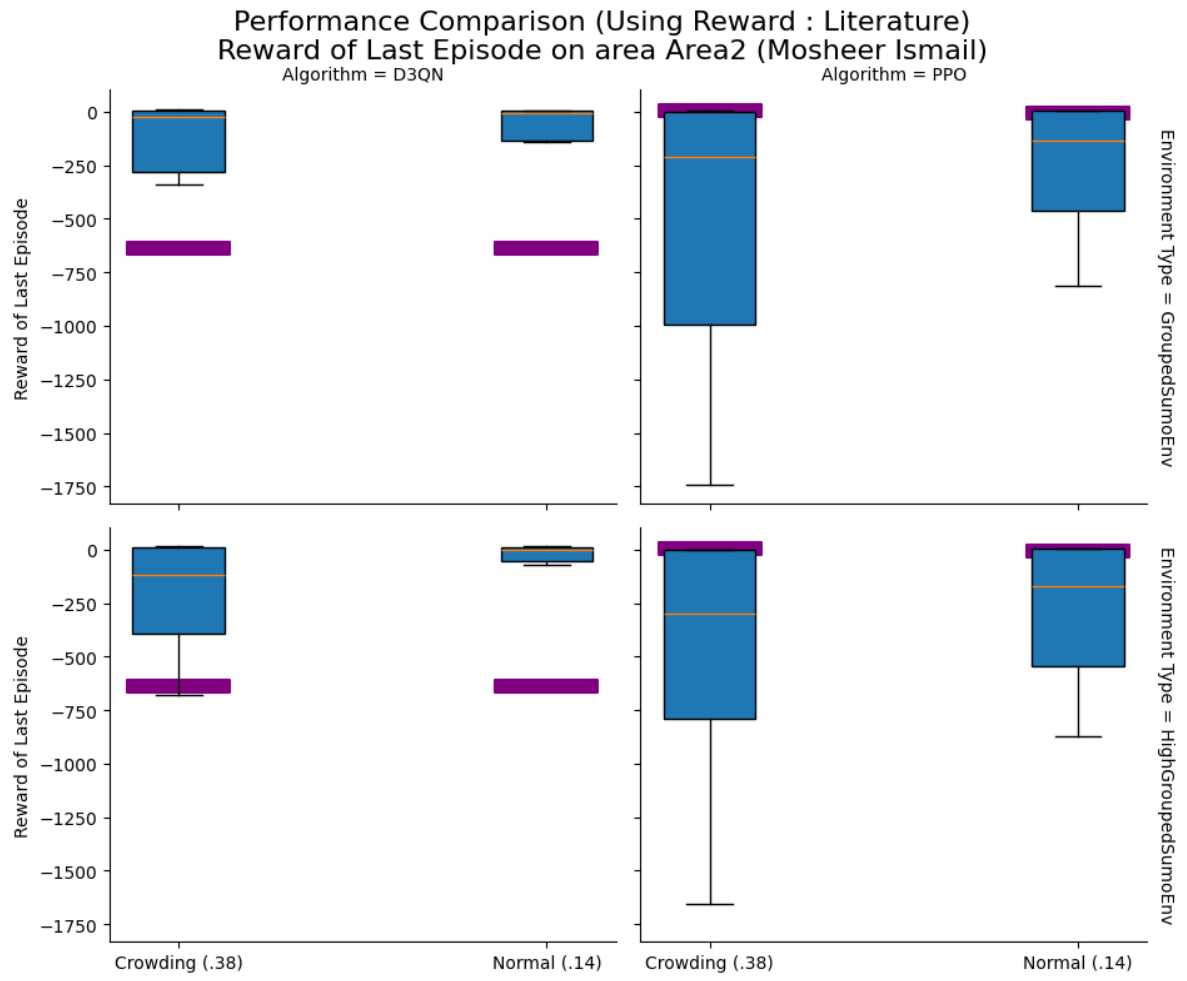
Metric	Defination	Direction
Reward of last episode	The total accumulated reward obtained by the agent during the final episode of training	Max
Average Reward for evaluated episodes	The mean of total rewards collected across multiple evaluation episodes.	Max/Equal Reward of last episode
Derivative of reward	Measures on how reward is changing over episodes — tells if the agent is improving.	Max
Waiting Time	The average time spent standing (seconds)	Min
Speed	The average trip speed (m/s)	Max
Depart Delay	The average time vehicles had to wait before starting their journeys (seconds)	Min
Time loss	The average time lost due to driving slower than desired (including waiting time) (seconds)	Min
Waiting Car	Number of vehicles with delayed insertion that were still waiting for insertion at simulation end	Min
Total Time	The total time taken to complete the full training or evaluation process, from start to finish. (Minutes)	-
Reward on Final Test	The average reward obtained during experiments conducted over a 3-hour simulation period.	Min
Waiting Time on Final Test	The average waiting time (in seconds) recorded during 3-hour experiments.	Min
Speed on Final Test	The average trip speed (in meters per second) observed during 3-hour experiments.	Max
Depart Delay on Final Test	The average departure delay (in seconds), represents the time vehicles waited before starting their journeys during 3-hour experiments.	Min
Time loss on Final Test	The average time loss (in seconds) resulting from vehicles traveling slower than their desired speed, including waiting time, during 3-hour experiments.	Min
Waiting Car on Final Test	The number of vehicles with delayed insertion remained uninserted by the end of the 3-hour simulation.	Min

5.4.3.3 Results

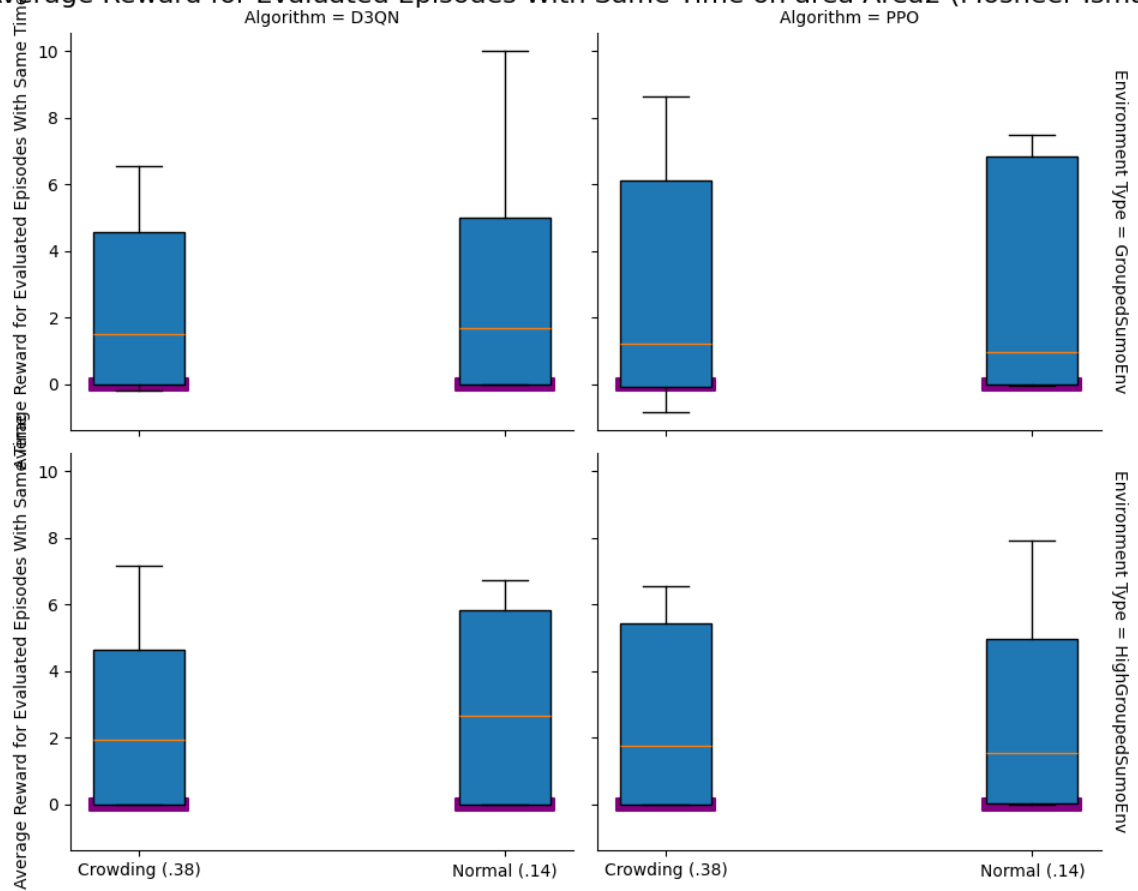
In the final version, the main objective was to make a direct comparison with the referenced SUMO-RL package (version 2.0.1) and to analyze the impact of the introduced modifications. The focus was placed on evaluating the overall system improvement and identifying configurations that achieved the most promising performance outcomes.

5.4.3.3.1 Results for Mosheer Ismail

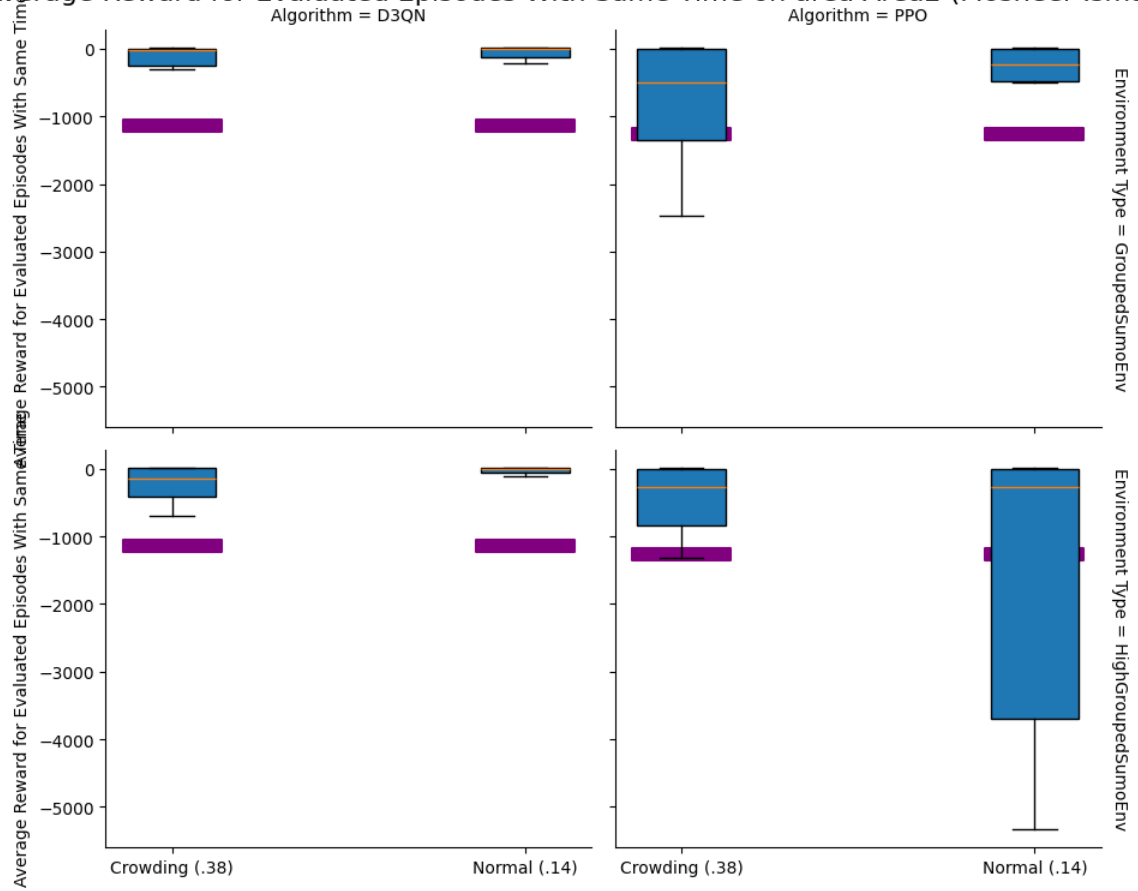




Performance Comparison (Using Reward : Proposed)
Average Reward for Evaluated Episodes With Same Time on area Area2 (Mosheer Ismail)

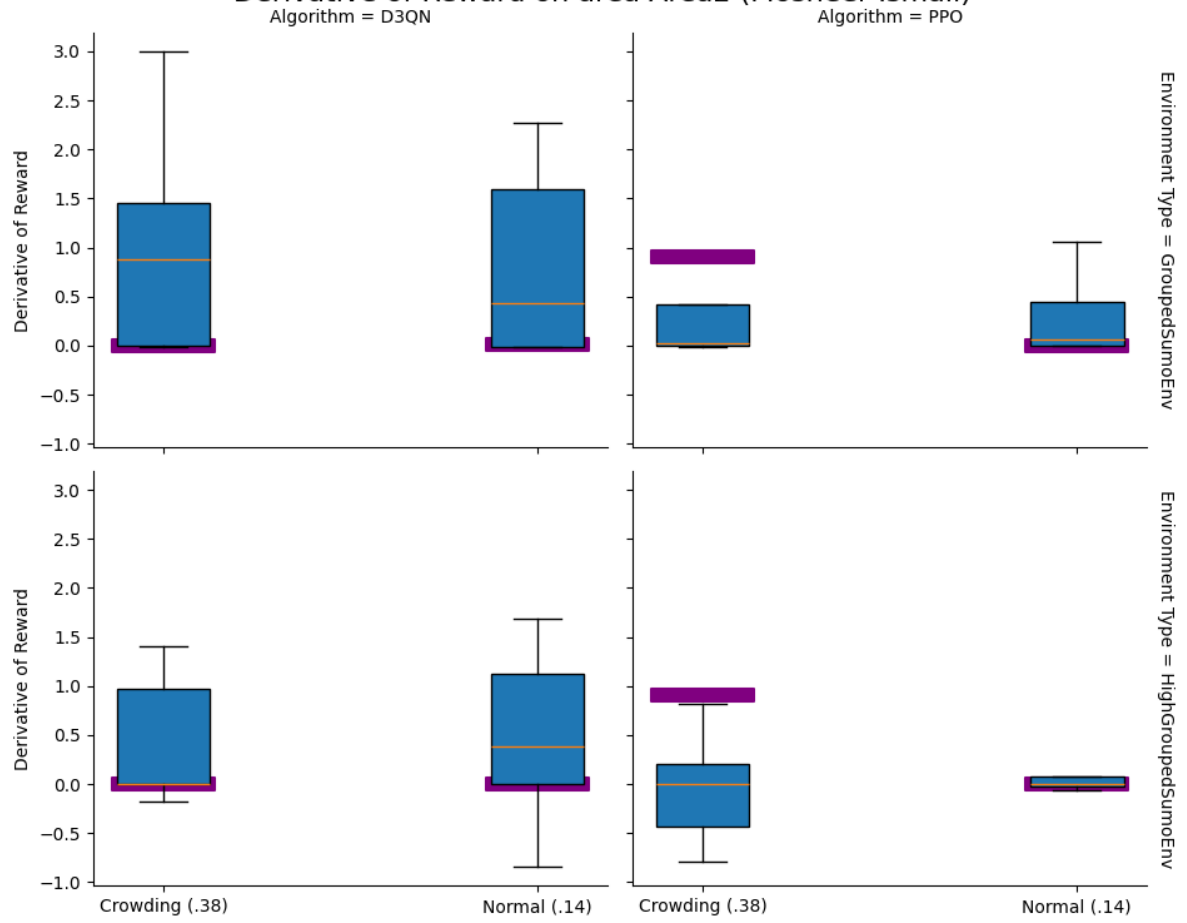


Performance Comparison (Using Reward : Literature)
Average Reward for Evaluated Episodes With Same Time on area Area2 (Mosheer Ismail)



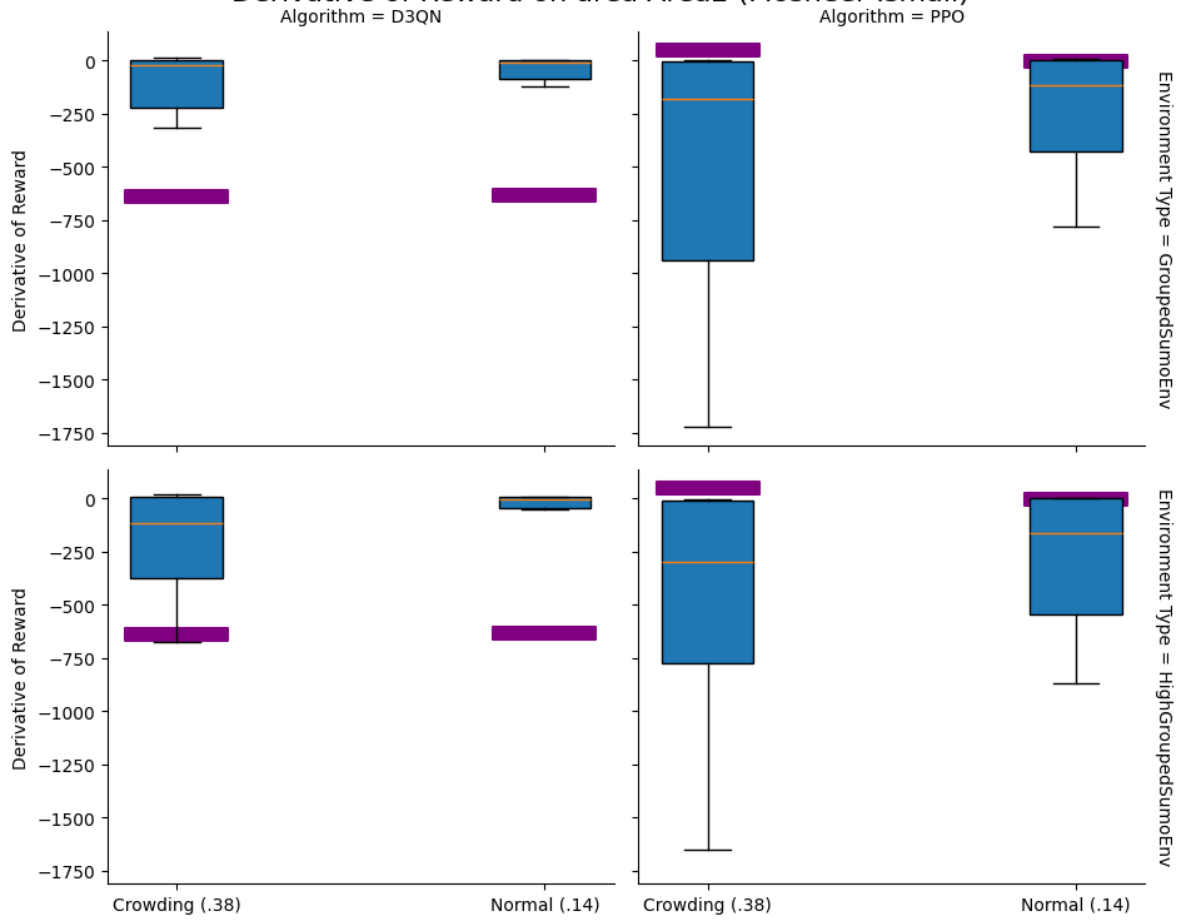
Performance Comparison (Using Reward : Proposed)

Derivative of Reward on area Area2 (Mosheer Ismail)

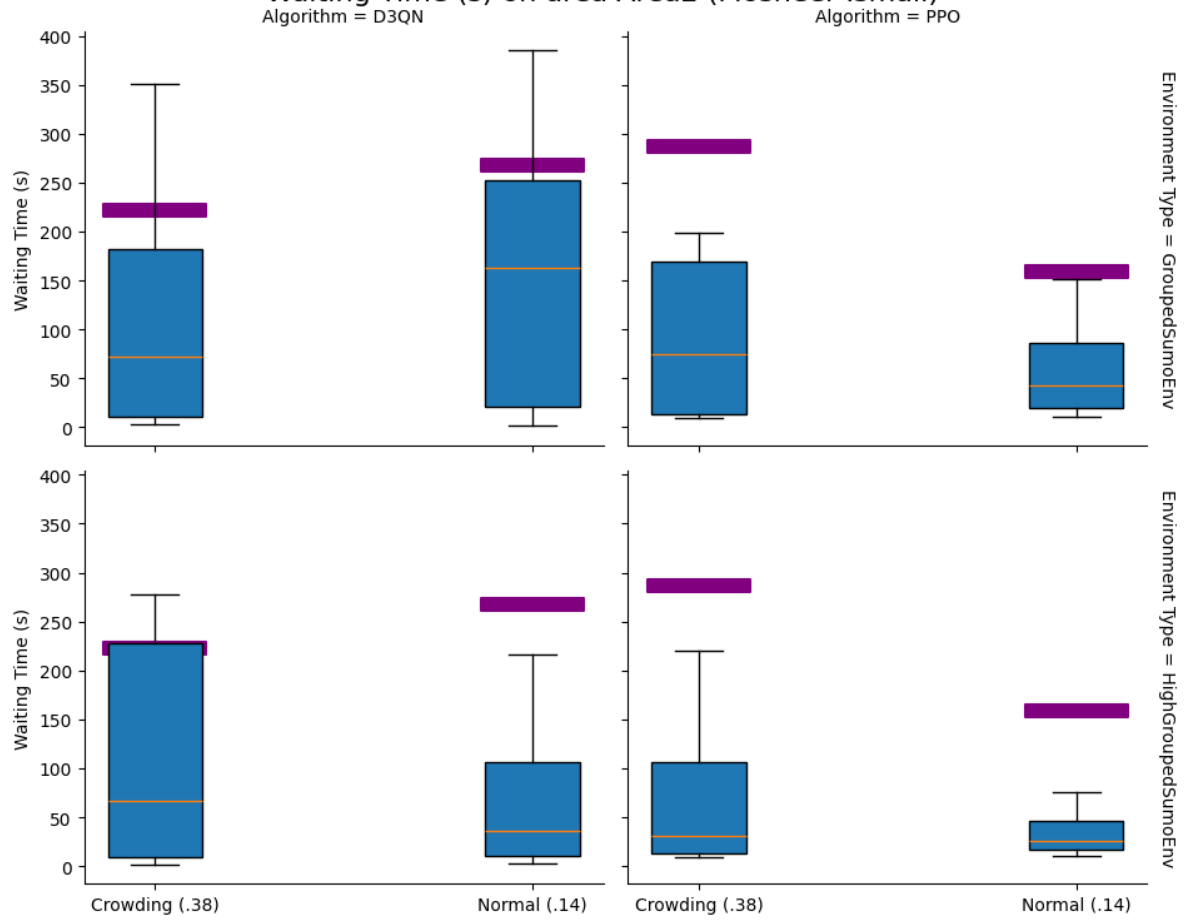


Performance Comparison (Using Reward : Literature)

Derivative of Reward on area Area2 (Mosheer Ismail)

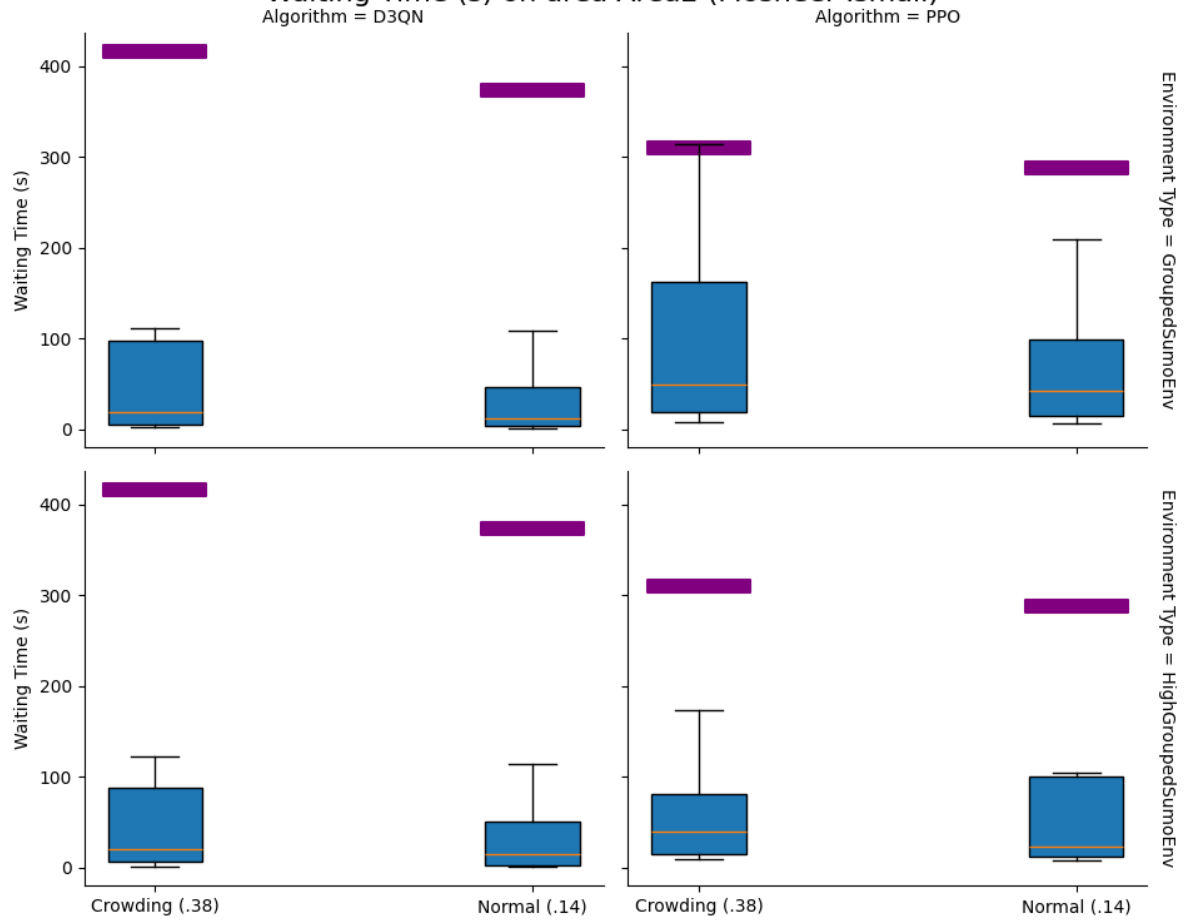


Performance Comparison (Using Reward : Proposed) Waiting Time (s) on area Area2 (Mosheer Ismail)

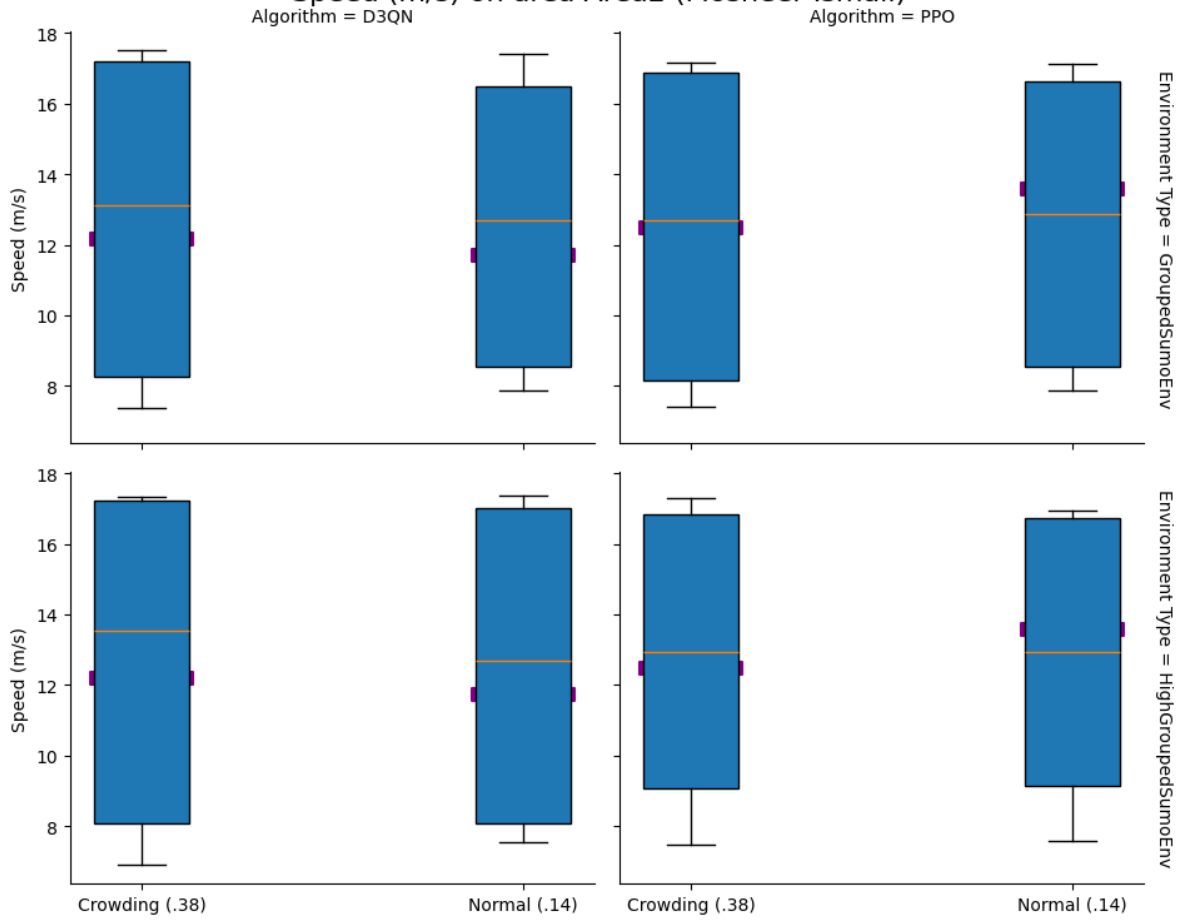


Performance Comparison (Using Reward : Literature)

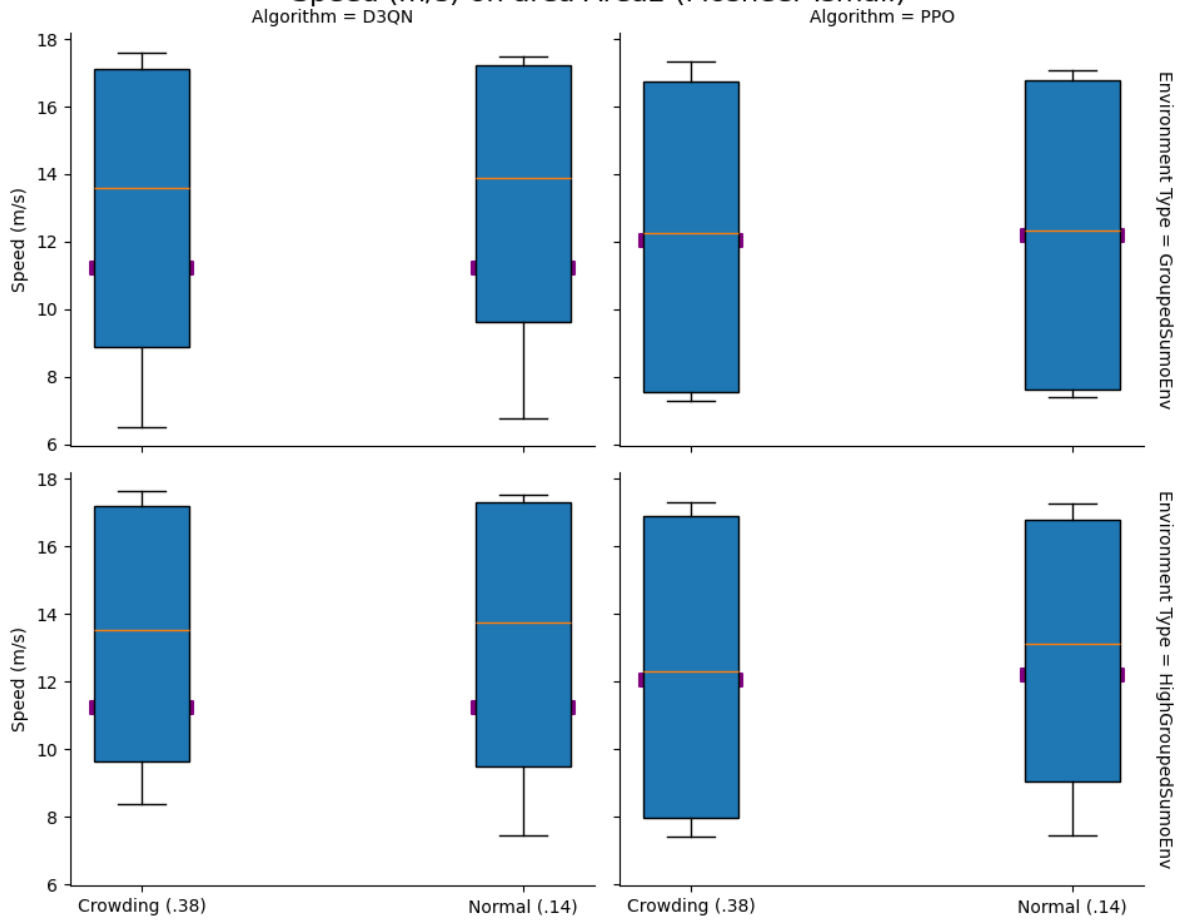
Waiting Time (s) on area Area2 (Mosheer Ismail)



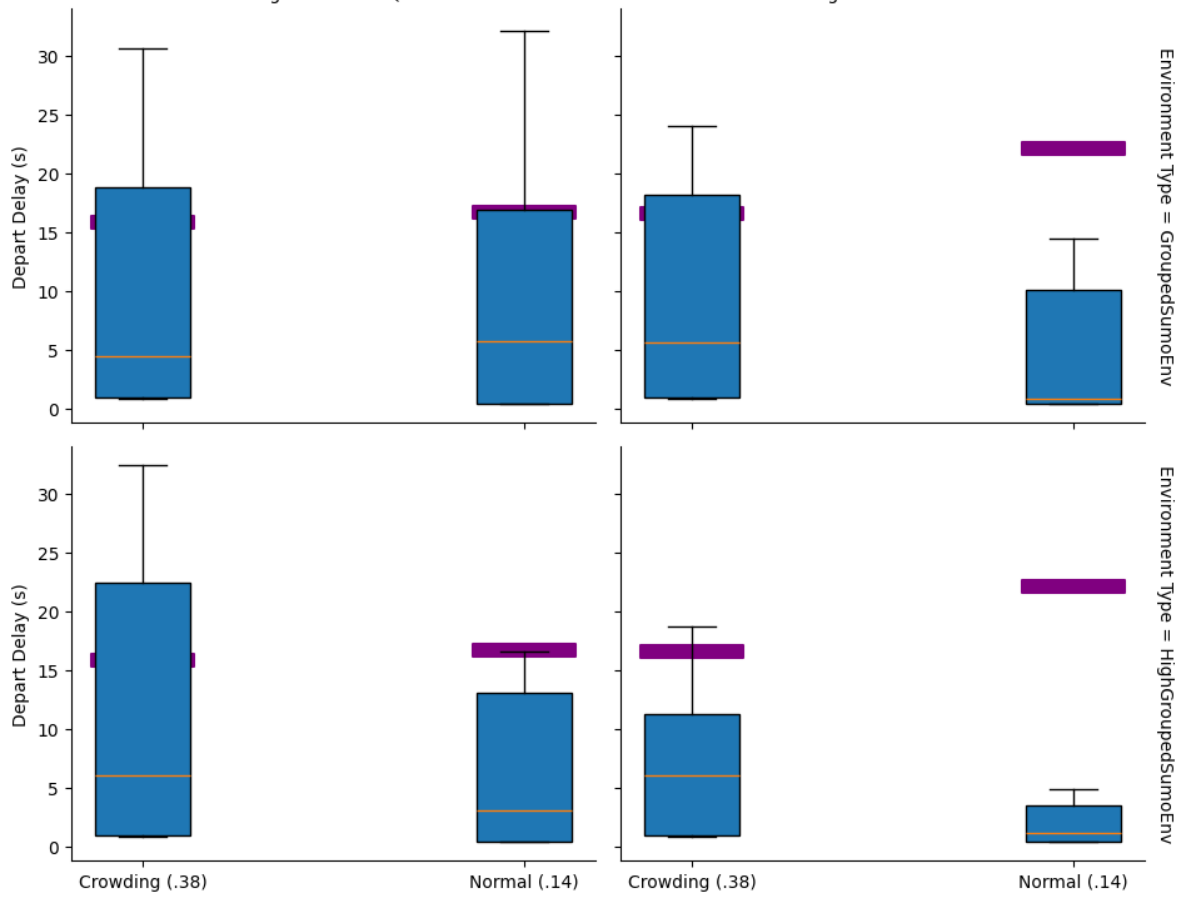
Performance Comparison (Using Reward : Proposed) Speed (m/s) on area Area2 (Mosheer Ismail)



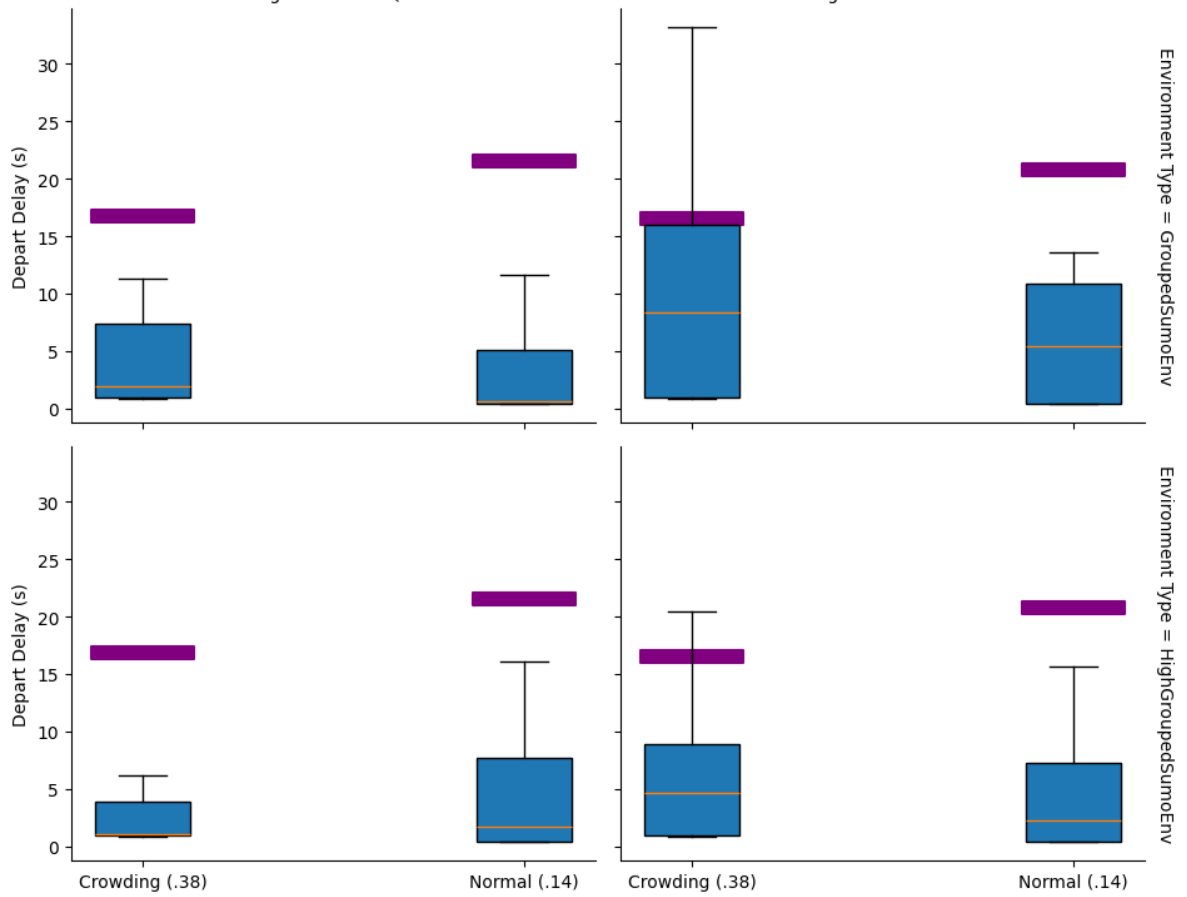
Performance Comparison (Using Reward : Literature) Speed (m/s) on area Area2 (Mosheer Ismail)



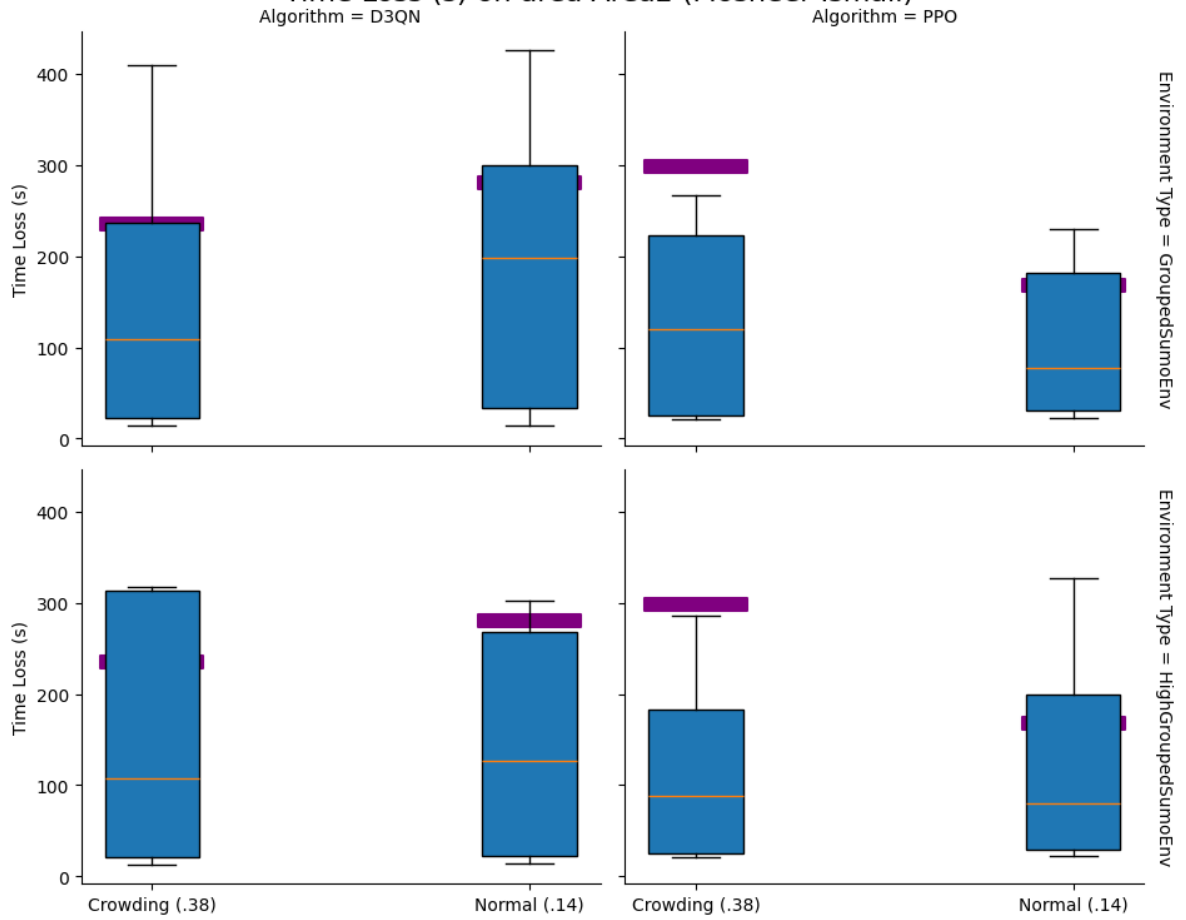
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



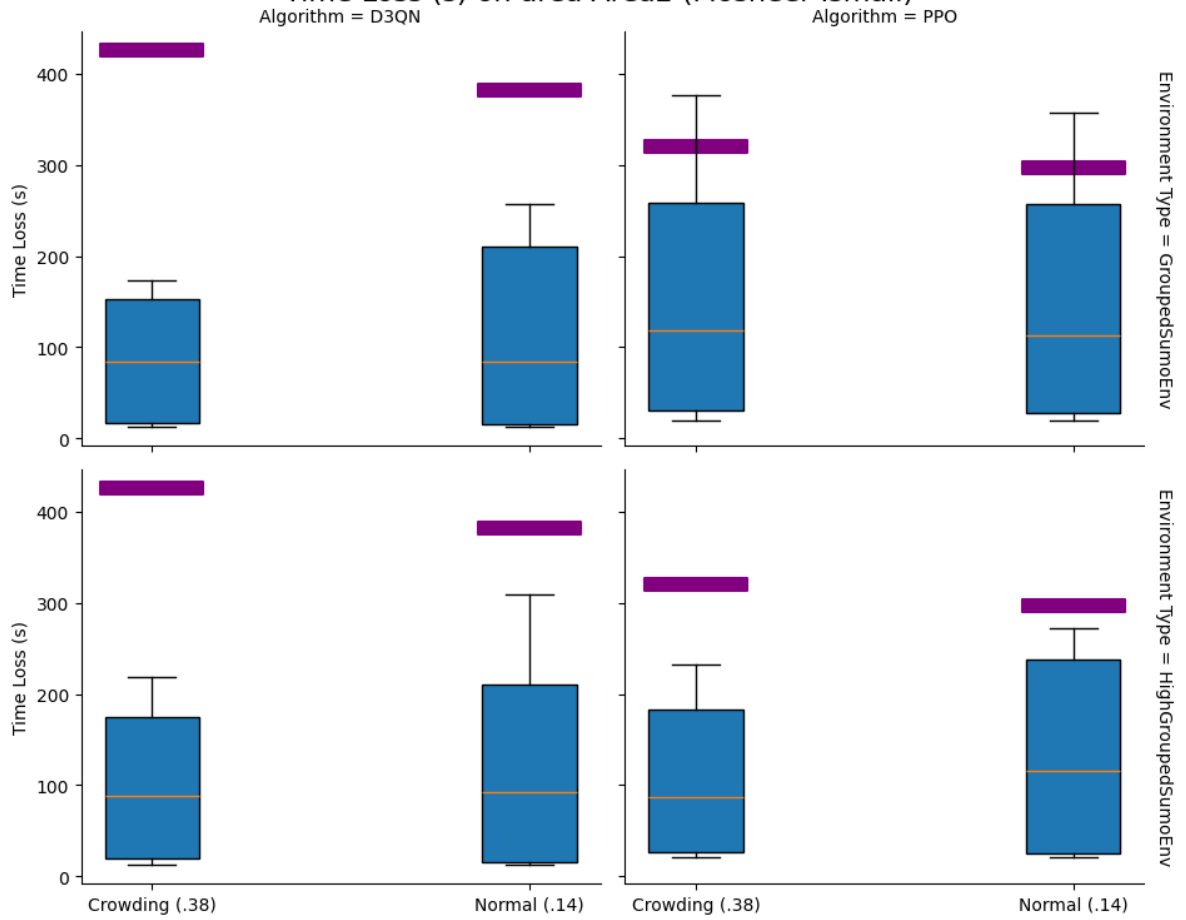
Performance Comparison (Using Reward : Literature)
 Depart Delay (s) on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



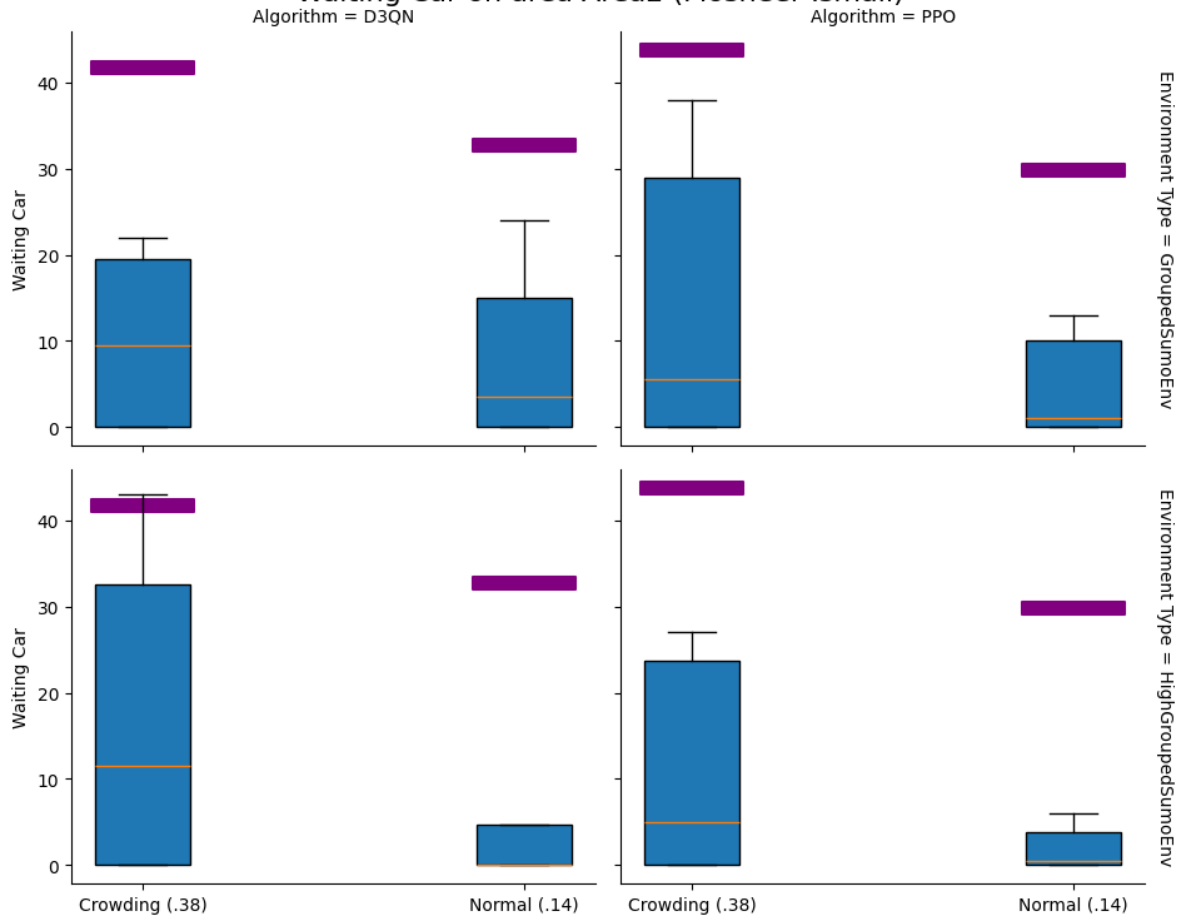
Performance Comparison (Using Reward : Proposed) Time Loss (s) on area Area2 (Mosheer Ismail)



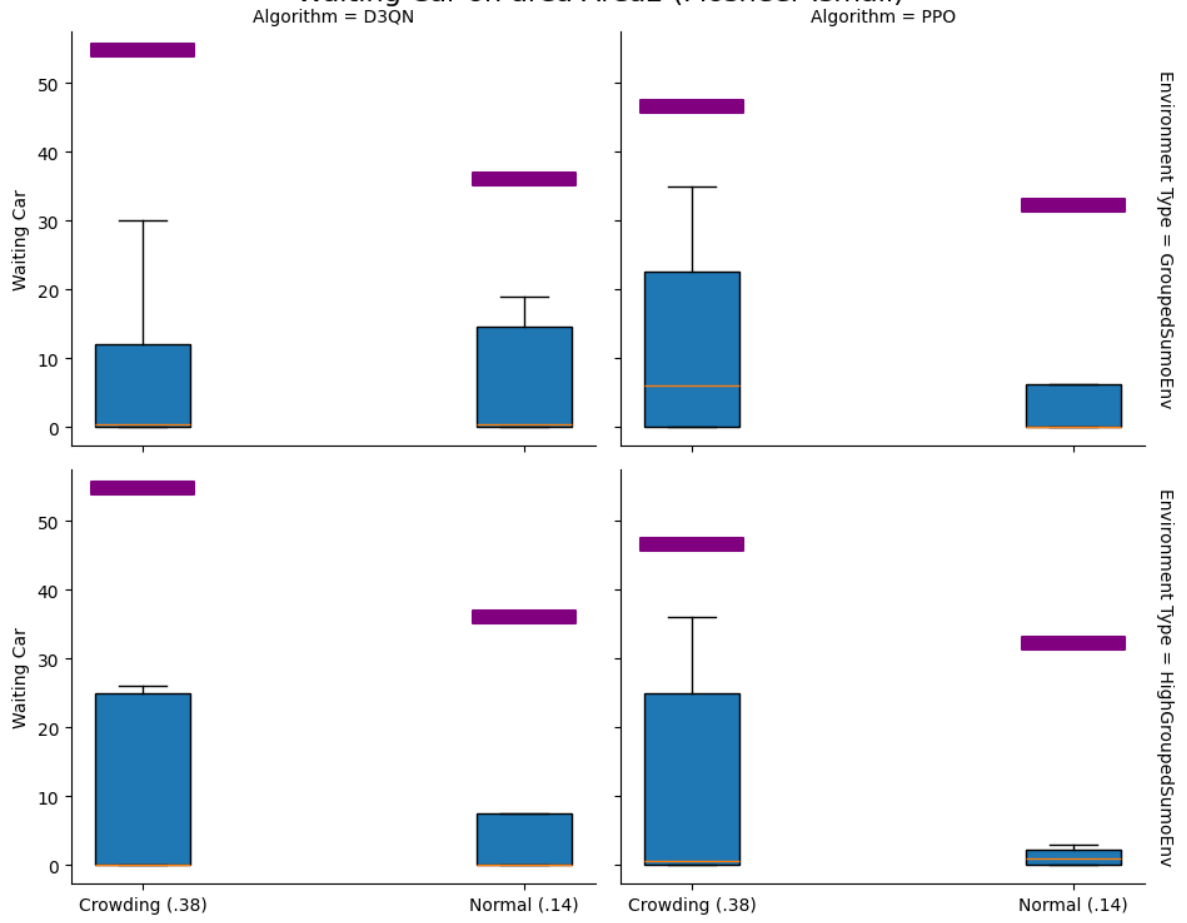
Performance Comparison (Using Reward : Literature) Time Loss (s) on area Area2 (Mosheer Ismail)



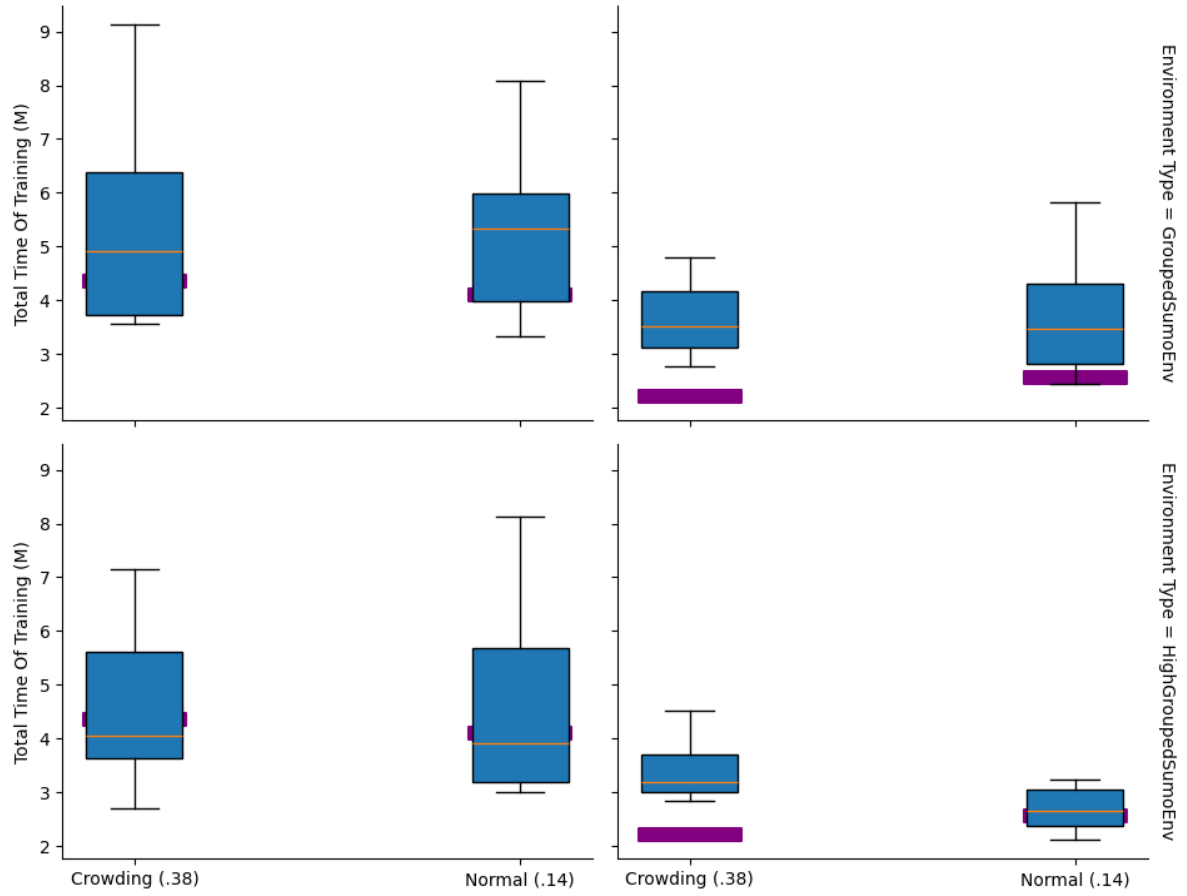
Performance Comparison (Using Reward : Proposed) Waiting Car on area Area2 (Mosheer Ismail)



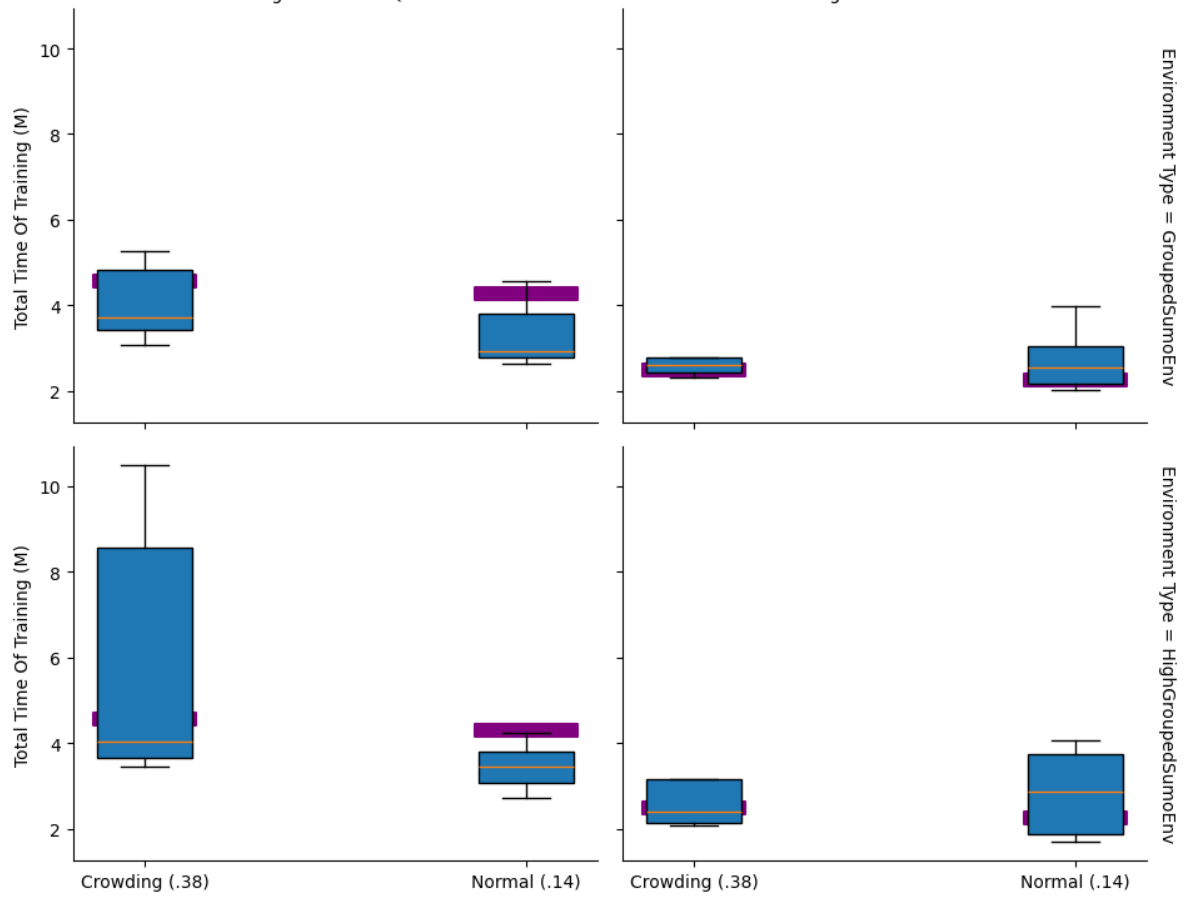
Performance Comparison (Using Reward : Literature) Waiting Car on area Area2 (Mosheer Ismail)



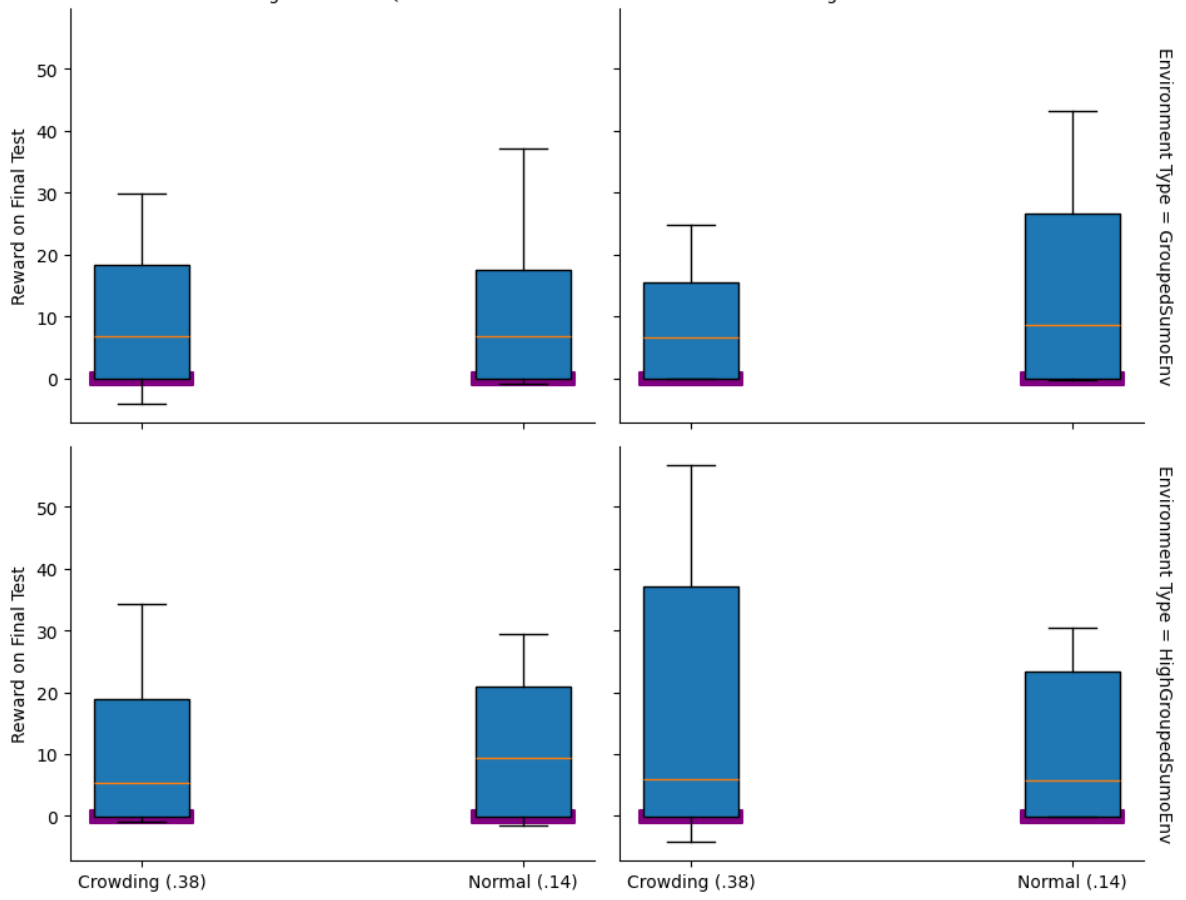
Performance Comparison (Using Reward : Proposed)
 Total Time Of Training (M) on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



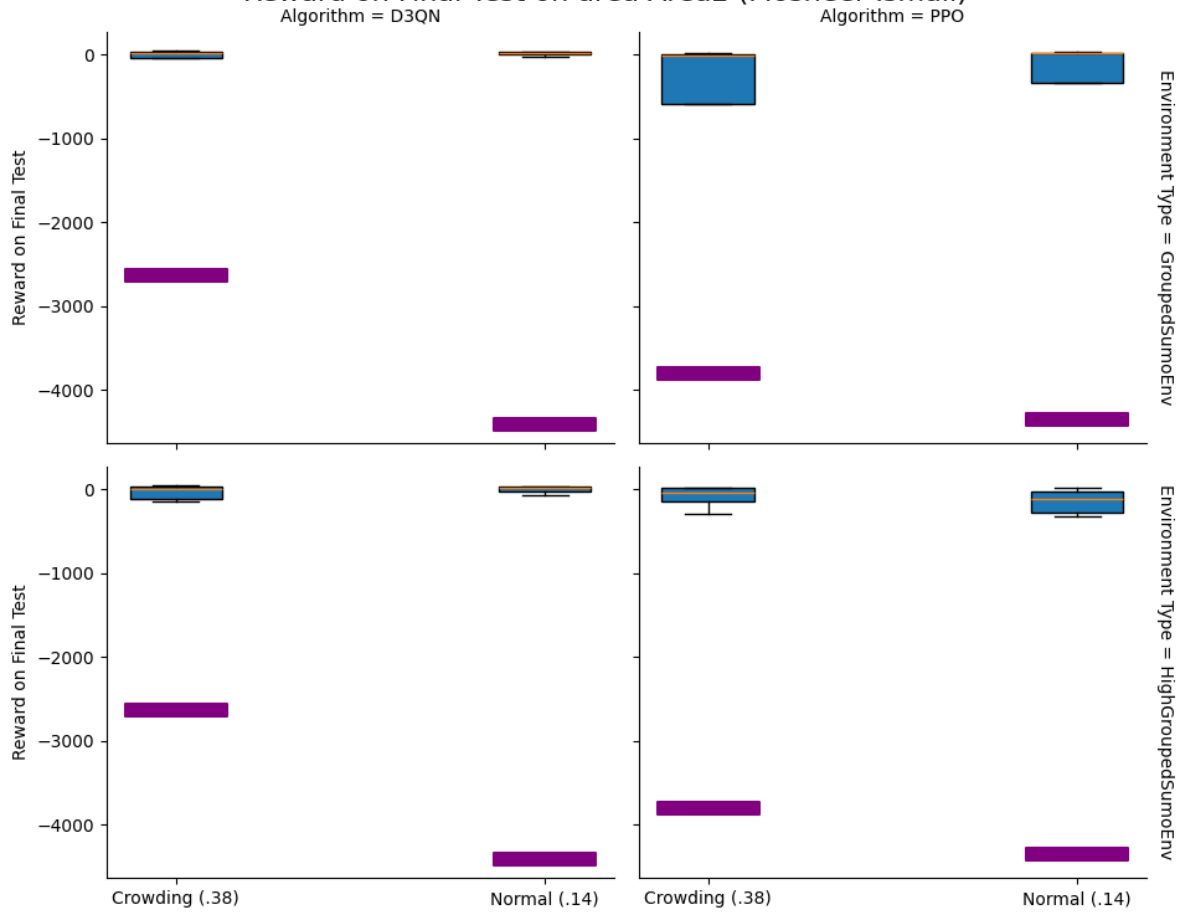
Performance Comparison (Using Reward : Literature)
 Total Time Of Training (M) on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



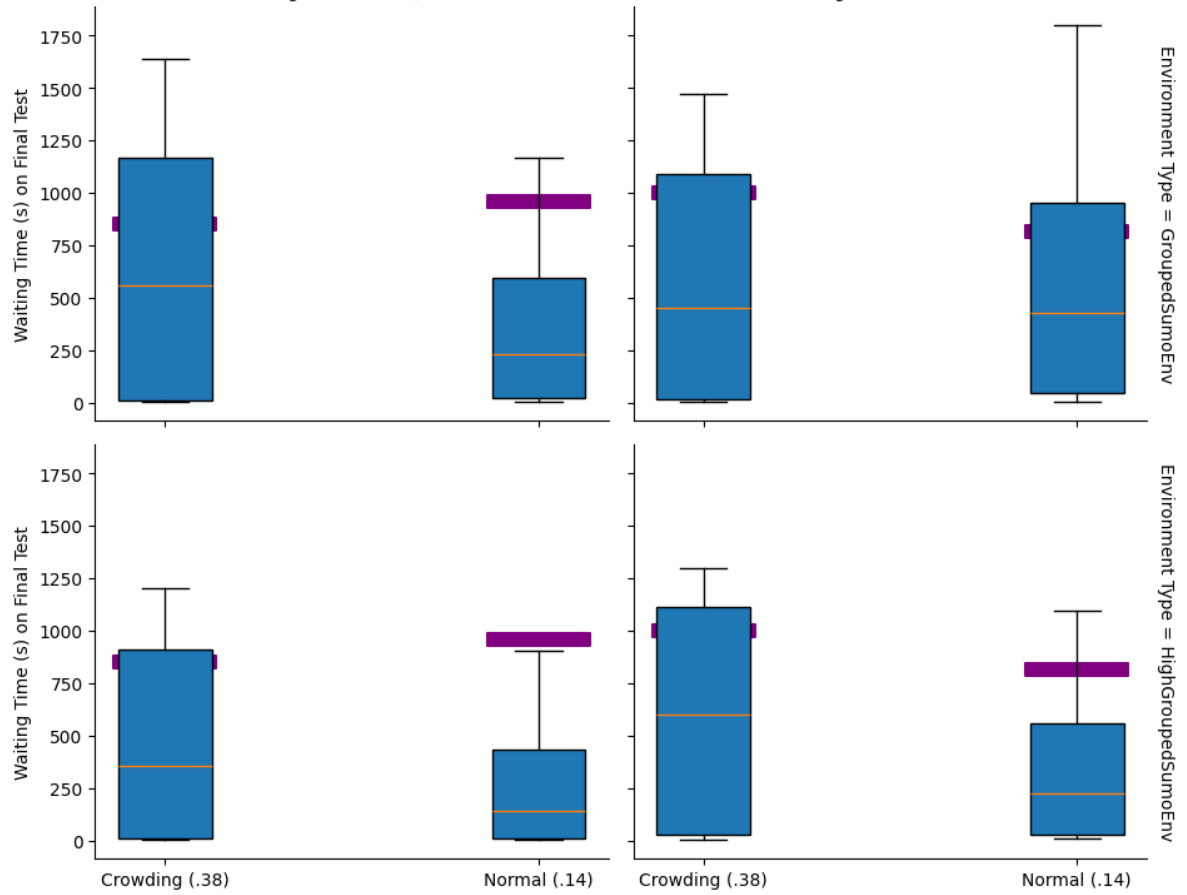
Performance Comparison (Using Reward : Proposed) Reward on Final Test on area Area2 (Mosheer Ismail)



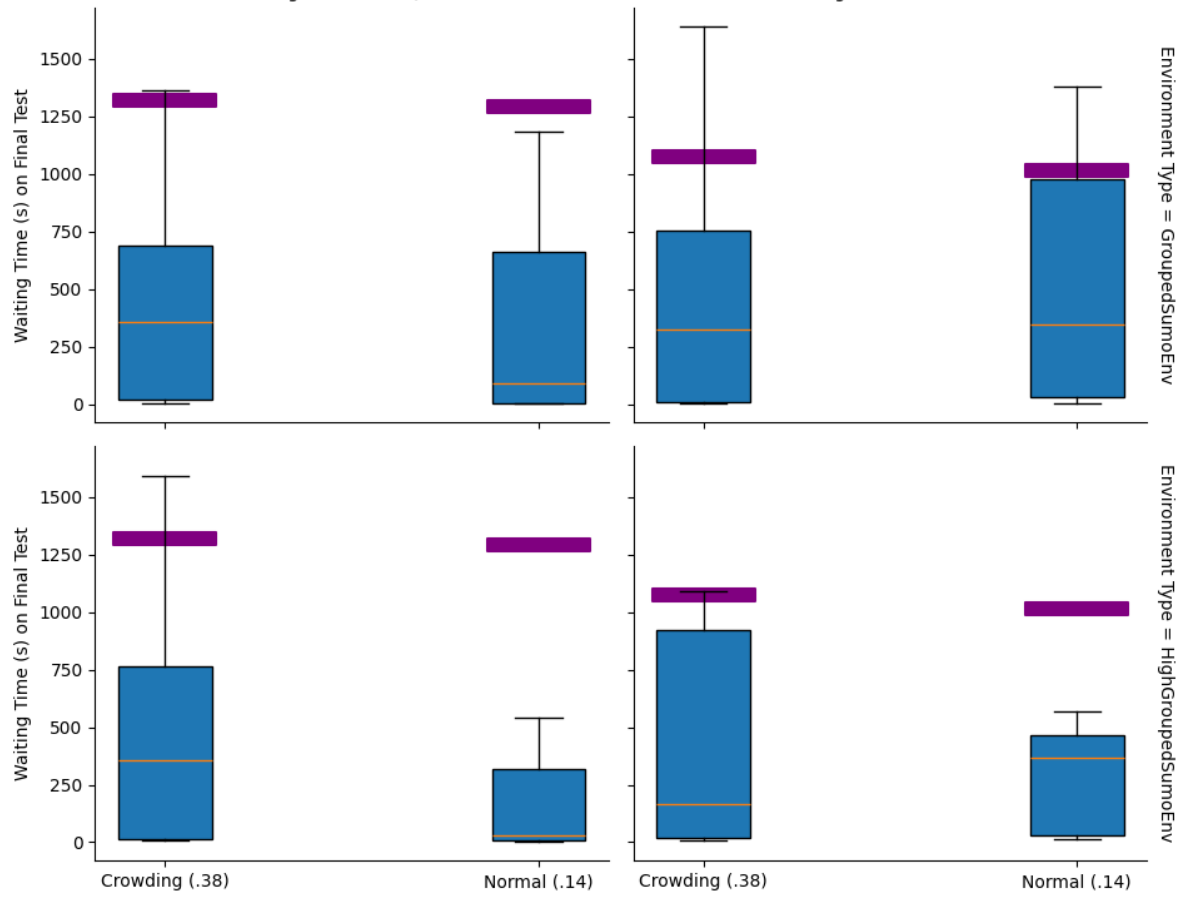
Performance Comparison (Using Reward : Literature) Reward on Final Test on area Area2 (Mosheer Ismail)



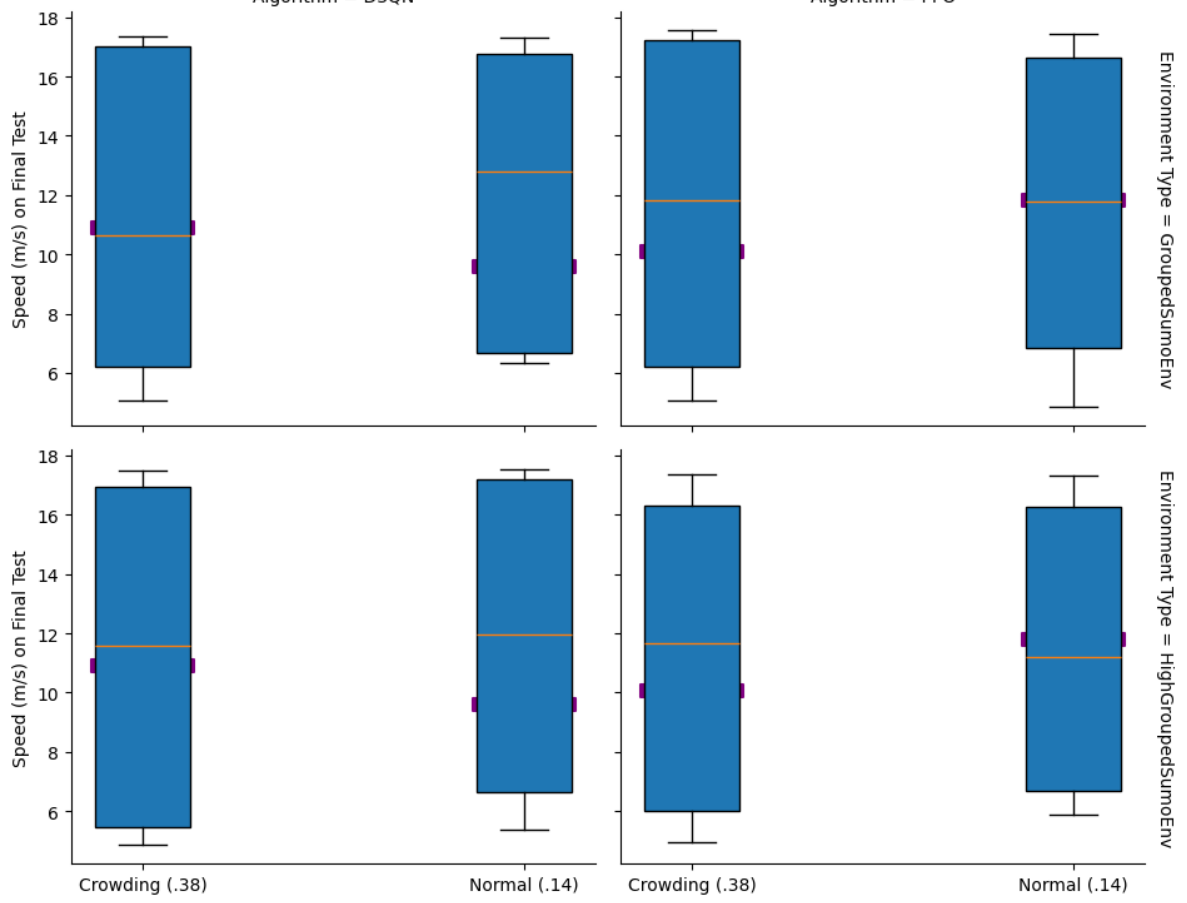
Performance Comparison (Using Reward : Proposed)
 Waiting Time (s) on Final Test on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



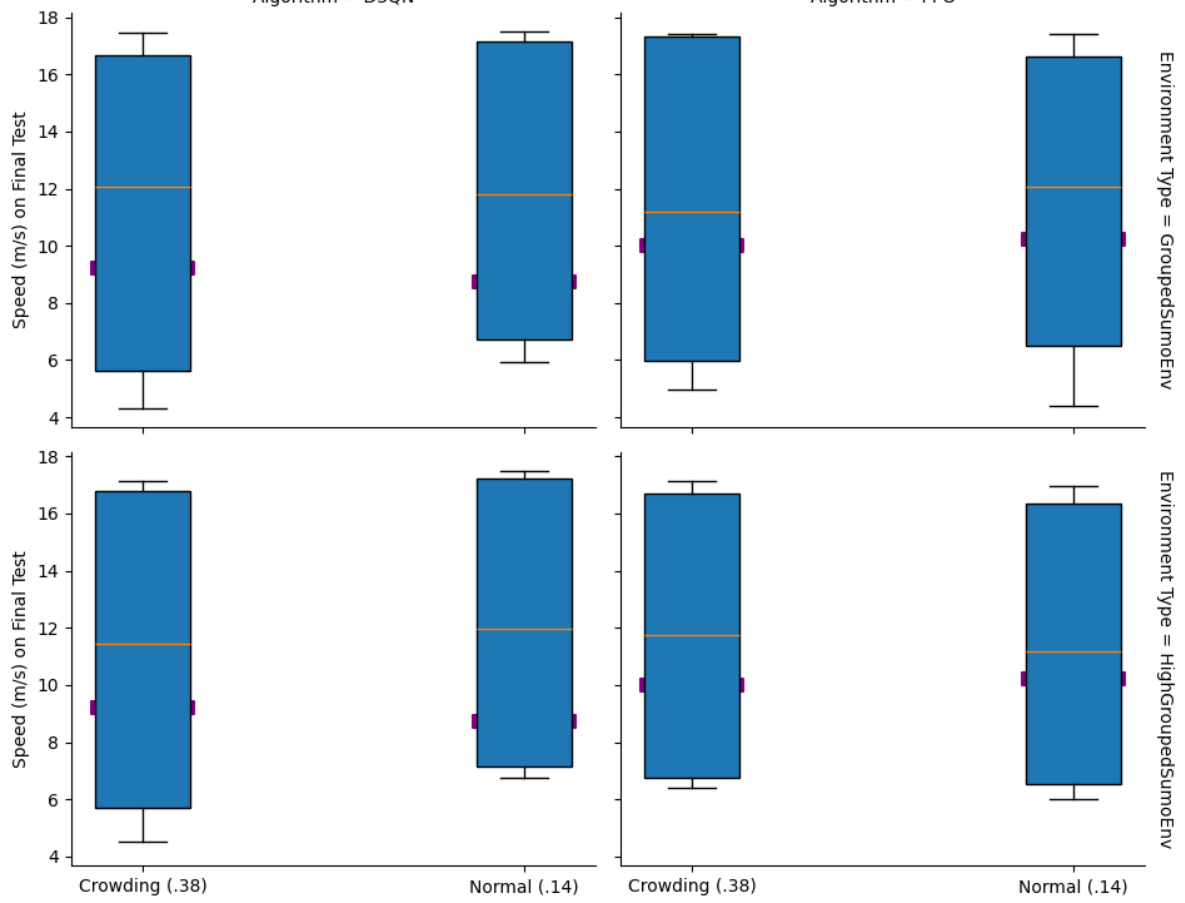
Performance Comparison (Using Reward : Literature)
 Waiting Time (s) on Final Test on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



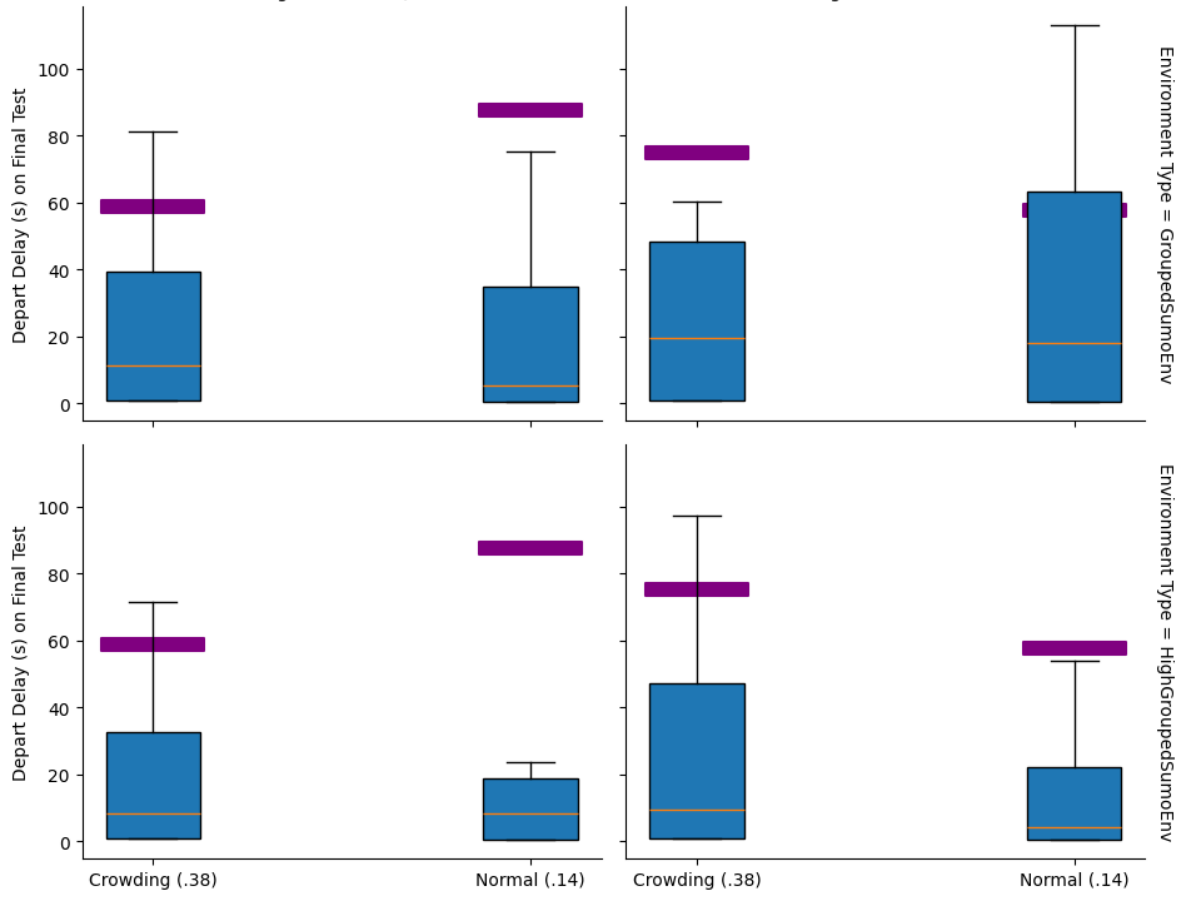
Performance Comparison (Using Reward : Proposed)
 Speed (m/s) on Final Test on area Area2 (Mosheer Ismail)



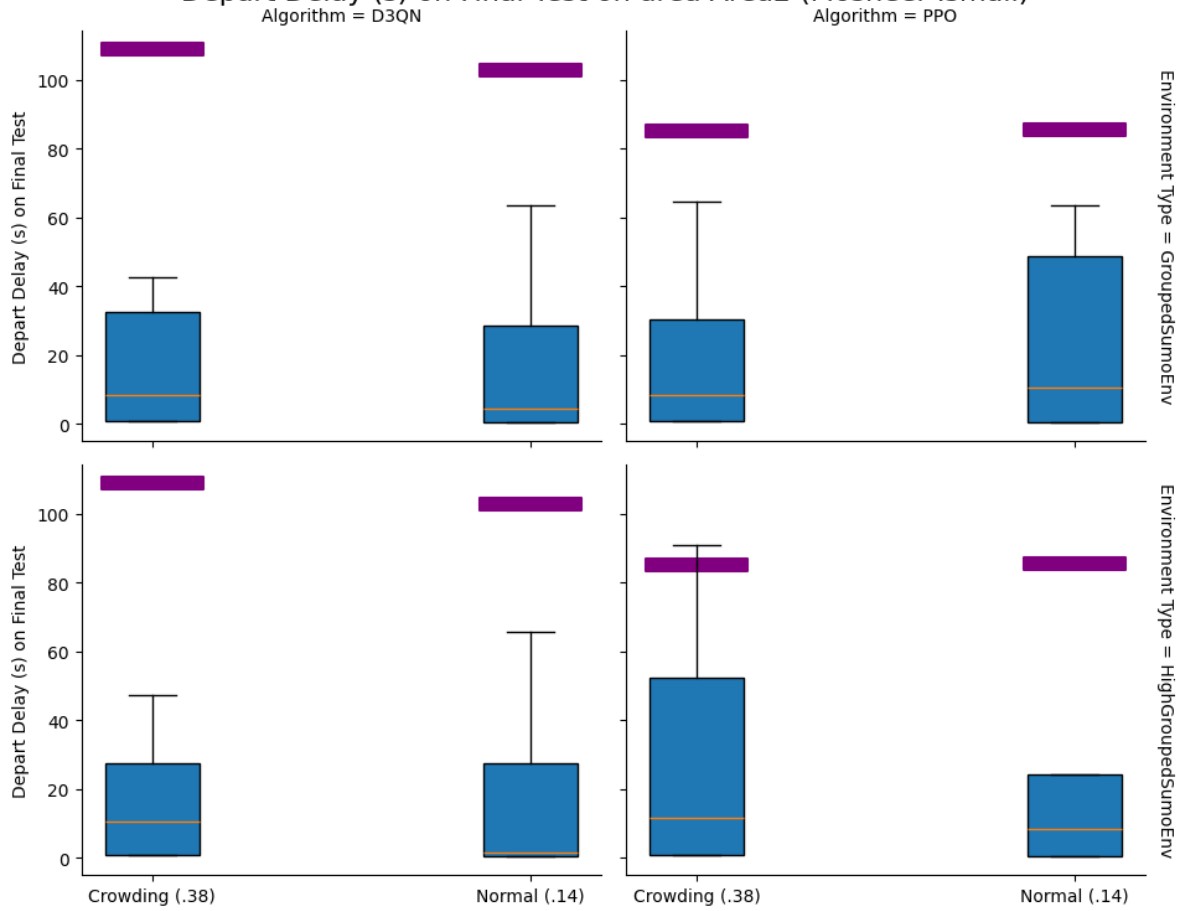
Performance Comparison (Using Reward : Literature)
 Speed (m/s) on Final Test on area Area2 (Mosheer Ismail)



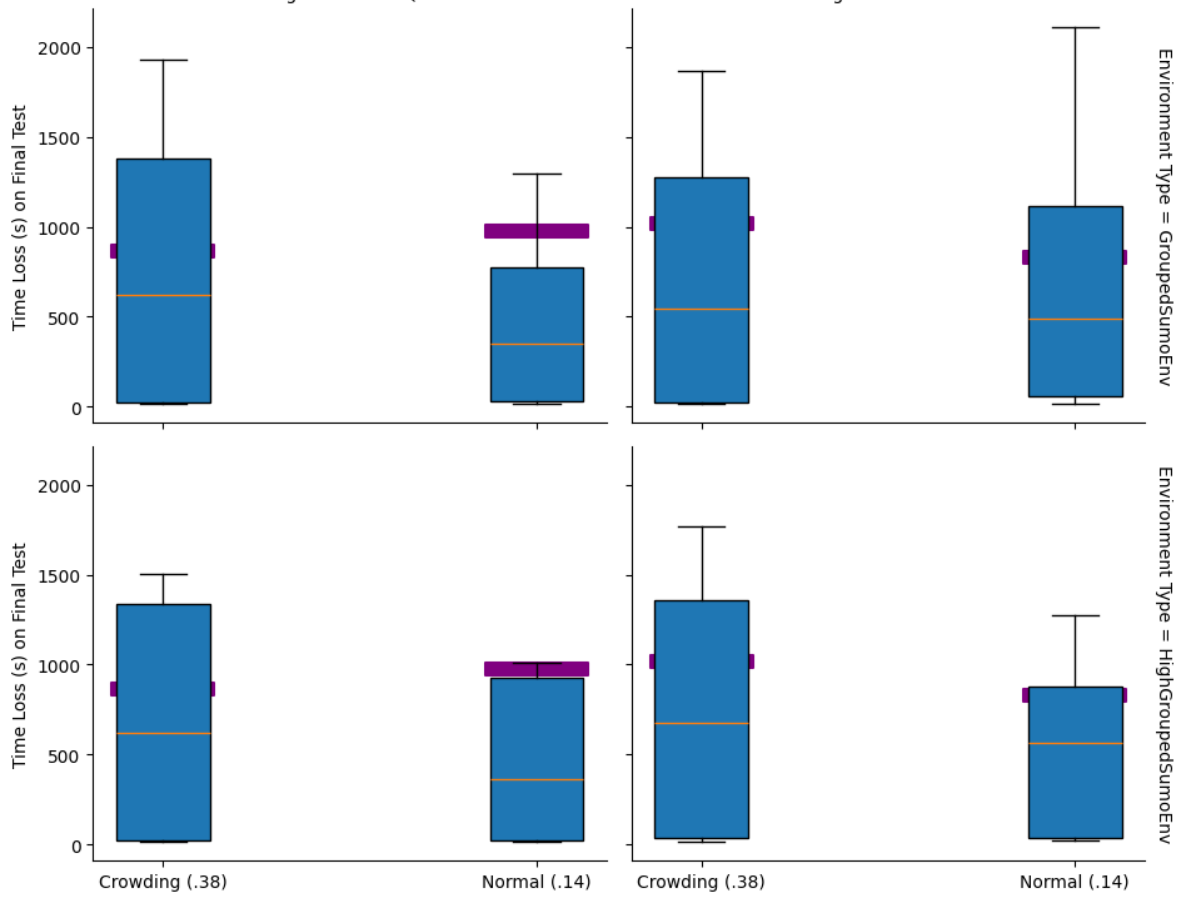
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) on Final Test on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



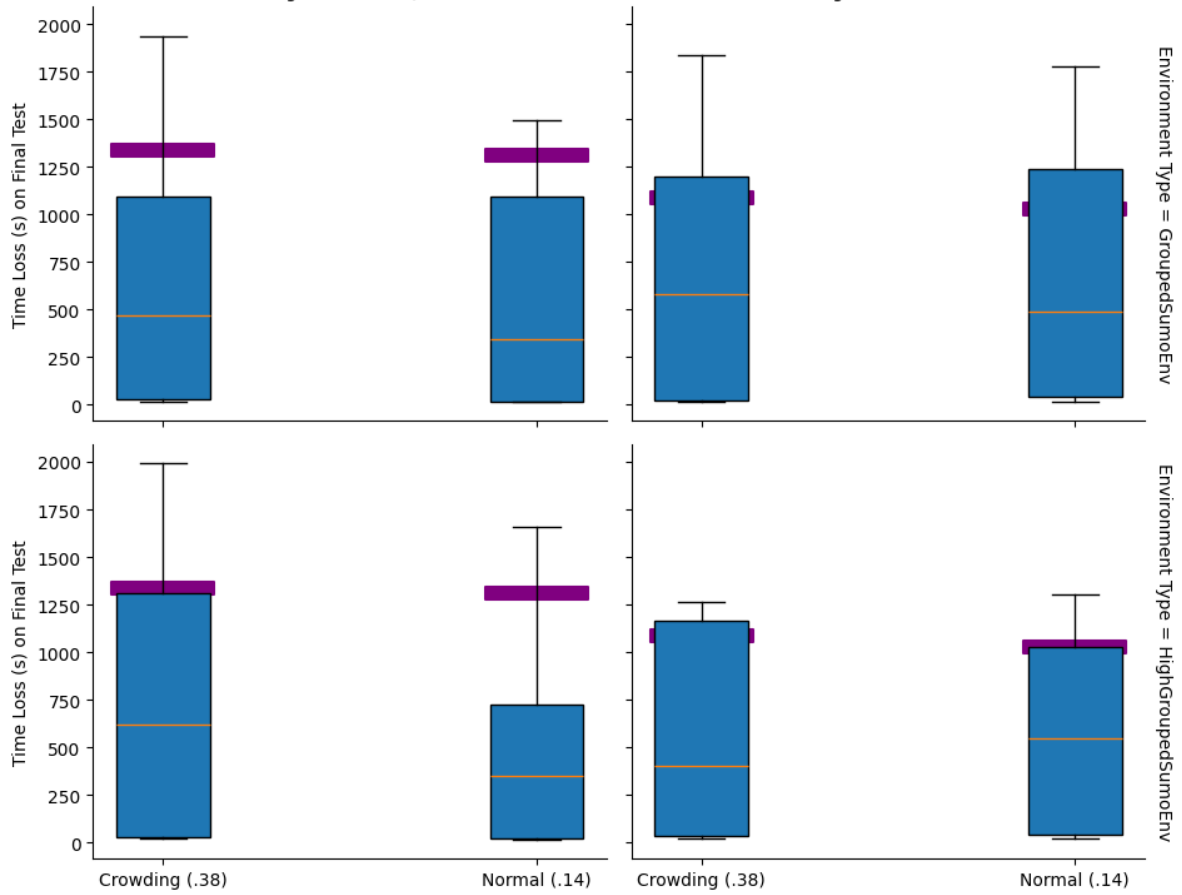
Performance Comparison (Using Reward : Literature) Depart Delay (s) on Final Test on area Area2 (Mosheer Ismail)



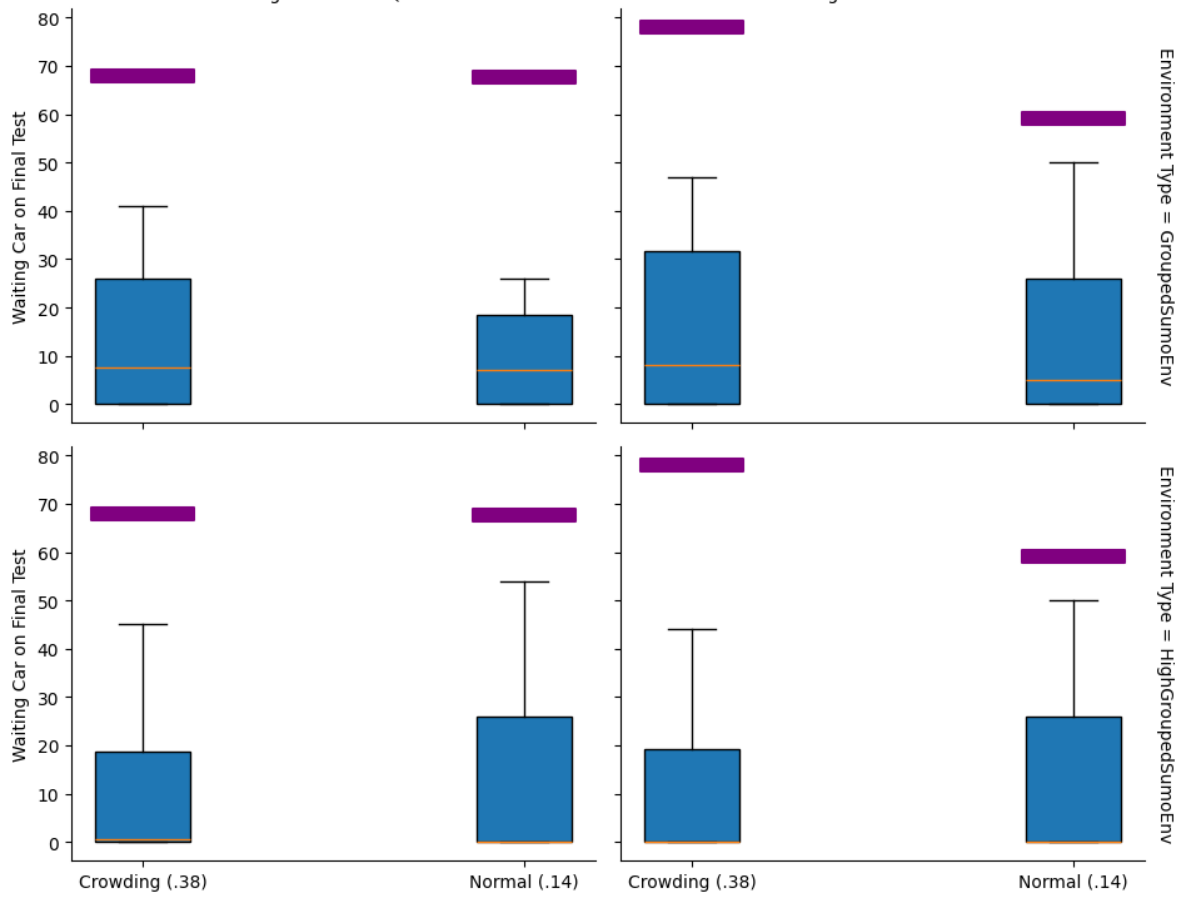
Performance Comparison (Using Reward : Proposed)
 Time Loss (s) on Final Test on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



Performance Comparison (Using Reward : Literature)
 Time Loss (s) on Final Test on area Area2 (Mosheer Ismail)
 Algorithm = D3QN Algorithm = PPO



Performance Comparison (Using Reward : Proposed) Waiting Car on Final Test on area Area2 (Mosheer Ismail)



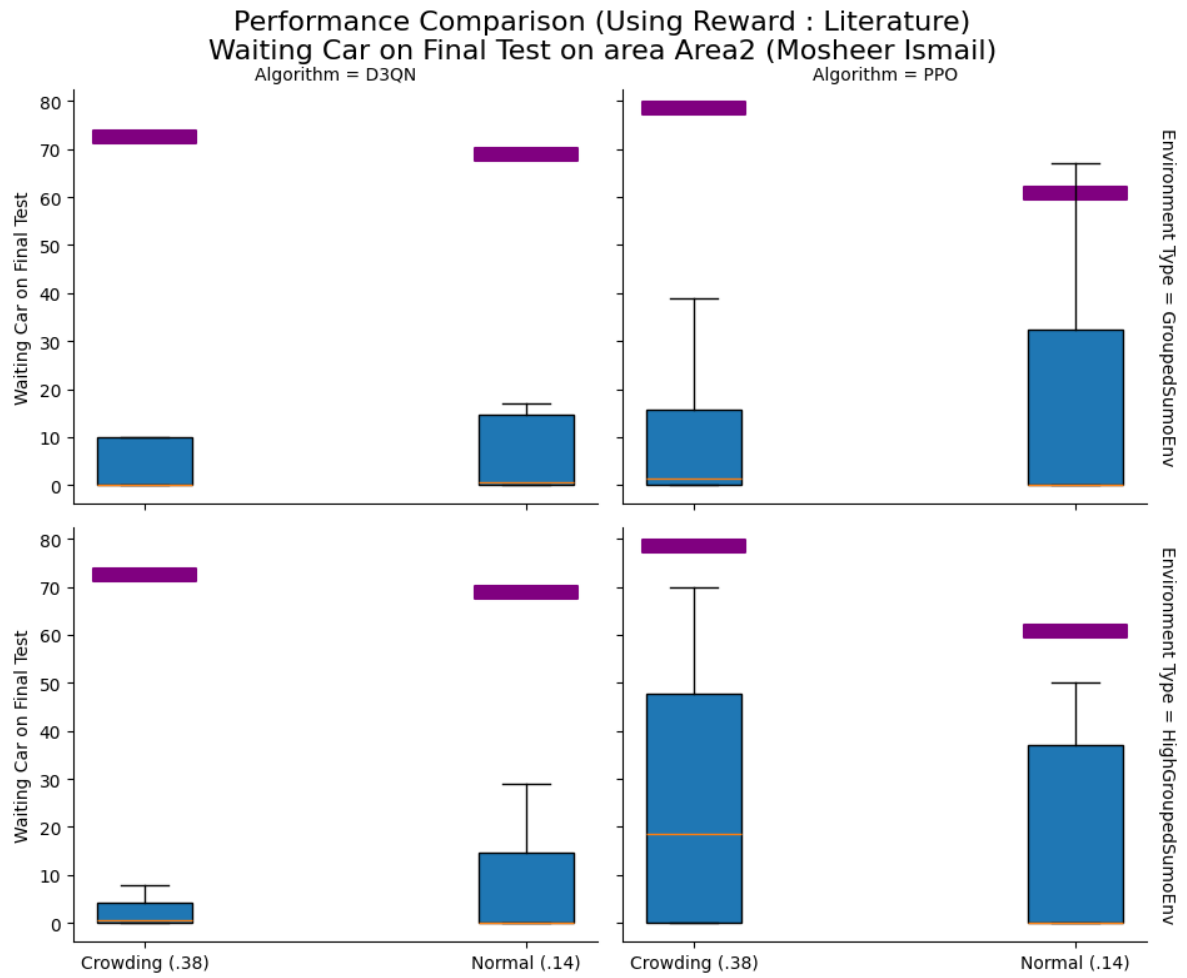
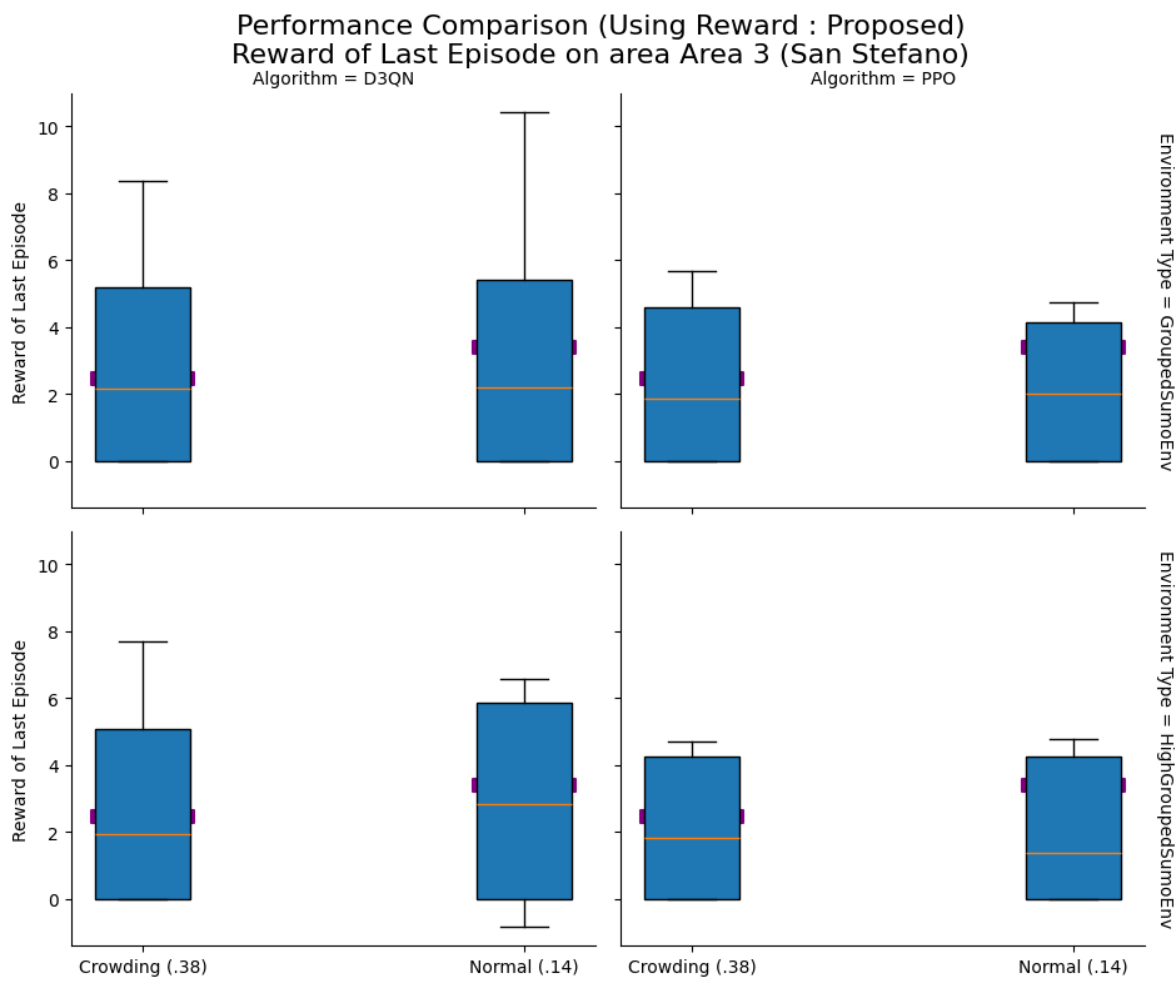
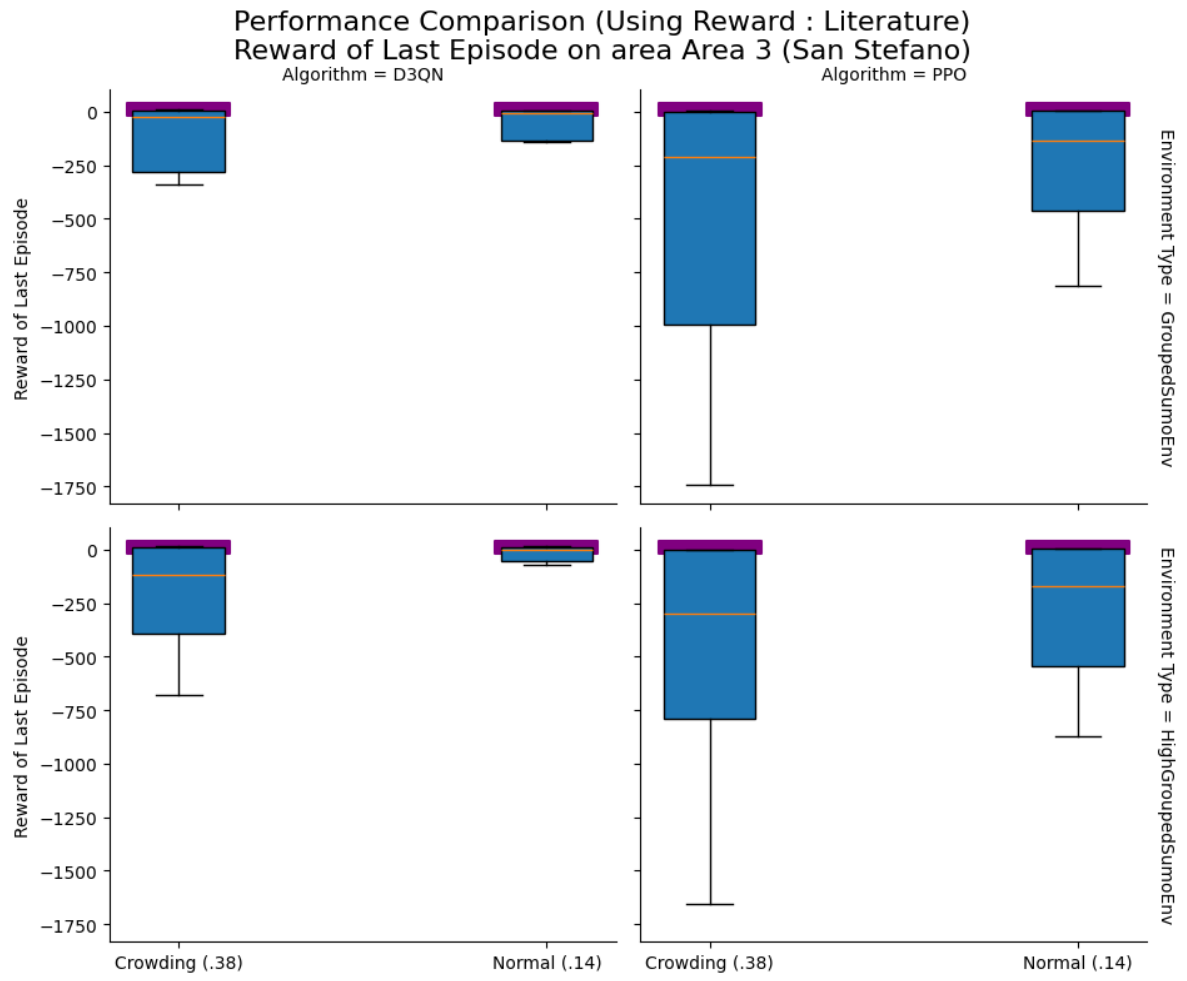


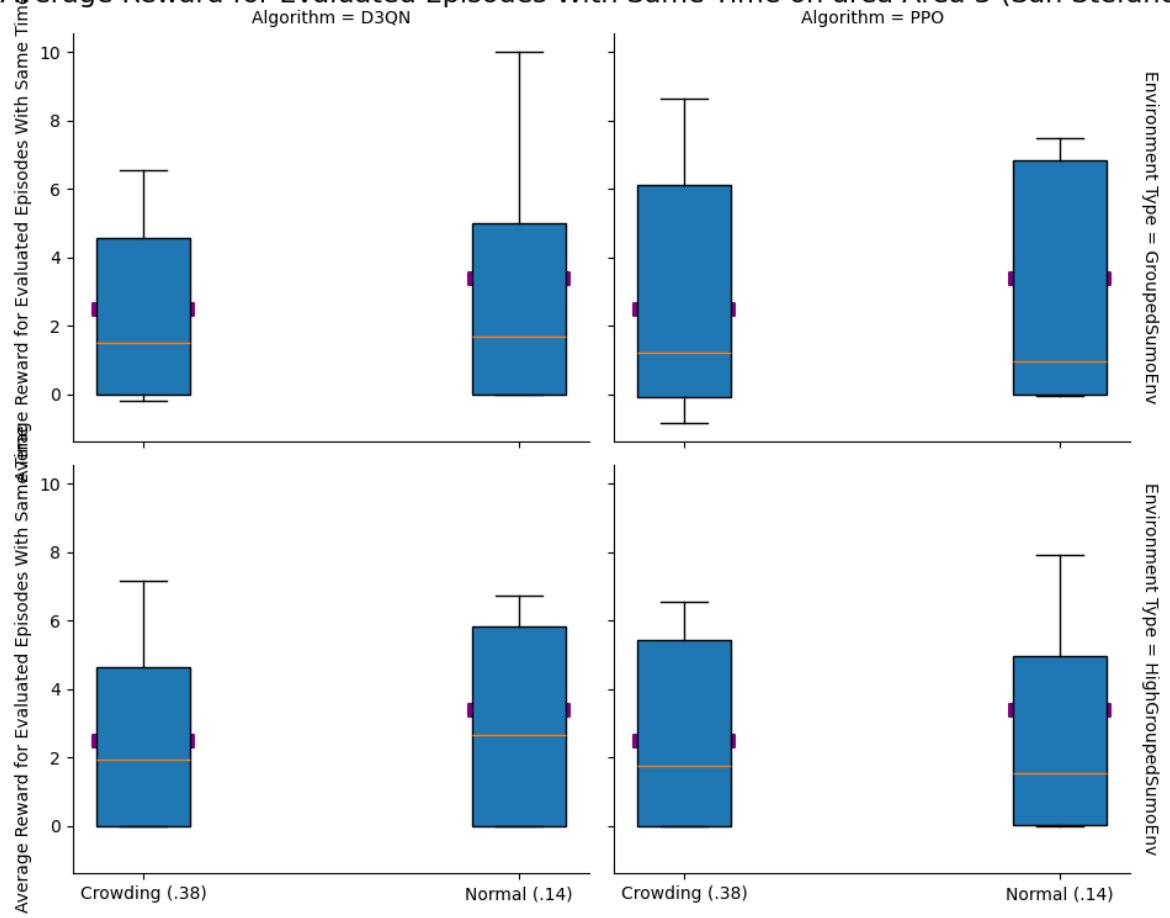
Figure 0.44 Results of Version 2.0.1 at Mosheer Ismail

5.4.3.3.2 Results for San Stefano

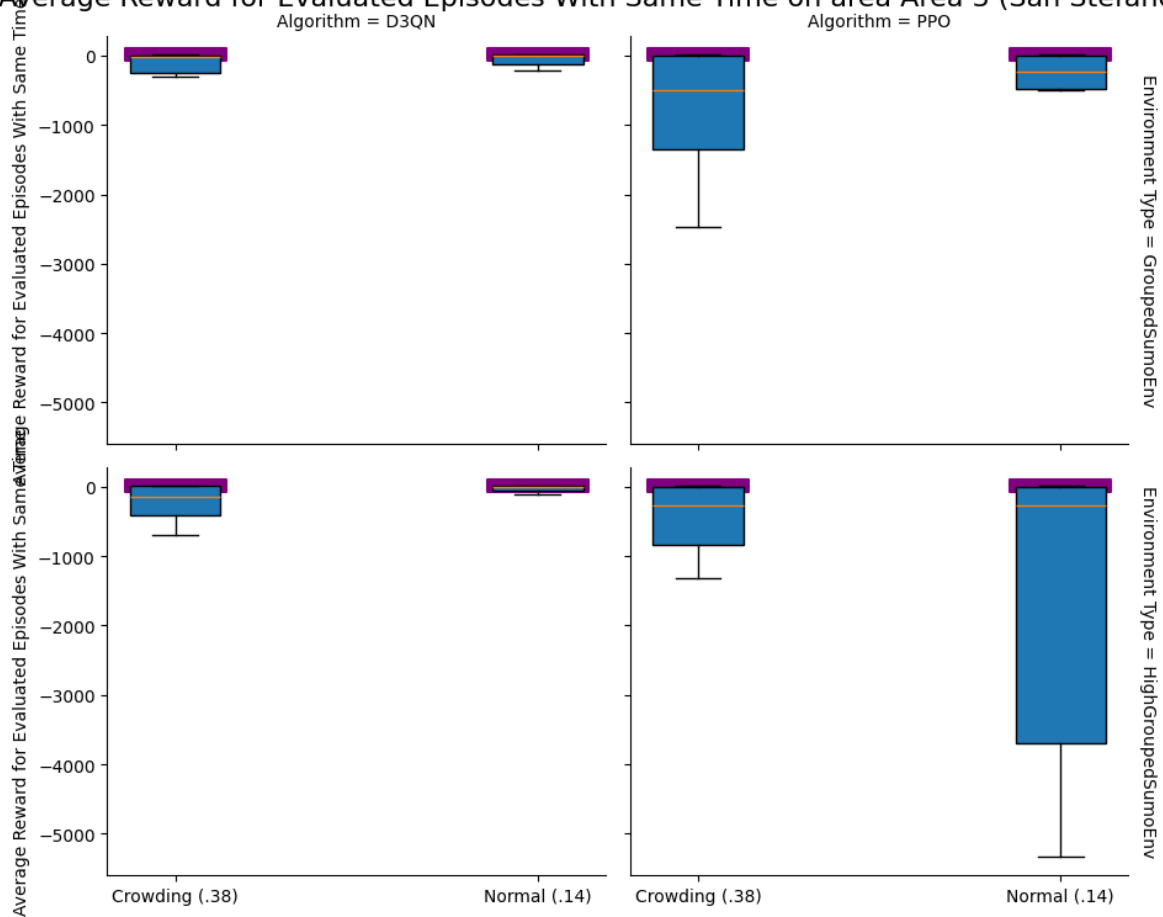




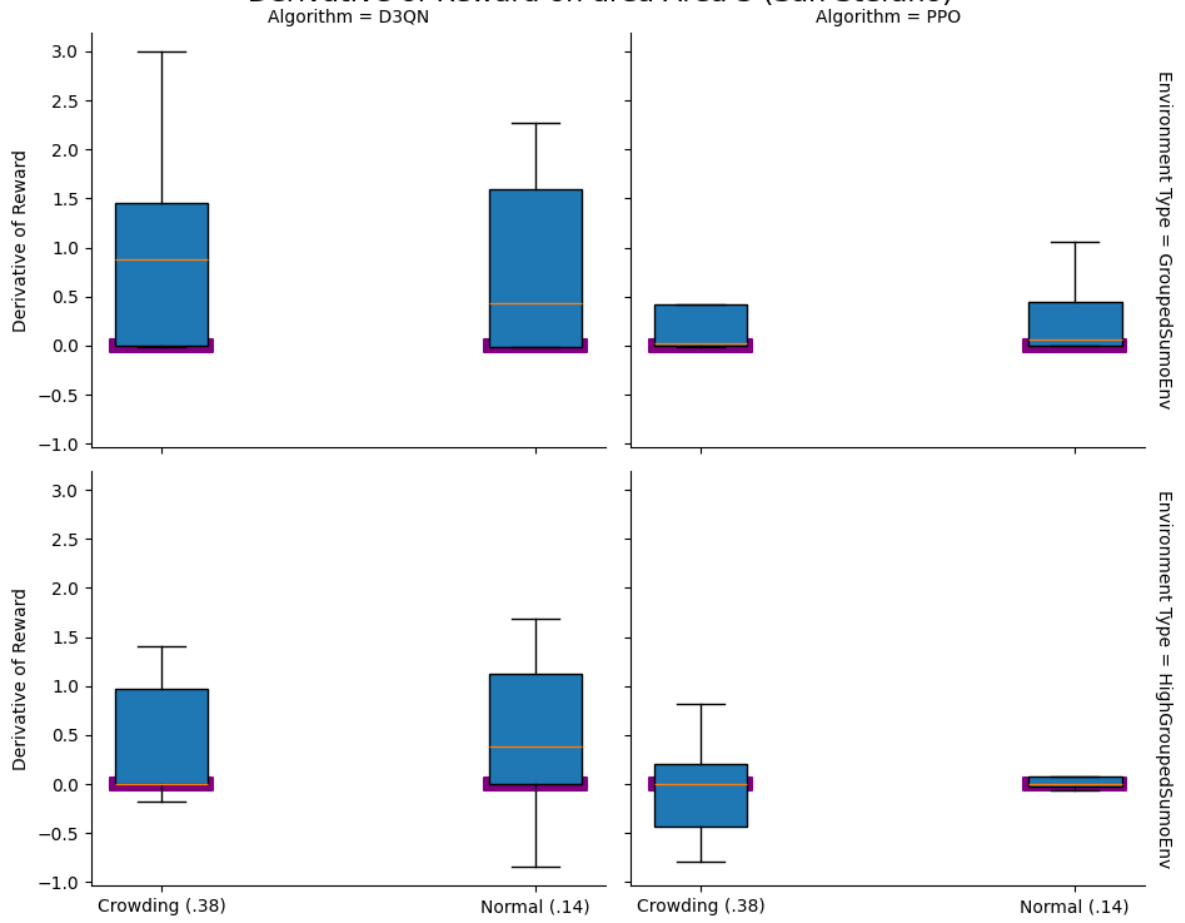
Performance Comparison (Using Reward : Proposed) Average Reward for Evaluated Episodes With Same Time on area Area 3 (San Stefano)



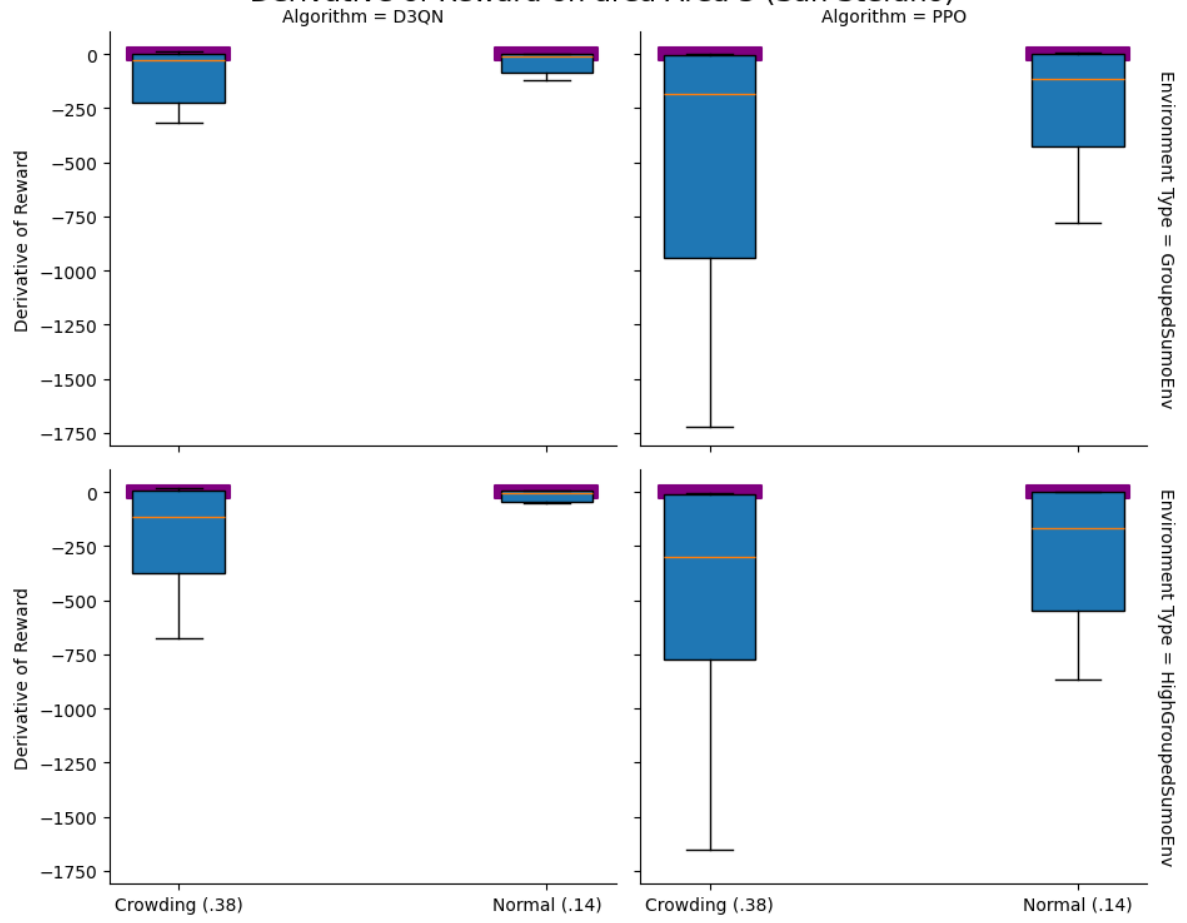
Performance Comparison (Using Reward : Literature) Average Reward for Evaluated Episodes With Same Time on area Area 3 (San Stefano)



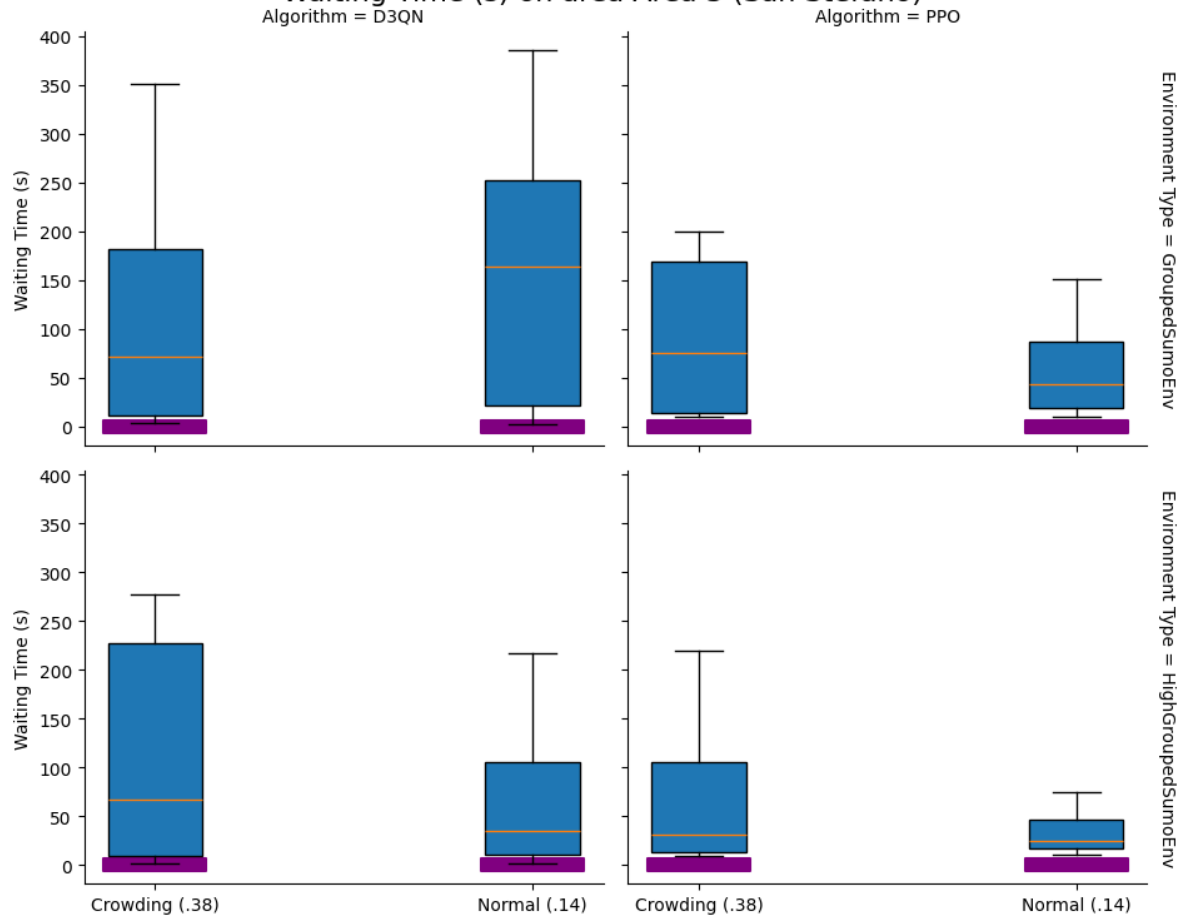
Performance Comparison (Using Reward : Proposed) Derivative of Reward on area Area 3 (San Stefano)



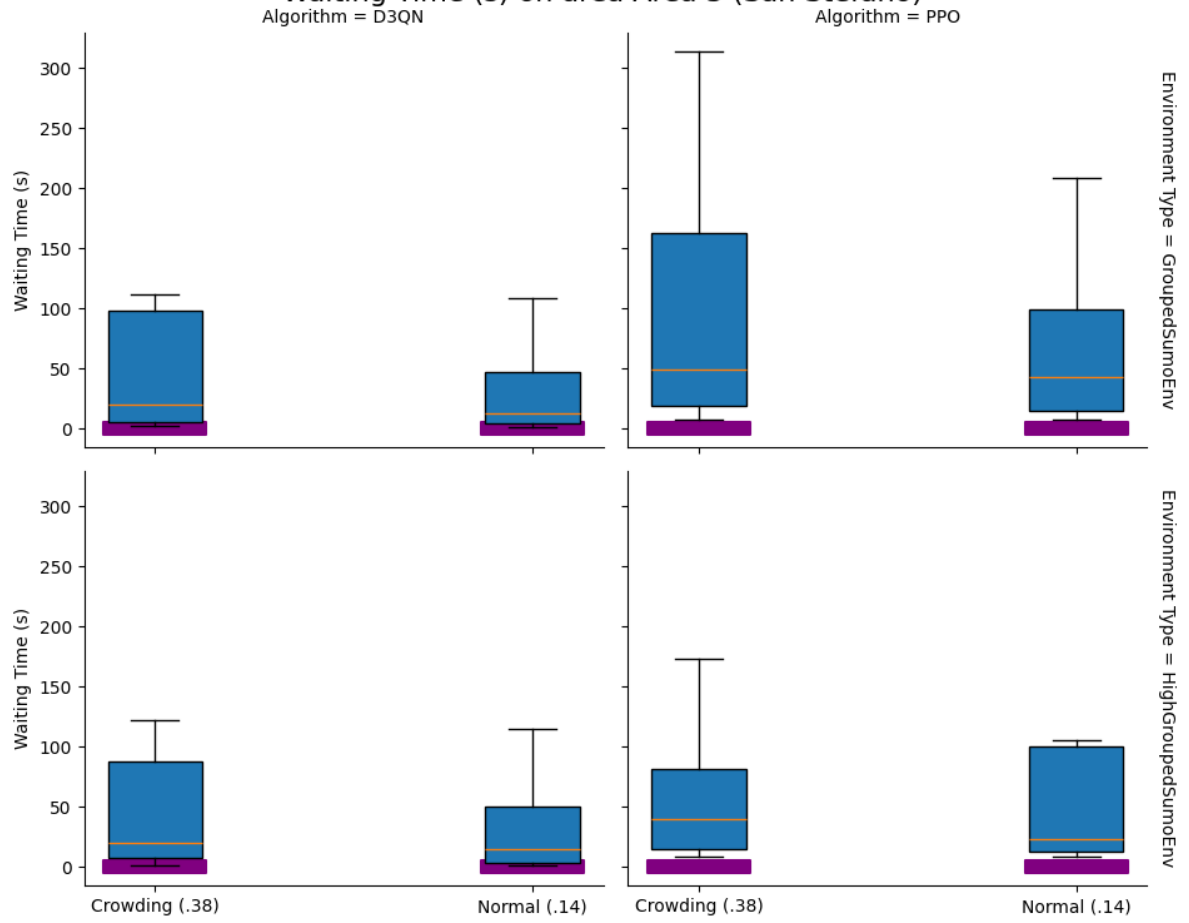
Performance Comparison (Using Reward : Literature) Derivative of Reward on area Area 3 (San Stefano)



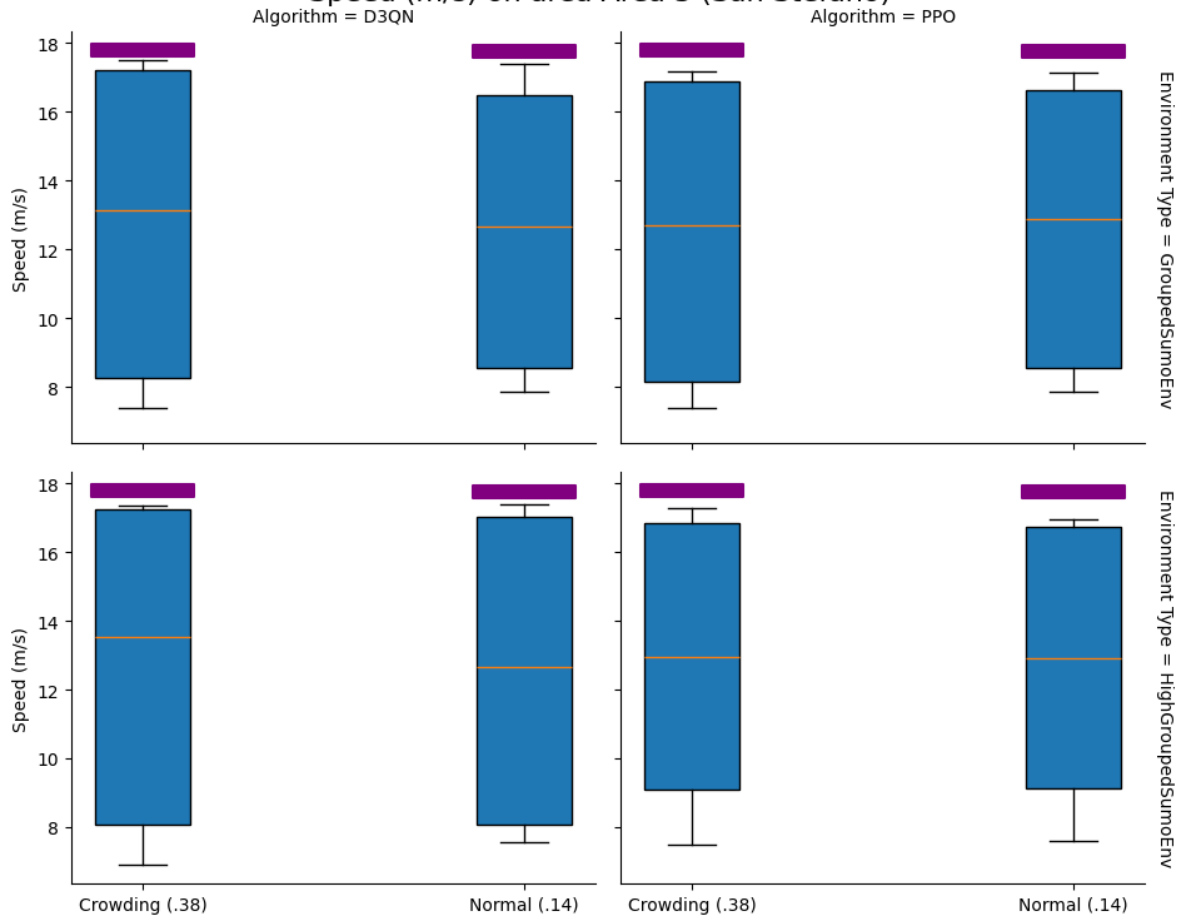
Performance Comparison (Using Reward : Proposed) Waiting Time (s) on area Area 3 (San Stefano)



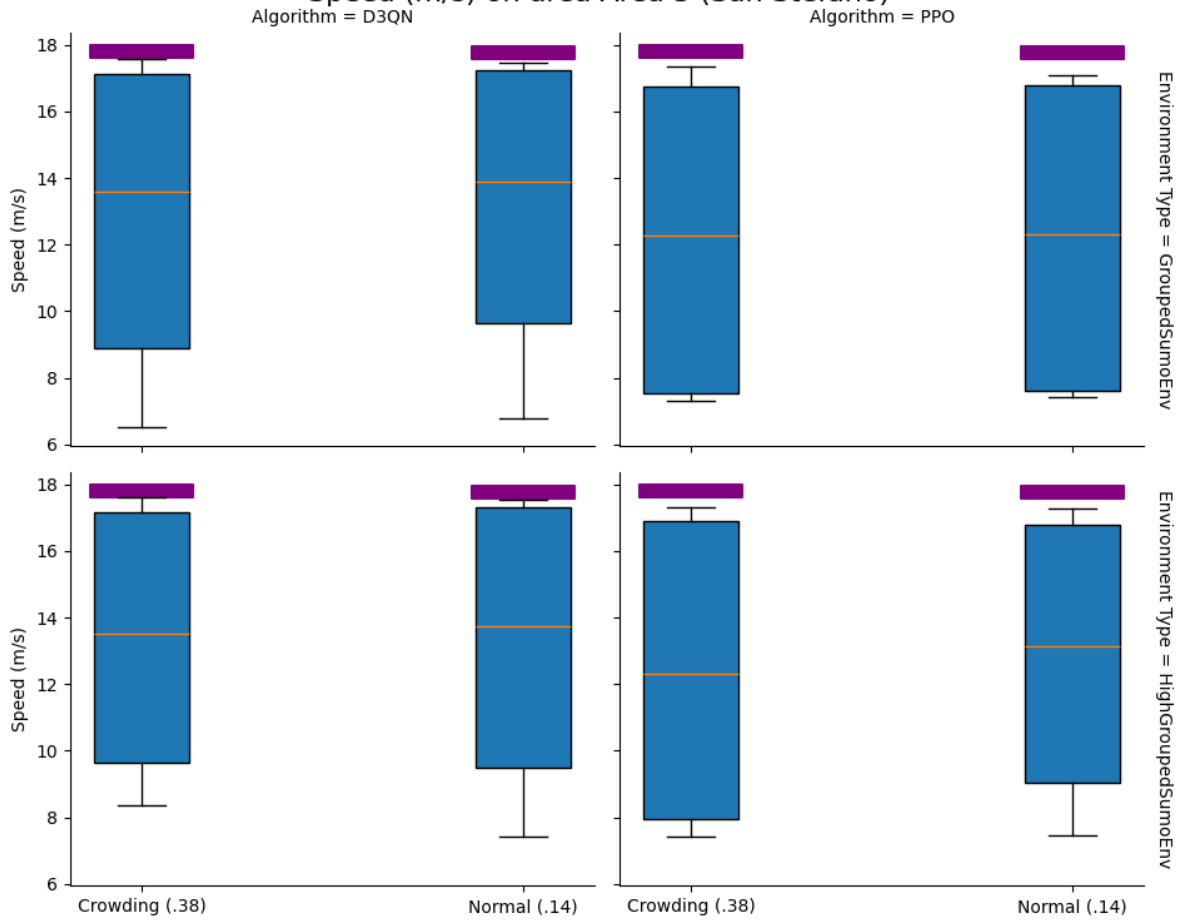
Performance Comparison (Using Reward : Literature) Waiting Time (s) on area Area 3 (San Stefano)



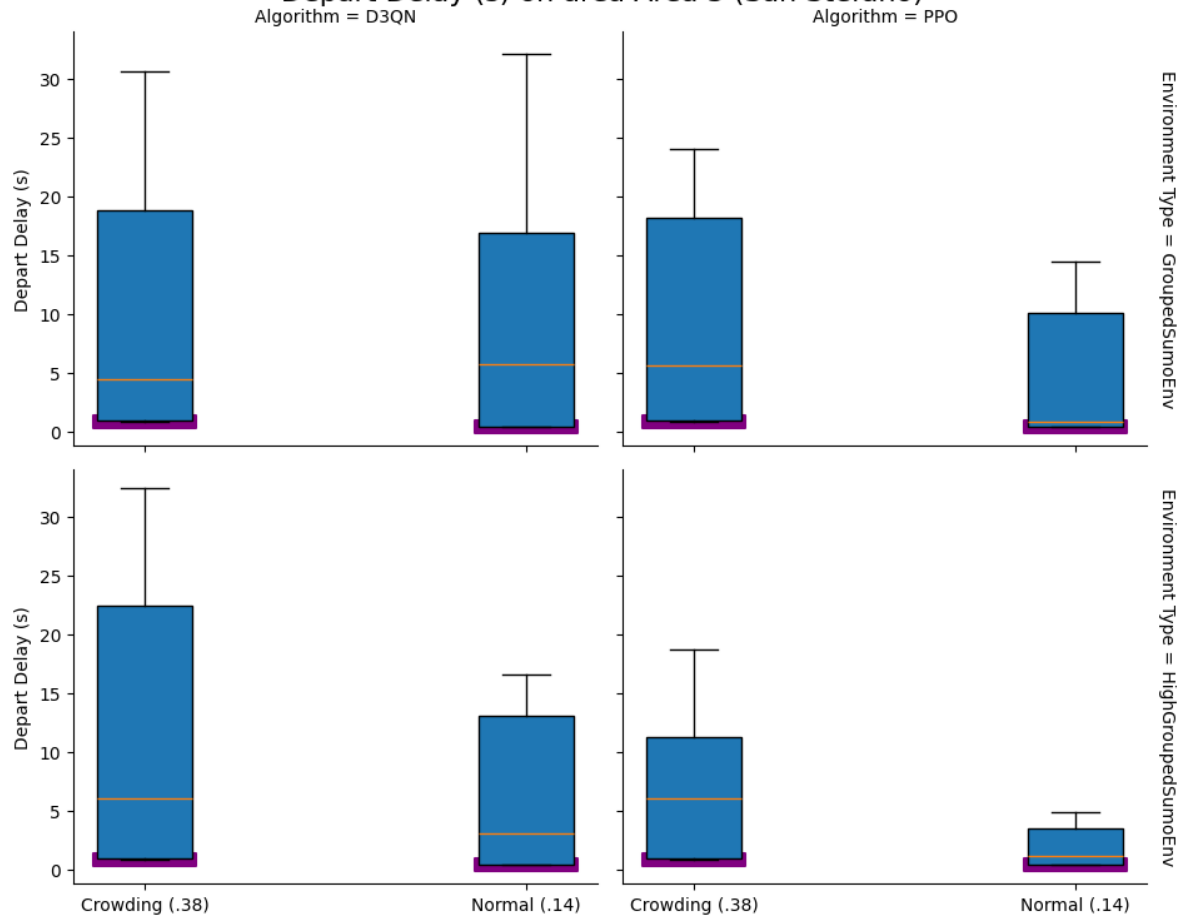
Performance Comparison (Using Reward : Proposed) Speed (m/s) on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Literature) Speed (m/s) on area Area 3 (San Stefano)



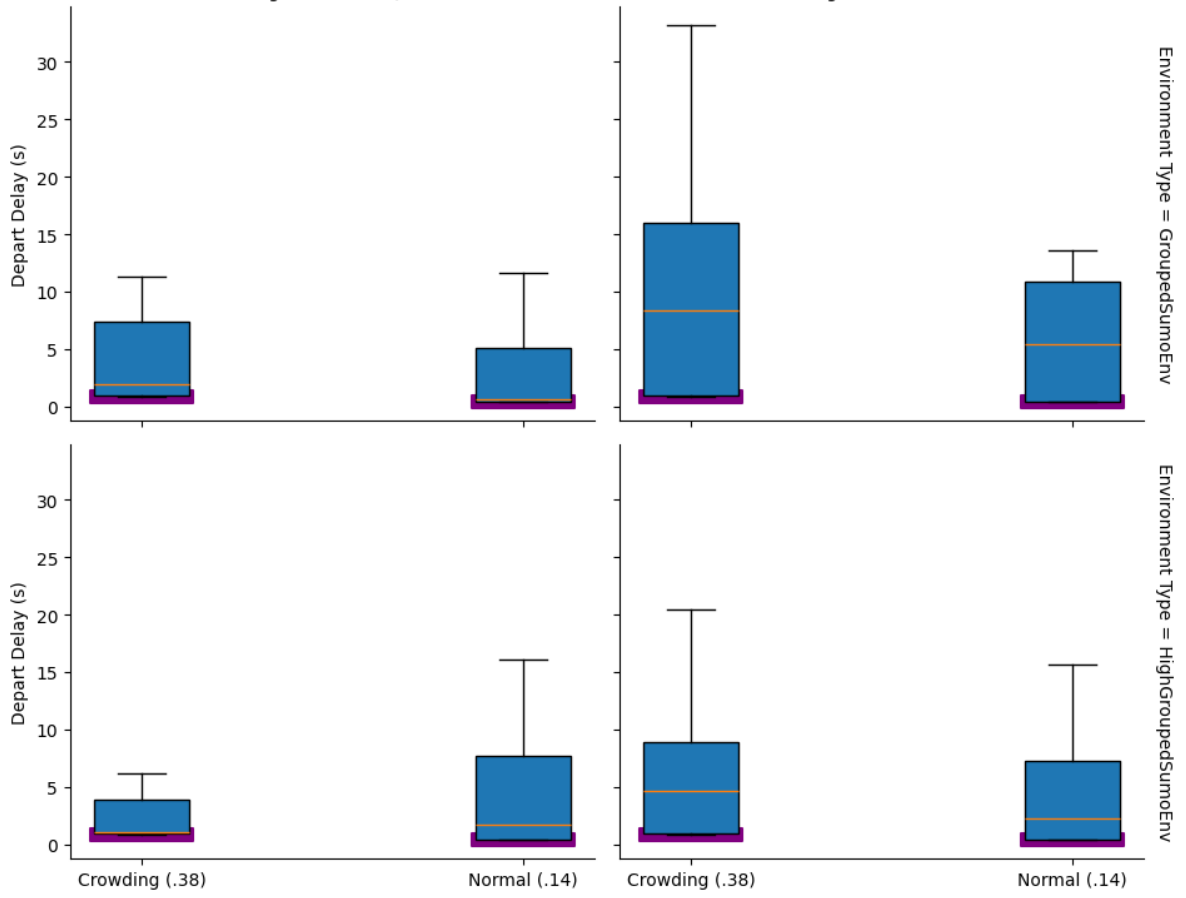
Performance Comparison (Using Reward : Proposed) Depart Delay (s) on area Area 3 (San Stefano)



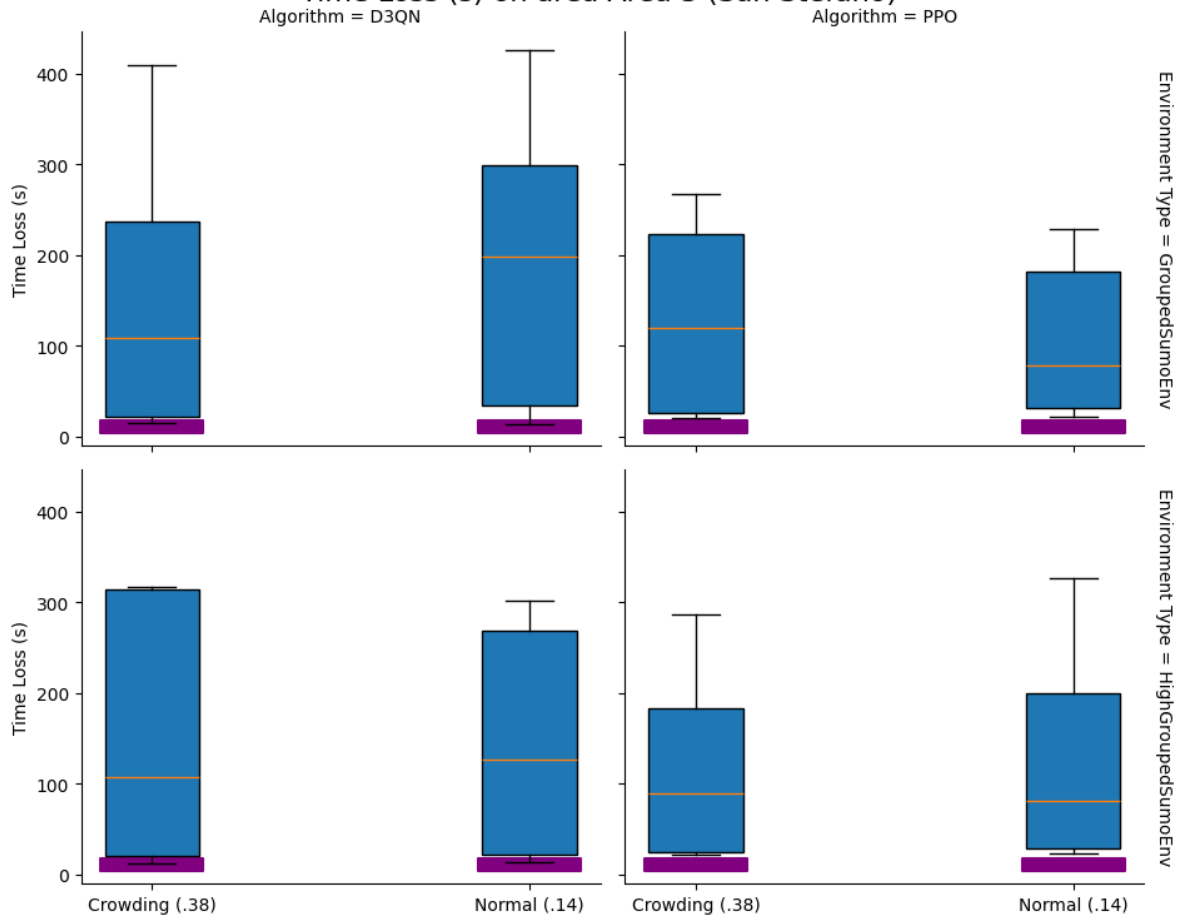
Performance Comparison (Using Reward : Literature) Depart Delay (s) on area Area 3 (San Stefano)

Algorithm = D3QN

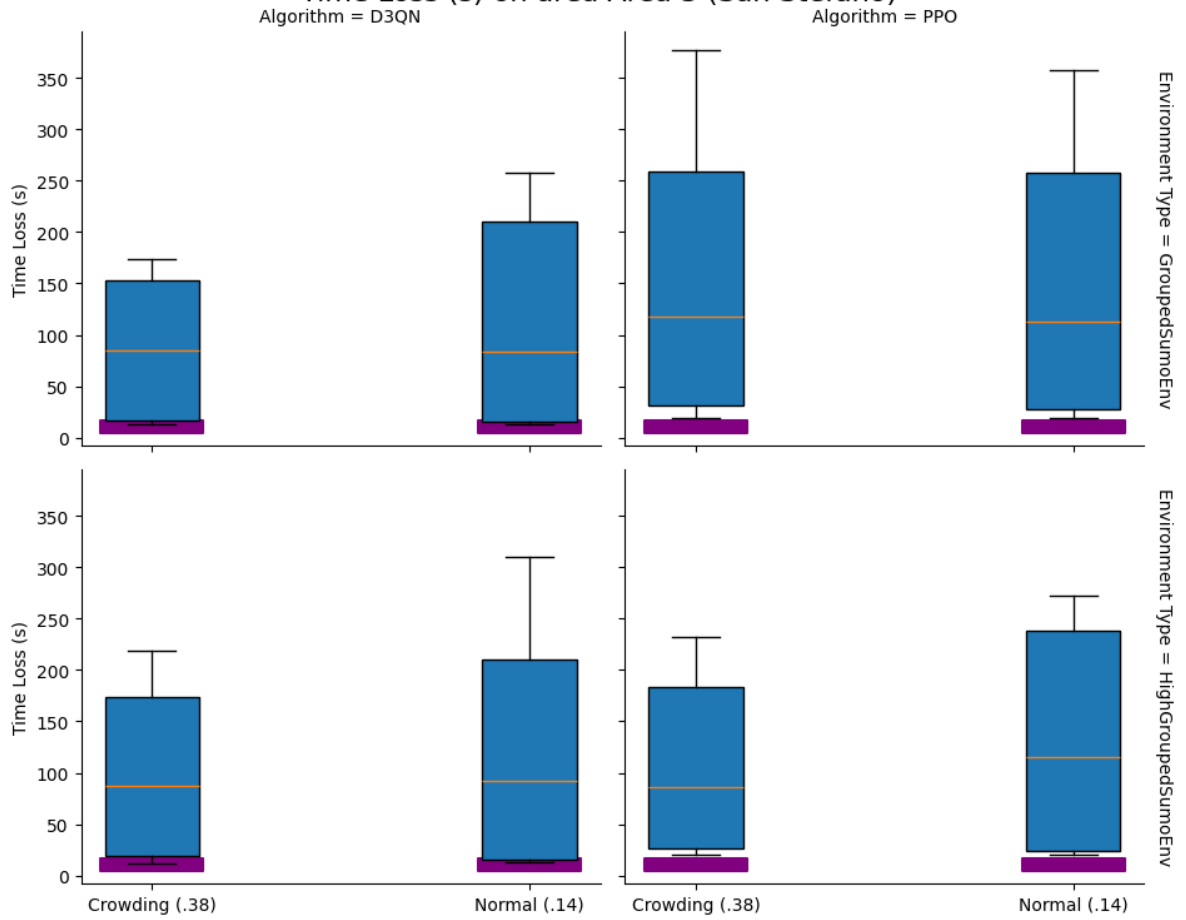
Algorithm = PPO



Performance Comparison (Using Reward : Proposed) Time Loss (s) on area Area 3 (San Stefano)



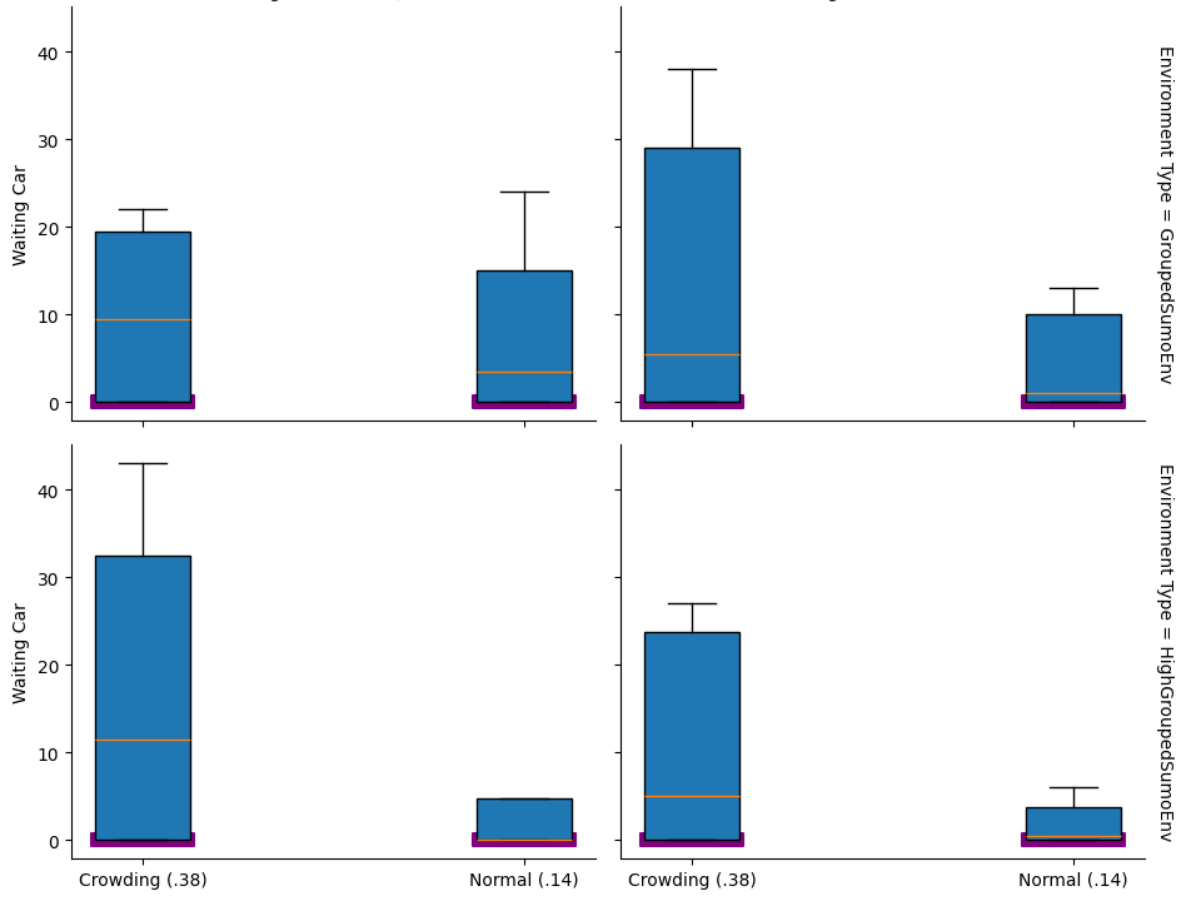
Performance Comparison (Using Reward : Literature) Time Loss (s) on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Proposed) Waiting Car on area Area 3 (San Stefano)

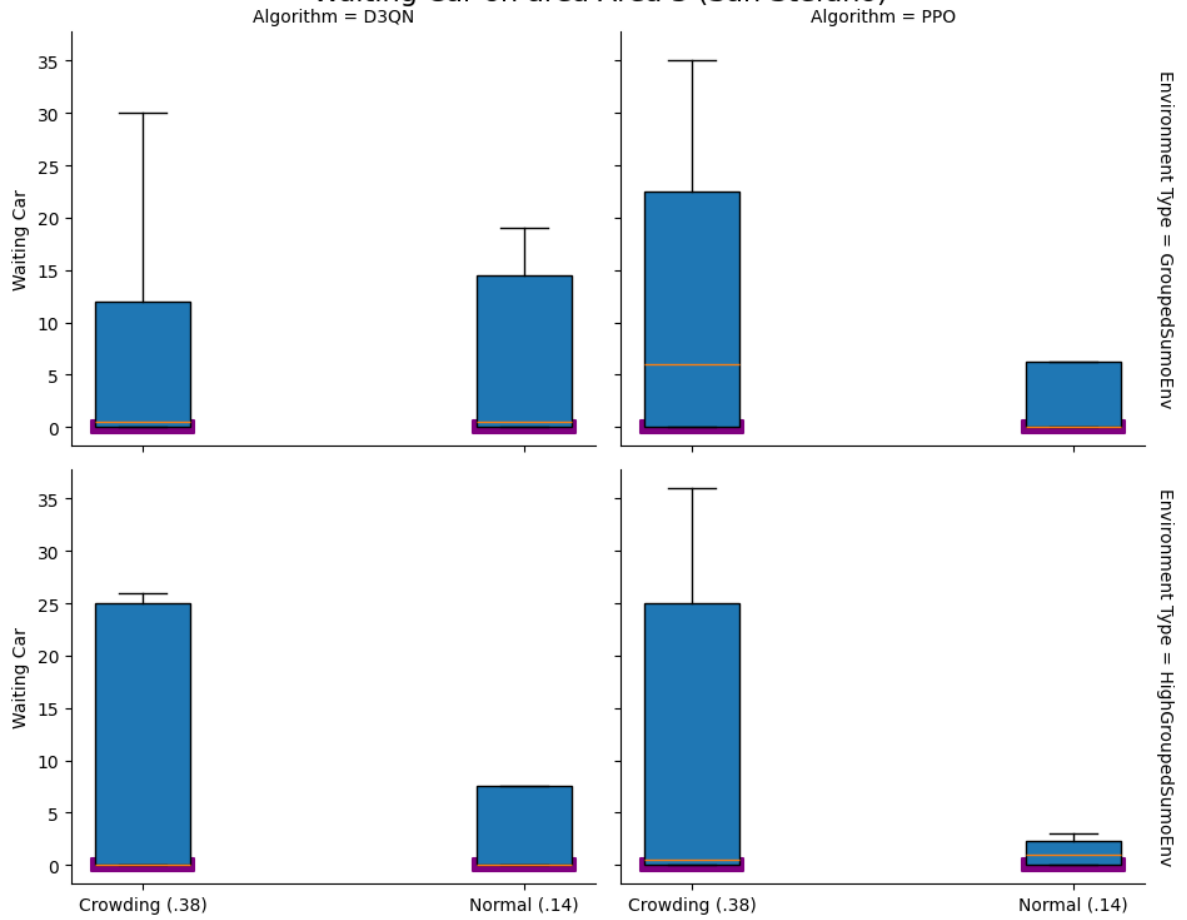
Algorithm = D3QN

Algorithm = PPO

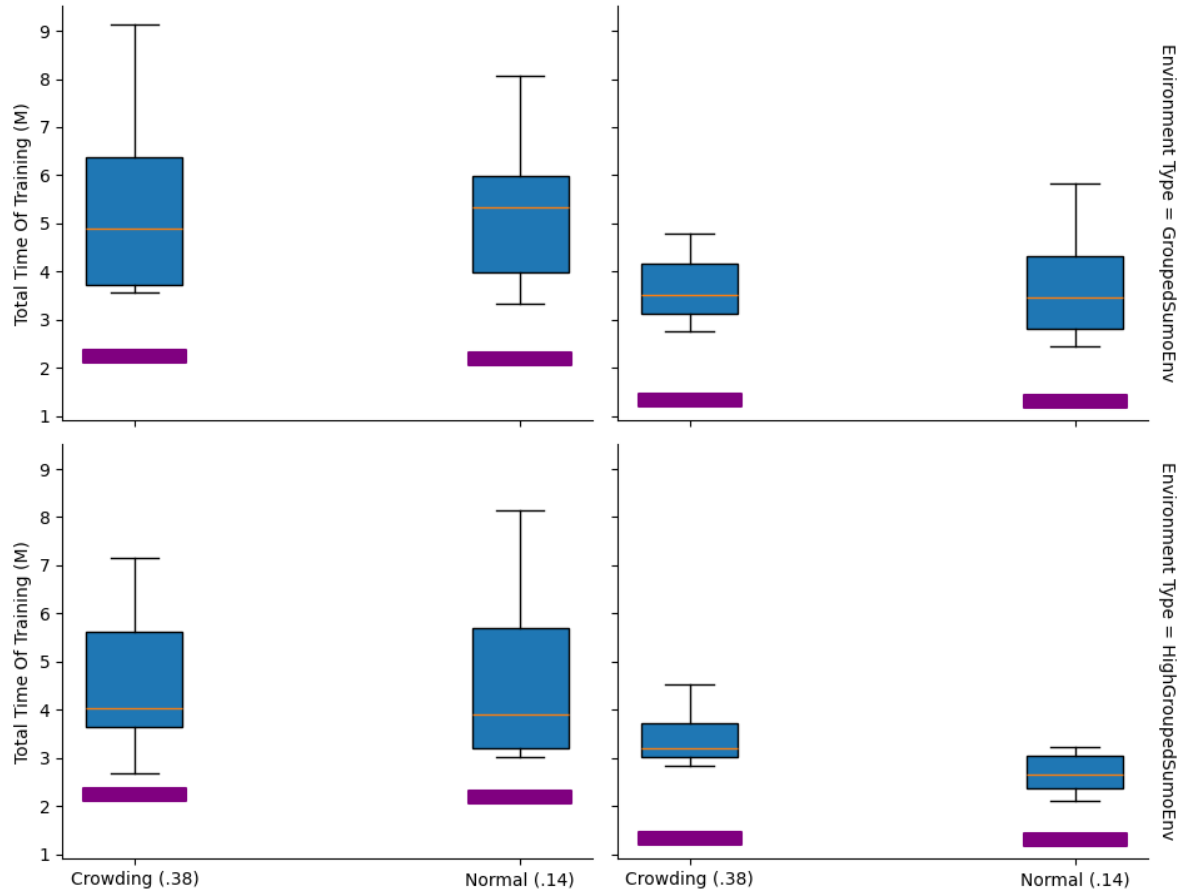


Performance Comparison (Using Reward : Literature)

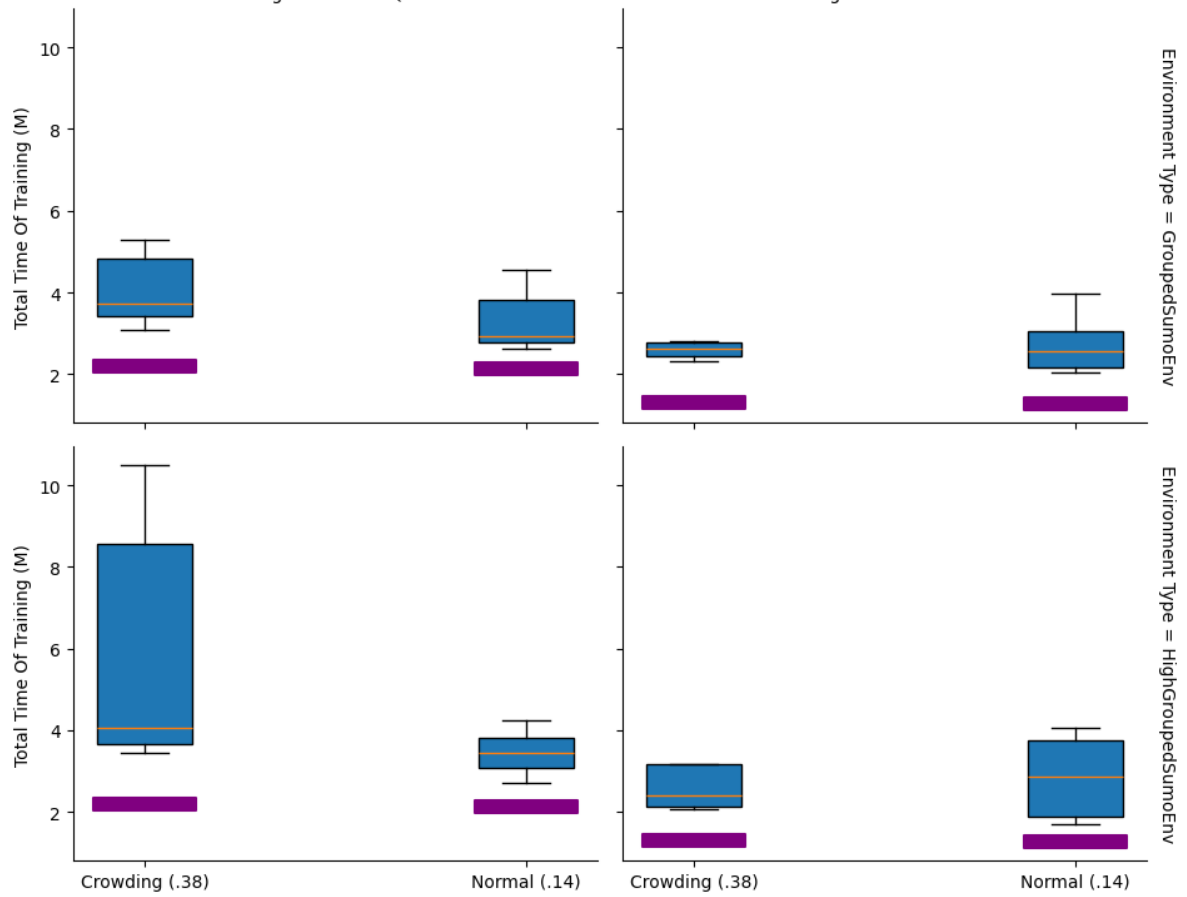
Waiting Car on area Area 3 (San Stefano)



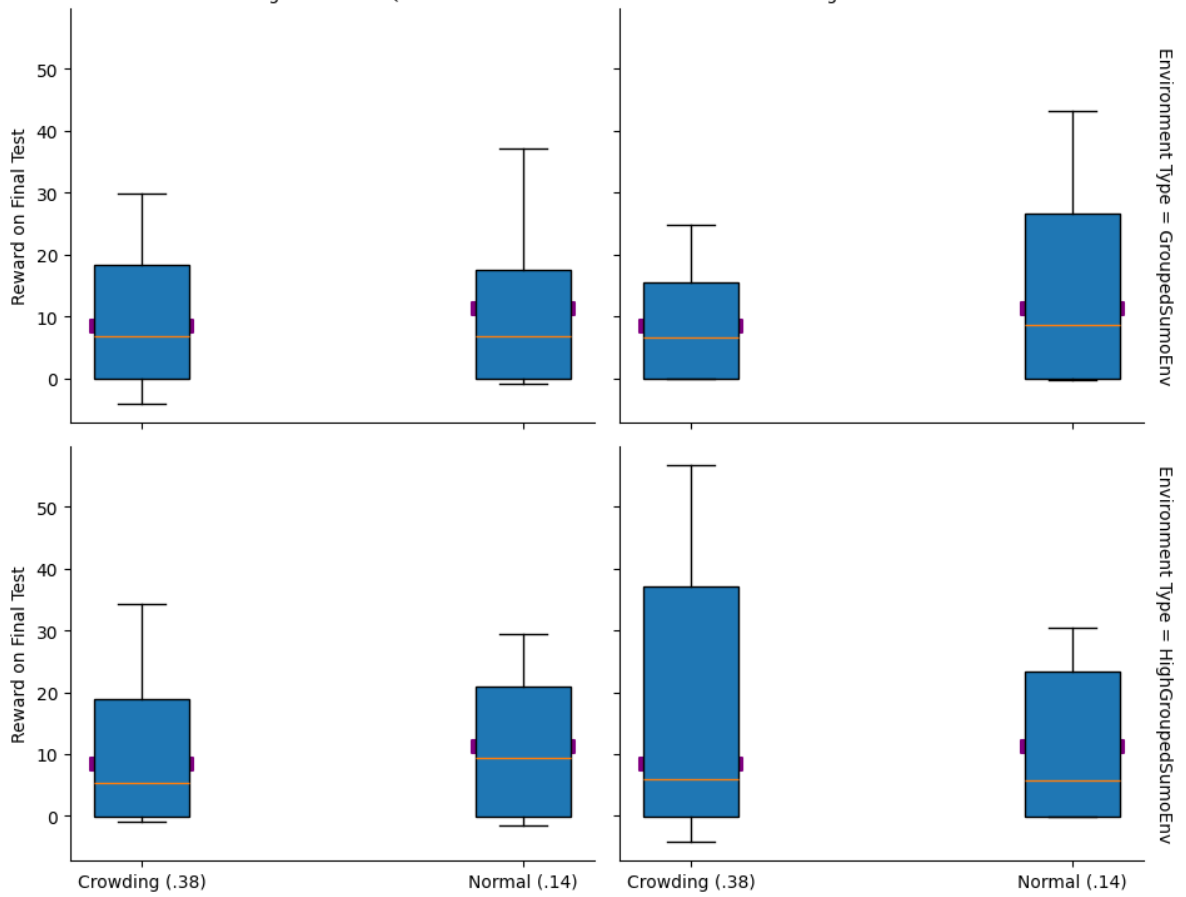
Performance Comparison (Using Reward : Proposed)
 Total Time Of Training (M) on area Area 3 (San Stefano)
 Algorithm = D3QN Algorithm = PPO



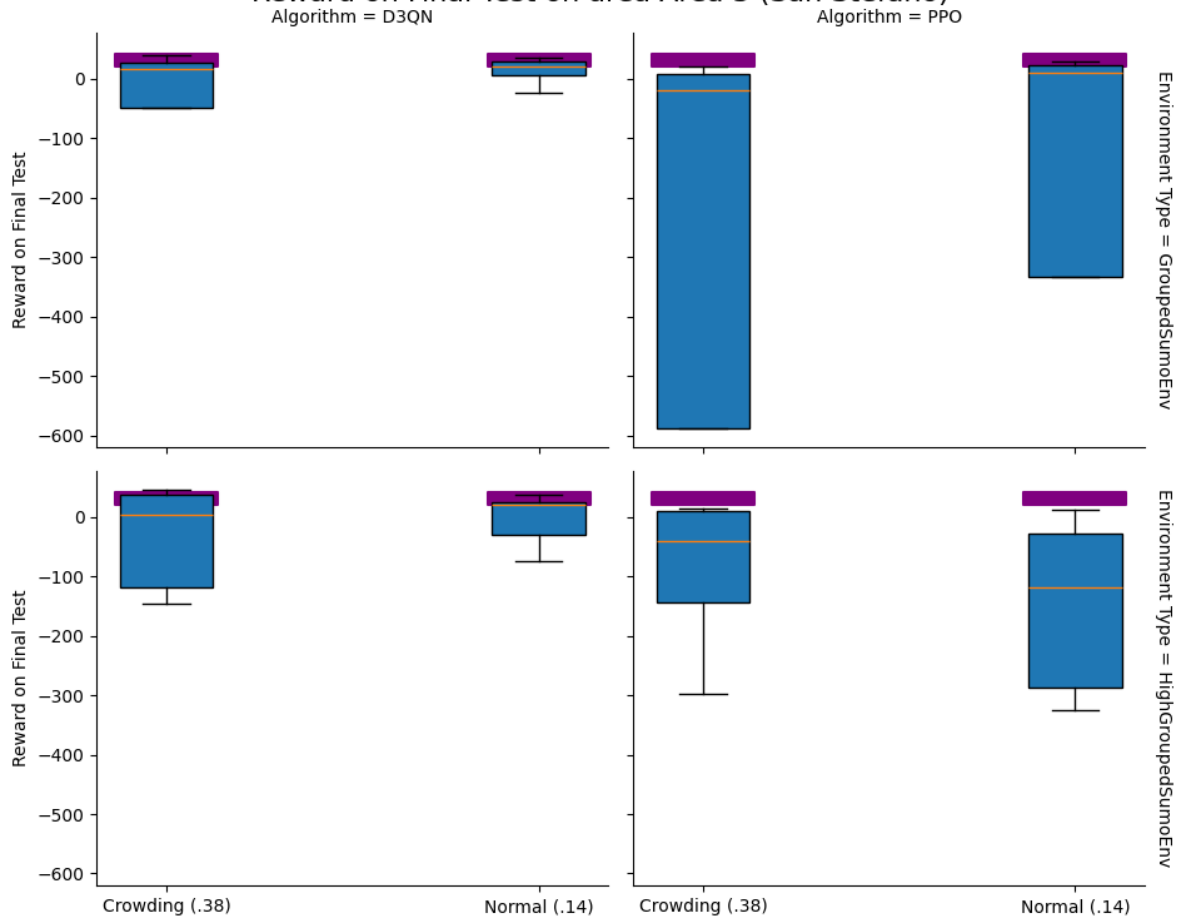
Performance Comparison (Using Reward : Literature)
 Total Time Of Training (M) on area Area 3 (San Stefano)
 Algorithm = D3QN
 Algorithm = PPO



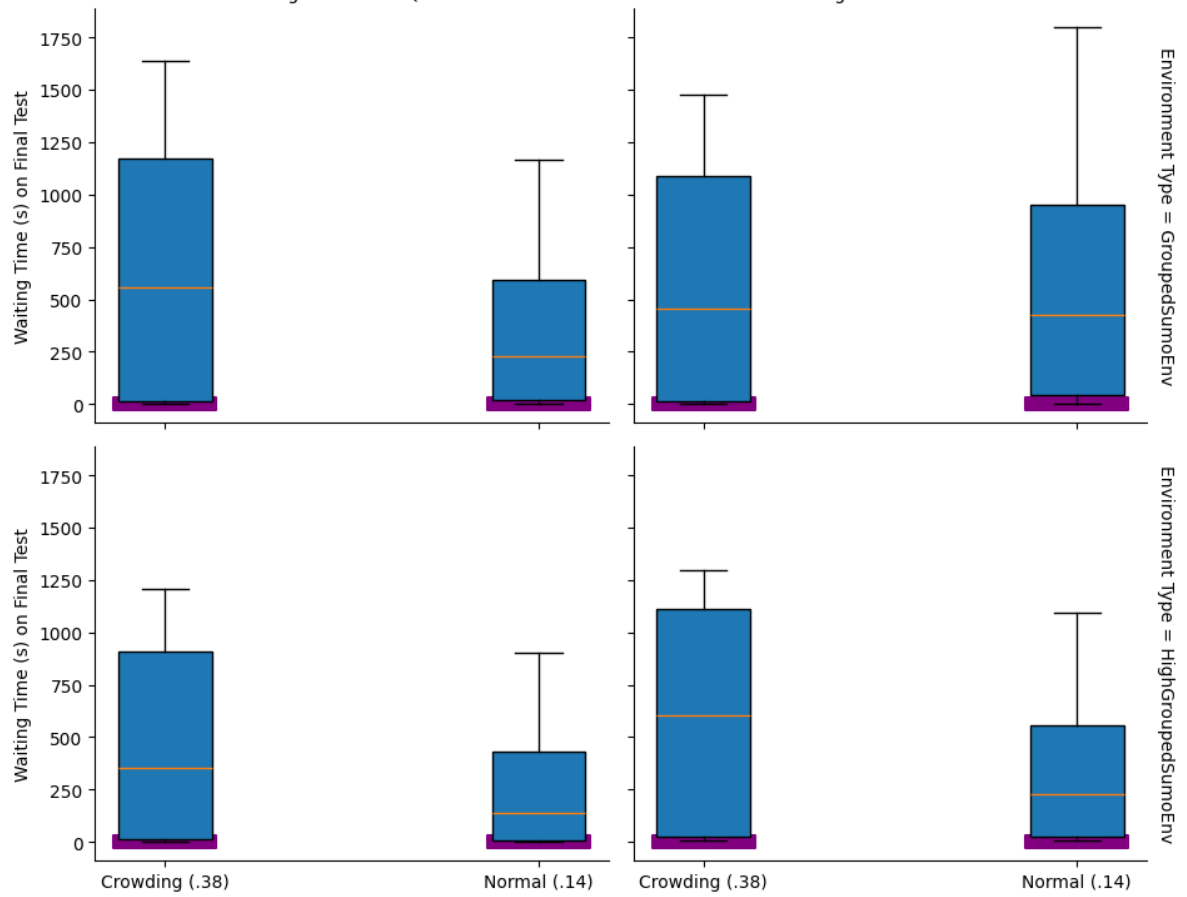
Performance Comparison (Using Reward : Proposed) Reward on Final Test on area Area 3 (San Stefano)



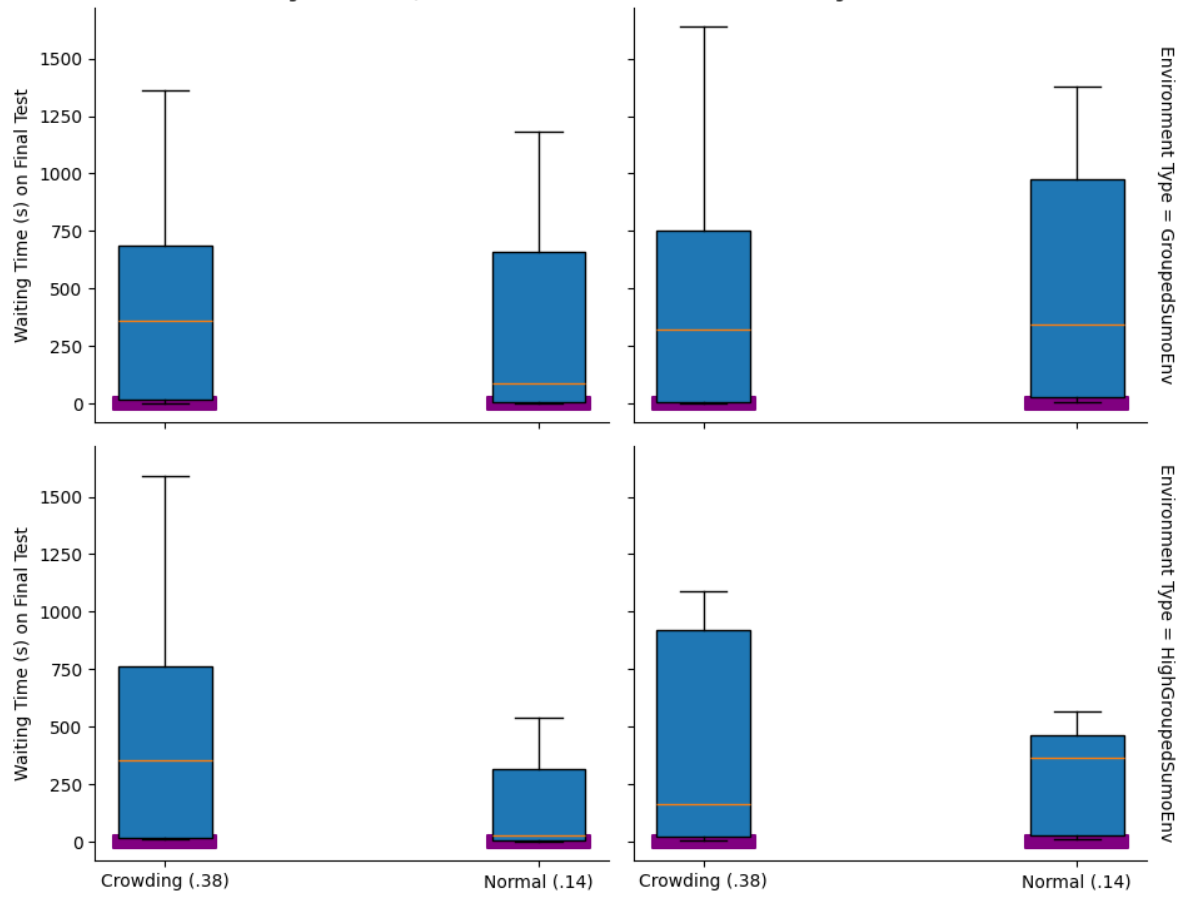
Performance Comparison (Using Reward : Literature) Reward on Final Test on area Area 3 (San Stefano)



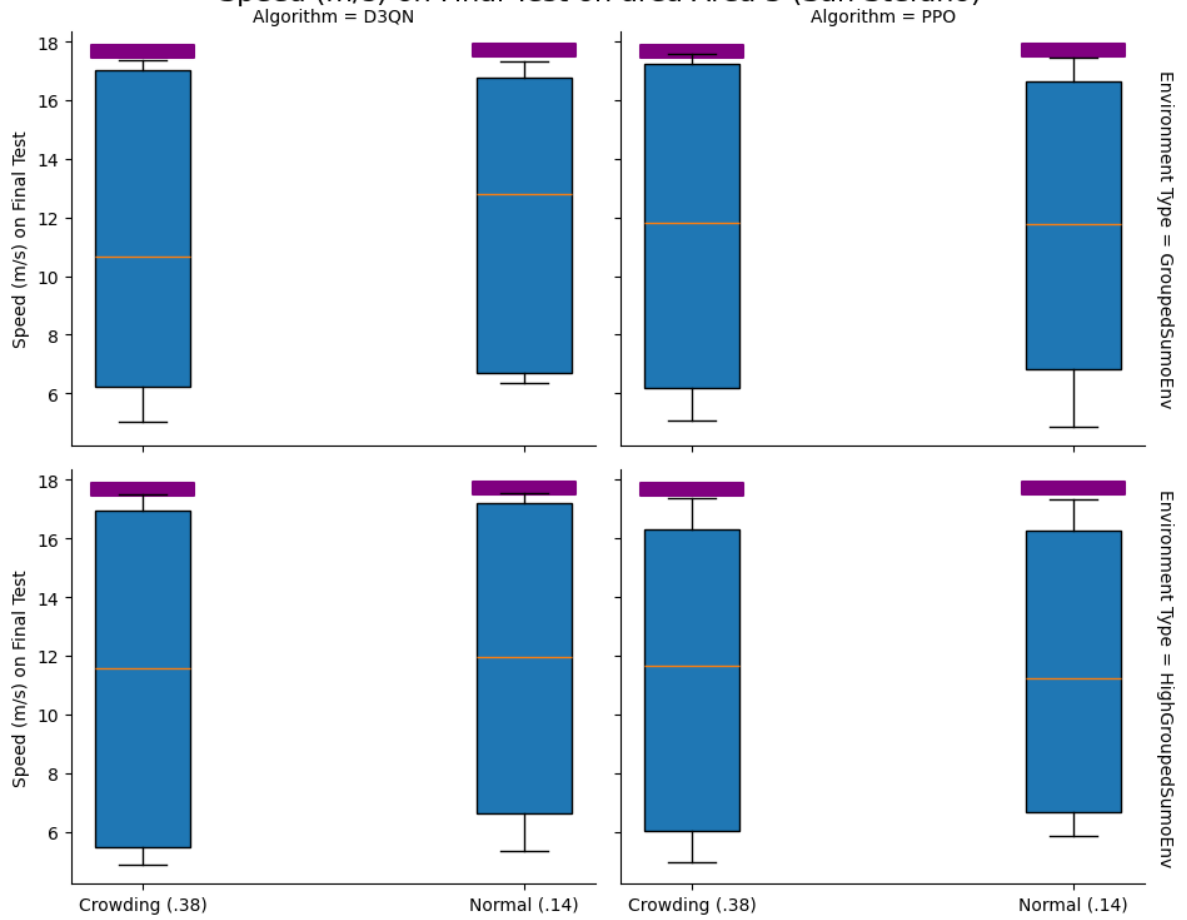
Performance Comparison (Using Reward : Proposed)
 Waiting Time (s) on Final Test on area Area 3 (San Stefano)
 Algorithm = D3QN Algorithm = PPO



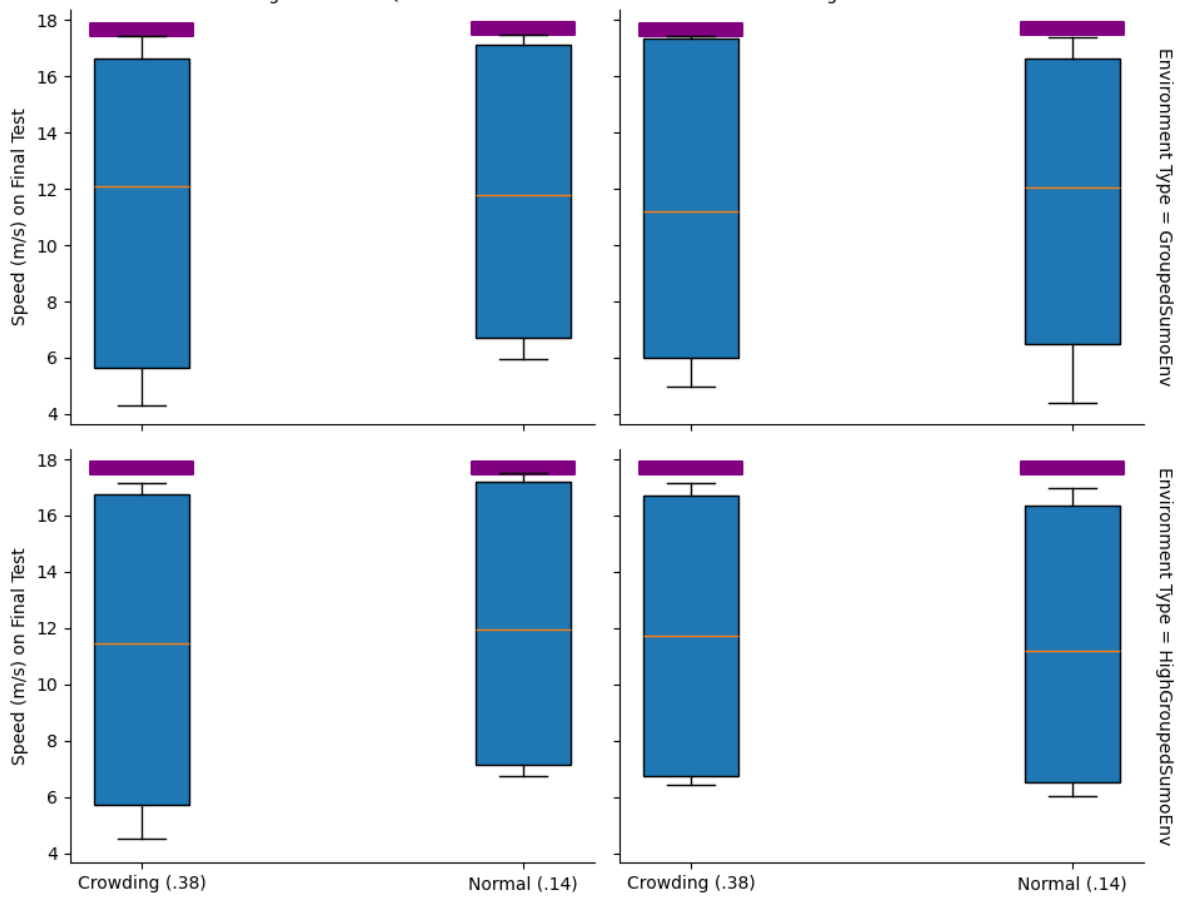
Performance Comparison (Using Reward : Literature)
 Waiting Time (s) on Final Test on area Area 3 (San Stefano)
 Algorithm = D3QN Algorithm = PPO



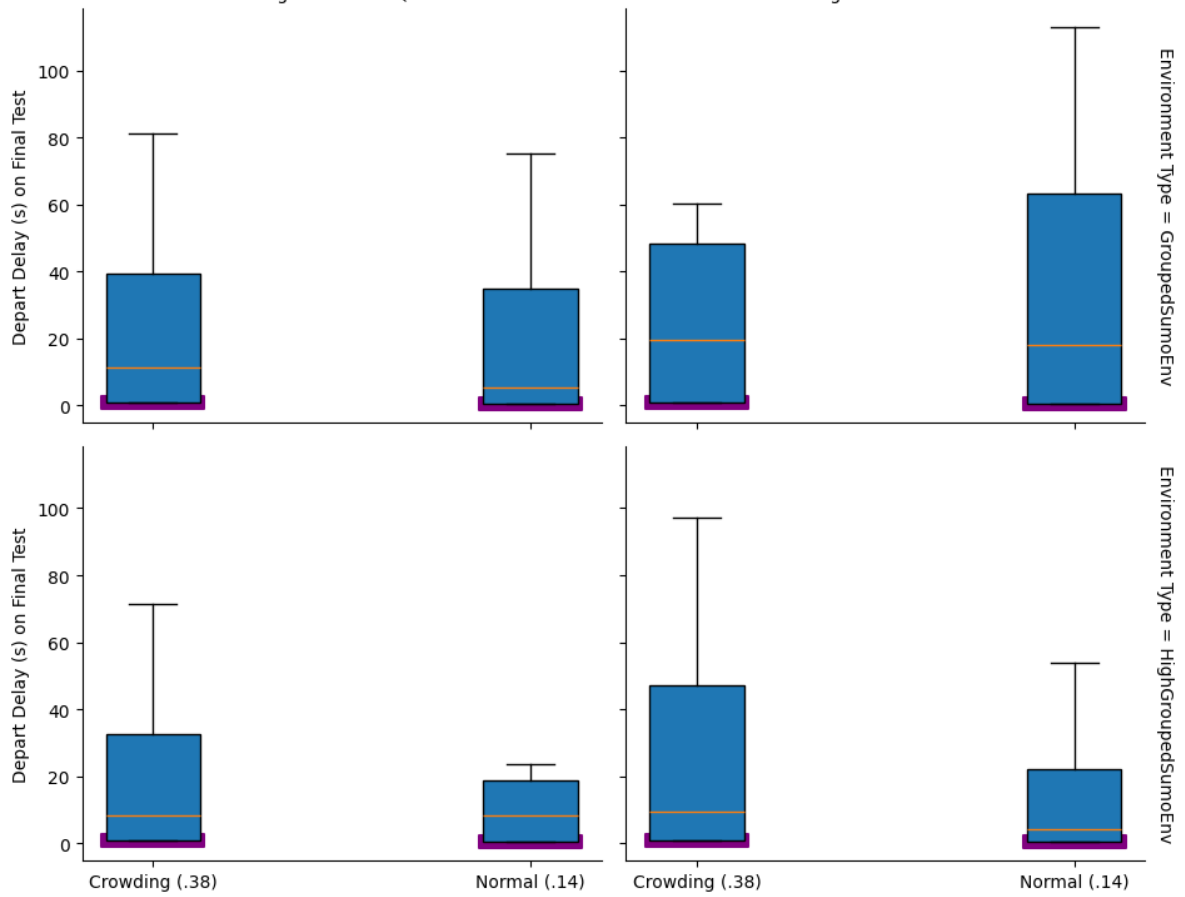
Performance Comparison (Using Reward : Proposed) Speed (m/s) on Final Test on area Area 3 (San Stefano)



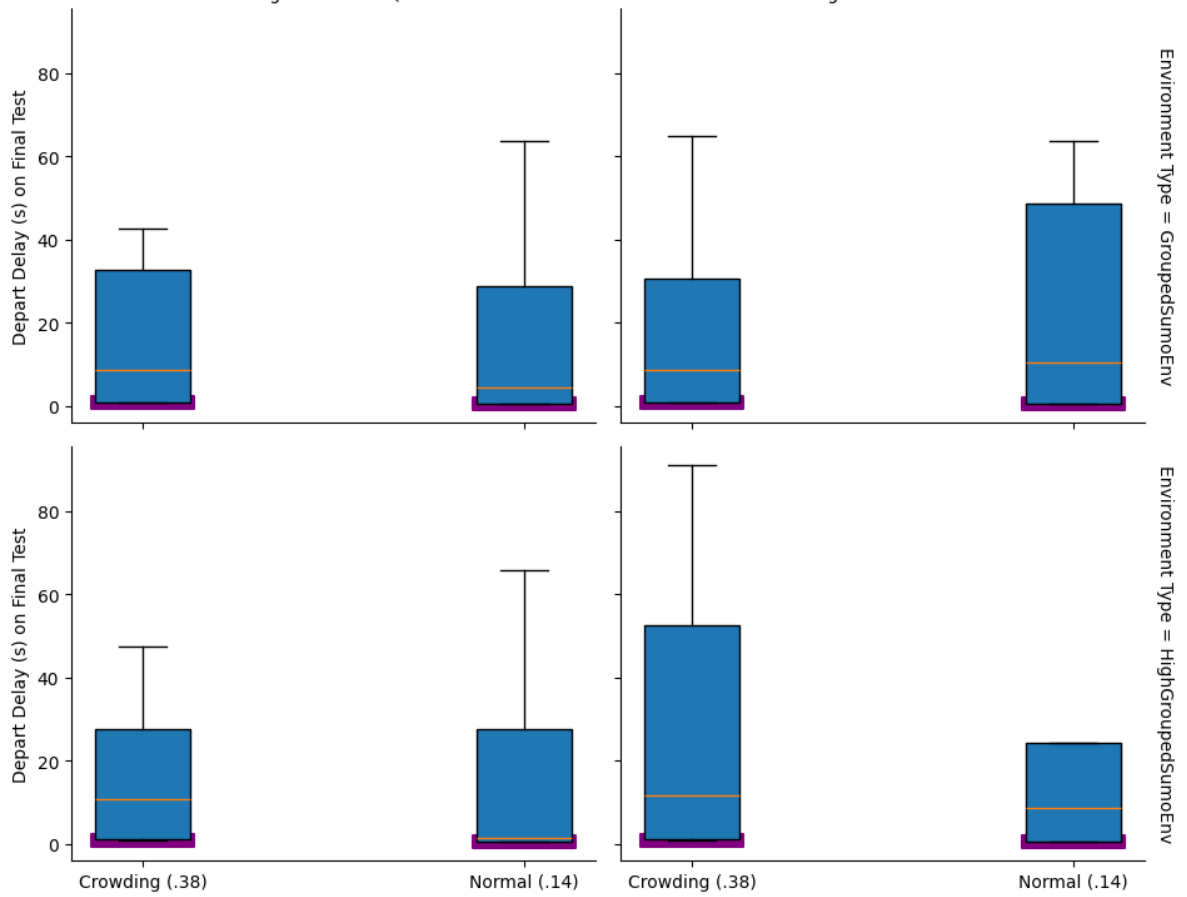
Performance Comparison (Using Reward : Literature)
 Speed (m/s) on Final Test on area Area 3 (San Stefano)



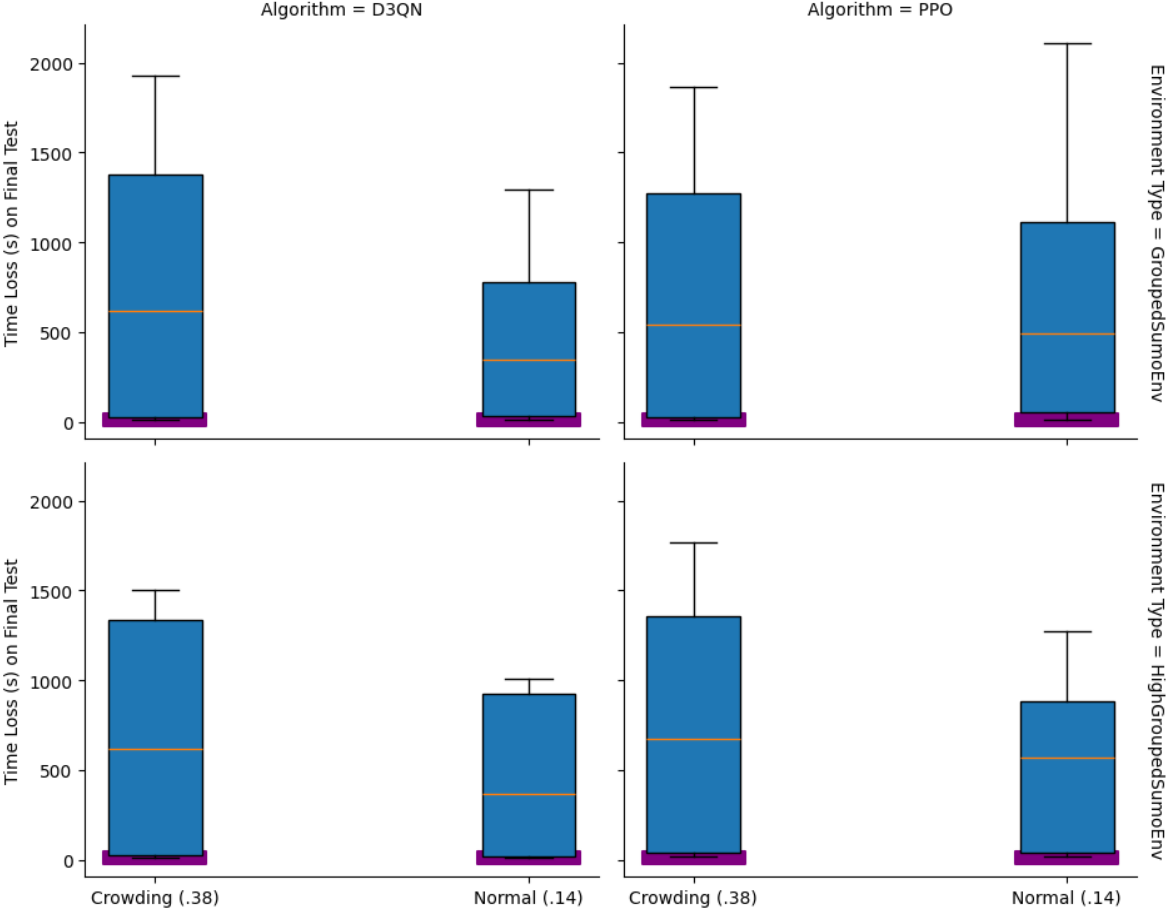
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) on Final Test on area Area 3 (San Stefano)



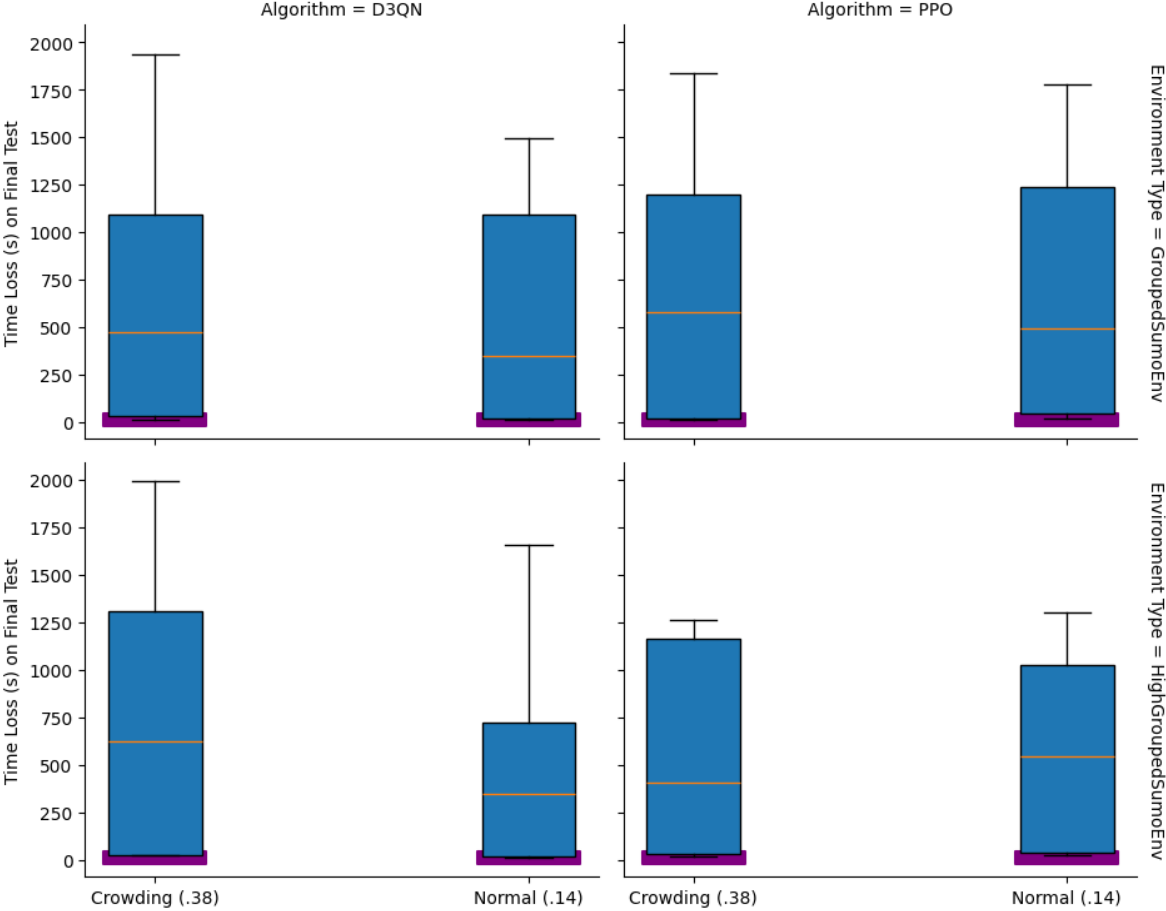
Performance Comparison (Using Reward : Literature)
 Depart Delay (s) on Final Test on area Area 3 (San Stefano)
 Algorithm = D3QN Algorithm = PPO



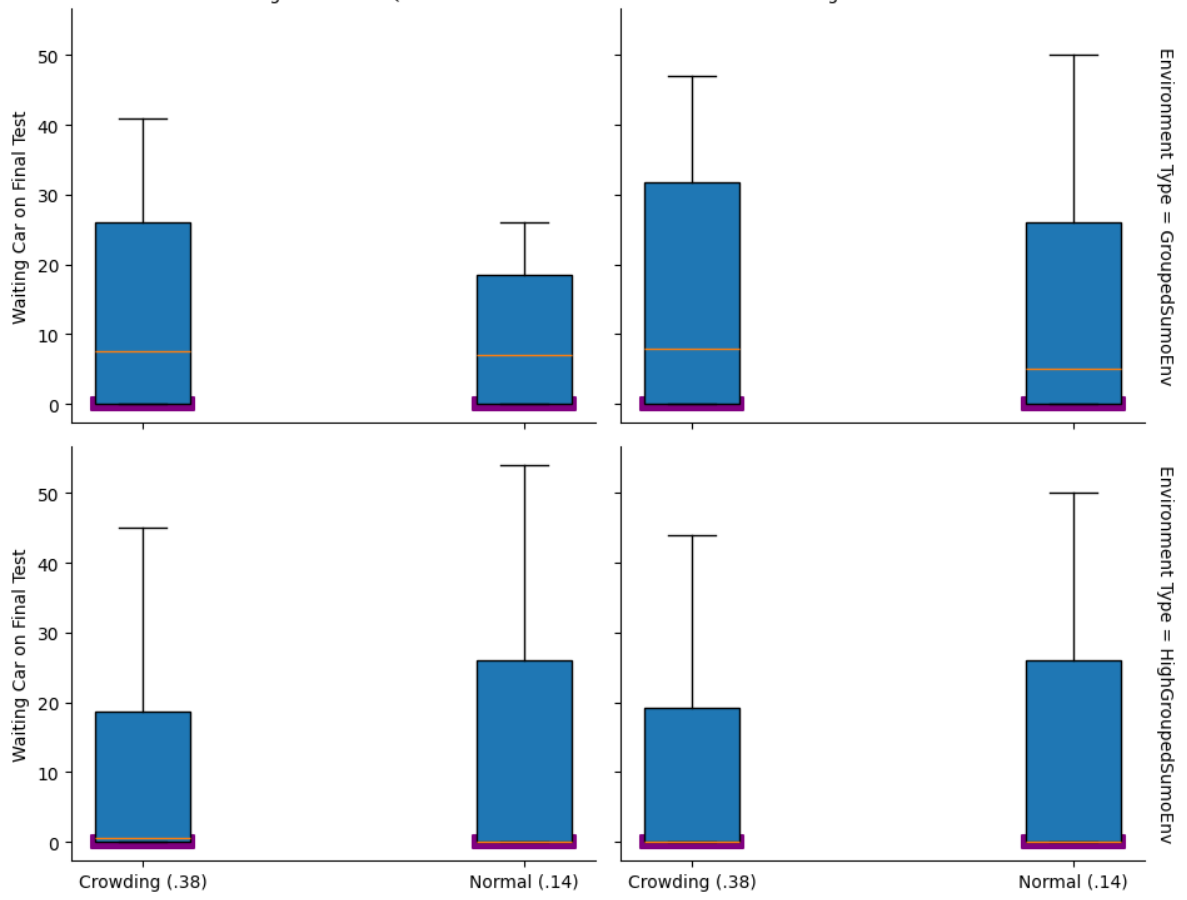
Performance Comparison (Using Reward : Proposed) Time Loss (s) on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Literature) Time Loss (s) on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Proposed)
 Waiting Car on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Literature)
 Waiting Car on Final Test on area Area 3 (San Stefano)

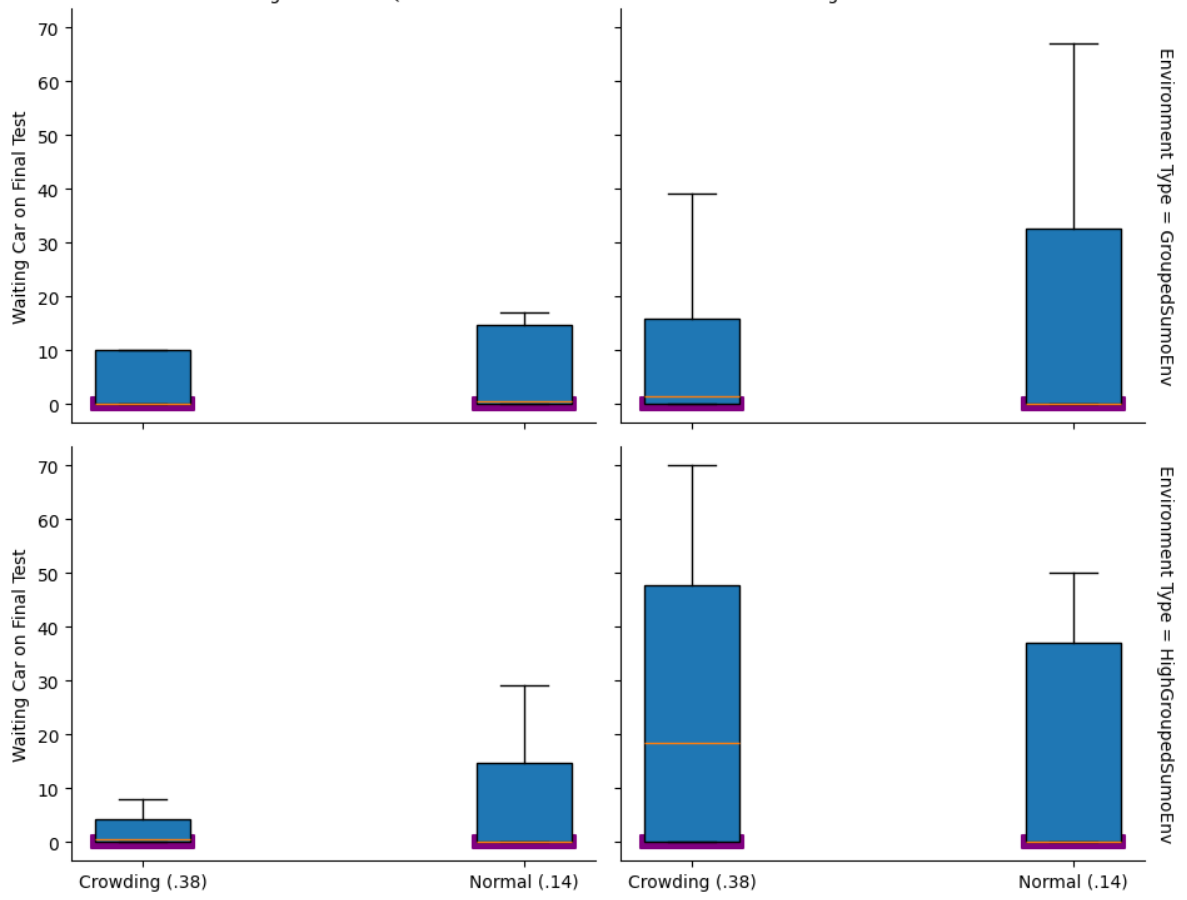


Figure 0.45 Results of Version 2.0.1 at San Stefano

5.4.3.3.3 Analysis of Variance (Anova)

After the visualization was completed, the data was later analyzed using ANOVA to address the question: *Does using a high-grouped environment yield better results compared to a grouped one?*

Table 0.17 Anova results for Version 2.0.1

Area	Reward	Traffic Scale	Algorithm	Reward of Last Episode	Waiting Time (s)	Speed (m/s)	Depart Delay (s)	Time Loss (s)
Area 3 (San Stefano)	Proposed	Normal (.14)	PPO	0.782093	0.861661	0.875426	1	0.869119
Area2 (Mosheer Ismail)	Proposed	Normal (.14)	PPO	0.102876	0.177899	0.795435	0.259052	0.456315
Area 3 (San Stefano)	Literature	Normal (.14)	PPO	0.300514	0.959981	0.766399	0.928534	0.946939
Area2 (Mosheer Ismail)	Literature	Normal (.14)	PPO	0.687844	0.287647	0.236412	0.624494	0.4027
Area 3 (San Stefano)	Proposed	Crowding (.38)	PPO	0.609186	0.903965	0.767729	0.823881	0.898314
Area2 (Mosheer Ismail)	Proposed	Crowding (.38)	PPO	0.252626	0.303793	0.240821	0.211223	0.396364
Area 3 (San Stefano)	Literature	Crowding (.38)	PPO	0.358621	0.626729	0.741556	0.853738	0.608224
Area2 (Mosheer Ismail)	Literature	Crowding (.38)	PPO	0.642148	0.17149	0.309016	0.151636	0.073529
Area 3 (San Stefano)	Proposed	Normal (.14)	D3QN	0.946177	0.324463	0.321173	0.762964	0.322403
Area2 (Mosheer Ismail)	Proposed	Normal (.14)	D3QN	0.315201	0.034894	0.276689	0.19007	0.182739
Area 3 (San Stefano)	Literature	Normal (.14)	D3QN	0.369999	0.420262	0.590412	1	0.440491
Area2 (Mosheer Ismail)	Literature	Normal (.14)	D3QN	0.86881	0.557502	0.970199	0.568192	0.649054
Area 3 (San Stefano)	Proposed	Crowding (.38)	D3QN	0.637802	0.660738	0.692497	0.715241	0.65831
Area2 (Mosheer Ismail)	Proposed	Crowding (.38)	D3QN	0.668569	0.998026	0.859955	0.776257	0.859563
Area 3 (San Stefano)	Literature	Crowding (.38)	D3QN	0.749003	0.843947	0.812639	1	0.848604

Area2 (Mosher Ismail)	Literature	Crowding (.38)	D3QN	0.035141	0.474805	0.45705	0.124167	0.638659
--------------------------	------------	-------------------	------	----------	----------	---------	----------	----------

With a significant level of 97%, most results were found not to show significant improvement. This was initially observed from the plot, same as before.

5.4.3.3.4 Analysis of results

5.4.3.3.4.1 Performance Results

- For the Mosher Ismail dataset, the proposed model demonstrated superior performance in contrast, for the San Stefano dataset, the SUMO-RL package achieved better overall results, however proposed environment still gives acceptable values.
- The SUMO-RL algorithm exhibited rapid convergence for the San Stefano dataset, reaching optimal performance without requiring derivative adjustments, indicating that initial weights provided sufficient learning. Across both rewards, SUMO-RL learned largely similar information, yielding nearly identical average results,
- Literature reward divergence approached zero in both datasets, this was anticipated, as previous studies [3] reported negative or zero rewards when applying simple algorithms, motivating the introduction of new models.
- Reward effectiveness appeared to vary across environments, with the proposed reward outperforming in the benchmark scenario but not in our environment in Mosher Ismail (with waiting time, waiting car, depart delay).
- The HighGrouped configuration proved more efficient, delivering superior results and lower variance in departure delays, particularly in area 2.
- Among the tested algorithms, the performance differences were relatively small; however, PPO exhibited noticeably higher stability, especially in the SUMO-RL environment. These observations are consistent with the ANOVA analysis, though the impact of literature-based reward functions was more distinctly visualized through comparative results.
- The proposed model behaved well in both normal and crowded cases but with more randomness than sumo-rl.
- Randomness inherent in RL models persisted, especially in later versions. The proposed model exhibited improved generalization, avoiding overfitting during extended test periods. Some metrics, such as waiting time timeloss reaching up to 2000 s, represented worst-case scenarios and were amplified by total simulation time; however, most performance metrics converged satisfactorily.

5.4.3.3.4.2 Best Performance Metrics

Mosher Ismail Dataset:

- **3000 s simulation:**

- Waiting time: 30 s (range 20s–8M)
- Speed: 13 m/s (range 8m/s–18m/s)
- Departure delay: 5 s (range 3s–30s)
- Timeloss: 90 s = 1.5M (range 50s–8M)
- Waiting cars: 10 (range 5–50)
- **10800s (3 h) simulation:**
 - Waiting time: 300 s = 5 M (range 50s–25M)
 - Speed: 12 m/s (range 8m/s–18m/s)
 - Departure delay: 11 s (range 3s–2M)
 - Timeloss: 500 s = 7.5 M (range 50s–33M)
 - Waiting cars: 10 (range 5–80)

San Stefano Dataset:

- **3000 s simulation:**
 - Waiting time: 20 s (range 0–6M)
 - Speed: 14 m/s (range 6–18)
 - Departure delay: 2 s (range 0–35s)
 - Timeloss: 90 s = 2M (range 0–6M)
 - Waiting cars: 3 (range 0–35)
- **10800 s simulation:**
 - Waiting time: 300s = 5 M (range 0–25M)
 - Speed: 12 m/s (range 4m/s–18m/s)
 - Departure delay: 9 s (range 0–1.3M)
 - Timeloss: 600 s = 10 M (range 0–33M)
 - Waiting cars: 3 (range 0–70)

So, our model achieved good results while not being overfit.

5.4.3.3.4.3 Comparison to Previous Versions

- Episode length did not significantly affect performance, indicating that early stopping was sufficient.
- D3QN exhibited high variability due to random seed effects, Unlike Version 1.0.0 PPO now is more stable with seeds especially in sumo-rl.

- Modifications to the simulation area using more realistic parameters substantially reduced the number of waiting cars; previously, high counts were partially attributable to environmental settings.
- Departure delays were significantly decreased from 1.5–13.3 minutes per half-hour to a maximum of one minute per 3000 s.
- Timeloss decreased, though it remained sensitive to simulation conditions.

Overall, the proposed model demonstrated robust performance across datasets, with reduced variance and improved generalization compared to previous versions, while SUMO-RL achieved rapid convergence and consistent results in certain scenarios.

As shown previously, the system’s performance is affected by a bug in the reward function. This issue will be addressed and resolved in the next version.

5.4.4 Version 2.0.2

5.4.4.1 Key Modifications

In this version, the reward issue identified in the previous release has been resolved. In addition, benchmarking was introduced using the RESCO benchmark on an alternative dataset.

5.4.4.1.1 Reward Improvement

In this version, the reward-related issues have been addressed.

- The variance issue caused by dividing by waiting time was resolved by reformulating the operation as a multiplication.
- Unit inconsistencies were corrected to ensure that all reward components use consistent units.
- Reward weights were refined to better align with SUMO’s expected behavior.

In addition, further reward-design experiments were conducted:

- The reward was adjusted to ensure balanced contributions across all scales or nearby scales, preventing dominance by any single component.
- Waiting time was identified as the most critical factor and given particular emphasis.

- A logarithmic transformation was applied to waiting time as it grows exponentially, preventing it from disproportionately affecting the overall reward.
- Since throughput and queue length are often correlated, the weight of queue length (w_2) was set higher than that of throughput (w_1). This choice reflects the fact that queue length serves as a leading indicator of congestion, whereas throughput is a lagging indicator; preventing queue formation is preferable to rewarding vehicles for clearing congestion after it occurs.

$$R(s) = \alpha \cdot \left(\frac{v}{v_{max}} \cdot \frac{1}{1 + \sigma_v^2} \right) - \beta * \ln \left(1 + \text{clip} \left(\frac{w}{w_{max}}, 0, 1 \right) \cdot (1 + \sigma_w^2) + 10^{-6} \right) + \gamma \left(\frac{w_1 \cdot \underline{t}}{t_{max}} - \frac{w_2 (\underline{q} - q_{min})}{q_{max}} \right)$$

Equation 0.8 Modified reward of version 2.0.2

Where:

$\underline{v}$ is average speed, σ_v^2 is variance of speed, $\underline{w}$ is the average waiting time, σ_w^2 is variance of waiting time, $\underline{t}$ is average of throughput, $\underline{q}$ is average of queue length.

$w_1, w_2, \alpha, \beta, \gamma$:is weight for terms, configured when testing the reward ($w_1 = .3, w_2 = .7, \alpha = 3, \beta = 3, \gamma = 4$), $v_{max}, w_{max}, q_{max}, t_{max}$ is max estimated speed, waiting time, queue length and throughput accordingly, $v_{max} = 30, w_{max} = 300, q_{max} = 20, q_{min} = 5, t_{max} = 50$.

Reward Function Validation

For reward testing, the test cases have been modified to better represent system states, ensuring that units are consistent and aligned with both SUMO and real traffic conditions.

Table 0.18 Validation of the reward of version 2.0.2

State	State Explain	Description	Expected Behaviour	Observed
(28.5, 0.5, 2.0, 1.0, 45.0, 0.5, 5.0)	Very high speed, very low speed variance, near-zero waiting, minimal queues, very high throughput, very low occupancy	High speed, near-zero waiting, high throughput. Perfect green wave.	Smooth uninterrupted traffic flow, signals well synchronized, vehicles rarely stop	3.570

(22.0, 2.0, 15.0, 5.0, 20.0, 2.0, 15.0)	Good speed, low variance, moderate waiting time, short queues, reasonable throughput, low occupancy	Stable traffic with occasional stops. Typical off-peak behavior.	Mostly smooth flow with brief stops at signals	0.846
(12.0, 8.0, 45.0, 20.0, 35.0, 10.0, 40.0)	Medium speed, higher variance, long waiting time, growing queues, high throughput, moderate occupancy	Heavy volume but still moving. High throughput but higher waiting times.	Congested yet moving traffic, signals under pressure	-3.996
(5.0, 25.0, 120.0, 150.0, 15.0, 18.0, 60.0)	Low speed, very high variance, long waiting times, large queues, low throughput, high occupancy	High speed variance and waiting time. Inefficient signal timing.	Stop-and-go traffic with frequent halts	-13.793
(0.5, 0.1, 1200.0, 500.0, 2.0, 29.0, 95.0)	Almost zero speed, negligible variance, extreme waiting, maximum queues, almost no throughput, near-full occupancy	Almost zero movement. Maximum queue and waiting time penalty.	Complete gridlock, traffic collapse	-21.922
(15.0, 12.0, 300.0, 800.0, 10.0, 15.0, 30.0)	Medium speed, high variance, extreme waiting on some lanes, long queues, low throughput	High variance in waiting time suggests one road is stuck while others move.	Severe imbalance between main and side roads	-21.106
(2.0, 40.0, 60.0, 300.0, 5.0, 25.0, 85.0)	Very low speed, extremely high variance, long waits, large queues, very low throughput, high occupancy	Low speed and massive variance. Likely just after a light turned red on a full lane.	Sudden congestion shock followed by heavy queuing	-15.018

5.4.4.1.2 SUMO-RL DATA Benchmark (2WaySingle)

In this version, a new benchmark paradigm was introduced to facilitate a more detailed evaluation of performance and to enable comparison with benchmark environments and benchmark datasets.

The SUMO-RL package provides multiple benchmark environments, including those based on RESCO and others. For the proposed environment, the most suitable configuration—considering a single-agent setup and appropriate code—was the benchmark environment with a single-direction layout and large simulation time.

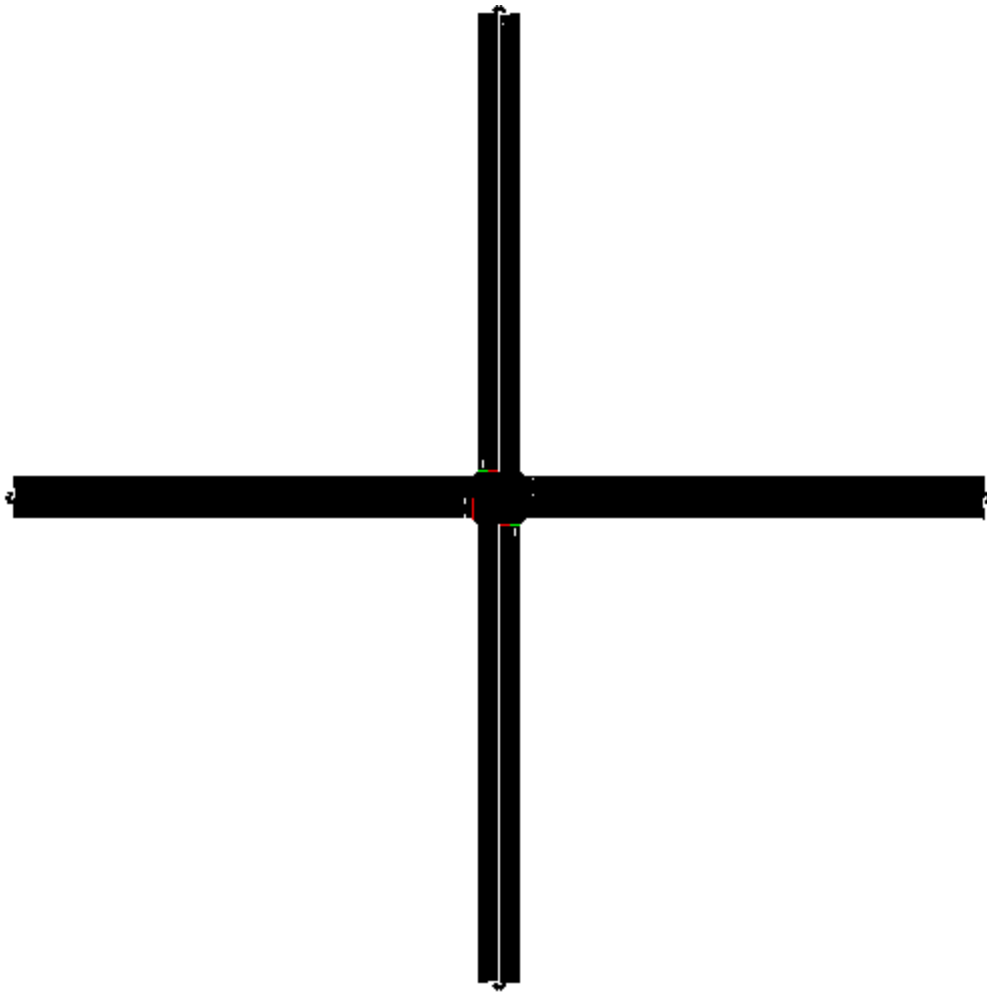


Figure 0.46 Area used in sumo-rl Bench

Sumo-RL Default Reward

The default reward function is the change in cumulative vehicle delay:

$$r_t = D_{a_t} - D_{a_t+1}$$

Equation 9 Sumo-RL Default Reward

That is, the reward is how much the total delay (sum of the waiting times of all approaching vehicles) changed in relation to the previous time-step.

5.4.4.1.3 SUMO-RL DATA Experiment Design

Table 0.19 SUMO-RL DATA Experiment Design

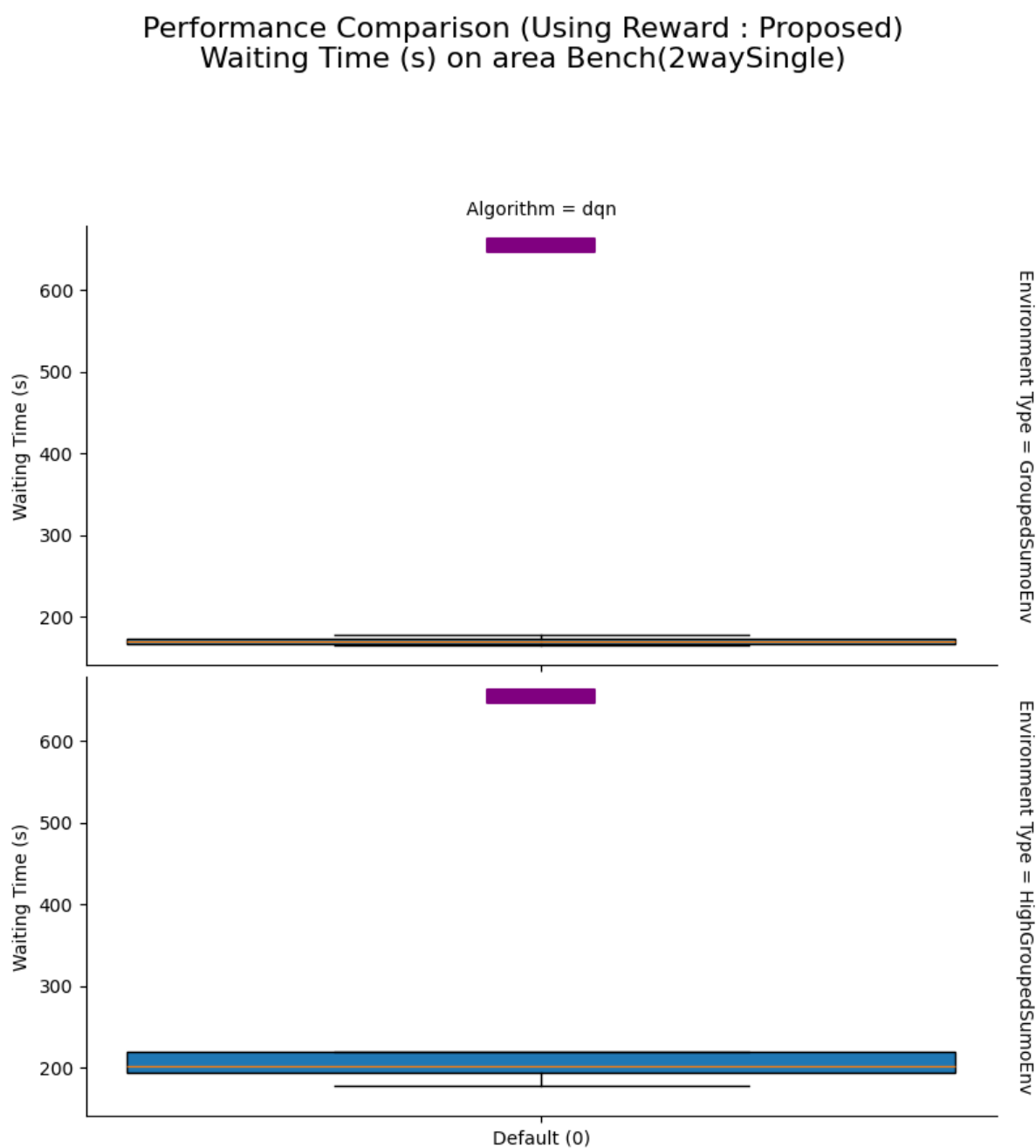
Parameter	Meaning	Real meaning	Range for experiment
-----------	---------	--------------	----------------------

Run	Experiment repetition count	The number of times the experiment is repeated to compare environments and ensure consistency	Sumo-rl , Our proposed environment.
Area	The dataset or environment used	Country-specific traffic data	2WaySingle
Reward	Type of reward function used	Determines how traffic control goals are prioritized (e.g., efficiency, fairness)	(Literature – Proposed) for proposed environment and Default reward for sumo-rl
Environment type	Type of SUMO environment used	Level of traffic control detail and intersection grouping	(GroupedSumoEnv HighGroupedSumoEnv) For proposed environment and Sumo-RL default reward
Algorithm	RL algorithm applied	Different learning strategies used to optimize traffic control	DQN
Episodes	Number of training episodes	Number of training cycles (longer = more learning)	1
Max sumo steps	Maximum number of simulation steps in SUMO	The number of times the experiment is repeated to compare environments and ensure consistency	100000s
Traffic Scale	Traffic load intensity	Adjusts the number of vehicles generated in the simulation	Default (1.0)
Seeds	Random seed for reproducibility	Run with different seeds to ensure stability and robustness against randomness	0,1,2,3

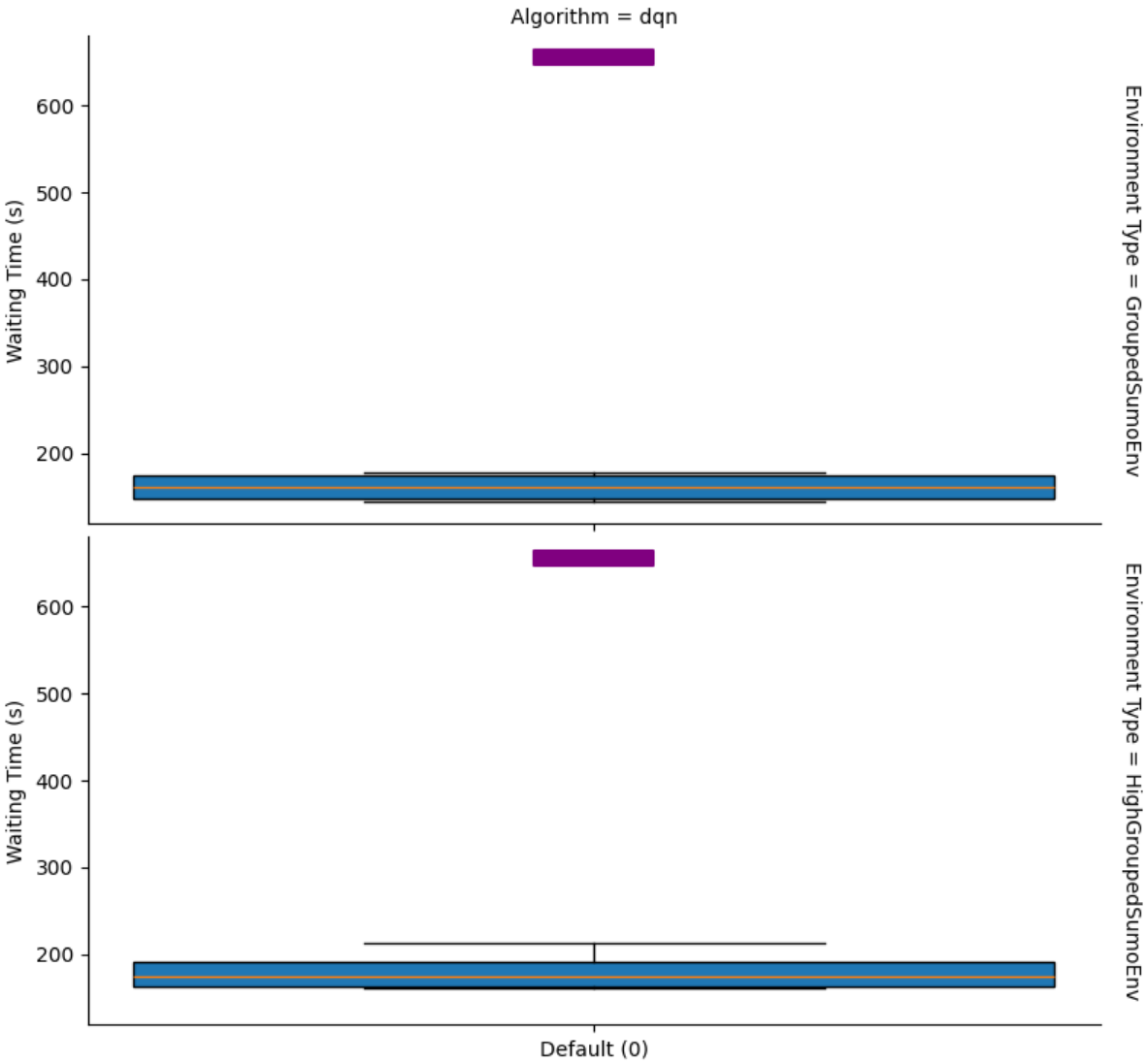
5.4.4.2 Results

5.4.4.2.1 Sumo-RL Bench

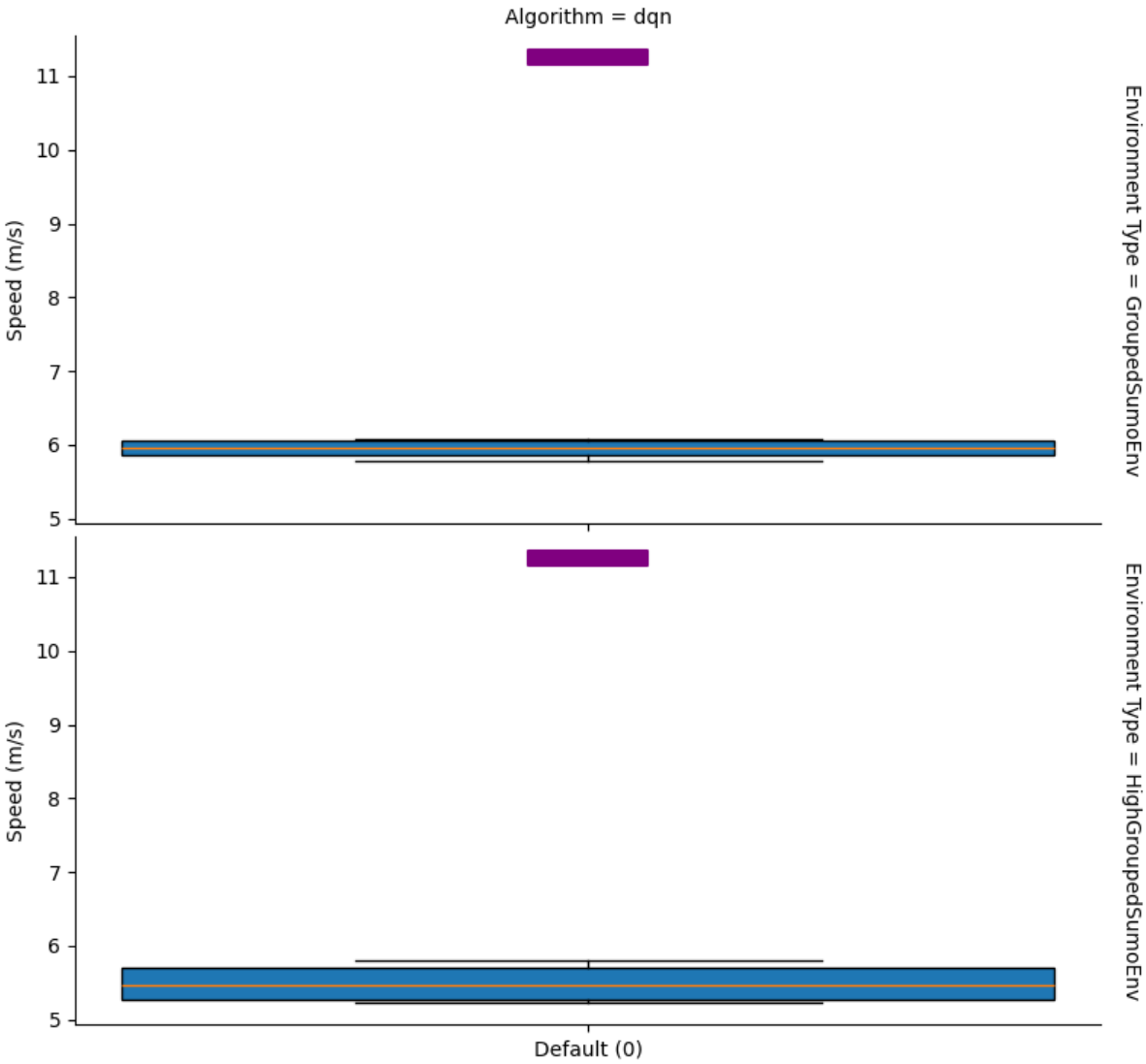
5.4.4.2.1.1 Results



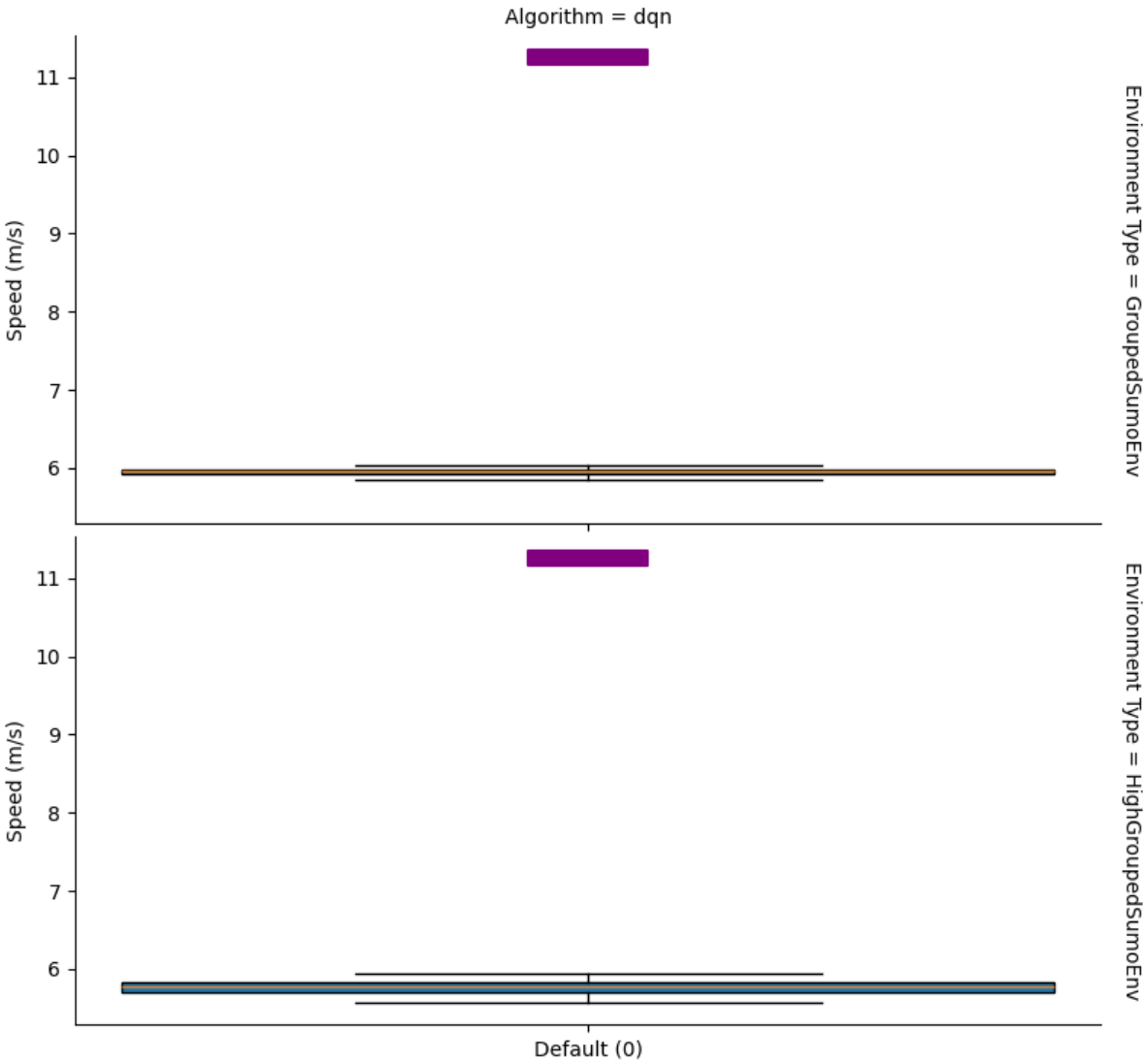
Performance Comparison (Using Reward : Literature)
Waiting Time (s) on area Bench(2waySingle)



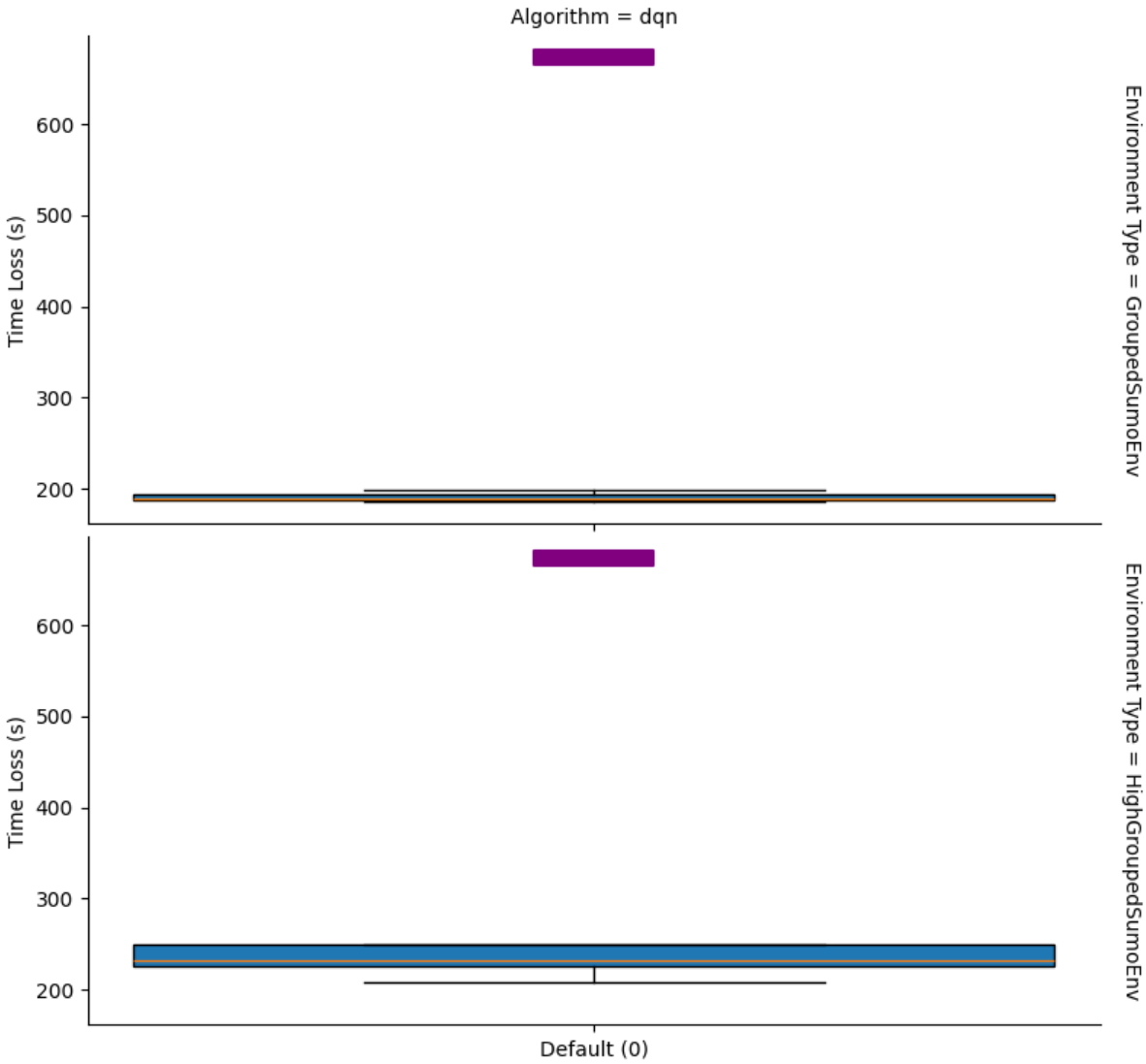
Performance Comparison (Using Reward : Proposed)
Speed (m/s) on area Bench(2waySingle)



Performance Comparison (Using Reward : Literature)
Speed (m/s) on area Bench(2waySingle)



Performance Comparison (Using Reward : Proposed)
Time Loss (s) on area Bench(2waySingle)



Performance Comparison (Using Reward : Literature) Time Loss (s) on area Bench(2waySingle)

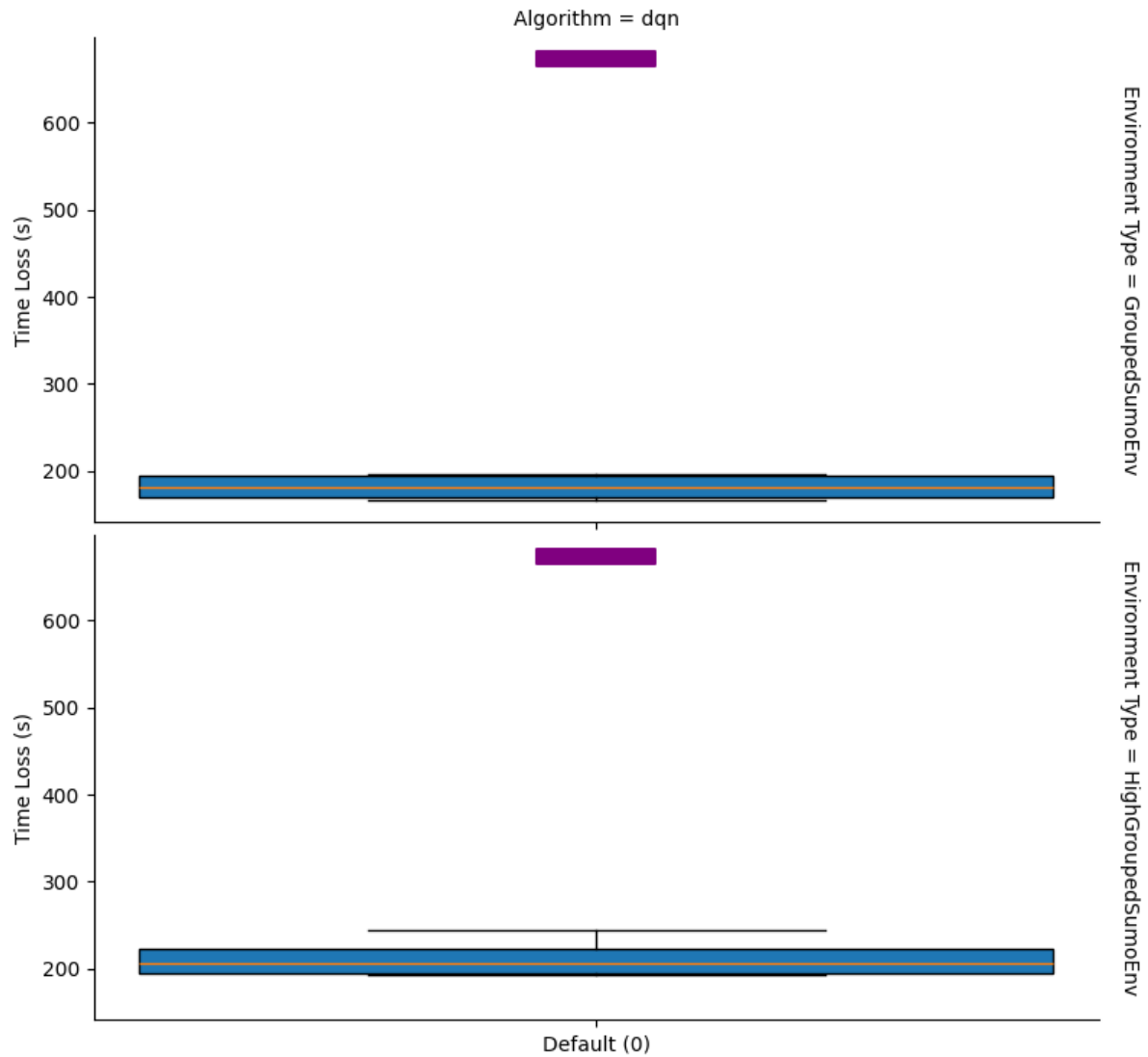


Figure 0.47 Results of Sumo-RL Bench

Reward is not plotted as different between one used in sumo-rl and the other on our environment.

5.4.4.2.1.2 Anova

Table 0.20 Anova for Sumo-rl Bench

Area	Reward	Traffic Scale	Algorithm	Reward of Last Episode	Waiting Time (s)	Speed (m/s)	Depart Delay (s)	Time Loss (s)
Bench(2waySingle)	Literature	Default (1.0)	dqn	0.030031	0.229831	0.068259	0.000208	0.072813
Bench(2waySingle)	Proposed	Default (1.0)	dqn	0.351531	0.084048	0.026958	0.003367	0.043407

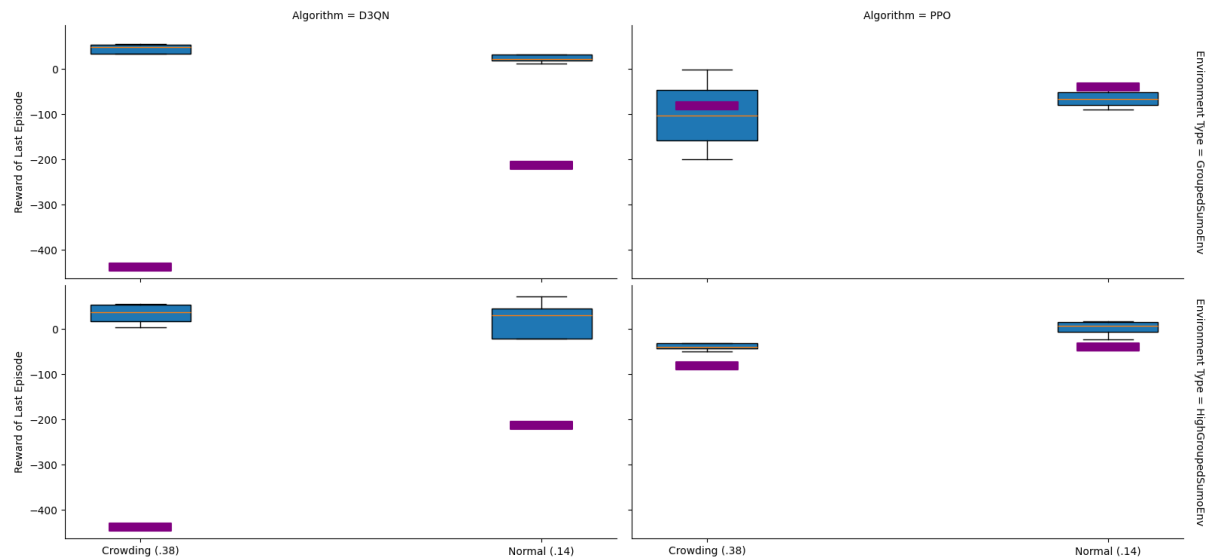
5.4.4.2.1.3 Discussion

- The proposed environment achieved lower waiting time and time loss; however, this improvement was associated with a reduction in average vehicle speed.
- The grouped environment demonstrated greater stability, particularly in terms of variance and randomness
- No statistically significant difference was observed between the environments with respect to the reward metric.

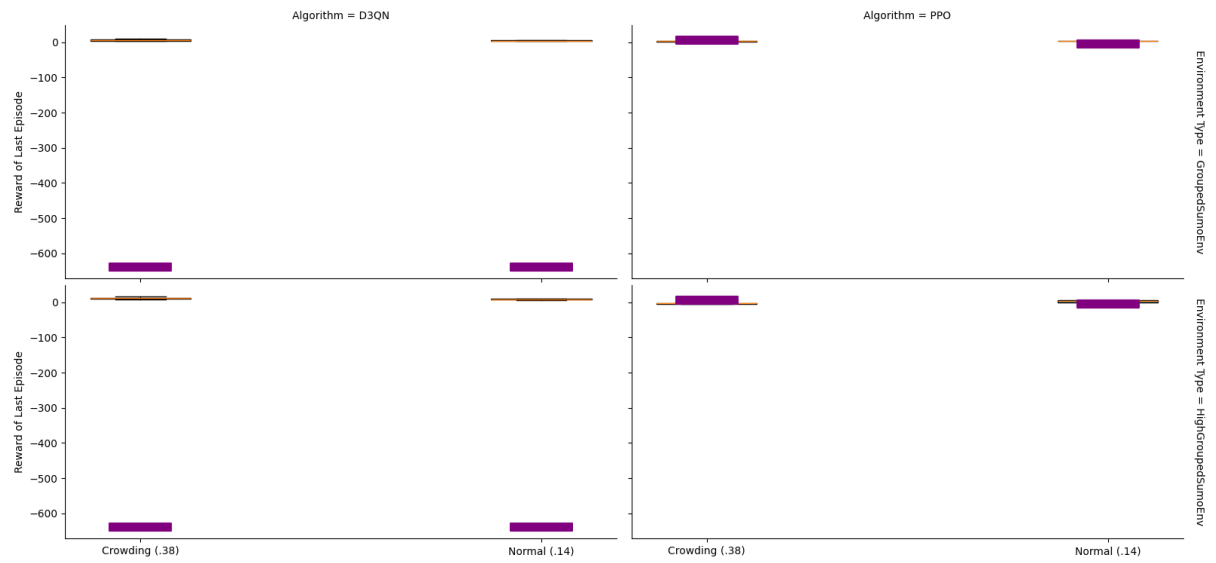
5.4.4.2.2 Environmental experiments

5.4.4.2.2.1 Mosher

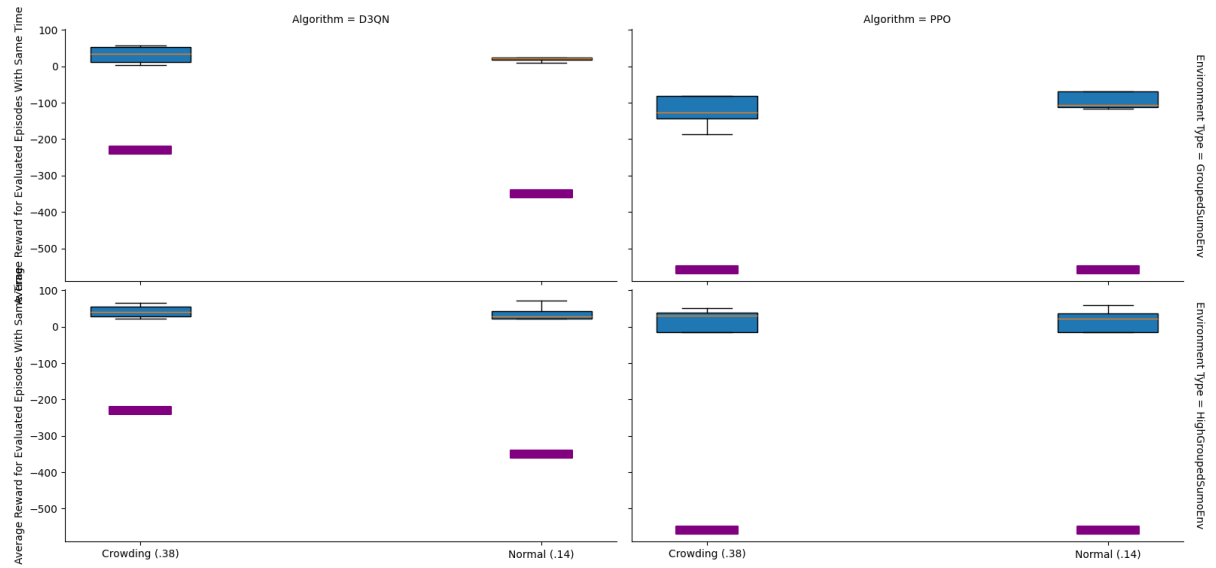
Performance Comparison (Using Reward : Proposed)
Reward of Last Episode on area Area2 (Mosheer Ismail)



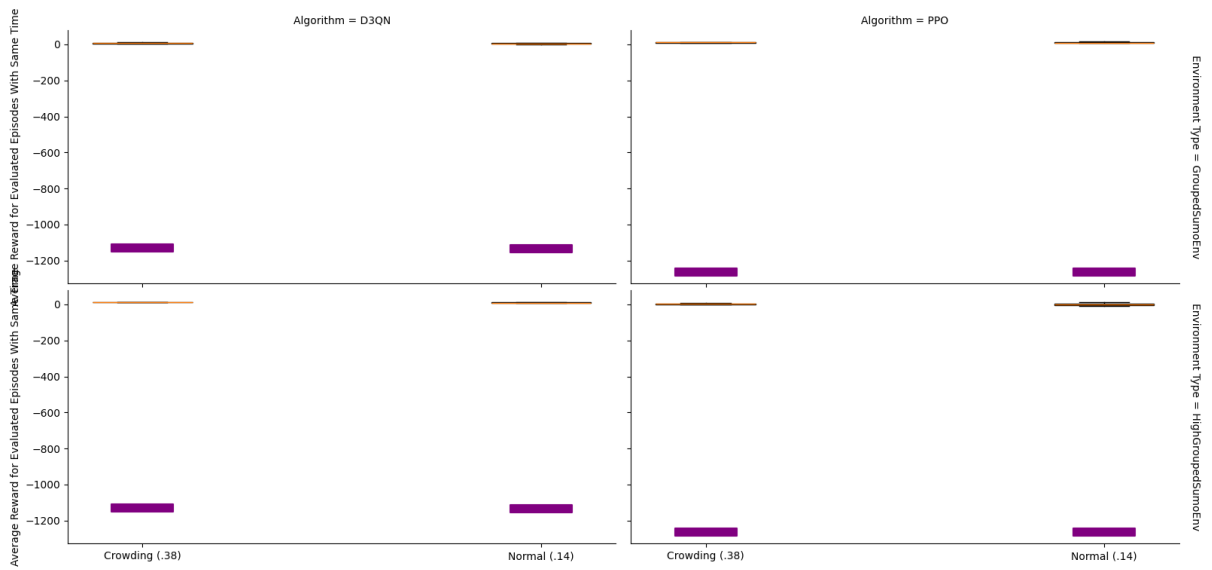
Performance Comparison (Using Reward : Literature)
Reward of Last Episode on area Area2 (Mosheer Ismail)



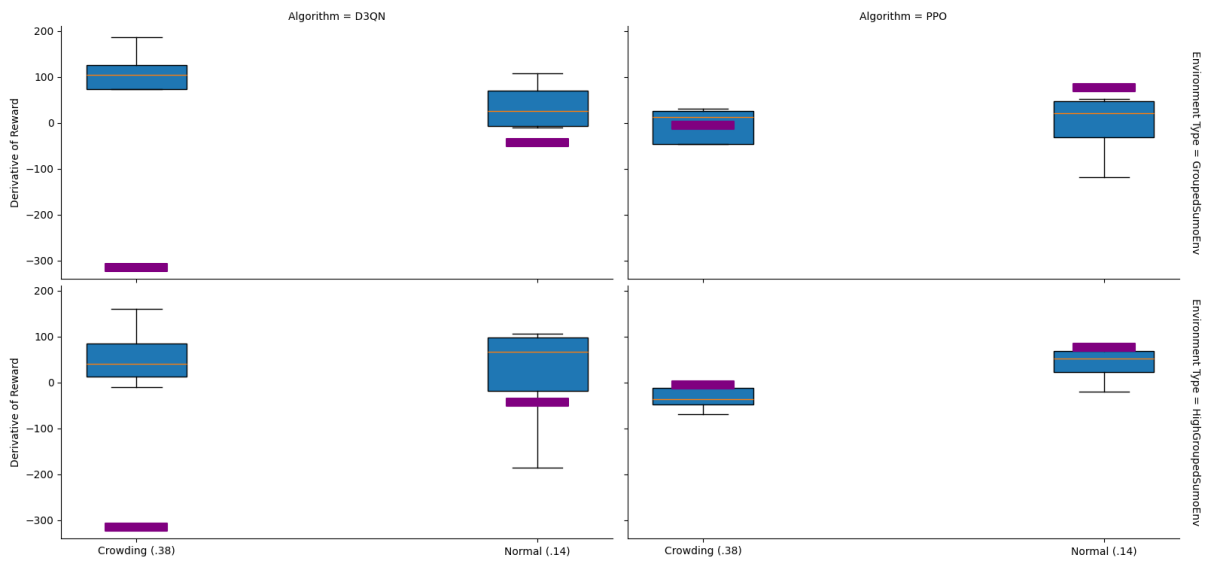
Performance Comparison (Using Reward : Proposed)
Average Reward for Evaluated Episodes With Same Time on area Area2 (Mosheer Ismail)



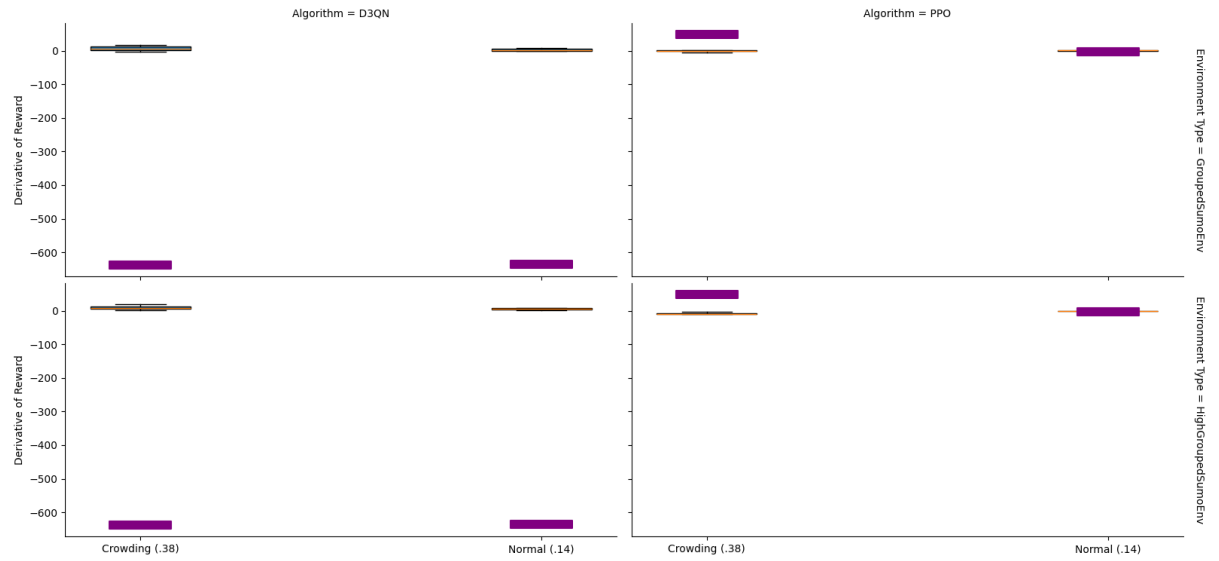
Performance Comparison (Using Reward : Literature)
Average Reward for Evaluated Episodes With Same Time on area Area2 (Mosheer Ismail)



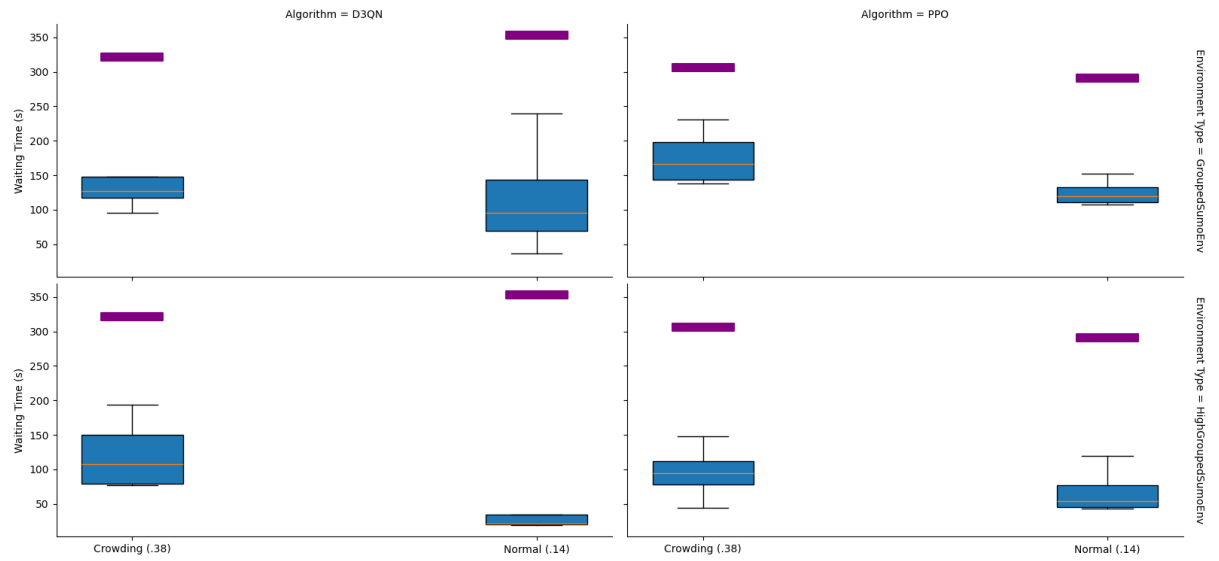
Performance Comparison (Using Reward : Proposed)
Derivative of Reward on area Area2 (Mosheer Ismail)



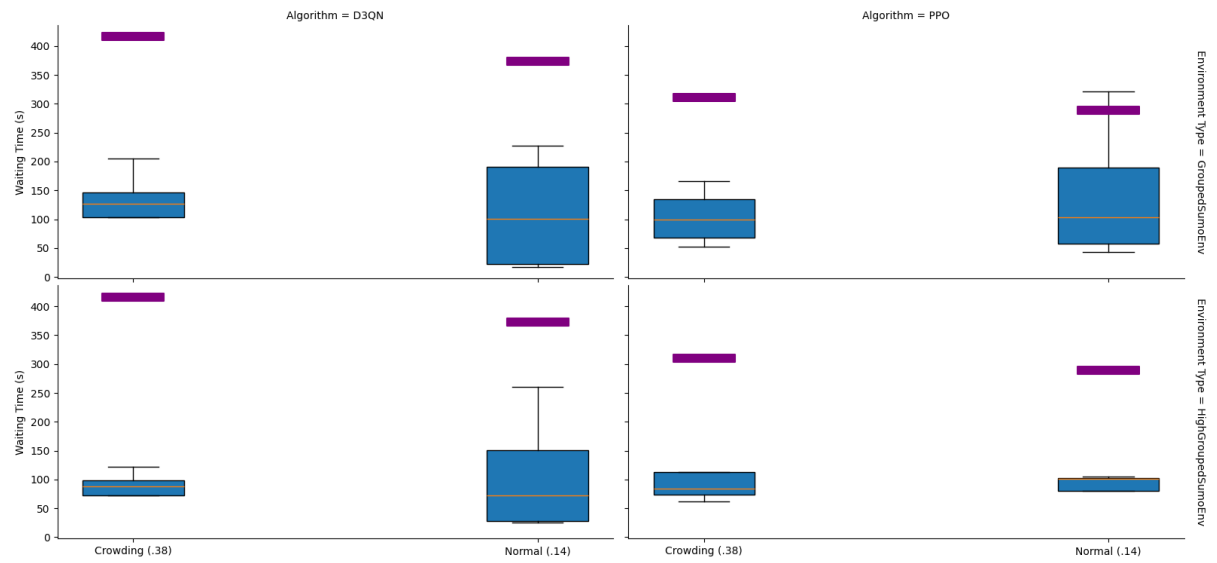
Performance Comparison (Using Reward : Literature)
Derivative of Reward on area Area2 (Mosheer Ismail)



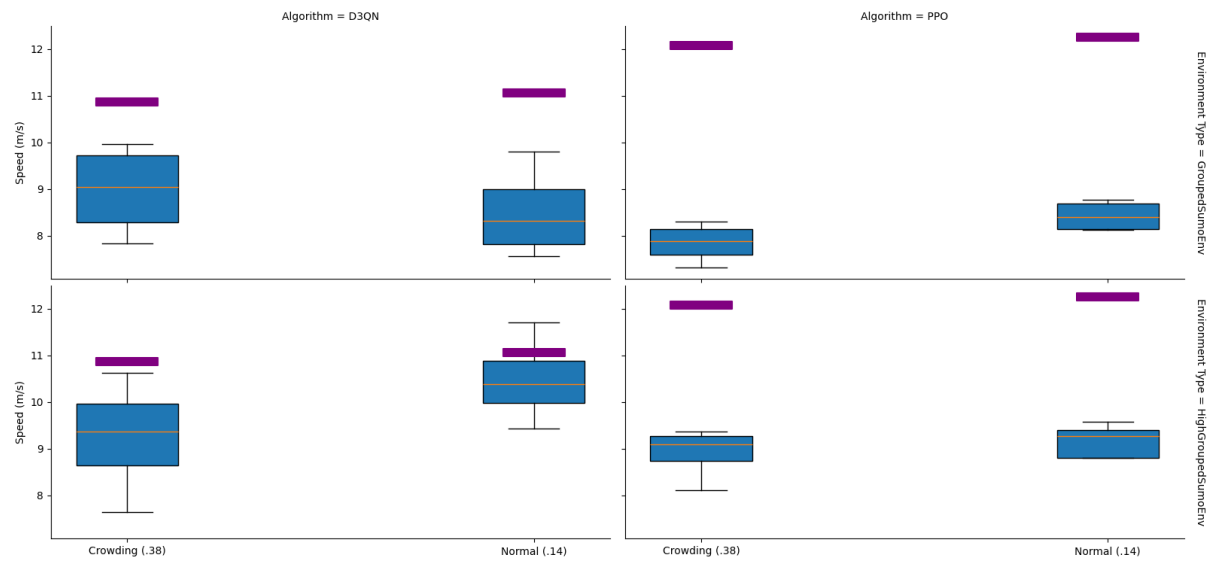
Performance Comparison (Using Reward : Proposed)
Waiting Time (s) on area Area2 (Mosheer Ismail)



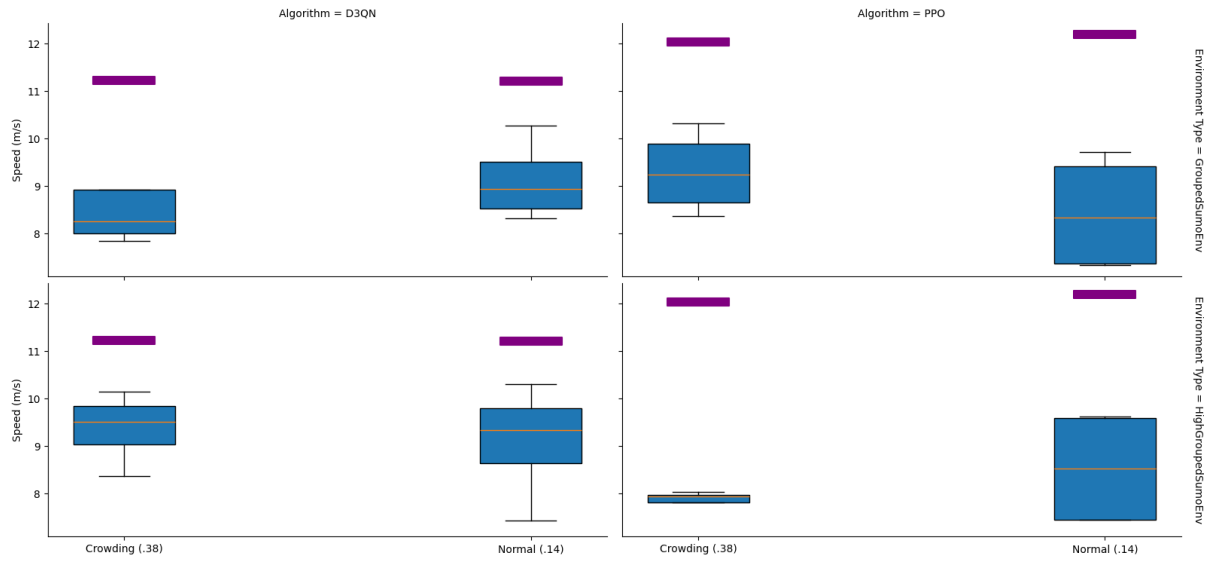
Performance Comparison (Using Reward : Literature)
Waiting Time (s) on area Area2 (Mosheer Ismail)



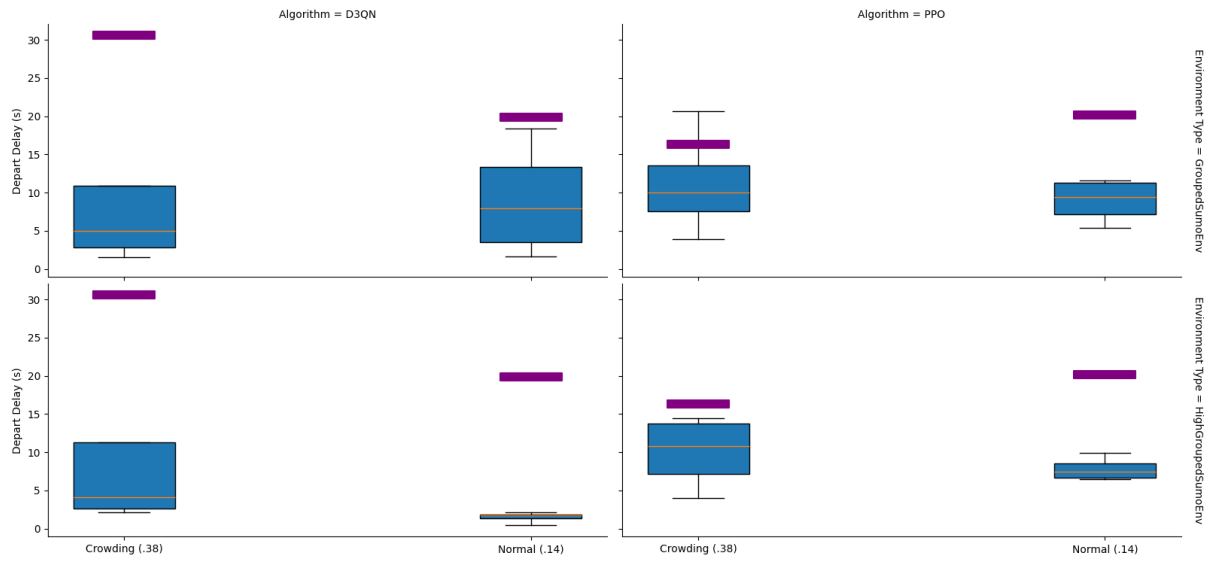
Performance Comparison (Using Reward : Proposed)
Speed (m/s) on area Area2 (Mosheer Ismail)



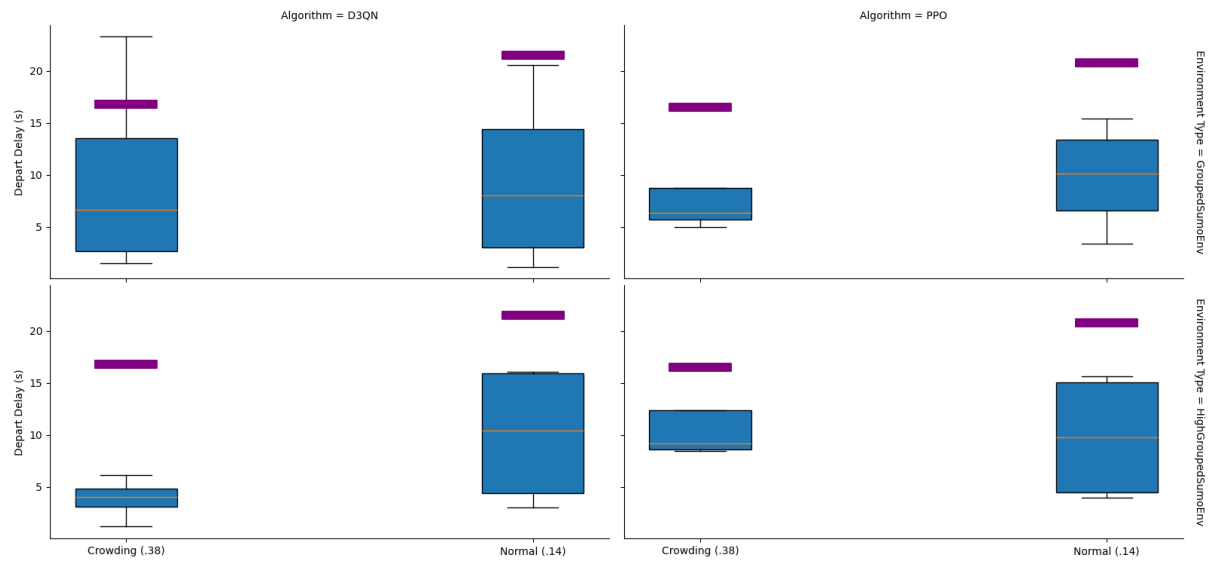
Performance Comparison (Using Reward : Literature)
Speed (m/s) on area Area2 (Mosheer Ismail)



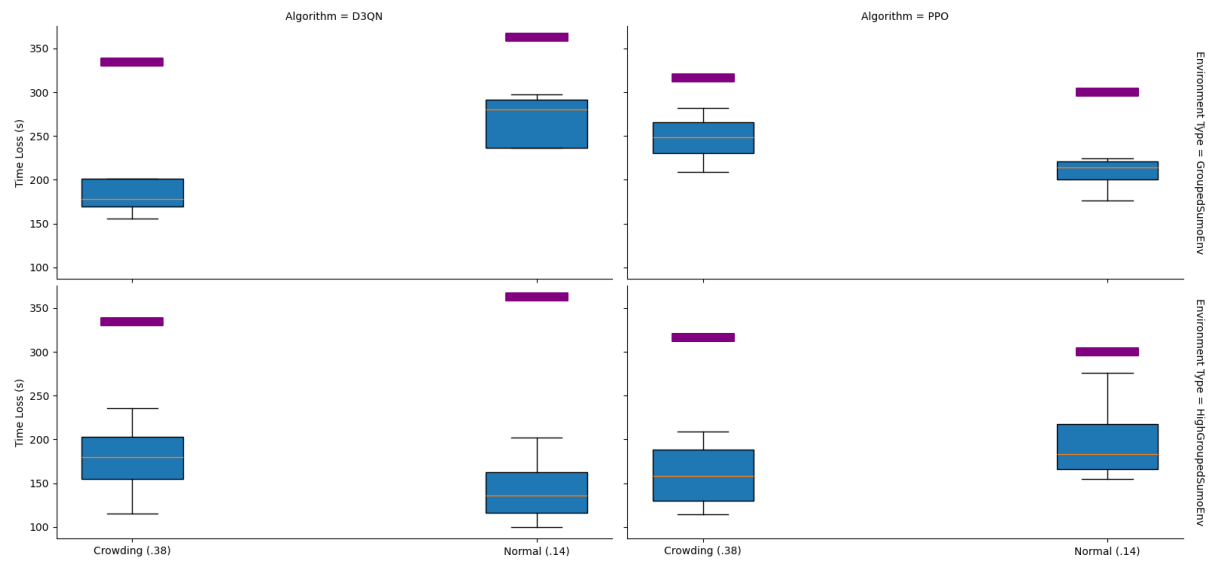
Performance Comparison (Using Reward : Proposed)
Depart Delay (s) on area Area2 (Mosheer Ismail)



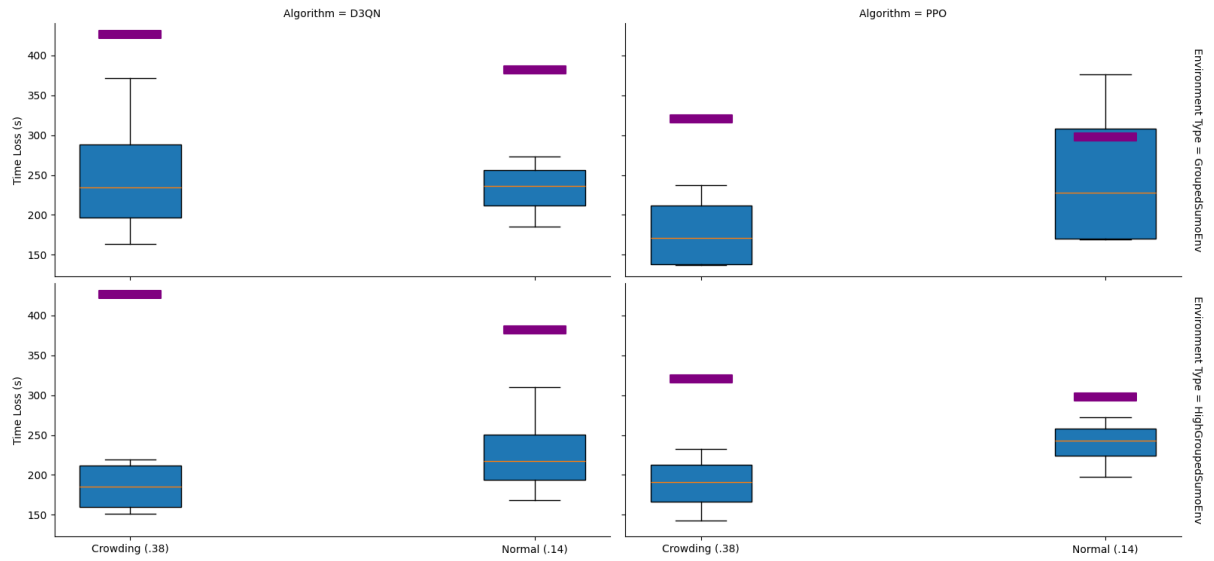
Performance Comparison (Using Reward : Literature)
Depart Delay (s) on area Area2 (Mosheer Ismail)



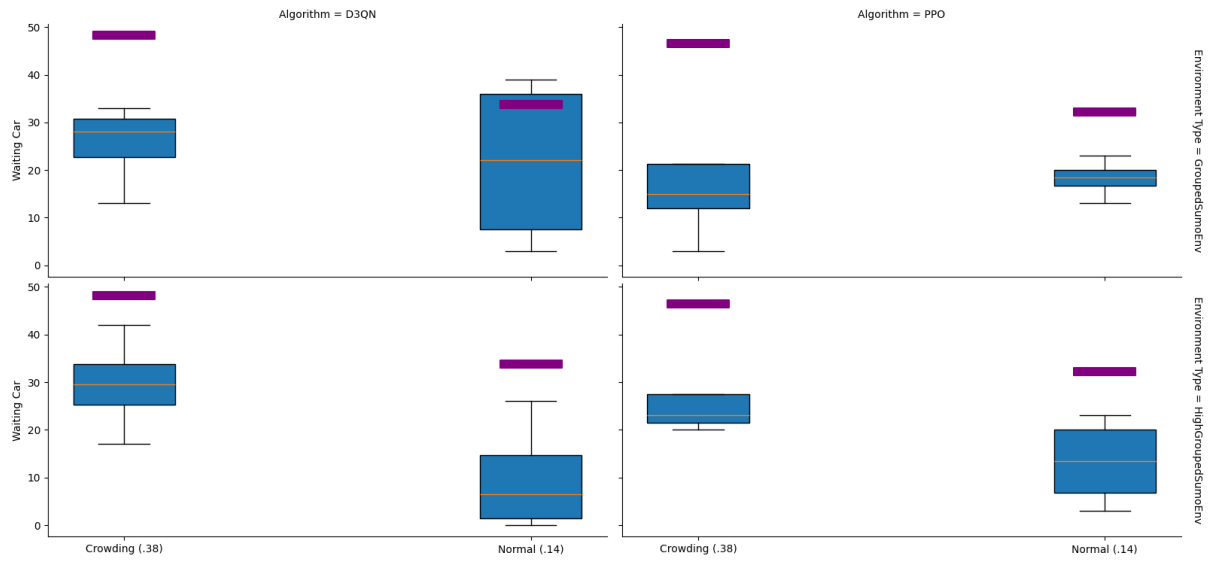
Performance Comparison (Using Reward : Proposed)
Time Loss (s) on area Area2 (Mosheer Ismail)



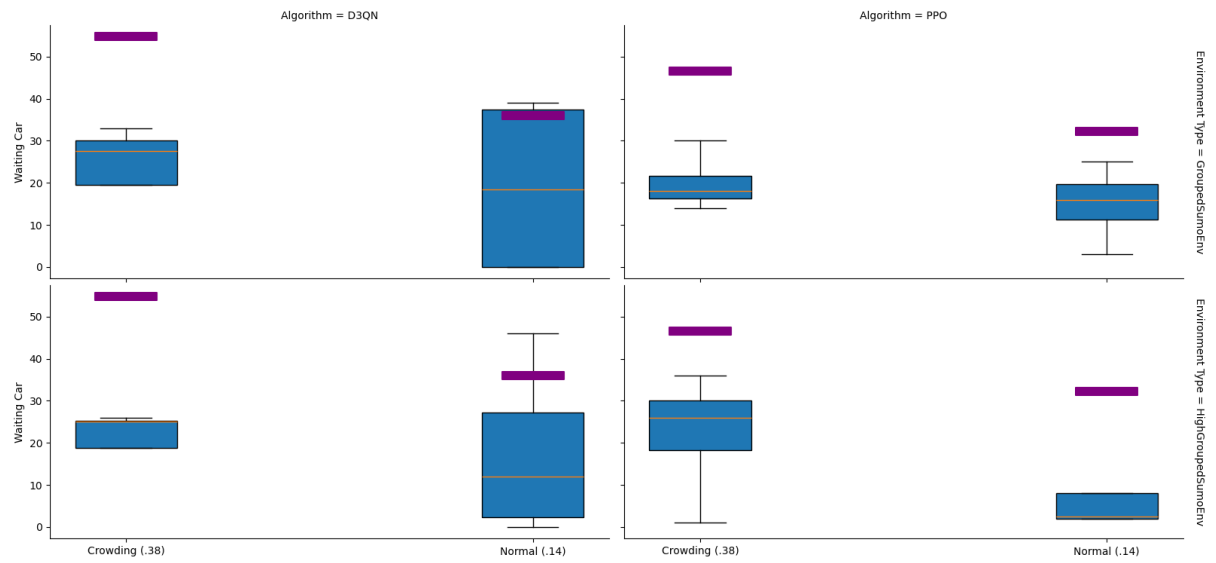
Performance Comparison (Using Reward : Literature)
Time Loss (s) on area Area2 (Mosheer Ismail)



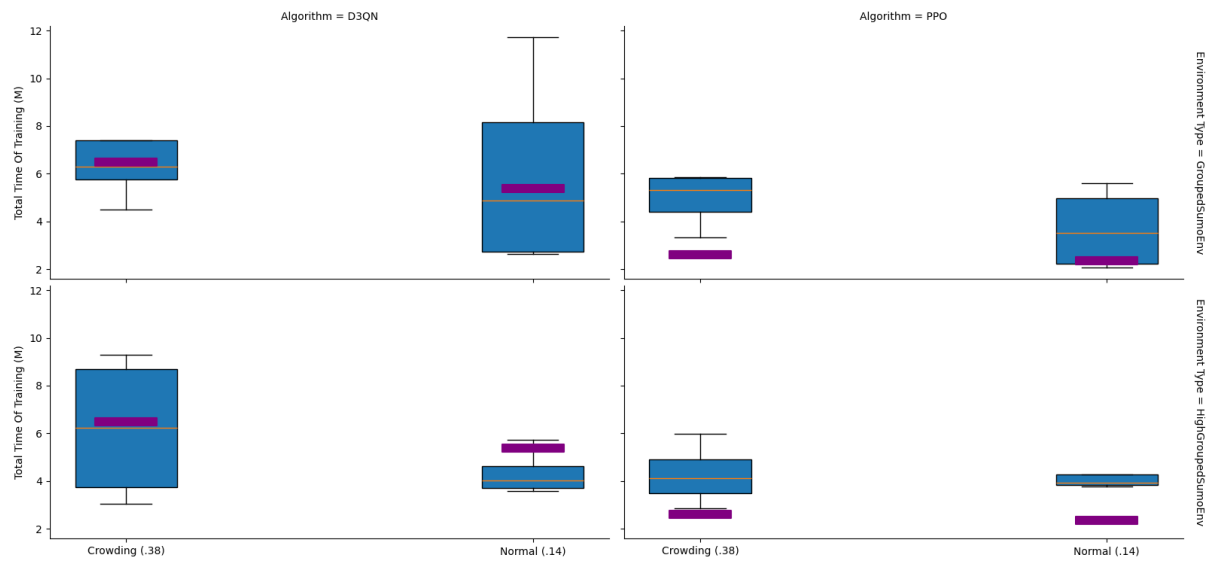
Performance Comparison (Using Reward : Proposed)
Waiting Car on area Area2 (Mosheer Ismail)



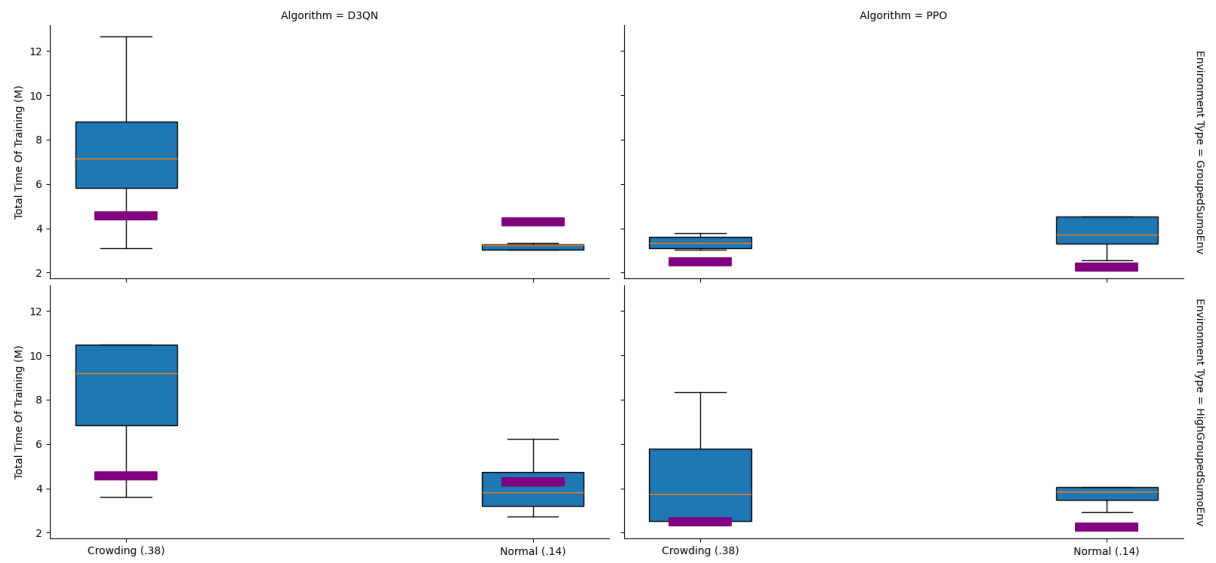
Performance Comparison (Using Reward : Literature)
Waiting Car on area Area2 (Mosheer Ismail)



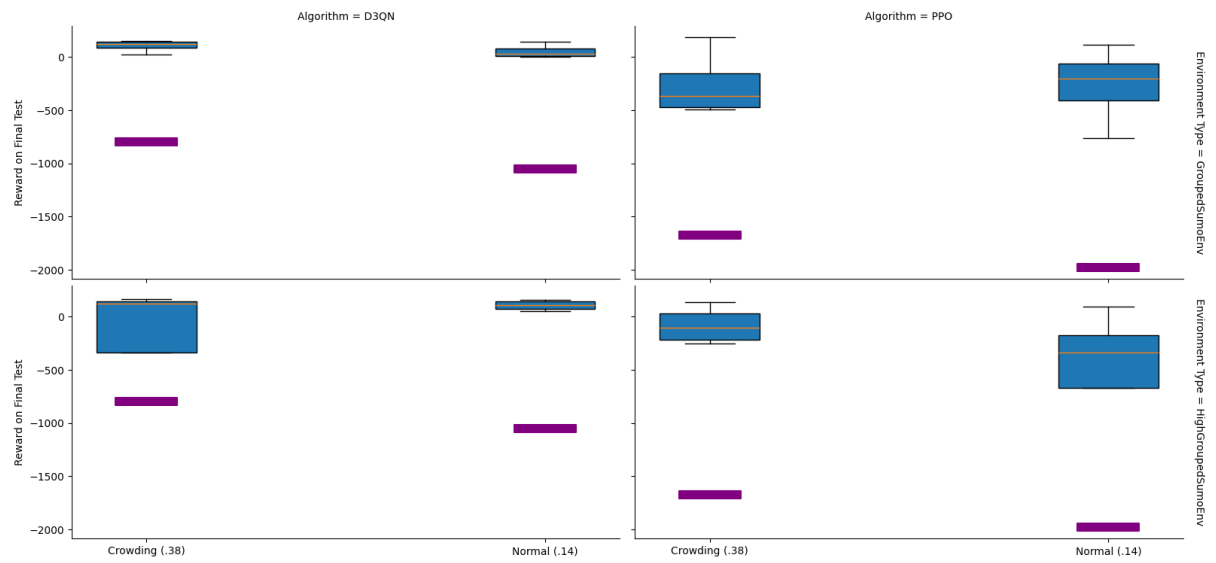
Performance Comparison (Using Reward : Proposed)
Total Time Of Training (M) on area Area2 (Mosheer Ismail)



Performance Comparison (Using Reward : Literature)
Total Time Of Training (M) on area Area2 (Mosheer Ismail)

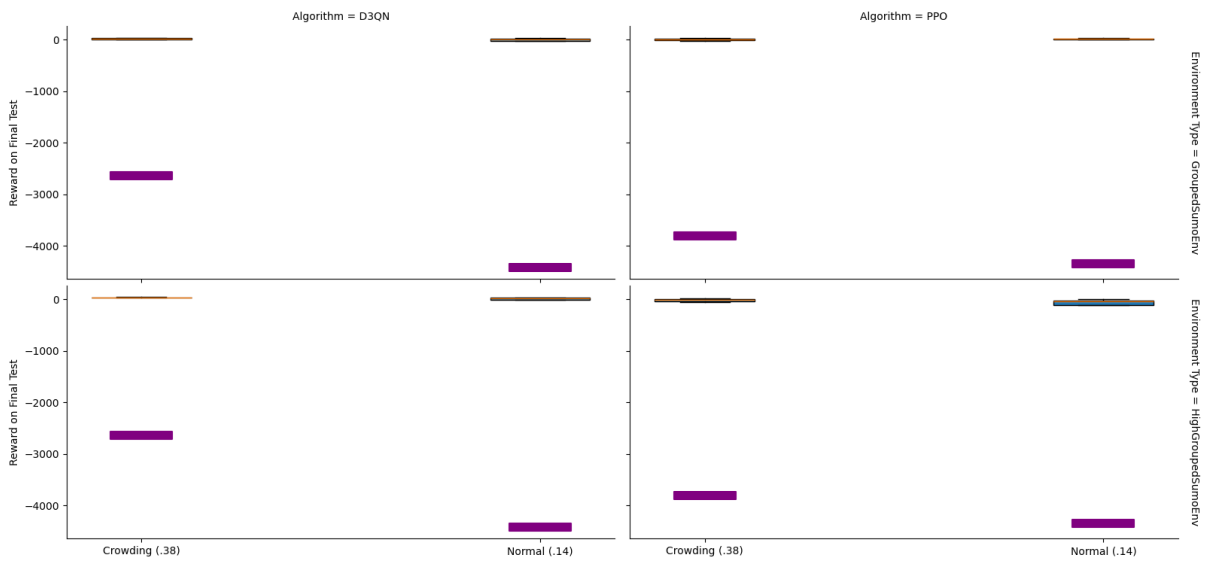


Performance Comparison (Using Reward : Proposed)
Reward on Final Test on area Area2 (Mosheer Ismail)

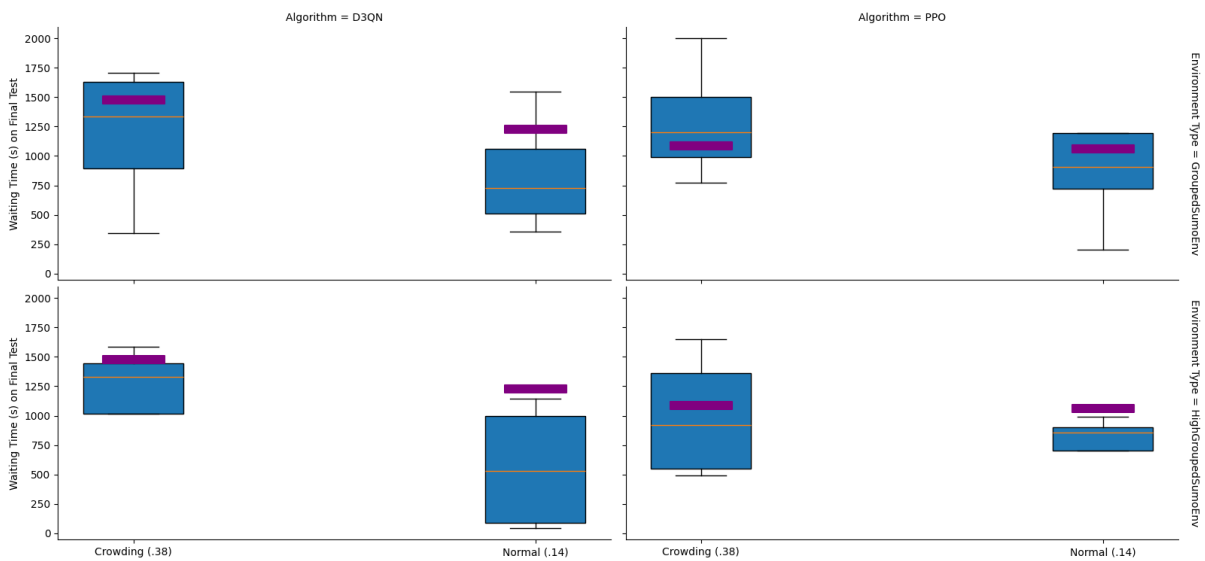


S

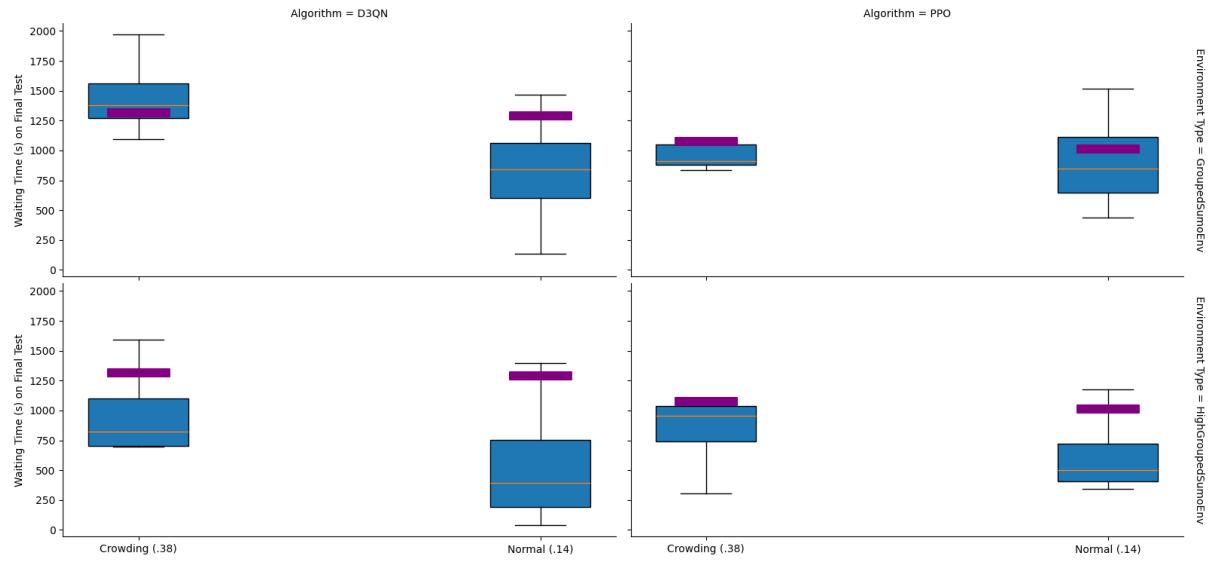
Performance Comparison (Using Reward : Literature)
Reward on Final Test on area Area2 (Mosheer Ismail)



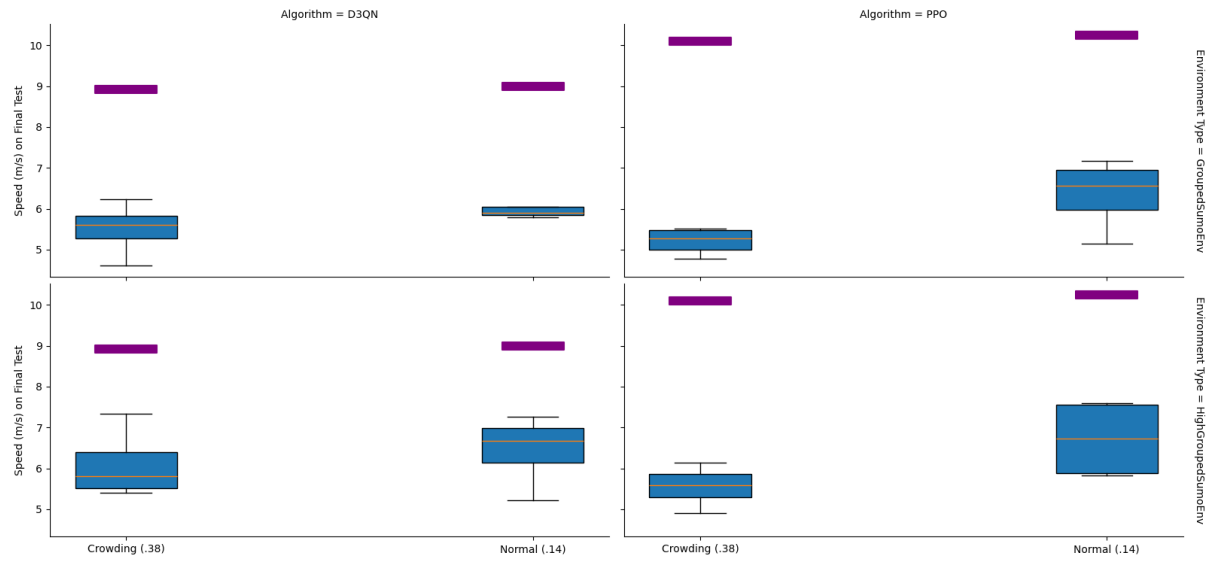
Performance Comparison (Using Reward : Proposed)
Waiting Time (s) on Final Test on area Area2 (Mosheer Ismail)



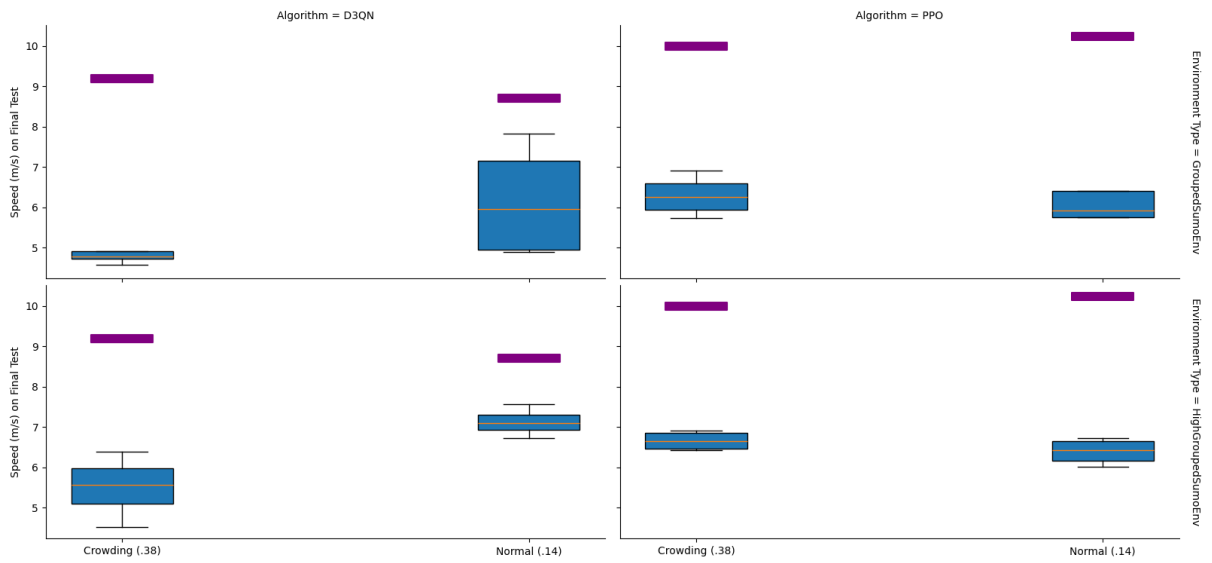
Performance Comparison (Using Reward : Literature)
Waiting Time (s) on Final Test on area Area2 (Mosheer Ismail)



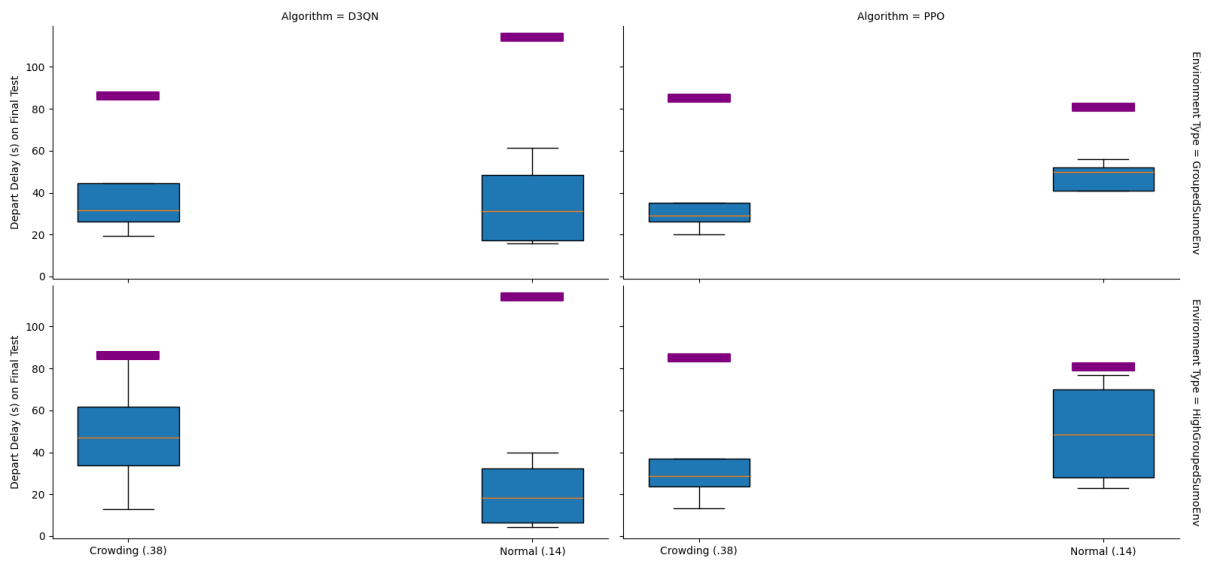
Performance Comparison (Using Reward : Proposed)
Speed (m/s) on Final Test on area Area2 (Mosheer Ismail)



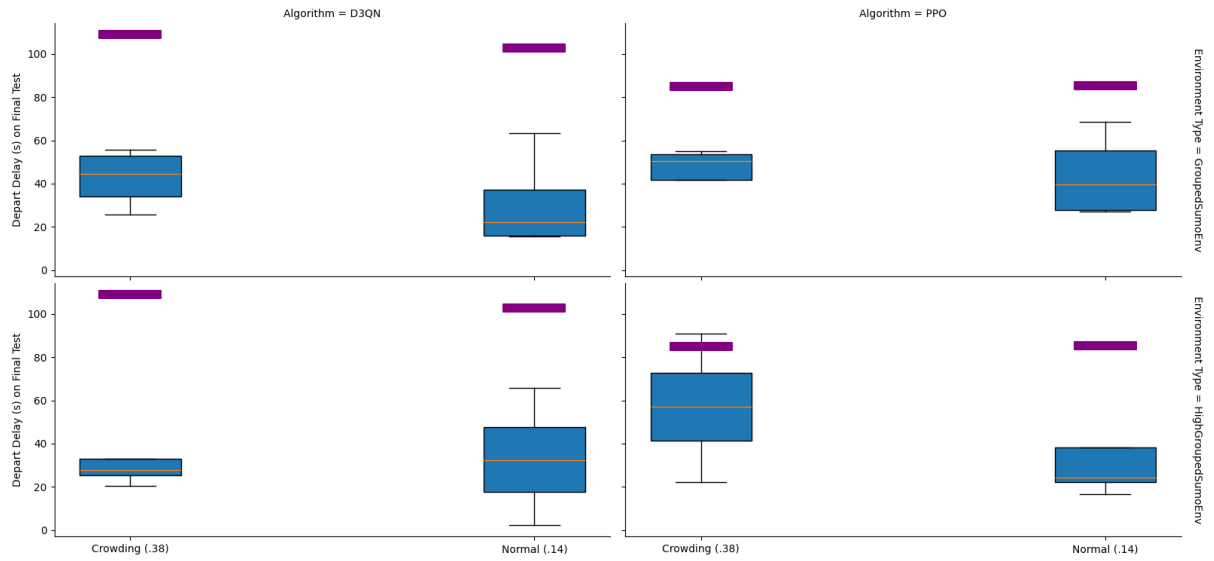
Performance Comparison (Using Reward : Literature)
Speed (m/s) on Final Test on area Area2 (Mosheer Ismail)



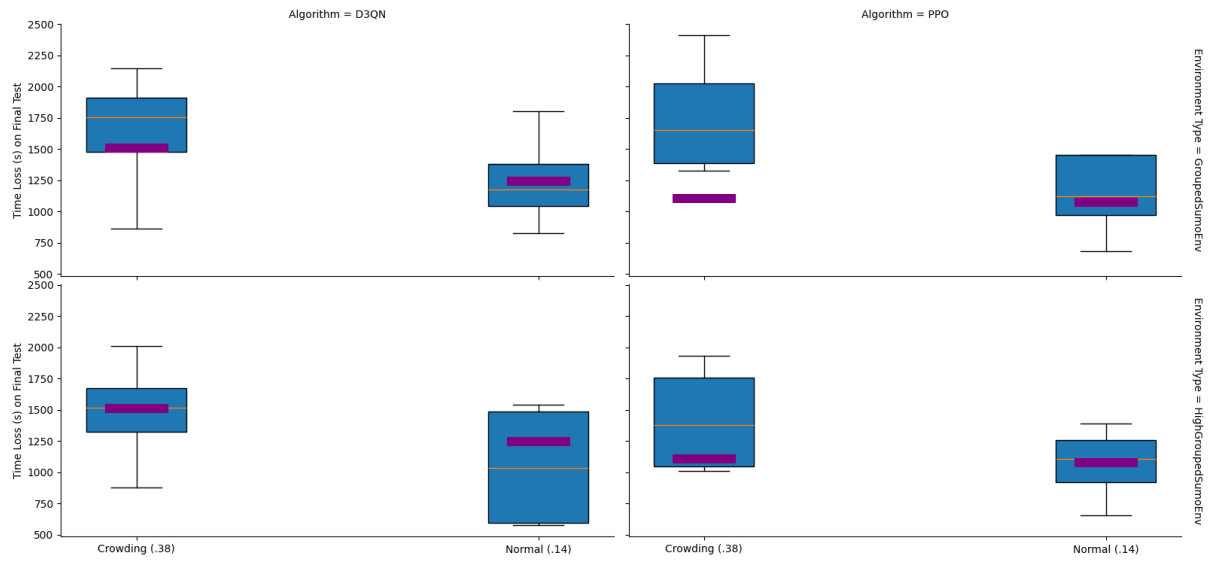
Performance Comparison (Using Reward : Proposed)
Depart Delay (s) on Final Test on area Area2 (Mosheer Ismail)



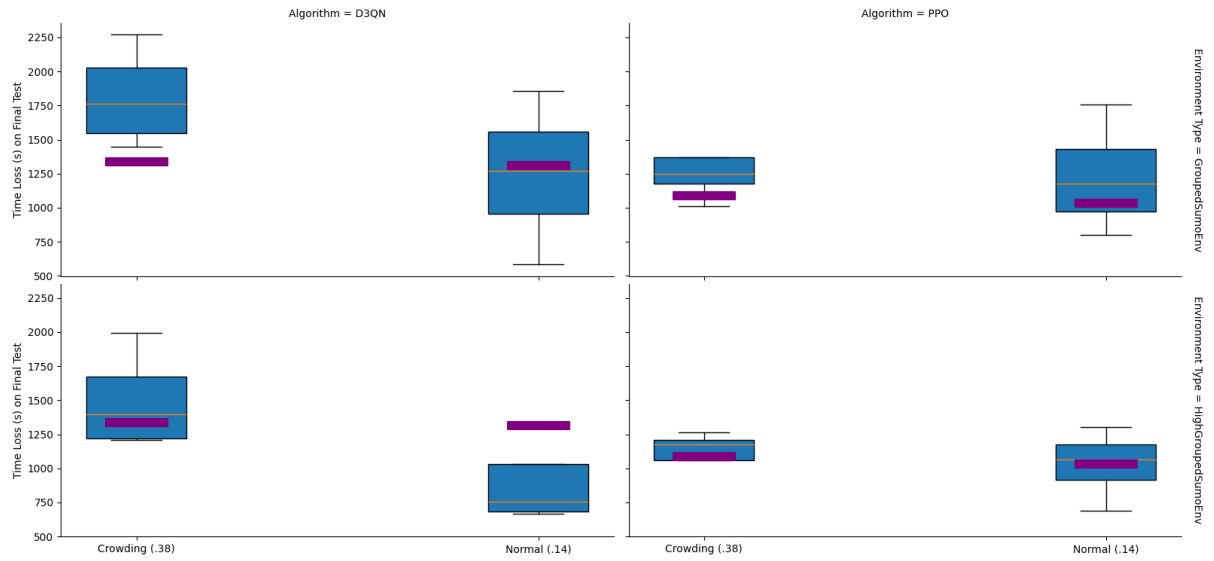
Performance Comparison (Using Reward : Literature)
Depart Delay (s) on Final Test on area Area2 (Mosheer Ismail)



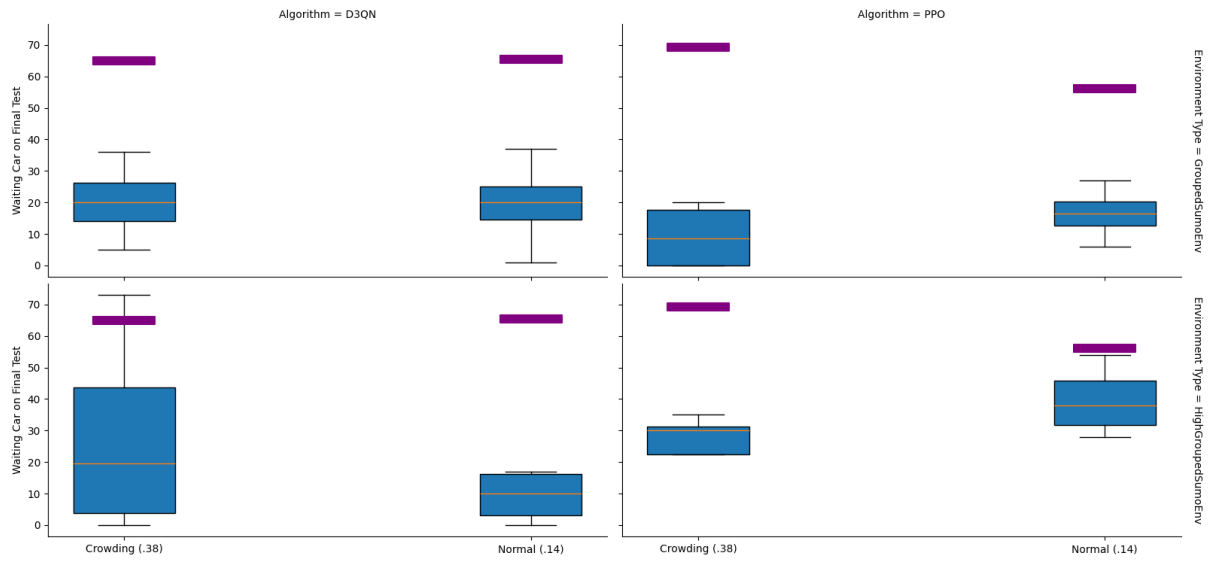
Performance Comparison (Using Reward : Proposed)
Time Loss (s) on Final Test on area Area2 (Mosheer Ismail)



Performance Comparison (Using Reward : Literature)
Time Loss (s) on Final Test on area Area2 (Mosheer Ismail)



Performance Comparison (Using Reward : Proposed)
Waiting Car on Final Test on area Area2 (Mosheer Ismail)



Performance Comparison (Using Reward : Literature)
Waiting Car on Final Test on area Area2 (Mosheer Ismail)

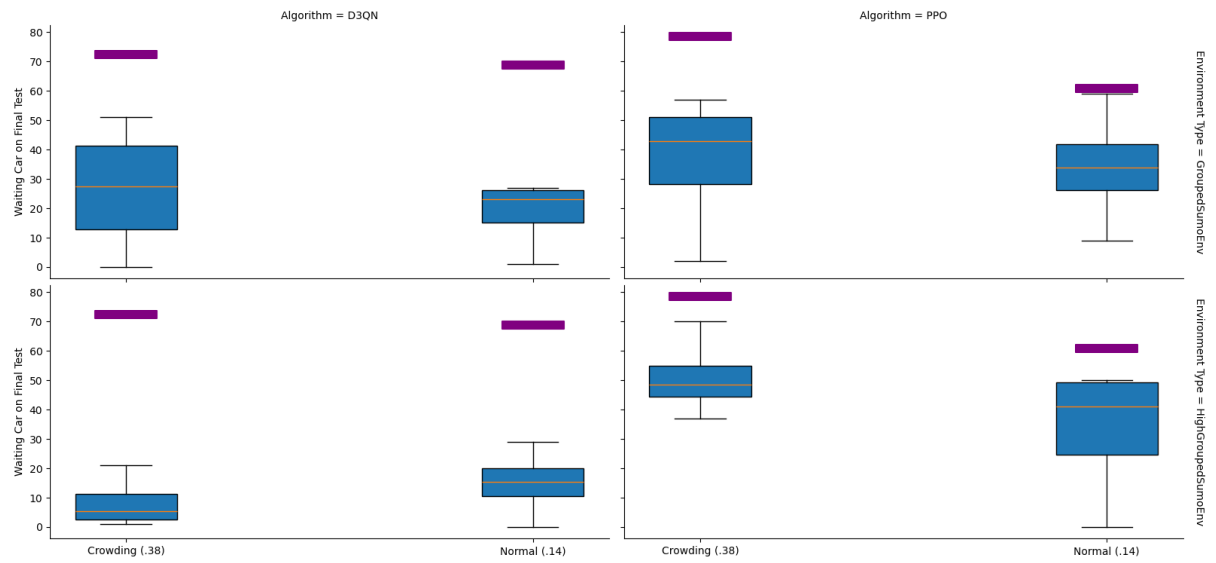
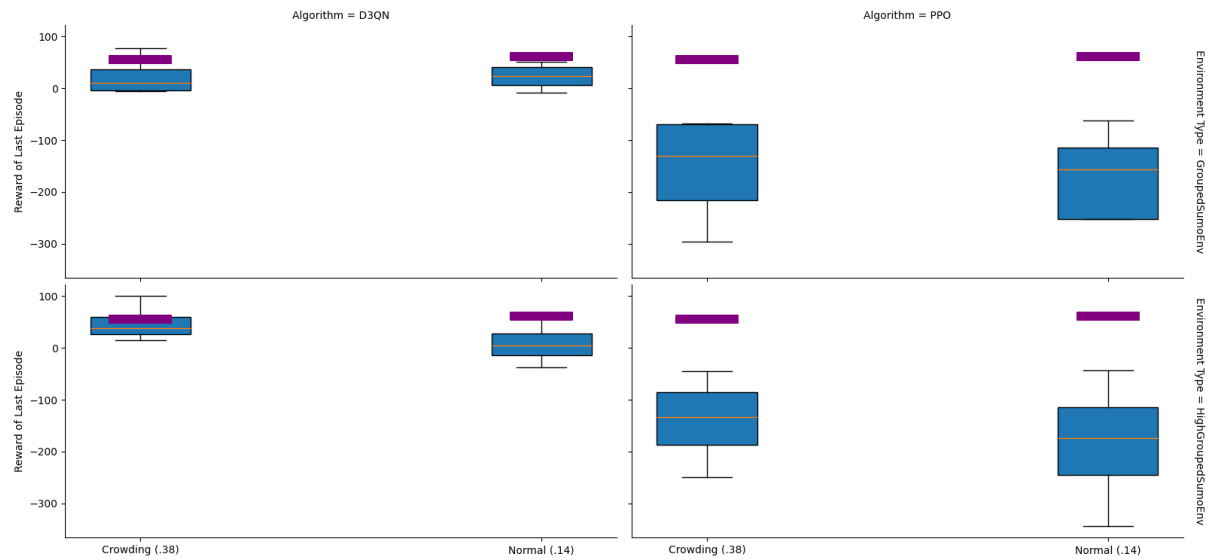


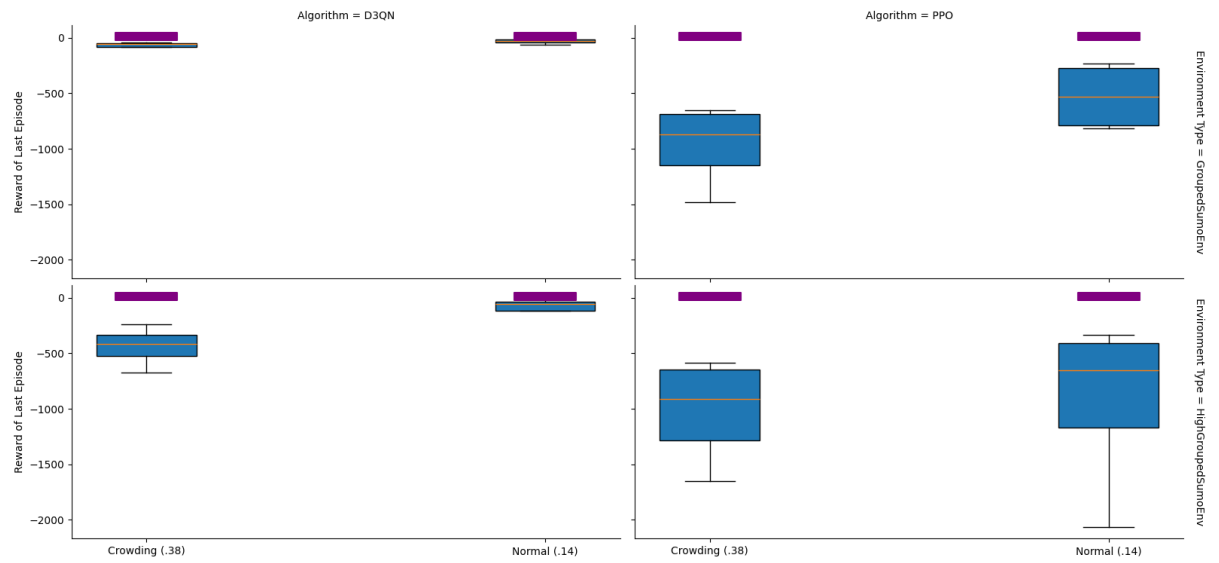
Figure 0.48 Results of Mosheer at version 2.0.2

5.4.4.2.2 Stefano

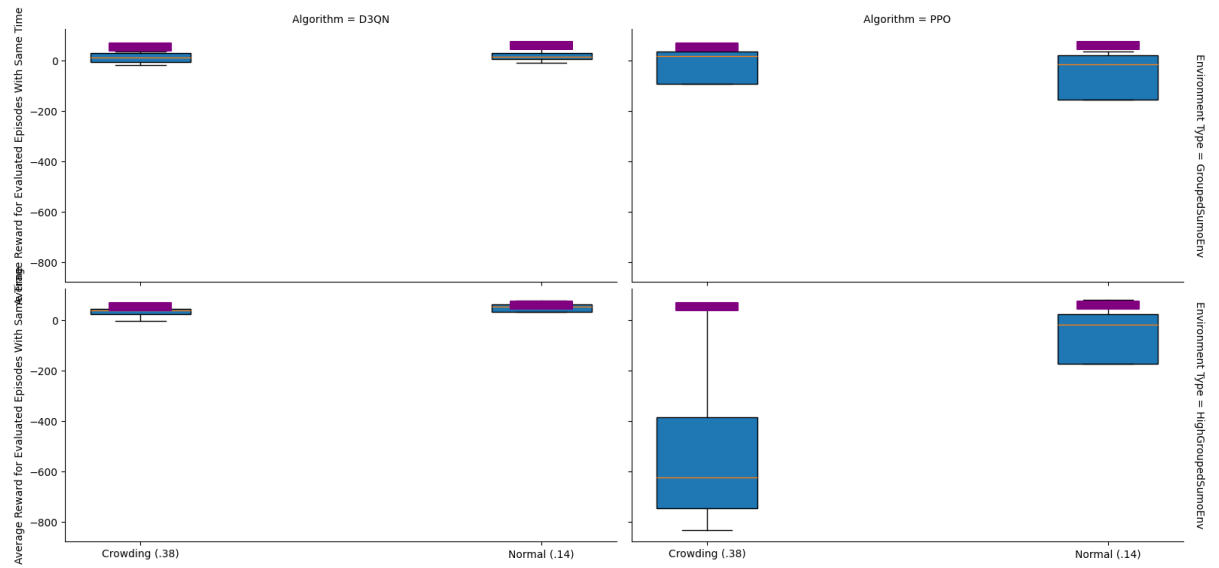
Performance Comparison (Using Reward : Proposed)
Reward of Last Episode on area Area 3 (San Stefano)



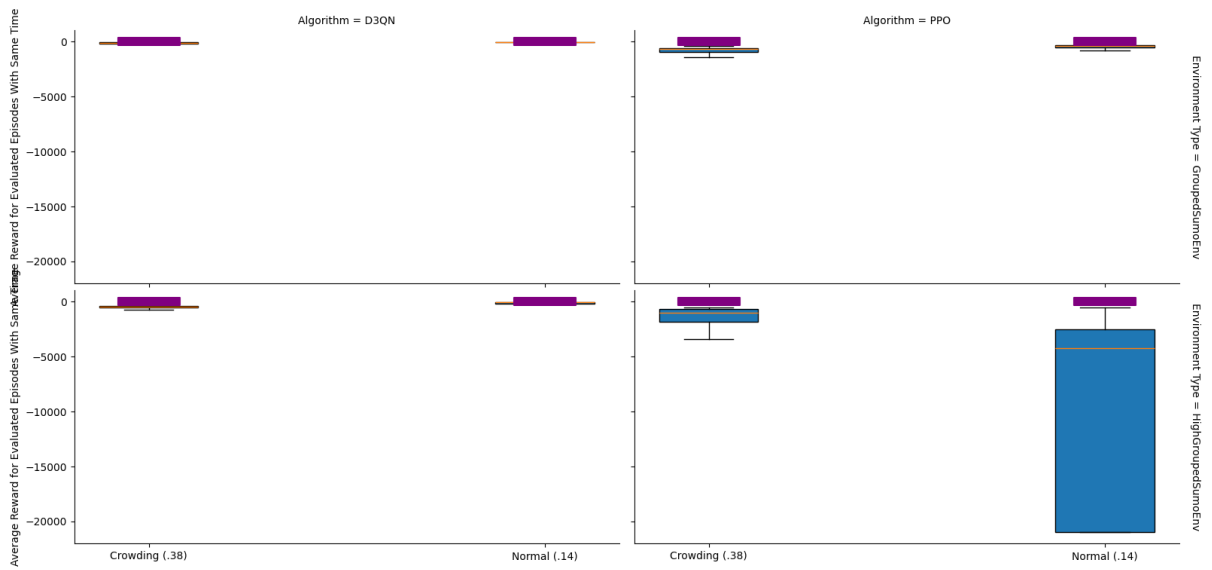
Performance Comparison (Using Reward : Literature)
Reward of Last Episode on area Area 3 (San Stefano)



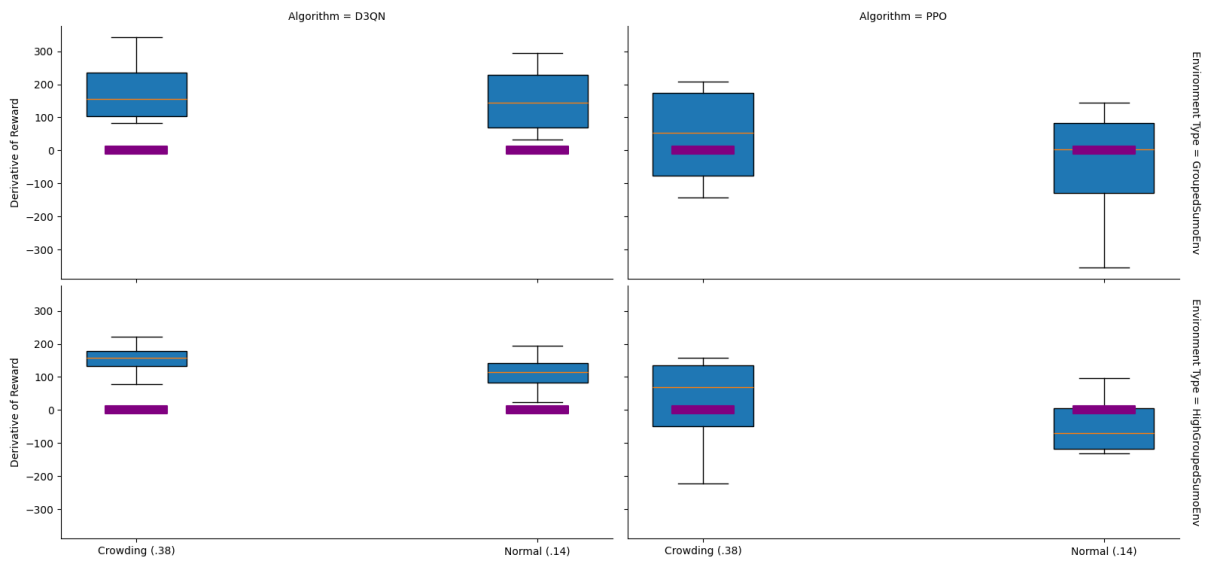
Performance Comparison (Using Reward : Proposed)
Average Reward for Evaluated Episodes With Same Time on area Area 3 (San Stefano)



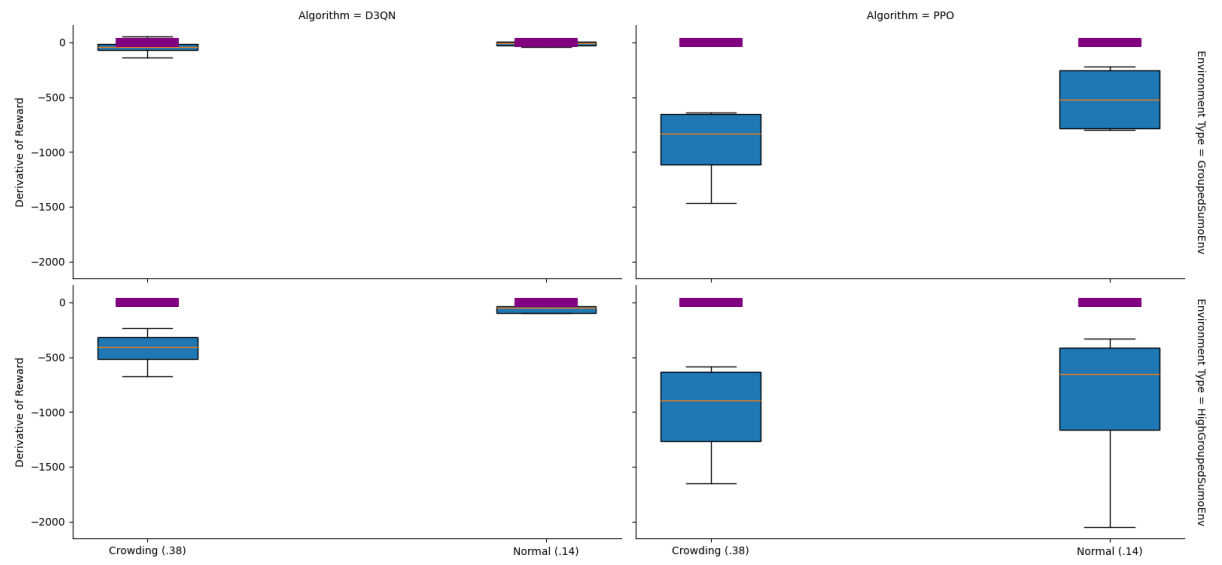
Performance Comparison (Using Reward : Literature)
Average Reward for Evaluated Episodes With Same Time on area Area 3 (San Stefano)



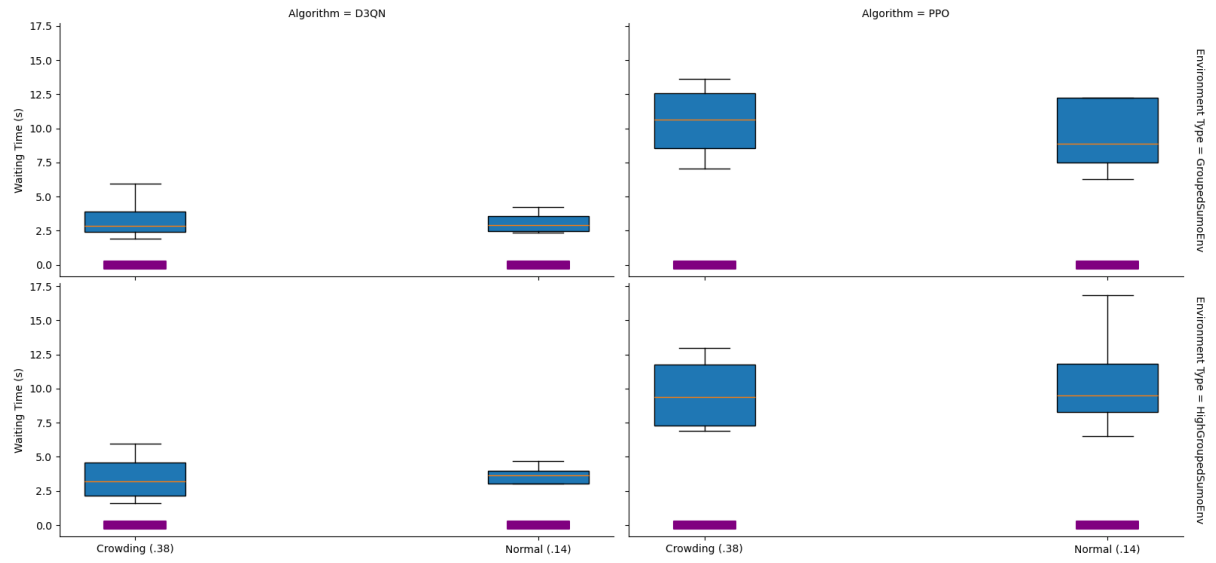
Performance Comparison (Using Reward : Proposed)
Derivative of Reward on area Area 3 (San Stefano)



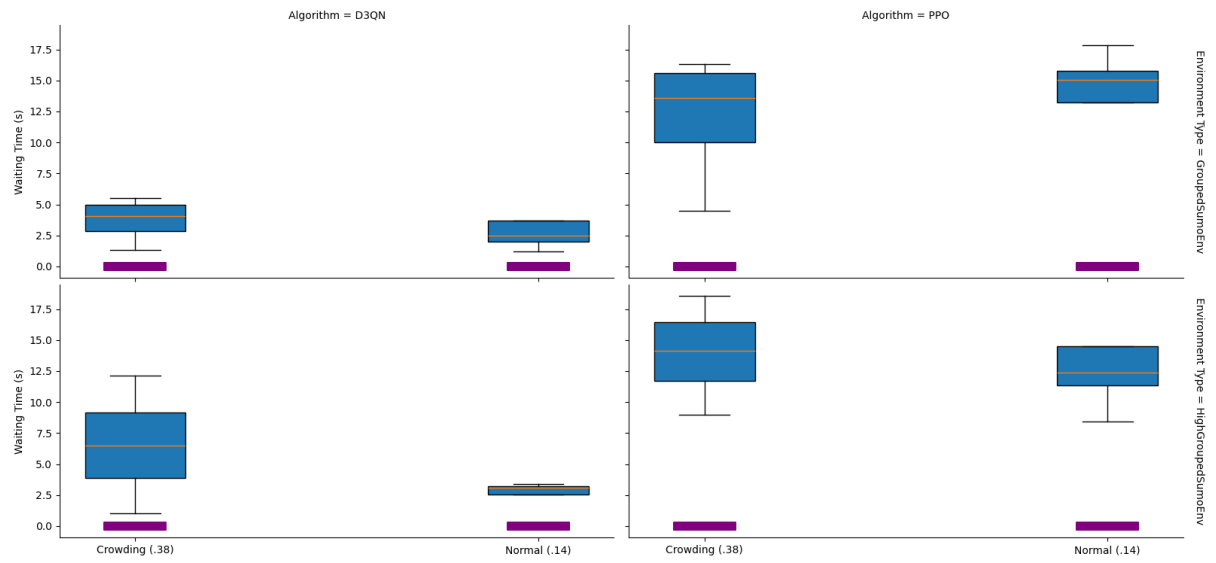
Performance Comparison (Using Reward : Literature)
Derivative of Reward on area Area 3 (San Stefano)



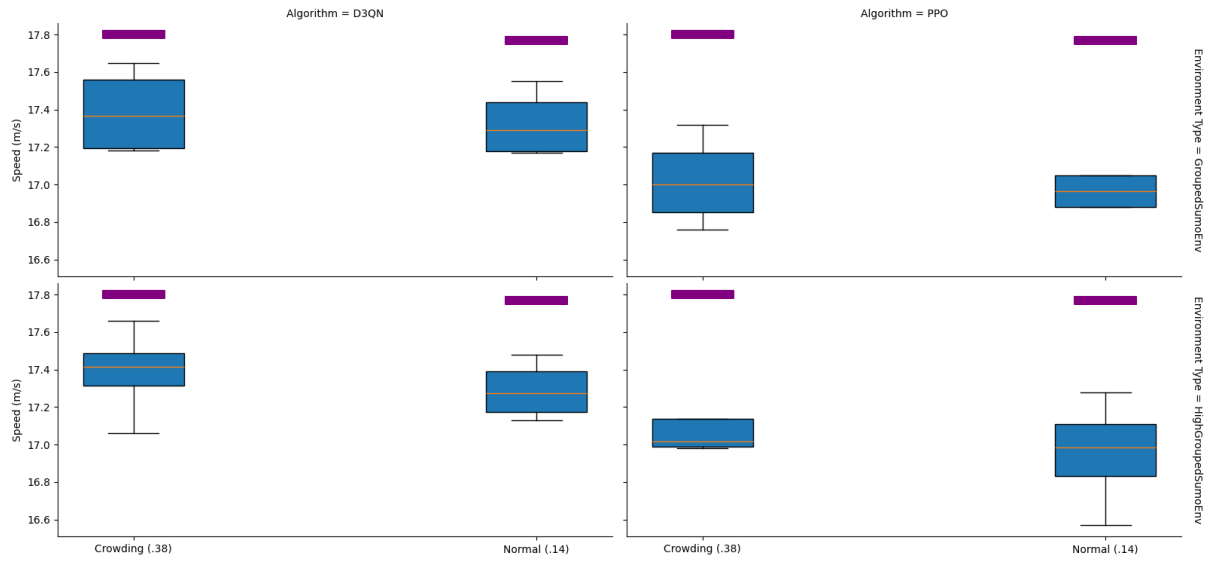
Performance Comparison (Using Reward : Proposed)
Waiting Time (s) on area Area 3 (San Stefano)



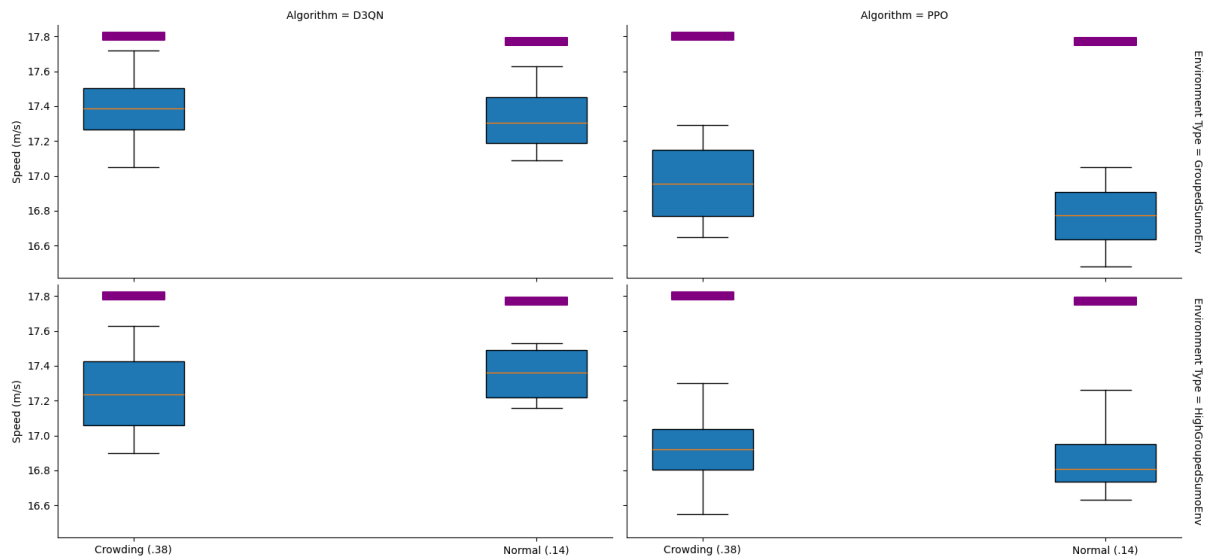
Performance Comparison (Using Reward : Literature)
Waiting Time (s) on area Area 3 (San Stefano)



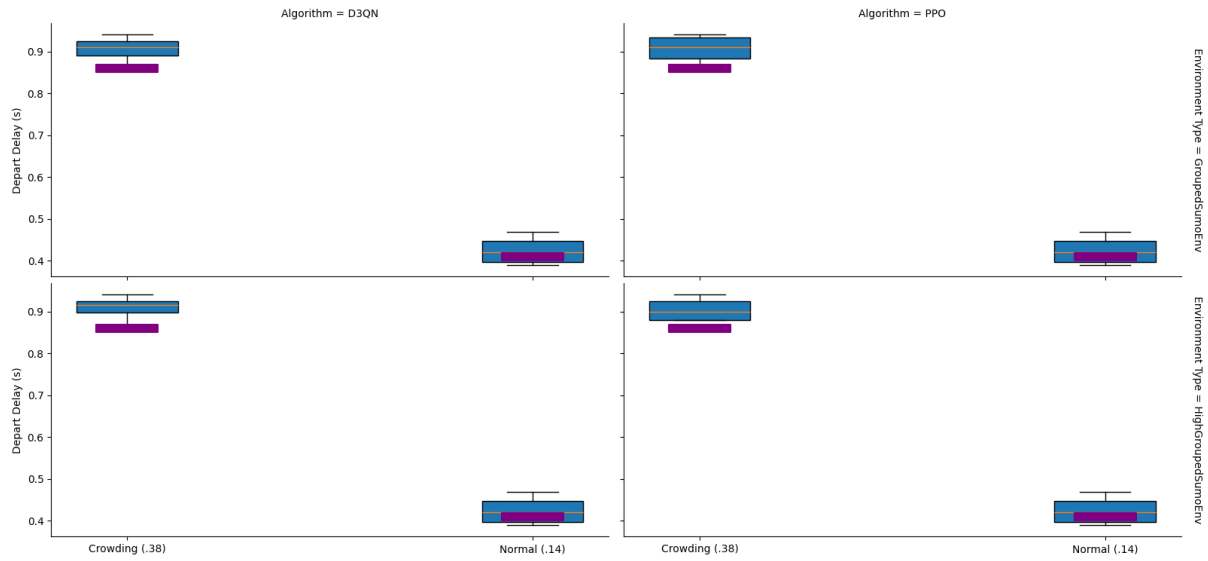
Performance Comparison (Using Reward : Proposed)
Speed (m/s) on area Area 3 (San Stefano)



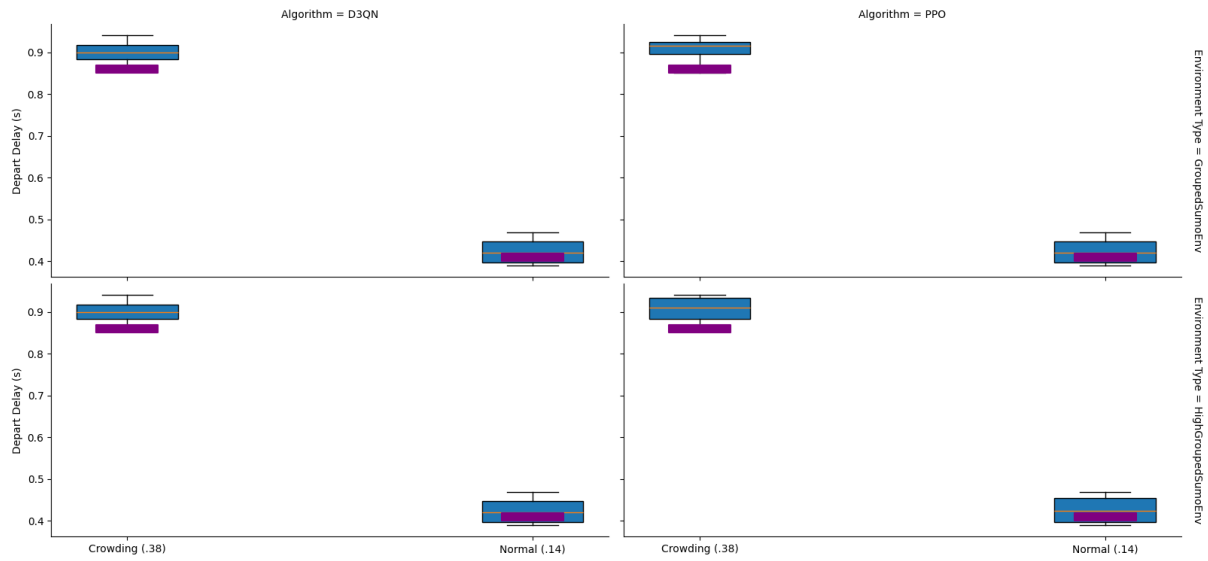
Performance Comparison (Using Reward : Literature)
Speed (m/s) on area Area 3 (San Stefano)



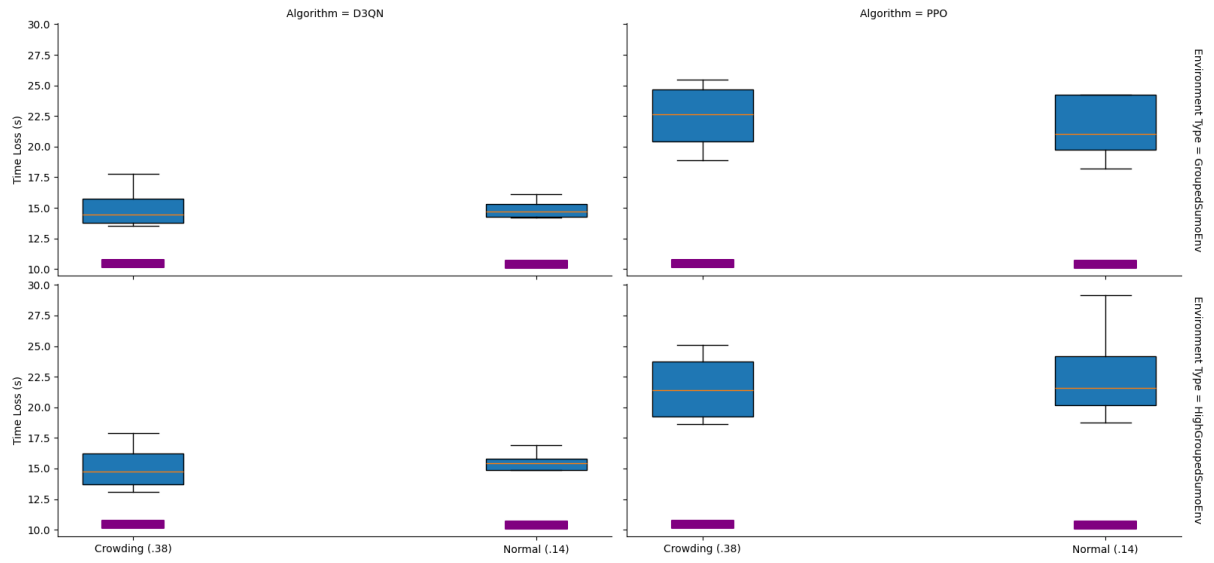
Performance Comparison (Using Reward : Proposed)
Depart Delay (s) on area Area 3 (San Stefano)



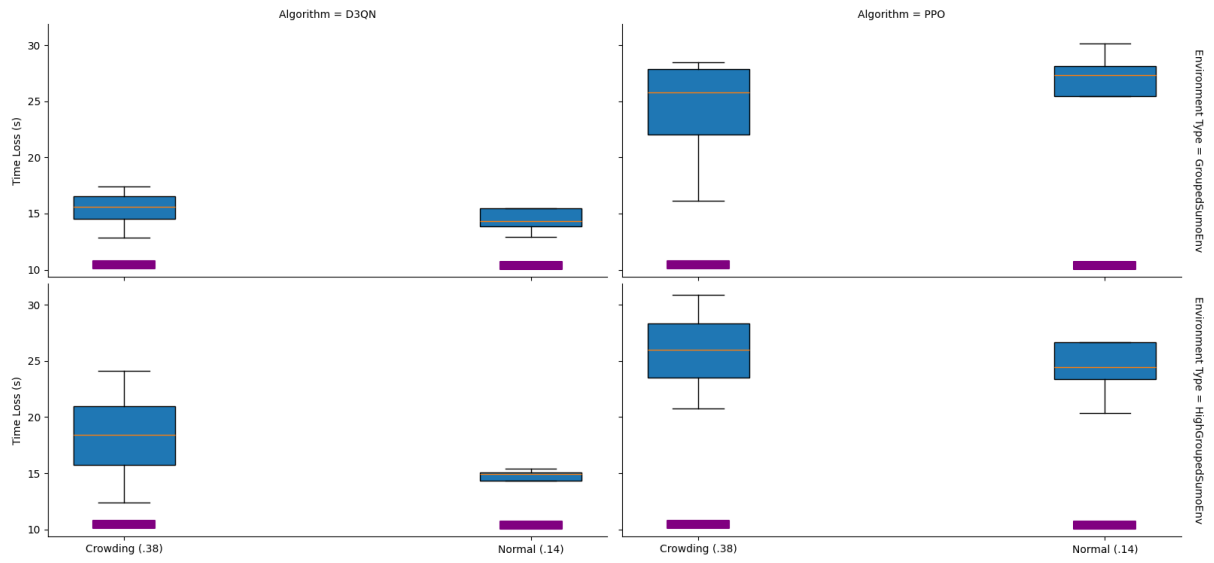
Performance Comparison (Using Reward : Literature)
Depart Delay (s) on area Area 3 (San Stefano)



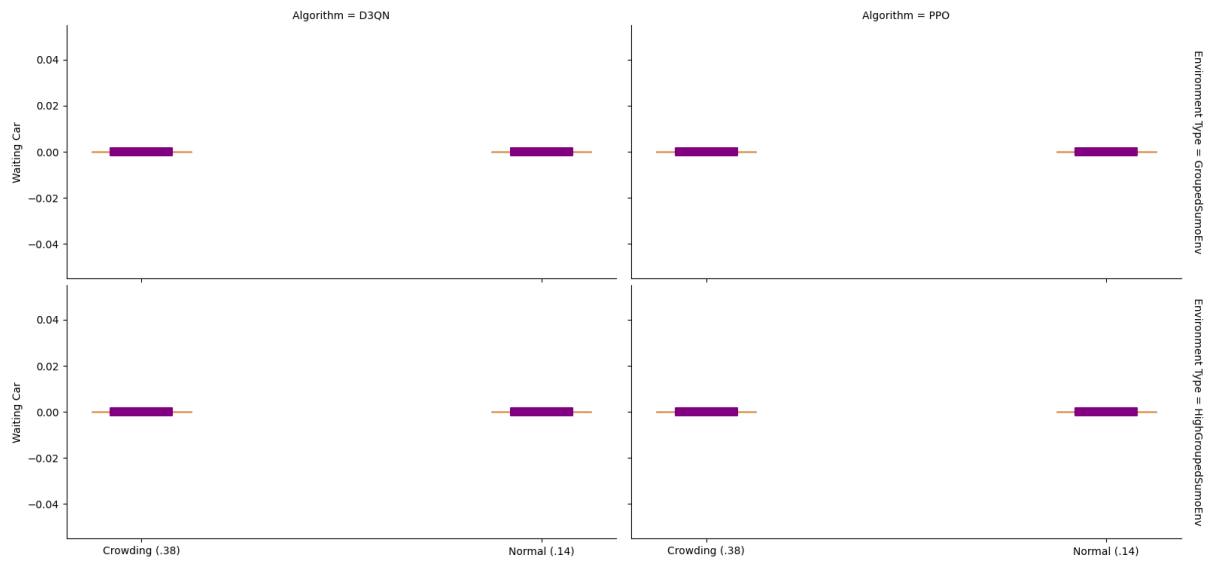
Performance Comparison (Using Reward : Proposed)
Time Loss (s) on area Area 3 (San Stefano)



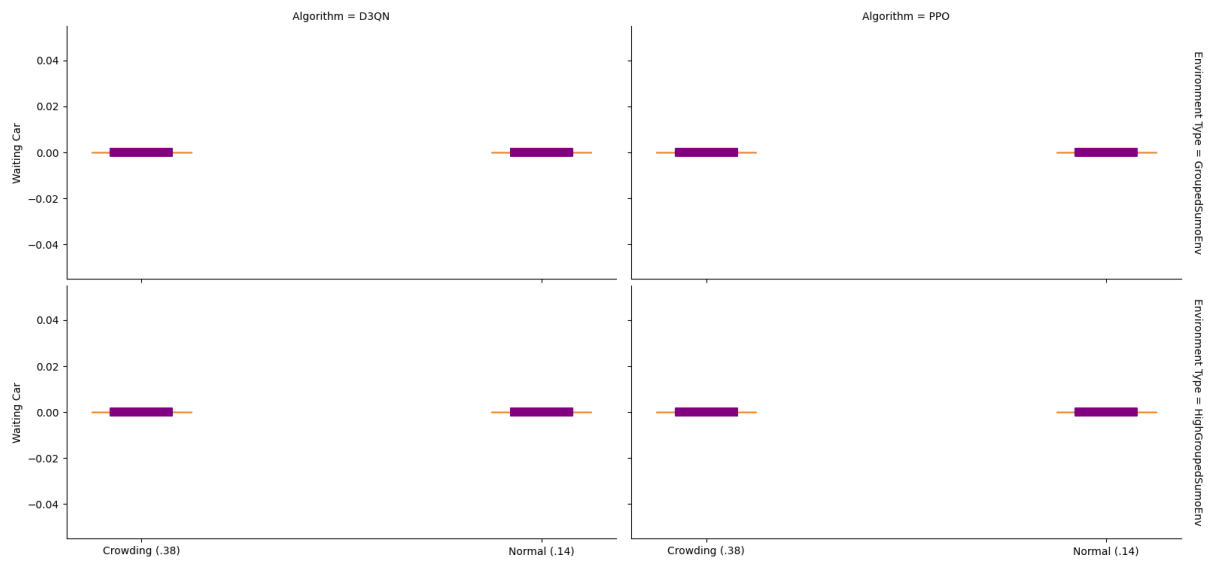
Performance Comparison (Using Reward : Literature)
Time Loss (s) on area Area 3 (San Stefano)



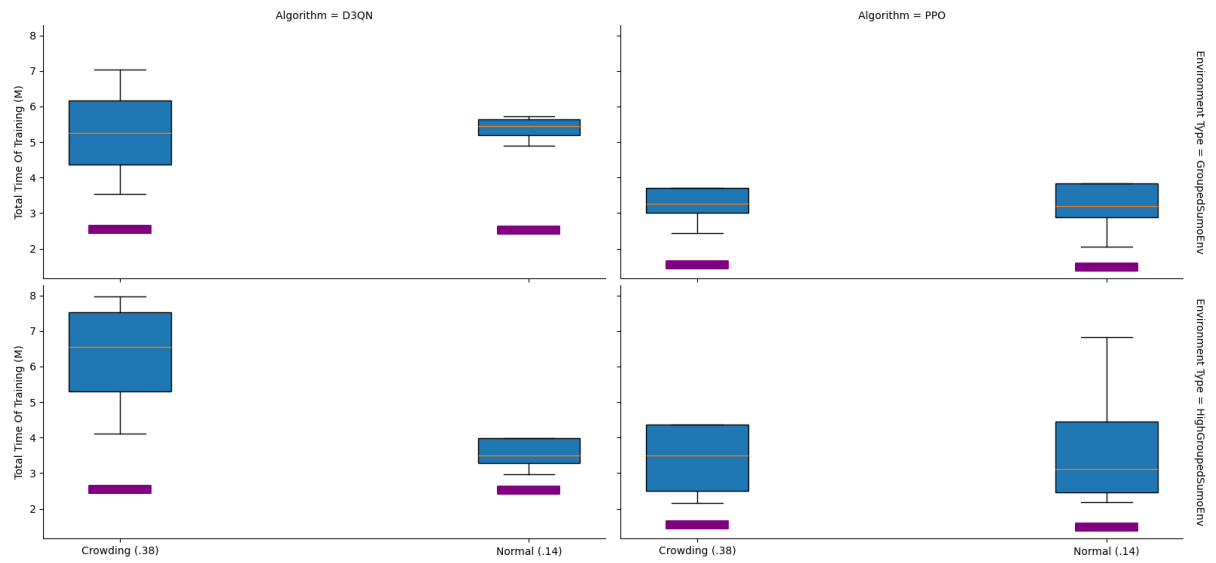
Performance Comparison (Using Reward : Proposed)
Waiting Car on area Area 3 (San Stefano)



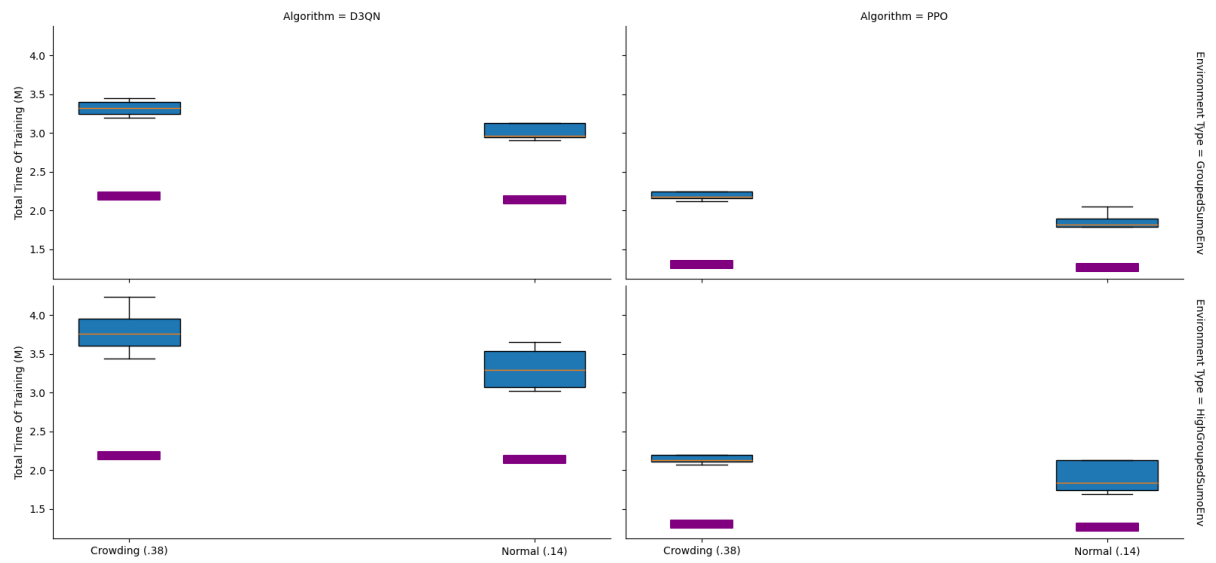
Performance Comparison (Using Reward : Literature)
Waiting Car on area Area 3 (San Stefano)



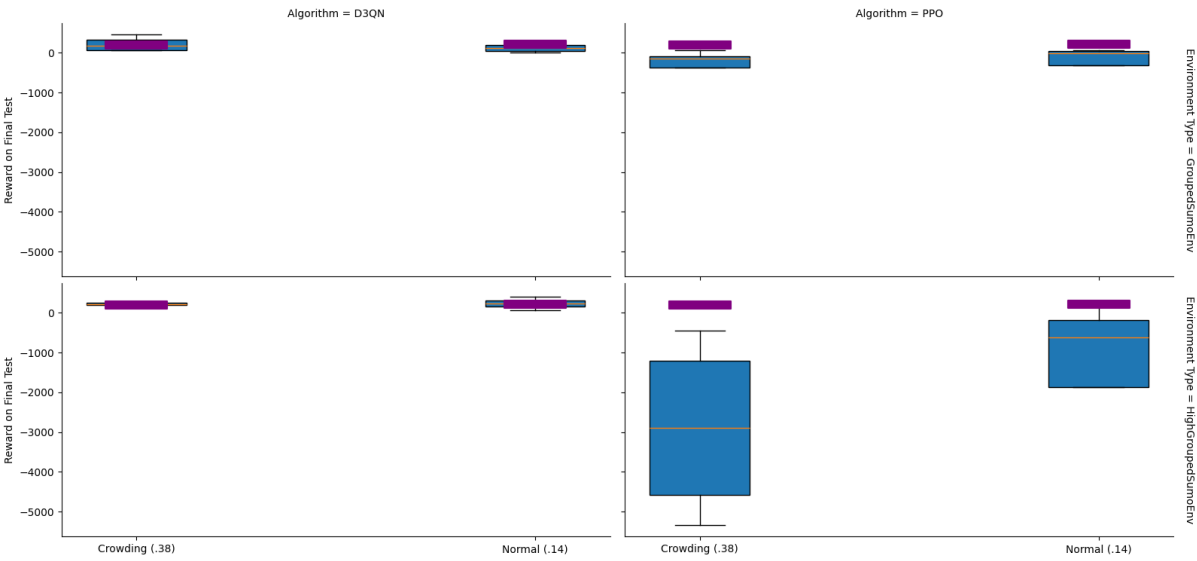
Performance Comparison (Using Reward : Proposed)
Total Time Of Training (M) on area Area 3 (San Stefano)



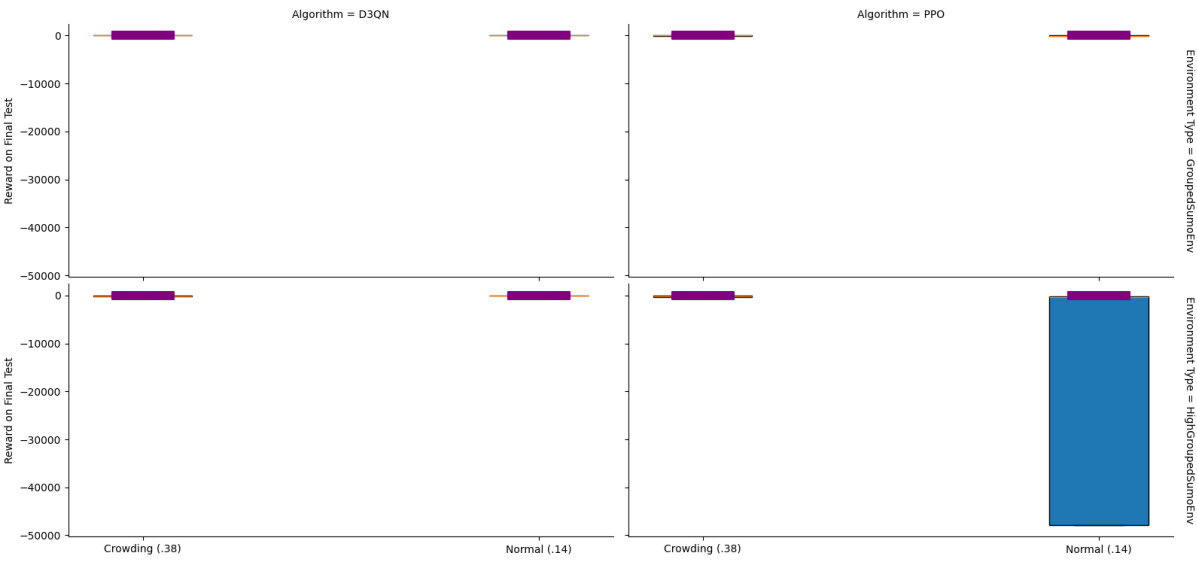
Performance Comparison (Using Reward : Literature)
Total Time Of Training (M) on area Area 3 (San Stefano)



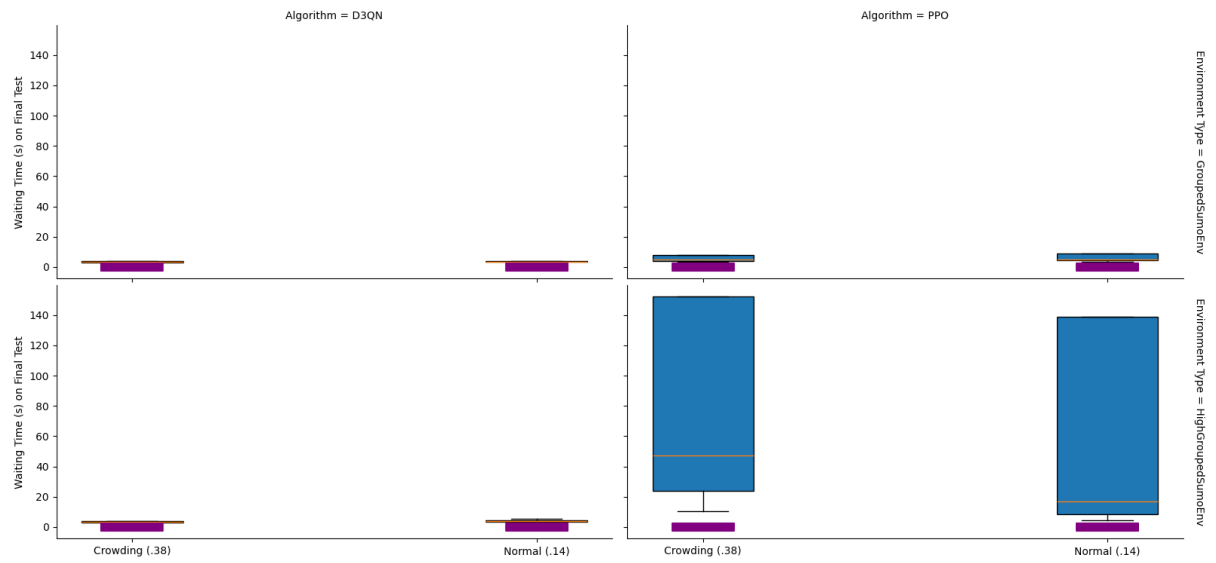
Performance Comparison (Using Reward : Proposed)
Reward on Final Test on area Area 3 (San Stefano)



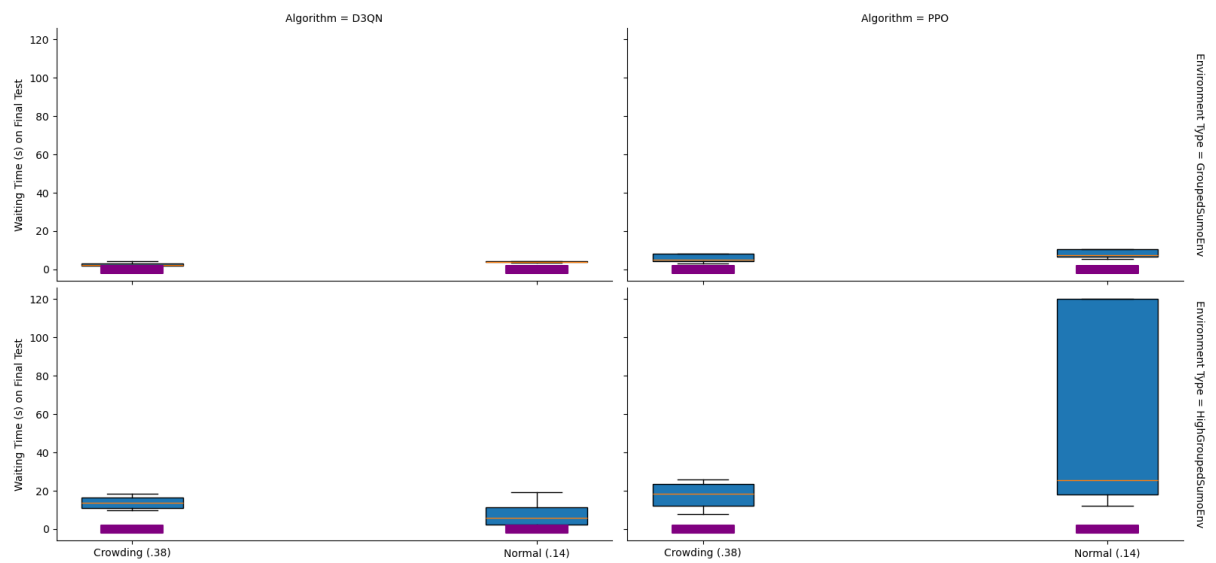
Performance Comparison (Using Reward : Literature)
Reward on Final Test on area Area 3 (San Stefano)



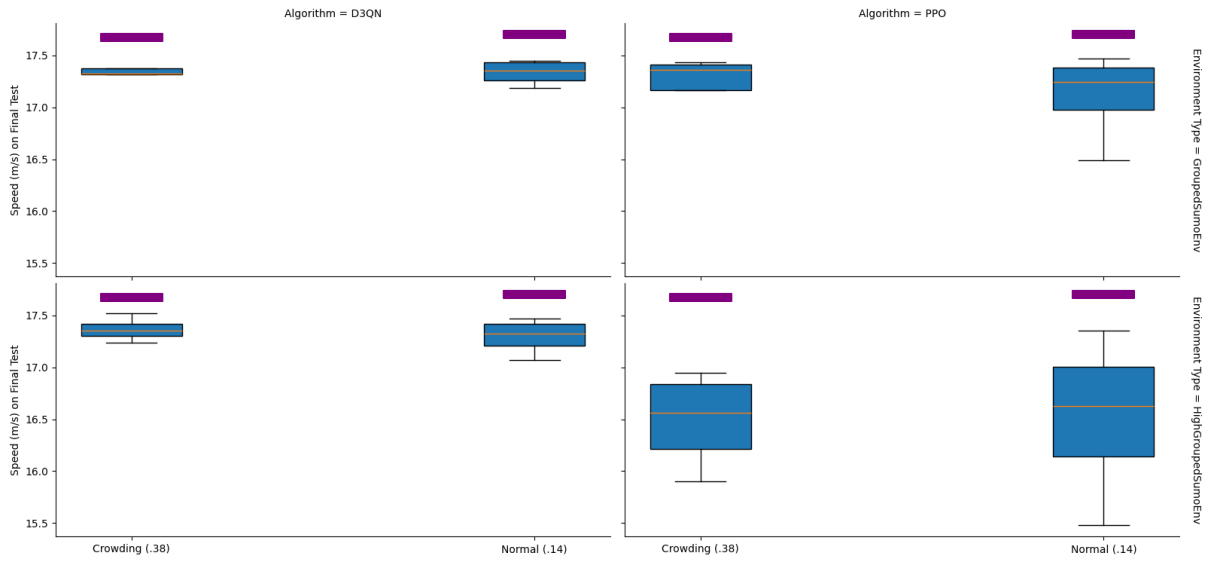
Performance Comparison (Using Reward : Proposed)
Waiting Time (s) on Final Test on area Area 3 (San Stefano)



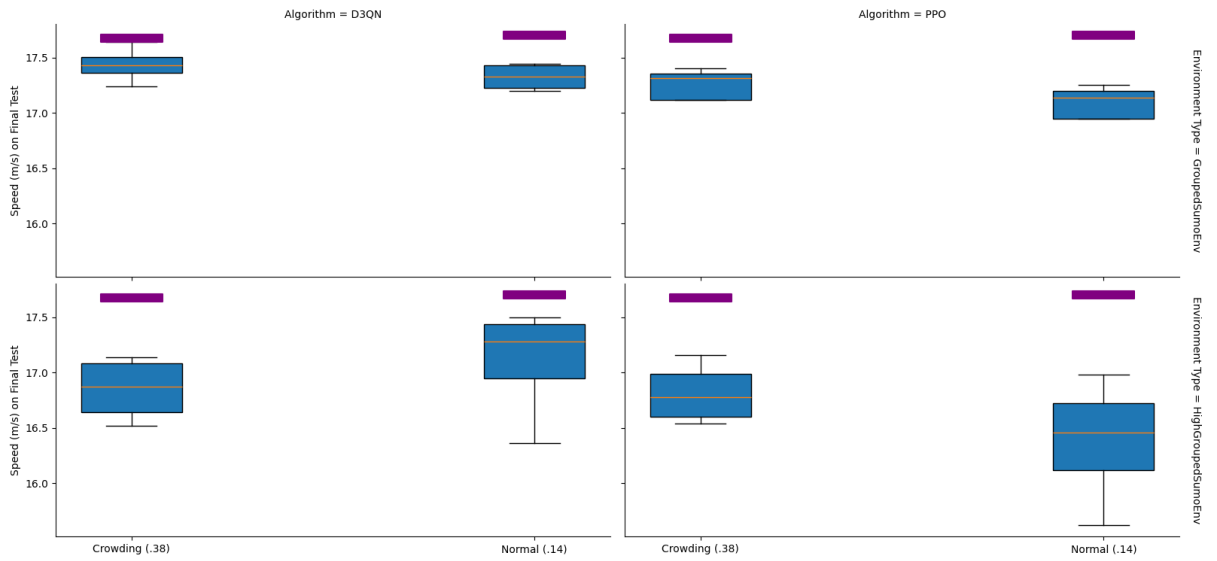
Performance Comparison (Using Reward : Literature)
Waiting Time (s) on Final Test on area Area 3 (San Stefano)



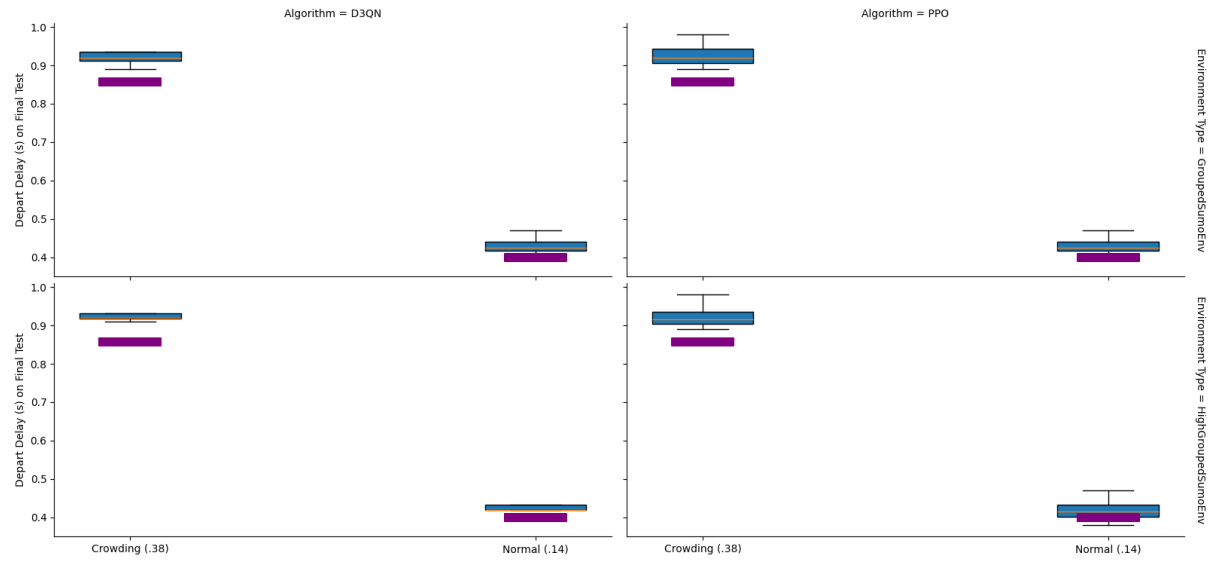
Performance Comparison (Using Reward : Proposed)
Speed (m/s) on Final Test on area Area 3 (San Stefano)



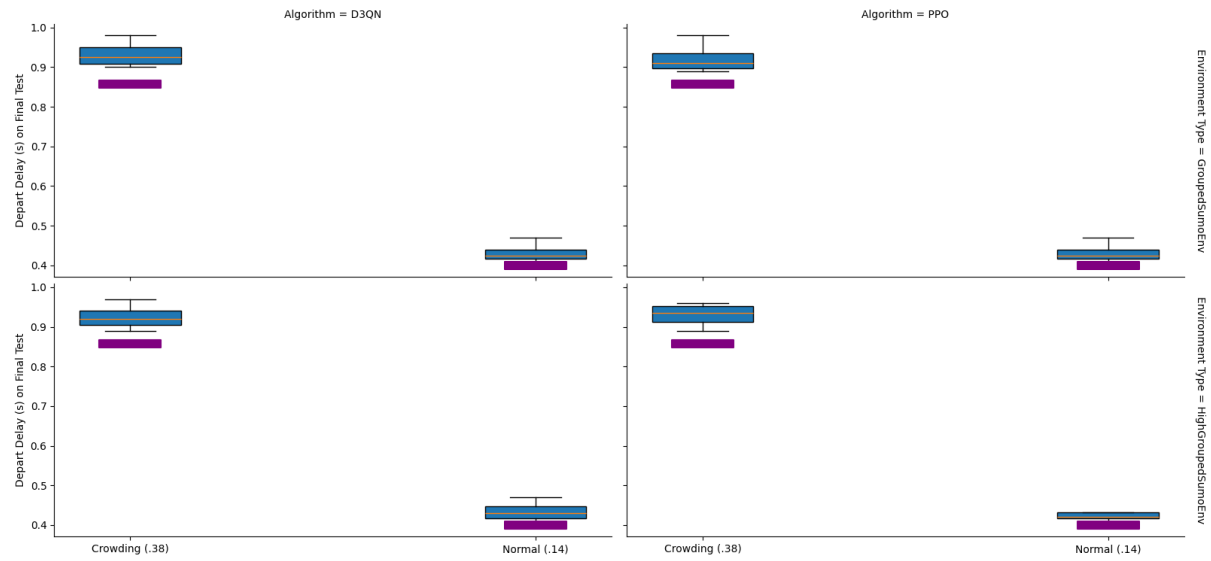
Performance Comparison (Using Reward : Literature)
Speed (m/s) on Final Test on area Area 3 (San Stefano)



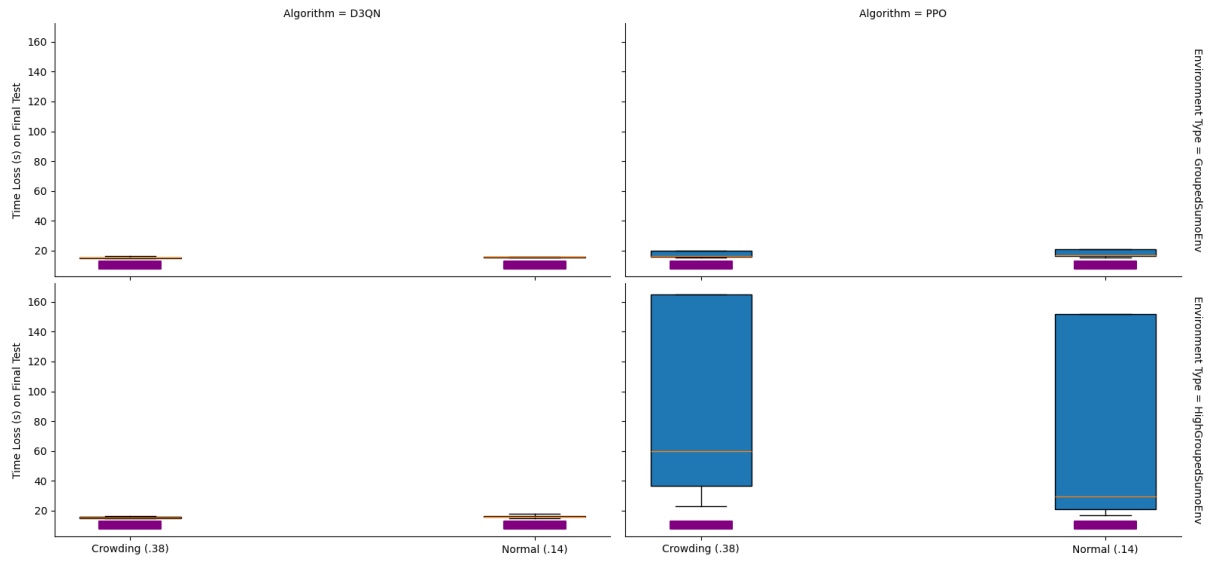
Performance Comparison (Using Reward : Proposed)
 Depart Delay (s) on Final Test on area Area 3 (San Stefano)



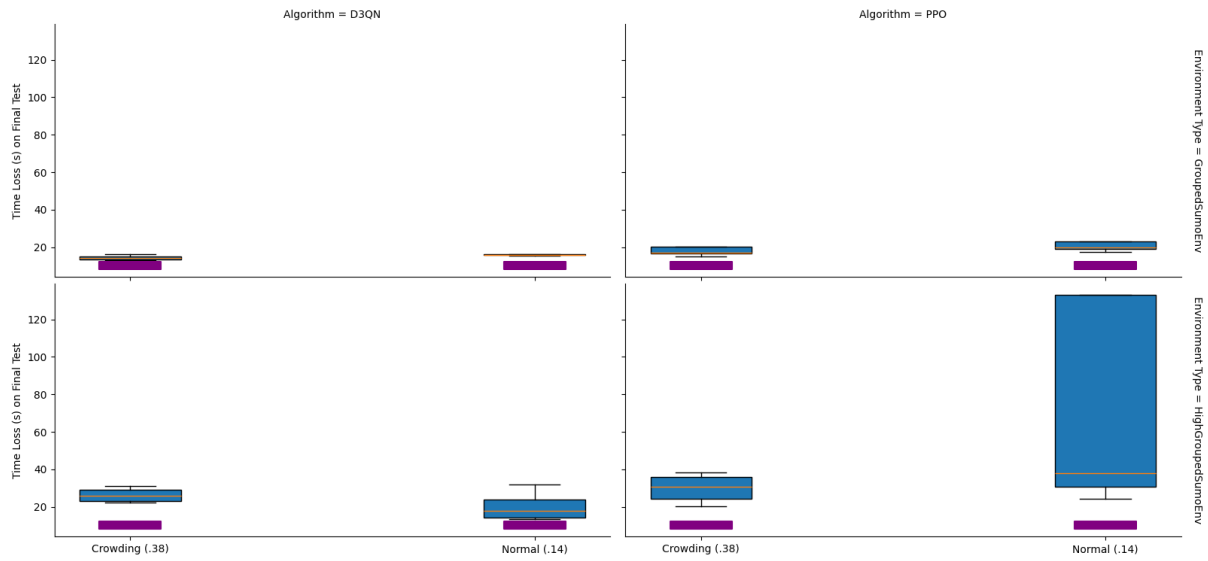
Performance Comparison (Using Reward : Literature)
 Depart Delay (s) on Final Test on area Area 3 (San Stefano)



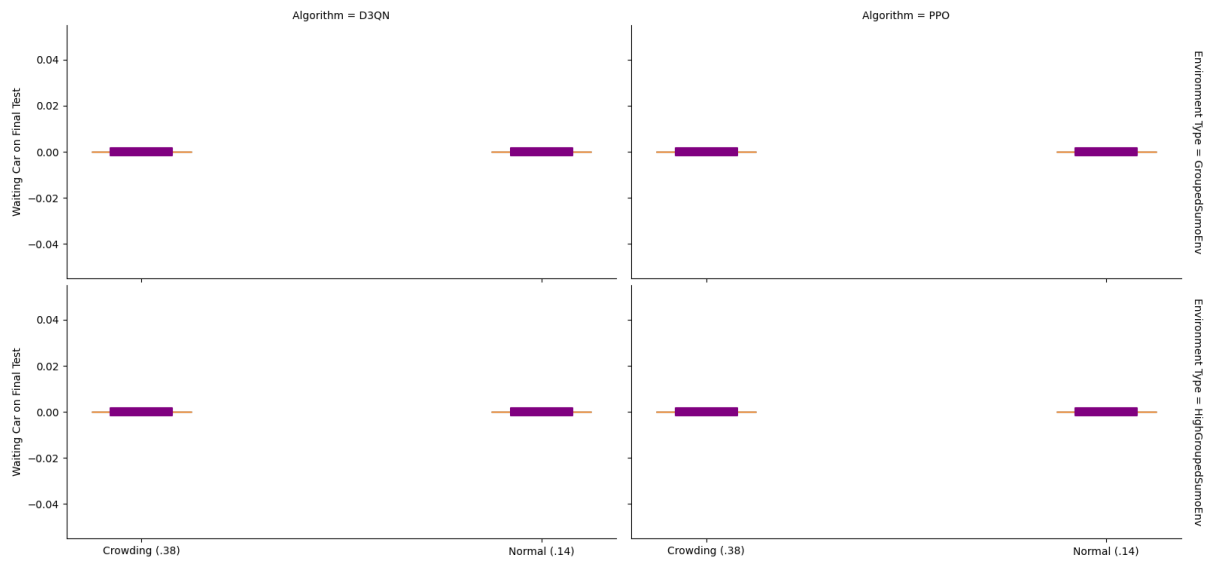
Performance Comparison (Using Reward : Proposed)
 Time Loss (s) on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Literature)
 Time Loss (s) on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Proposed)
Waiting Car on Final Test on area Area 3 (San Stefano)



Performance Comparison (Using Reward : Literature)
Waiting Car on Final Test on area Area 3 (San Stefano)

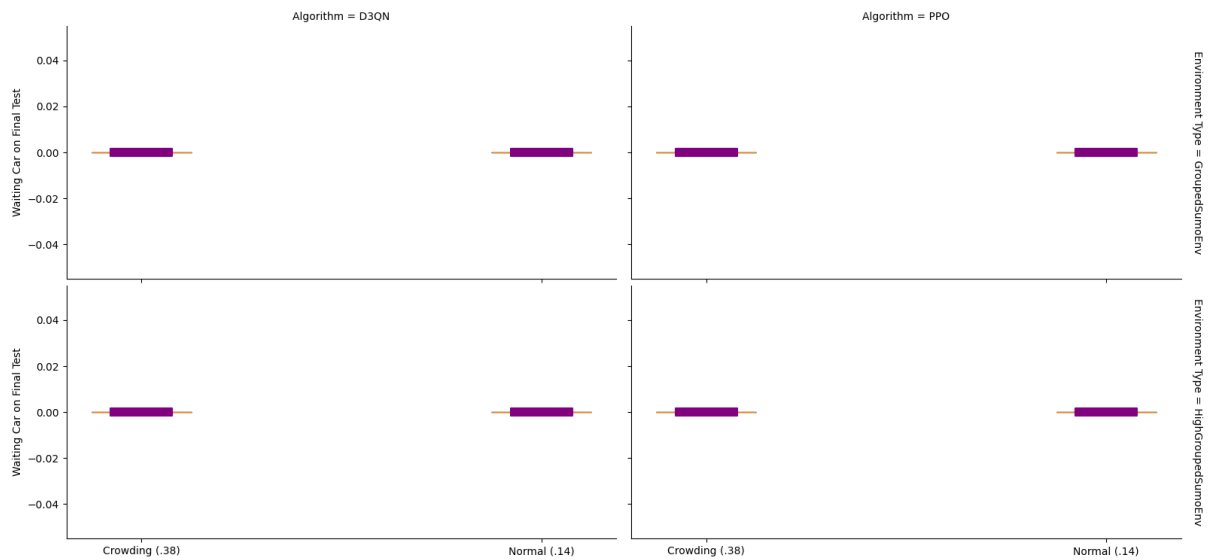


Figure 0.49 Results of Stefano at version 2.0.2

5.4.4.2.2.3 Anova

Area	Reward	Traffic Scale	Algorithm	Reward of Last Episode	Waiting Time (s)	Speed (m/s)	Depart Delay (s)	Time Loss (s)	Waiting
Area2 (Mosheer Ismail)	Proposed	Normal (.14)	PPO	0.174093	0.031968	0.330064	0.517569	0.800414	0.3643
Area 3 (San Stefano)	Proposed	Normal (.14)	PPO	0.960184	0.951592	0.951052	1	0.963889	
Area2 (Mosheer Ismail)	Literature	Normal (.14)	PPO	0.051094	0.400111	0.917906	0.99631	0.835703	0.3202
Area 3 (San Stefano)	Literature	Normal (.14)	PPO	0.291977	0.898301	0.576019	0.928534	0.856301	
Area2 (Mosheer Ismail)	Proposed	Crowding (.38)	PPO	0.218491	0.038619	0.024489	0.805826	0.016779	0.4117
Area 3 (San Stefano)	Proposed	Crowding (.38)	PPO	0.100271	0.709171	0.604623	1	0.728166	

Area2 (Mosheer Ismail)	Literature	Crowding (.38)	PPO	0.059923	0.933453	0.020783	0.344741	0.760575	0.7949
Area 3 (San Stefano)	Literature	Crowding (.38)	PPO	0.357383	0.584193	0.855345	1	0.623974	
Area2 (Mosheer Ismail)	Proposed	Normal (.14)	D3QN	0.184975	0.116823	0.028446	0.097345	0.058172	0.3196
Area 3 (San Stefano)	Proposed	Normal (.14)	D3QN	0.373108	0.790994	0.783433	1	0.717158	
Area2 (Mosheer Ismail)	Literature	Normal (.14)	D3QN	0.102138	0.960094	0.987272	0.928452	0.903518	0.9247
Area 3 (San Stefano)	Literature	Normal (.14)	D3QN	0.246665	0.683067	0.896514	1	0.700729	
Area2 (Mosheer Ismail)	Proposed	Crowding (.38)	D3QN	0.567614	0.64958	0.744536	0.897984	0.667898	0.5761
Area 3 (San Stefano)	Proposed	Crowding (.38)	D3QN	0.254087	0.946033	0.988809	0.92079	0.954675	
Area2 (Mosheer Ismail)	Literature	Crowding (.38)	D3QN	0.054542	0.34432	0.352183	0.307016	0.220703	0.7698
Area 3 (San Stefano)	Literature	Crowding (.38)	D3QN	0.01578	0.313325	0.54331	1	0.314806	

5.4.4.2.2.4 Discussion

5.4.4.2.2.4.1 Discussion of results

- For the Mosher Ismail dataset, the proposed model outperformed the baseline approach across multiple performance metrics, including reward value, waiting time, departure delay, time loss, and the number of vehicles waiting. Conversely, for the San Stefano dataset, the SUMO-RL framework achieved slightly better overall performance, although the difference between the two environments was low.

Mosheer:

- At moderate congestion (scale 1.14), traffic conditions may fluctuate across approaches, leading to suboptimal phase switching. Under heavier congestion (scale 1.38), dominant traffic streams become more apparent, allowing the controller to allocate green time more efficiently and reduce overall delay metrics.

- Within the SUMO-RL framework, PPO achieved superior performance in terms of time loss, departure delay, and average speed. When applied to the proposed environment, PPO demonstrated greater stability, particularly with respect to reduced stochastic randomness. This can be attributed to the clipping mechanism in PPO, which enhances robustness in complex and highly dynamic environments.

- The proposed reward formulation yielded better results for departure delay and time loss compared to the literature baseline, although it resulted in slightly lower average speed. This outcome is expected, as the reward function was specifically tailored to the designed state space and traffic scale factors, prioritizing congestion-related metrics over speed maximization.

- Both environment levels produced comparable overall results, with only marginal differences across key performance metrics.

- The observed performance distribution remained consistent even under higher simulation durations and larger traffic demand, indicating robustness of the experimental findings.

Stefano:

- The SUMO-RL algorithm demonstrated rapid convergence on the San Stefano dataset, attaining near-optimal performance without the need for additional gradient-based tuning, suggesting that the initial parameter initialization was sufficient for effective learning.
- Higher traffic demand led to increased departure delays, as expected under congested conditions. Conversely, low numbers of vehicles resulted in minimal waiting time, indicating that the model effectively optimizes traffic flow across both low- and high-demand scenarios.
- Reduced stochastic variability was observed when using the grouped SUMO environment combined with the D3QN algorithm. and also in some cases, performance metrics such as waiting time are better. The D3QN's dueling architecture proved particularly effective for simpler datasets, contributing to more stable and reliable outcomes.
- The choice of reward function did not significantly affect overall performance, indicating that the model's behavior is robust across different reward formulations.

5.4.4.2.2.4.2 Best Performance Metrics**Mosher Ismail Dataset:****Best Combination**

Grouped (due to complex nature of env) - PPO – Proposed

3000 s simulation:

- * Waiting time: 150 s (range 100–220)
- * Speed: 8.5 m/s(range 6–9)
- * Departure delay: 7 s (range 3–20 s)
- * Time loss: 220 s = 3.6 M (range 170–260 s)
- * Waiting cars: 12 (range 5–25)

10800 s simulation:

- * Waiting time: 960 s = 16 M (range 4M- 29M) =
- * Speed: 5 m/s (range 4-8)
- * Departure delay: 22 s (range 20–50s)
- * Timeloss: 1600 s = 26 M (range 13M-41M)
- * Waiting cars: 10 (range 5–30)

San Stefano Dataset:

Best Combination

grouped - D3QN - Proposed (as smoothly integrate)

3000 s simulation:

- * Waiting time: 3 s (range 2–6)
- * Speed: 17.4 m/s
- * Departure delay: .9s
- * Timeloss: 15 s = (range 13,17.5)
- * Waiting cars: 1

10800 s simulation:

- * Waiting time: 5 s
- * Speed: 17.3 m/s
- * Departure delay: 1s
- * Timeloss: 20s
- * Waiting cars: 2

5.4.4.2.2.4.3 Comparison to last version

- The proposed environment achieved lower waiting time and time loss; however, this improvement was associated with a reduction in average vehicle speed.
- The grouped environment demonstrated greater stability, particularly in terms of variance and randomness

- No statistically significant difference was observed between the environments with respect to the reward metric.

5.4.4.2.2.4.4 Discussion of Results

In this section the results of version 2.0.2 will be discussed, in terms of showing what was expected and what was seen, with reason and suggestions.

Table 0.21 Discussion of environmental results

Metric	Expected	Reason (Expected)	Found	Reason (Found)
Algorithm	D3QN	It was selected due to its frequent use in related work and its reported stability.	(depends on environment)	Minimal differences were observed between the two algorithms; although less common in this domain, PPO converged adequately and therefore produced usable results.
Reward	Proposed	It was designed to incorporate additional traffic-related metrics that accurately describe road.	Proposed-based reward	—
Environment	HighGrouped	This behavior was considered the default pattern observed in Egyptian streets.	Grouped	The groups represent the actions assigned to each lane direction, thereby making the control actions more organized and efficient.
Episodes	Large	A high number of episodes were expected due to the typical	Few	The environment was not highly complex for a single agent setting, and the

		requirements of reinforcement learning training.		selected algorithm demonstrated sufficient learning with fewer episodes. It's also common in related work to focus on training on large time instead episodes
--	--	--	--	---

5.5 Final Results of System

For Both Mosheer Ismail and San Stefano

Best Algorithm: Mosher: PPO, Stefano: D3QN

Best Configuration Grouped

Best Reward Function Proposed-defined reward

With best values as described in 5.4.4.2.2.4.3

Results of System

- The proposed environment achieved strong performance on both the Mosher and San Stefano datasets, particularly in reducing the number of waiting vehicles, this happens for most metrics. However, in highly congested scenarios, where increased time loss and reduced average speed were recorded.
- Significant variations in certain metrics (e.g., waiting time, time loss, and speed) can be attributed to differences in space design and the interaction mechanism between the agent and the environment.
- Both the literature reward and the proposed reward achieved high results, indicating that the reward formulation is effective. The proposed reward structure is more suitable for real-world deployment as it can be integrated smoothly into operational systems.

- The grouped environment exhibited greater stability in the San Stefano dataset. This may be explained by its coordinated signal strategy, where phases serving the same traffic direction are activated together, resulting in smoother traffic flow.
- The effectiveness of the learning algorithm varied across datasets, highlighting the influence of traffic characteristics and network structure on model performance so using specific algorithm needs on the dataset (area).
- Some degree of stochastic variability was observed across experiments, which is typical in reinforcement learning. However, compared to earlier versions, the proposed model demonstrated improved stability and reduced randomness.
- Stochastic variability inherent to reinforcement learning models remained evident, particularly in later training stages. Nevertheless, the proposed model showed stronger generalization capability, maintaining stable performance over extended testing periods and exhibiting reduced signs of overfitting.

Results of Prototype

- In the prototype, the system was demonstrated to be operable outside SUMO, allowing real traffic light control.
- In *DBOnline*, the environment was shown to be retrievable through a database connection.
- By using the mobile application, it was demonstrated that the system can be manually activated or deactivated, proving it is not uncontrollable.
- The system was designed to be scalable—from a single agent to any number of lanes.
- Computer vision was used; however, for lower latency, physical road sensors are recommended.

As shown in [APPENDIX VI: File Structure], the *Core* files are detailed there.

In summary, as explained in this section and in Sections 5.4.4.2.2.4.3(Best Performance Metrics) and 5.1.1 (Problem and Environment Design), the main goal is to obtain a system with very low waiting time and straightforward deployment, and these objectives have been achieved.

5.9 Conclusion

In this chapter, the environmental design was discussed, including the data, reward structure, state representation, and action space. The chapter was then dived into the

presentation of experiment and results. The next chapter will have final summary and future work.

CONCLUSION AND FUTURE WORK

In the previous chapter, the environment design, the data used, the state space, the action space, the reward structure, and the chosen model were presented. The selection of hyperparameters and the detailed environment setup were then explained. This was followed by the presentation of the environment's results and the implementation of the system prototype.

In this chapter, the final conclusions of the report will be presented, along with an outline of potential directions for future work.

6.1 Conclusion

Urban traffic management has been recognized as a critical issue influencing daily life and energy consumption. Traditional systems have been constrained by inefficient energy utilization and suboptimal traffic flow. In this study, a system was designed using reinforcement learning to optimize traffic while accounting for emergencies and accidents. The system was iteratively improved until waiting time, time loss, and the number of waiting vehicles were reduced to only a few seconds. Furthermore, the design was focused on ensuring deploy ability without encountering the reinforcement learning generalization problem.

6.2 Future work

According to section 3.4 Future work:

- Improve reward by solving the anomaly in stability [5.4.3.1.1 Normalizing reward and testing it].
- Scaling to dependent multi-agent systems will be implemented.
- Consideration will be given to state, temporal, spatial, and nearby information as a new trend [3].
- Improvements are needed in the calculation of the step function. Specifically, the handling of the greatest common divisor (GCD) of durations requires refinement, as some unexpected behaviors were observed [V2.0.1]. Additionally, the initial action should be executed through the step function rather than as a separate first action. [5.4.2.1 Key Modifications].
- A minor bug was observed during experiment execution: the max step was set to 100, which is smaller than the maximum duration plus yellow phase, which may cause the system to operate faster than intended [5.4.3.1 Key Modifications].
- The process of identifying and selecting the signal that aligns with the user's direction will be further investigated as described in [4.5.4 Mobile implementation].

- Signal duration and delay adjustments will be automated to reduce manual configuration and improve responsiveness as described in [4.5.4 Mobile implementation].
- User experience in the mobile app will be improved by notifying users, helping them select roads, predicting better routes, and gathering feedback [32].
- Cybersecurity measures will be considered as described in [4.8 Basic implemented security][32], [33].
- Control will be established for Egypt's GPS to identify traffic violators.

REFERENCES

- [1] K. Sabra, M. Moustafa, et al., "Intelligent Traffic Analysis," B. Sc. Degree, Graduation Project, Computer and Systems Engineering Department, Faculty of Engineering, Alexandria University, 2023.
- [2] L. N. Alegre, "SUMO-RL," GitHub repository, 2019. [Online]. Available: <https://github.com/LucasAlegre/sumo-rl>. [Accessed: Dec. 7, 2024].
- [3] M. M. Dhanvijay and S. C. Patil, "Energy efficient deep reinforcement learning approach to control the traffic flow in IoT networks for smart city," J. Ambient Intell. Human Comput., vol. 15, pp. 3945–3961, Dec. 2024, doi: 10.1007/s12652-024-04869-w.
- [4] TWI, "What is simulation?," TWI Global. [Online]. Available: <https://www.twi-global.com/technical-knowledge/faqs/faq-what-is-simulation#WhatDoesitMean>. [Accessed: Dec. 1, 2024].
- [5] "Gymnasium Documentation," Gymnasium. <https://gymnasium.farama.org/> (accessed Jan. 20, 2025).
- [6] DLR, "SUMO Tutorials," SUMO Documentation, Available: <https://sumo.dlr.de/docs/Tutorials/>, Accessed: Nov. 18, 2024
- [7] GeeksforGeeks, "Machine Learning," GeeksforGeeks, [online]. Available: <https://www.geeksforgeeks.org/machine-learning/>. [Accessed: 1-Dec-2024].
- [8] Cornell University, "Heuristic Algorithms," accessed Dec. 20, 2024. [online]. Available: https://optimization.cbe.cornell.edu/index.php?title=Heuristic_algorithms&utm_source=chatgpt.com
- [9] Coursera, *Introduction to Machine Learning Specialization*, Available, Accessed: Nov. 18, 2024
- [10] IBM, "What are decision trees?" IBM, [online]. Available: <https://www.ibm.com/topics/decision-trees>. [Accessed: 1-Dec-2024].
- [11] ResearchGate, "A single neuron model," accessed Dec. 20, 2024. [online]. Available: https://www.researchgate.net/figure/A-single-neuron-model_fig1_317428704
- [12] Hugging Face, *Deep Reinforcement Learning Course*, Hugging Face, Available, Accessed: Nov. 18, 2024.
- [13] Hugging Face, "Offline and Online Reinforcement Learning," Hugging Face, Available, Accessed: Nov. 18, 2024.
- [14] H. Liu, J. Niles-Weed, K. S. Tai, and E. Wong, "Loss Landscape Smoothing: A Generalized Perspective to Sharpness-Based Regularization," arXiv, Available, Accessed: Nov. 18, 2024
- [15] Hugging Face, "The Bellman Equation," Hugging Face, Available, Accessed: Nov. 18, 2024.
- [16] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3, pp. 279-292, 1992, doi: 10.1007/BF00992698.
- [17] H. Hasselt, "Double Q-learning," in Advances in Neural Information Processing Systems 23 (NIPS 2010), 2010, pp. 1-9.
- [18] Hugging Face, "Advantage Actor-Critic," Deep Reinforcement Learning Course, accessed Dec. 20, 2024. [Online]. Available: <https://huggingface.co/learn/deep-rl-course/unit6/advantage-actor-critic>

- [19] C. Wu, I. Kim, and Z. Ma, "Deep Reinforcement Learning Based Traffic Signal Control: A Comparative Analysis," in Proc. 14th Int. Conf. Ambient Syst., Netw. Technol. (ANT), Leuven, Belgium, Mar. 15-17, 2023. Available: <https://www.sciencedirect.com/proceedings/browse>.
- [20] M. Abdoos and A. L. Bazzan, "Hierarchical traffic signal optimization using reinforcement learning and traffic prediction with long short-term memory," *Expert Systems with Applications*, vol. 171, p. 114580, 2021.
- [21] N. Kumar, S. S. Rahman, and N. Dhakad, "Fuzzy inference enabled deep reinforcement learning-based traffic light control for intelligent transportation system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 8, pp. 4919–4928, 2020.
- [22] T. Hu, Z. Hu, Z. Lu, and X. Wen, "Dynamic traffic signal control using mean field multi-agent reinforcement learning in large scale road-networks," *IET Intelligent Transport Systems*, 2023.
- [23] Hugging Face, "Variance Problem," Deep Reinforcement Learning Course, accessed Dec. 20, 2024. [Online]. Available: <https://huggingface.co/learn/deep-rl-course/unit6/variance-problem>
- [24] M. Wang, L. Wu, J. Li, and L. He, "Traffic signal control with reinforcement learning based on region-aware cooperative strategy," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 7, pp. 6774–6785, 2021.
- [25] Hugging Face, "Clipped Surrogate Objective," Deep Reinforcement Learning Course, accessed Dec. 20, 2024. [Online]. Available: <https://huggingface.co/learn/deep-rl-course/unit8/clipped-surrogate-objective>
- [26] S. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2012.
- [27] R. Kundu, "YOLO: Algorithm for Object Detection Explained [+Examples]," V7. <https://www.v7labs.com/blog/yolo-object-detection>
- [28] Q. U. Jamil et al., "Urban traffic signal control optimization through Deep Q Learning and double Deep Q Learning: a novel approach for efficient traffic management," pp. 1-24, 2024.
- [29] M. Al-Turki, M. T. Kashifi, and N. T. Ratrout, "Traffic signal optimization framework using interpretable machine learning technique under heterogeneous-autonomy traffic environment," *Neural Computing and Applications*, vol. 36, no. 22, pp. 13761–13781, Aug. 2024, doi: 10.1007/s00521-024-09694-y.
- [30] L. Wang, Z. Ma, C. Dong, and H. Wang, "Human-centric multimodal deep (HMD) traffic signal control," *IET Intelligent Transport Systems*, vol. 17, no. 4, pp. 744-753, 2023.
- [31] "Traffic Management Software Development," Intellias. <https://intellias.com/traffic-management-softwaredevelopment/> (accessed Dec. 25, 2022)
- [32] Advanced traffic management system software TOPXVIEW | Telegra (telegra-europe.com)
- [33] Iteris, "A leader in smart mobility," [Online]. Available: <https://www.iteris.com/>. [Accessed: Nov. 19, 2024].
- [34] Kapsch TrafficCom, "Specializes in intelligent transportation systems, offering solutions for tolling, traffic management, and vehicle-to-infrastructure communication," [Online]. Available: <https://www.kapsch.net/de>. [Accessed: Nov. 19, 2024].

- [35] Software Engineering Competence Center (SECC), SPIG Product Suite Handbook V1.2, Software Process Improvement Guide (SPIG), 1st ed., Giza, Egypt: Information Technology Industry Development Agency (ITIDA), Ministry of Communications and Information Technology (MCIT), 2010.
- [36] C. Liechti, "pySerial: Python Serial Port Access Library," GitHub repository. [Online]. Available: <https://github.com/pyserial/pyserial>. [Accessed: Jun. 10, 2025].
- [37] Streamlit Inc., "Streamlit," ver. 1.35, [Online]. Available: <https://streamlit.io/>. [Accessed: Jun. 10, 2025].
- [38] T. Simonini, "Generalization in Reinforcement Learning," Deep Reinforcement Learning Course. [Online]. Available: <https://huggingface.co/learn/deep-rl-course/unitbonus3/generalisation>. Accessed: Jun. 11, 2024.
- [39] OpenStreetMap contributors, "Planet dump," OpenStreetMap, 2017. [Online]. Available: <https://planet.osm.org>. [Accessed: Day Month Year].
- [40] "Reinforcement Learning Tips and Tricks — Stable Baselines 2.10.3a0 documentation." https://stable-baselines.readthedocs.io/en/master/guide/rl_tips.html
- [41] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement learning Implementations," 2021. <http://jmlr.org/papers/v22/20-1364.html>
- [42] TomTom, "Alexandria Traffic," *TomTom Traffic Index*, accessed Jun. 12, 2024. [Online]. Available: <https://www.tomtom.com/traffic-index/alexandria-traffic/>
- [43] https://sumo.dlr.de/docs/Tools/Trip.html?utm_source=chatgpt.com
- [44] https://www.sciencedirect.com/science/article/pii/S1110016820300715?utm_source=chatgpt.com
- [45] https://www.tomtom.com/traffic-index/about/?utm_source=chatgpt.com
- [46] https://sumo.dlr.de/docs/Vehicle_Type_Parameter_Defaults.html?utm_source=chatgpt.com
- [47] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 92–96.

APPENDIX I: CHALLENGE FACED

1. Project Complexity and Design

To address the inherent complexity of the system, several open-source projects were referenced during development. A review of relevant research papers was conducted to understand prior methodologies. Development began with an initial phase involving a simplified environment, which was gradually expanded. The SPI model was followed to progressively incorporate more complex components into the system

2. Action Space Complexity

A hierarchical, three-level action space was designed to manage decision complexity and enhance control mechanisms. This structure allowed modular handling of agent behavior and facilitated scalability [5.1.4 Action Space].

3. Compatibility with SUMO and Real-World Data

To ensure compatibility between SUMO simulations and real-world video data, the logic associated with SUMO was decoupled from the main environment code. A unified Connection class was introduced, serving as a parent to both SumoConnection and RealConnection (for computer vision input). All environmental communication was routed through this abstraction [4.4.5 Class Diagram]. Additionally, certain road-related information, such as lane configurations and directions, was extracted from the road network and stored in a file so that it remained independent of SUMO.

4. Object Serialization Issues in Configuration Passing

Due to serialization issues when passing objects such as Connection during environment configuration, a workaround was implemented. The connection was instead set and retrieved using setter and getter methods, avoiding direct object transfer.

5. Frame Processing Latency with YOLO

Significant processing delays were observed due to the size of the YOLO model. To mitigate this, IP-based video capture was employed. Frames were requested based on synchronized time intervals aligned with the processing cycle, and frame timing was monitored for performance tuning [4.5.1 Computer Vision Implementation].

6. Thread-Safety Issues in IP Video Access

Accessing IP video feeds concurrently from both the browser and backend introduced thread-safety issues. A locking mechanism was implemented to ensure exclusive access to the video stream by a single thread at any given time [4.5.1 Computer Vision Implementation].

7. Backend Communication for IoT Devices

To support continuous listening by IoT devices without overwhelming the server or overloading low-end hardware, MQTT streaming was adopted. This allowed efficient, lightweight communication between clients and the server [4.5.3 Backend Implementation].

8. Shared State Management in the Backend

A mechanism was required for maintaining a shared variable representing the current system state. A centralized state file was created where shared variables and requests were globally defined and accessed as needed.

9. Access Control and User Management

To enforce secure access to the application, a user verification system was implemented. Only administrative users were granted the ability to register or add new users, ensuring controlled access to system features [4.5.3 Backend Implementation].

10. Large Departure Delay and Values

During the initial stage (Version 1), when the first experiment was conducted, a significant departure delay was observed. This issue was mitigated by applying the step-based approach as described in [30] [5.4.3.1 Key Modifications of version 2.0.0].

11. I/O Bottleneck

A bottleneck was encountered during the reset process. In each reset, the output was stored in `osm.statistics.xml`, and the metrics were subsequently extracted by converting the file into CSV format, which was then read using a data processing library. When repeated for many episodes (e.g., 350 or 1000), this approach introduced substantial overhead. The problem was resolved by directly parsing the XML file, thereby eliminating the intermediate conversion step.

12. Unexpected Lane Identifiers

In some cases, unexpected lane identifiers were generated by the simulation, where lane IDs appeared to be duplicated. This issue was identified as a deployment-related problem. To address it, the edges connected to the reported lane ID were explicitly verified, and the correct lanes were determined based on these connections.

13. SUMO-RL many resets

In SUMO-RL, the benchmark package was automatically reset, which caused the OSM metric results to be deleted. This issue was resolved by conducting all experiments in SUMO-RL using a test environment, where a continuous loop was maintained with a high iteration limit, restarting only when the maximum simulation time was reached.

14. Reward testing

Proper validation of the reward function must be ensured. This issue was addressed through mathematical modeling rules, followed by systematic testing of the reward and benchmarking it against other reward functions.

APPENDIX II: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1.Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the Traffic Manager System, which integrates Artificial Intelligence (AI) and Reinforcement Learning (RL) to optimize urban traffic flow. The system aims to reduce congestion, enhance mobility, and provide real-time decision-making for traffic management using IoT-enabled infrastructure.

1.2 Scope

The system shall dynamically adjust traffic signals based on real-time conditions, prioritize emergency vehicles, and offer real-time traffic updates via a mobile application. The system will be designed to scale from a single intersection prototype to a city-wide intelligent traffic control system.

1.3 References

[1],[2],[3],[22],[24],[32],[34]

2. System Requirements

2.1 Functional Requirements (FR)

FR-01: Intelligent Traffic Light Control

- The system shall utilize Reinforcement Learning (RL) to dynamically optimize traffic signals based on real-time sensor inputs.
- The system shall aim to minimize vehicle waiting times, depart delay, timeLoss and Maximize speed.

FR-02: IoT-Enabled Data Collection

- The system shall collect real-time traffic data from cameras.
- The system shall integrate computer vision to detect vehicle movement.
- The system shall store collected data securely for long-term analysis and AI model improvement.

FR-03: Emergency Vehicle Priority Management

- The system shall enable registered emergency vehicles (ambulances, fire trucks) to log in, select a route, and transmit their location.
- The system shall automatically adjust traffic signals along the emergency route by extending green light durations and minimizing red light durations.
- The system shall ensure priority handling does not disrupt overall traffic efficiency.
- The system shall allow only registered emergency users (e.g., ambulance drivers, police, fire personnel) to report accidents by specifying the location and severity.

FR-04: Accident-Based Traffic Signal Control

- The system shall allow authenticated users to report accidents by specifying the location and severity.
- Upon receiving a valid accident report, the system shall automatically adjust traffic signals near the reported location by extending red light durations and minimizing green light durations to restrict vehicle movement.

FR-05: Data-Driven Traffic Simulation and Optimization

- The system shall utilize SUMO (Simulation of Urban Mobility) to simulate traffic behavior and train initial traffic control models.
- The system shall be improved using collected offline traffic experience data (e.g., historical patterns) to refine decision-making algorithms.
- The system shall apply and validate the optimized models using real-time traffic data for live analysis and control.
- The system shall enable continuous monitoring of system performance, including metrics such as traffic flow efficiency, model accuracy.

2.2 Non-Functional Requirements (NFR)

NFR-01: Real-Time Processing & Low Latency

- The system shall process incoming data and optimize traffic flow in real-time.
- The system shall maintain low-latency decision-making for seamless traffic signal adjustments.

NFR-02: Scalability & Expandability

- The system shall be designed to support deployment at any number of intersections, regardless of whether they follow the same or different traffic flow directions.
- The system shall be designed for deployment at a single intersection with the ability to scale to city-wide deployment.
- The system shall support coordination between multiple traffic lights.

3. Tools & Environment

- Simulation Framework: SUMO, OpenAI Gym
- Machine Learning Framework: Python, TensorFlow, StableBasline3, Ultralytics
- Mobile Development: Flutter
- Hardware: IoT sensors (cameras)
- Video Processing: OpenCV
- Deployment: Aiven, Huggingface, fastapi

4. Conclusion

This document outlines the critical system requirements for the Traffic Manager Project, ensuring alignment with IEEE 1233-1998 standards. It provides a structured breakdown of functional and non-functional requirements, tools, and key system components. The system aims to deliver intelligent, adaptive, and sustainable traffic management solutions for modern urban environments.

APPENDIX III: MINUTES OF MEETING

Minutes of meetings are considered formal records of the discussions and decisions made during a meeting, essential for ensuring clarity and accountability. The date and time of the meeting, the location, and a list of attendees are typically included. The key points discussed, decisions made, and any action items assigned are captured, along with the responsible individuals for those tasks. Additionally, the time of the next scheduled meeting is documented, providing a clear reference for future follow-ups and ensuring that all participants are aligned on upcoming activities and deadlines. This comprehensive documentation helps ensure transparency and continuity across the project or organizational activities.

Minutes of Meetings – Tuesday 12/Nov/2024

Place	Online
Date and time	12 th of November 2024 at 8:30 pm
Attendees	Taha Fawzy Anwar Elshrif Ahmed Ibrahim Hassan Kerolss Bolis Shehata Remon Ehab Ramses Youssef Ibrahim Hanfy Marwa Shebl Emad Mohammed Fawzy Mostafa Hussian Mostafa Eldeep Mayan Islam Mohamed Ahmed Bichay Adel Ramses
Minutes	
The points listed are not in the order of discussion but capture the essence of the meeting.	
1	Reviewed week work.

Figure III.1 screen of minutes of meeting

APPENDIX IV: WEEKLY TIMESHEET

A weekly timesheet is a record that is used to track the work hours of everyone for a given week. The tasks completed and the time spent on each activity are captured. This helps ensure progress is monitored, accountability is maintained, and an overview of productivity is provided.

1	Graduation Project Weekly Timesheet										
2	Project	From 1/12/2024		To 8/12/2024							
3											
4	Name	fods Graduation Project 2024_2025			Department	Data Science					
5											
6	Team Member	Taha Fawzy	Ahmed Ibrahim	Emad Mohammed	Mayan Islam	Kerolis Bails	Ramon Ehab	Masata Hussian	Marwa Shebl	Youssef Ibrahim	Bichai adel
7	Activity										
8	Writing/Updating Presentation	4	2	1	4	1	0	1	0	6	0

Figure IV.1 screen of a weekly timesheet

APPENDIX V: JIRA PLAN

To ensure effective project management and task coordination, we utilized JIRA as our primary planning tool. The following screenshot provides an overview of our workflow, highlighting task allocation, progress tracking, and milestone planning throughout the development lifecycle.

Traffic Manager										
Supervisor :		Dr. Amira A. Alshazly								
ACTIONS	ACTIVITIES	PREPARED BY	Progress	REVIEWED BY	OCT					
					7	14	21	30		
Researching Related Works and Applications	Build a strong scientific background by reading papers, summarizing them.	ALL	100%	ALL						
	Search about reliable open-sources that we can use in our project.	Taha	100%	ALL						
	Re-iterate the comparisons between related work while trying to add more extensions to the current work and finalizing the project scope.	ALL	100%	ALL						
3 Learning	Learn SUMO (Simulation of Urban MObility)	Mayan , Kerolos, Marwa,Bichai	100%						8 weeks	
	Study reinforcement learning	Taha, Ahmed ,Marwa, Mostafa ,Remon ,kerolos	100%							
	Run a portion of the simulations.	ALL	100%							

Figure V.1 screen of a JIRA Plan

APPENDIX VI: File Structure

This appendix provides detailed descriptions of the code files used in the project:

Core

- **AIST_Cleaned/AIST:** The dataset used in Alex preprocessing is stored here. *AIST* refers to version 2.0.1 of the data, which is optimized for real-world scenarios.
- **Connections/:** Scripts for managing different types of connections are included.
- **Sensors.py and Observations/:** The state space configuration is defined here—*Sensors* for Version 1 and *Observation* for Version 2.0.1.
- **SumoEnvSingleAgent.py:** The main environment implementation for single-agent SUMO experiments is contained here.
- **info_road.xlsx:** Road-related information generated from OpenStreetMap (OSM) data is defined here for repeated use. The area must be defined first using the code in *automated_experiments_running_py.py* under *enable_editing*.
- **config.yaml:** Configuration parameters for experiments and environment setup are stored here.
- **Additional files required for modular and functional design:**
 - **data_parser.py:** Data parsing operations, such as reading and processing Excel files, are handled.
 - **callback.py:** Callbacks used to control algorithmic behavior during training are defined.
 - **Utils_reporting.py:** Utility functions for formatted and readable output reporting are provided.
 - **Utils_running_singleAgent.py:** Shared utility functions used across modules, such as lane direction detection and GCD computation, are provided.

Benchmark

- **benechmark-project:** Files for the modified benchmark project [1] are contained here.
- **Automation Files:**
 - **Sumo:** Files for running basic SUMO experiments are contained here (not automated; parameters must be modified manually each time).
 - **benechmark-project:** The *experiments_test_dqn* script is used for running experiments (not automated; parameters must be modified manually each time).
 - **automated_experiments_running.ipynb/automated_experiments_running.py:** Environmental experiments are executed through these files, It also defines concrete example of using the environment, It runs experiments defined in *sheet_full_environment_experiments* (Search of first row where waiting time is null).
 - **SUMO-RL-Test:** Files for running SUMO-RL experiments are contained here.
 - **For rerunning Optuna:** The `--optuna` flag is used, and the `--save-parameters` flag saves the results in the *output/* folder. (This works for *automated_experiments_running* and *sumo-rl-test* only.)

- **results/:** Experiment result scripts and HTML reports of the best-performing runs are stored here.

Prototypes

- **Proto_type_files/:** Prototype files implemented using Arduino and computer vision are contained here. (Additional modules are provided through the GitHub links referenced in the README file.)
- **DBOnline/:** Files used for the database prototype are contained here.
- **Working/:** Scripts used during development—such as data cleaning, OpenStreetMap exploration, and video streaming applications—are contained here. These are included for demonstration purposes only and do not contribute to the final system.